JavaScript Handbook — Vol 1

the language — setup, types and operators, control flow, functions and closures, objects, strings and regex, arrays, collections and generators, this/prototypes/classes, modules and tooling (Ch 1–10)

10 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — Introduction and Setup**
   - 1.1 What JavaScript is
   - 1.2 ECMAScript and TC39
   - 1.3 JavaScript engines — how your code actually runs
   - 1.4 Where JavaScript runs — hosts
   - 1.5 Setting up and running code
   - 1.6 Hello, World! — your first program and the basic rules
   - 1.7 Comments
   - 1.8 Strict mode
   - Chapter 1 TL;DR
2. **Chapter 2 — Variables, Types and Operators**
   - 2.1 Variables: `let`, `const` and `var`
   - 2.2 Hoisting and the Temporal Dead Zone
   - 2.3 Data types and `typeof`
   - 2.4 Primitives vs objects
   - 2.5 Arithmetic operators
   - 2.6 Comparison operators and equality
   - 2.7 Logical operators, `??` and `?.`
   - 2.8 Bitwise and assignment operators
   - 2.9 Operator precedence
   - 2.10 Numbers in depth
   - 2.11 Booleans, truthy and falsy
   - 2.12 Type conversion summary
   - Chapter 2 TL;DR
3. **Chapter 3 — Control Flow and Loops**
   - 3.1 Conditional statements — making decisions
   - 3.2 `if`, `else` and `else if`
   - 3.3 The conditional (ternary) operator
   - 3.4 `switch`
   - 3.5 Loops — why and which
   - 3.6 The `for` loop
   - 3.7 The `while` loop
   - 3.8 The `do…while` loop
   - 3.9 The `for…in` loop
   - 3.10 The `for…of` loop
   - 3.11 `break`, `continue` and labels
   - 3.12 Choosing the right loop
   - Chapter 3 TL;DR
4. **Chapter 4 — Functions, Scope and Closures**
   - 4.1 What a function is
   - 4.2 Declarations vs expressions (and hoisting)
   - 4.3 Arrow functions
   - 4.4 Parameters: defaults, rest, `arguments` and spread
   - 4.5 Scope: global, function and block
   - 4.6 Hoisting — the complete picture
   - 4.7 Closures
   - 4.8 Higher-order functions and callbacks
   - 4.9 IIFE — Immediately Invoked Function Expressions
   - 4.10 Recursion
   - 4.11 Pure functions
   - Chapter 4 TL;DR
5. **Chapter 5 — Objects, Math, Date and JSON**
   - 5.1 What an object is
   - 5.2 Reading, adding, updating and deleting properties
   - 5.3 Methods and `this`
   - 5.4 Spread, rest and copying objects
   - 5.5 Destructuring objects
   - 5.6 `Object.*` static methods, descriptors and locking
   - 5.7 Getters and setters
   - 5.8 Optional chaining `?.`
   - 5.9 JavaScript's built-in (native) objects
   - 5.10 `Math`
   - 5.11 `Date`
   - 5.12 JSON
   - Chapter TL;DR
6. **Chapter 6 — Strings and Regular Expressions**
   - 6.1 What a string is
   - 6.2 Creating strings
   - 6.3 Template literals
   - 6.4 Working with strings: indexing, immutability, coercion, String objects
   - 6.5 Unicode: code units, code points and graphemes
   - 6.6 String methods
   - 6.7 Regular expressions: the pattern language
   - 6.8 Regex methods and techniques
   - 6.9 Modern regex features and practical recipes
   - Chapter TL;DR
7. **Chapter 7 — Arrays**
   - 7.1 Why arrays?
   - 7.2 Creating arrays
   - 7.3 Reading, changing, adding and removing
   - 7.4 The methods table: mutating vs non-mutating
   - 7.5 Iteration methods: map, filter, reduce and friends
   - 7.6 Sorting correctly
   - 7.7 Spread, rest and destructuring with arrays
   - 7.8 Arrays vs objects (vs Map/Set)
   - 7.9 Typed arrays (a short note)
   - Chapter TL;DR
8. **Chapter 8 — Maps, Sets, Iterators and Generators**
   - 8.1 `Map`
   - 8.2 `Set`

- 8.3 `WeakMap`, `WeakSet`, `WeakRef` and `FinalizationRegistry`
- 8.4 Iterators and the iterable protocol
- 8.5 Generators
- 8.6 Iterator helpers (ES2025)
- 8.7 Grouping: `Object.groupBy` and `Map.groupBy` (ES2024)
- Version summary (verified September 2026)
- Chapter TL;DR

9. **Chapter 9 — `this`, Prototypes and Classes**
   - 9.1 `this` — the call-time context
   - 9.2 `call`, `apply` and `bind`
   - 9.3 Prototypes and the prototype chain
   - 9.4 Constructor functions and `new`
   - 9.5 Classes
   - 9.6 Inheritance with `extends` and `super`
   - 9.7 `instanceof` and type checks
   - 9.8 Mixins
   - 9.9 Composition vs inheritance
   - Chapter 9 TL;DR
10. **Chapter 10 — Modules and Tooling**
    - 10.1 Why modules?
    - 10.2 `export` and `import` syntax
    - 10.3 How ES modules load
    - 10.4 Dynamic `import()`
    - 10.5 Top-level `await`
    - 10.6 `import.meta` and JSON modules
    - 10.7 Modules in the browser
    - 10.8 CommonJS vs ES modules in Node
    - 10.9 npm and `package.json`
    - 10.10 Semantic versioning, ranges and lockfiles
    - 10.11 Bundlers
    - 10.12 Transpilers and polyfills
    - 10.13 Linters and formatters
    - 10.14 Putting it together — a modern project
    - Chapter 10 TL;DR

Chapter 1 — Introduction and Setup

JavaScript makes web pages *do* things (open a menu, validate a form, update a chat without reloading) and today also runs web servers, command-line tools, desktop and mobile apps. This chapter covers what JavaScript is, who defines it, what runs it, how to run your first programs (browser and Node.js), comments and strict mode.

No programming experience is assumed — you need a modern browser and, for Node examples, Node installed (§1.5). The book's goal: read and write real-world JavaScript and be ready for frameworks such as React (a **framework** is a ready-made structure of code your app plugs into).

What you'll learn: what JS is and why learn it; JavaScript vs ECMAScript and TC39; engines (V8, SpiderMonkey, JavaScriptCore); hosts (browser vs Node); four ways to run code; `console.log`, statements and semicolons; comments; strict mode.

All runnable code is in `code/`. Every "Output:" block was produced with Node 24.

1.1 What JavaScript is

What it is

JavaScript (JS) is a high-level, general-purpose language. *High-level*: you write in terms of numbers, text, lists and functions, not memory and processor; the language even frees unused memory for you (**garbage collection**). (A **variable** is a name holding a value; **compiling** translates source code into a lower-level form the machine runs.)

Term	Meaning for JavaScript
Interpreted / JIT-compiled	No separate compile step: the engine compiles the source on the fly ("just in time") while running it (§1.3).
Dynamically typed	The *value* has a type, not the variable; one variable can hold a number now and text later (Chapter 2).
Multi-paradigm	Imperative, object-oriented (Chapter 9) or functional style (Chapter 4).
Single-threaded with an event loop	Code runs one piece at a time on one thread (one sequence of instructions); slow work (network, timers, files) is handed off and its result returns later as an event (Chapter 12).

On a web page, **HTML** is the structure, **CSS** the decoration, **JavaScript** the behaviour.

A short history

Year	Event
1995	Brendan Eich writes it at Netscape in about ten days: Mocha → LiveScript → **JavaScript** (a marketing tie-in with Java; the languages are otherwise unrelated).
1996–97	Handed to **Ecma International** (Nov 1996); first **ECMAScript** standard (ECMA-262) published June 1997.
1999	ES3 — the version most of the early web used.
2008	Ambitious ES4 abandoned. Google releases Chrome with the fast **V8** engine.
2009	ES5: strict mode, JSON (plain-text data format), array methods like `map`. Ryan Dahl releases **Node.js** (V8 outside the browser).
2015	**ES2015 (ES6)** — biggest update: `let` / `const`, arrow functions, classes, modules, promises, template literals.
2016 → now	A new edition every June, named by year (ES2016, ES2017 …). This book targets **ES2025**.

Why it matters — reasons to learn JavaScript

- **Everywhere:** the only language every browser runs natively (directly, or via something that compiles to it, like TypeScript — JS plus type labels checked before running).
- **Many platforms:** front ends (what users see); back ends (server side) with **Node.js**; mobile with **React Native**; desktop with **Electron** (VS Code, Slack, Discord); **serverless** functions (provider runs your uploaded function on demand) at the **edge** (servers near users, e.g. Cloudflare Workers) (System Design Fundamentals).
- **Ecosystem:** npm, the largest software registry, hosts reusable **packages**; most problems already have a **library** (someone else's code you call) (Chapter 10).
- **Jobs:** front-end, back-end and full-stack roles; top of Stack Overflow's "most used language" for over a decade.
- **Gentle start:** runs in a browser tab with nothing installed. It has sharp corners — **coercion** (automatic type conversion, Chapter 2), `this` (Chapter 9) — but the basics are approachable.

Common mistakes

- **"JavaScript is Java for the web."** Different language; the name is a marketing accident.
- **"JavaScript only runs in browsers."** It runs wherever there is an engine; Node.js is the most common non-browser host.
- **Language vs browser:** `document.getElementById` belongs to the browser, not the language (§1.4) — the root of many "why doesn't this work in Node?" questions.

TL;DR

(TL;DR = "too long; didn't read".) JavaScript is a high-level, dynamically typed, garbage-collected, multi-paradigm language, standardised as ECMAScript, running in every browser and — via Node.js and friends — almost everywhere else.

1.2 ECMAScript and TC39

What it is

ECMAScript (ES) is the official specification (ECMA-262): exactly what every piece of syntax means and how every built-in (`Math`, `Array`, `Promise` …) behaves. **JavaScript** is the everyday name for implementations of it ("JavaScript" is an Oracle trademark — one reason the standard has another name). The spec is the rule book; Chrome, Firefox and Node are different sets that must play by it.

TC39 (Technical Committee 39 of Ecma International) maintains the spec: browser makers, heavy JS users and invited experts, working in public at github.com/tc39.

How it works — how a feature gets into the language



- A **champion** (TC39 member) presents a problem; Stage 1 = worth solving, Stage 2 = draft.
- Stage 2.7 (added 2024): design approved, conformance tests (shared tests that check engines follow the spec) being written.
- Stage 3: engines ship it, often behind a flag first; real-world feedback arrives.
- Stage 4: two compatible shipping implementations plus full tests → included in the next June edition.

Why it matters

- **Version names:** "ES6" = "ES2015"; after that, years ("ES2020 added `??` and `?.`").
- **Compatibility:** features ship before their edition is published; whether *your users'* browsers support one is answered by MDN compatibility tables / caniuse.com, and tools can translate new syntax to old (Chapter 10).

- **"Don't break the web":** old code must keep working, so quirks like `typeof null === "object"` are never fixed, only worked around (strict mode, `===`, `let`).

Editions you will hear about

Edition	Headline features (a selection)
ES5 (2009)	strict mode, `JSON`, `Array.prototype.forEach/map/filter`, `Object.defineProperty`
ES2015 / ES6	`let` / `const`, arrow functions, classes, modules, promises, template literals, destructuring, `Map` / `Set`, `for…of`
ES2017	`async` / `await`
ES2020	`??`, `?.`, `BigInt`, `globalThis`, `Promise.allSettled`
ES2022	class fields and `#private`, top-level `await`, `.at()`, `Error` `cause`
ES2023	`toSorted`, `toReversed`, `with`, `findLast`
ES2024	`Object.groupBy`, `Map.groupBy`, `Promise.withResolvers`
ES2025	iterator helpers (`.map/.filter` on iterators), new `Set` methods (`union`, `intersection` …), `Promise.try`, `RegExp.escape`, JSON modules/import attributes, `Float16Array`
ES2026	`Array.fromAsync`, `Error.isError`, `Math.sumPrecise`, `Uint8Array` base64/hex, `Map.prototype.getOrInsert` (upsert), `Iterator.concat`, `JSON.parse` source text access — only some are in Node 24
ES2027 (next)	**Temporal** (modern dates), `using` declarations (explicit resource management), `Atomics.pause`, `Iterator.zip` — finished proposals waiting for the June 2027 edition

Each feature is taught in its own chapter; this table is only a map.

In the interview

> **30-second answer — "What's the difference between JavaScript and ECMAScript?"** ECMAScript is the standard (ECMA-262) maintained by TC39; JavaScript is the language as implemented by engines such as V8 and SpiderMonkey, plus, in practice, the host APIs around it (an **API** is the set of functions and objects one piece of software offers other code). Since 2015 a new edition ships yearly via a staged proposal process; Stage 4 features go into the next edition.

- *Is `document` part of JavaScript?* No — it is the host's **DOM** (Document Object Model: the browser's tree of objects representing the page); ECMAScript doesn't define it.
- *What does Stage 3 mean?* Design complete, engines implementing; later changes are rare and driven by implementation feedback.

TL;DR

ECMAScript is the spec, JavaScript its implementations; TC39 moves proposals through Stages 0–4; a new edition ships every June.

1.3 JavaScript engines — how your code actually runs

What it is

A **JavaScript engine** reads JavaScript source and executes it.

Engine	Made by	Used in
V8	Google	Chrome, Edge, Opera, Brave, **Node.js**, Deno
SpiderMonkey	Mozilla	Firefox (the first JS engine ever, descended from Eich's original)
JavaScriptCore (JSC)	Apple	Safari, most iOS browsers (Apple long required it; EU rules now allow others), **Bun**

How it works — from text to execution (V8 as the example)



1. **Parse** — check the grammar. A grammar error (missing `)`) is a `SyntaxError`, reported *before any line of that script runs*.
2. **AST** — the code becomes a tree (**abstract syntax tree**) representing its meaning.
3. **Interpret bytecode** — V8's **Ignition** interpreter turns the tree into compact **bytecode** (low-level instructions for the engine's own virtual machine) and runs it immediately, so start-up is fast.
4. **Profile and optimise** — the engine tracks "hot" functions and the value types they see; hot code is recompiled into machine code by **Sparkplug** (quick, non-optimising baseline), then **Maglev** and **TurboFan** (optimising, built on assumptions like "this variable is always a small integer").
5. **Deoptimise** — if an assumption breaks (a string arrives), optimised code is discarded and execution falls back to bytecode. Results stay correct; only speed changes.
6. A **garbage collector** frees memory nothing references any more.

This is **just-in-time (JIT) compilation**; other engines use the same idea with differently named tiers.

Why it matters

- You ship source (usually **minified** — whitespace/comments removed, names shortened, behaviour unchanged); the engine does the rest.
- **Syntax errors stop the whole file**; runtime errors (e.g. calling a non-function) stop at the line where they happen (Chapter 11).
- Consistent value types help the optimiser — but clarity comes first.

Engine vs runtime

An engine alone runs only the core language (variables, functions, `Math`, `Array`, `JSON`, promises) — no screen, files or network. The program that *embeds* an engine and adds those is the **host** or **runtime** (§1.4). The **event loop** — which runs your **callbacks** (functions handed over to be called later) when a timer fires or data arrives — also belongs to the host (Chapter 12).

Self-check

1. Engine in Node.js? → V8.
2. When is a `SyntaxError` reported? → While parsing, before any code in that script runs.
3. "Deoptimisation"? → Discarding optimised machine code because a type assumption proved false; execution continues in bytecode.

TL;DR

An engine (V8, SpiderMonkey, JavaScriptCore) parses code into an AST, runs bytecode and JIT-compiles hot parts. Everything beyond the core language comes from the host.

1.4 Where JavaScript runs — hosts

What it is

A **host environment** (**runtime**) embeds a JS engine and adds **host APIs** for the outside world.

Host	Engine	Adds (examples)	Typical use
Web browser	V8 / SpiderMonkey / JSC	`window`, `document` (the **DOM**), `fetch`, `localStorage`, events, `setTimeout`	Interactive web pages (Ch. 13, Ch. 14)
Node.js	V8	`process`, `fs` (files), `http` (servers), `path`, `Buffer`, plus `fetch` and `setTimeout`	Servers, CLIs (command-line programs), build tools (Ch. 15)
Deno	V8	Web-style APIs, built-in TypeScript, permission system	Servers and scripts
Bun	JavaScriptCore	Node-compatible APIs, bundler (combines many code files into a few), test runner	Servers, tooling
Others	various	React Native (mobile), Electron (desktop = Chromium + Node), Cloudflare Workers (edge)	Apps and edge functions

The **language** is identical everywhere; only host objects differ. The **global object** (whose properties are global variables) is `window` in browsers and `global` in Node, so ES2020 added `globalThis`, which works everywhere.

Example — asking "where am I?"

```
// code/04_where_am_i.js
console.log("typeof globalThis :", typeof globalThis);  // standard name for the global object everywhere
console.log("typeof window     :", typeof window);      // only in browsers
console.log("typeof document   :", typeof document);    // only in browsers (the DOM)
console.log("typeof process    :", typeof process);     // only in Node.js
console.log("Node version      :", process.version.split(".")[0] + ".x");
console.log("V8 is the engine  :", typeof process.versions.v8 === "string");
console.log("Math.max(3, 7)    :", Math.max(3, 7));     // ECMAScript built-in
console.log("typeof setTimeout :", typeof setTimeout);  // host API (browser and Node both have it)
console.log("typeof fetch      :", typeof fetch);       // web API, also built into Node 18+
```

Output (Node 24):

```
typeof globalThis : object
typeof window     : undefined
typeof document   : undefined
typeof process    : object
Node version      : v24.x
V8 is the engine  : true
Math.max(3, 7)    : 7
typeof setTimeout : function
typeof fetch      : function
```

`typeof` on an undeclared name returns `"undefined"` instead of throwing — the classic way to **feature-detect** (check a feature exists before using it).

Client-side vs server-side

- **Client-side** JS runs in the user's browser, reacting to input and updating the page *without reloading*.
- **Server-side** JS (usually Node.js) handles requests (usually HTTP), talks to databases and sends responses (System Design Fundamentals). One language on both sides lets a team share code (e.g. validation rules).

Common mistakes

- `document.querySelector` in a Node script → `ReferenceError: document is not defined`. (jsdom emulates the DOM in Node for tests — used in Chapter 13.)
- `require` (Node's older file loading, Chapter 10) in browser code, or `window` in Node.

TL;DR

Engine = runs the language; host = engine + APIs (DOM in browsers; files, processes, servers in Node). `globalThis` is the portable global object.

1.5 Setting up and running code

1. The browser console (zero install)

Open any page (even `about:blank`), open **DevTools**: `F12`, or `Ctrl+Shift+J` / `Cmd+Option+J` (Chrome, Edge); `Ctrl+Shift+K` / `Cmd+Option+K` (Firefox); Safari: enable *Settings → Advanced → Show features for web developers*, then `Cmd+Option+C`. In the **Console** tab type `2 + 3` → `5`. The console is a **REPL** (**R**ead, **E**valuate, **P**rint, **L**oop). Good for experiments and inspecting a live page; code is lost when the tab closes.

2. A `<script>` tag in an HTML page

A page loads JS with `<script>` — inline, or from a file via `src`:

```
<!-- code/hello.html (key parts) -->
<head>
  <meta charset="utf-8">
  <title>Hello from JavaScript</title>
  <!-- defer: download now, run after the HTML is parsed, so the #out element exists -->
  <script src="hello.js" defer></script>
</head>
<body>
  <h1>My first page with JavaScript</h1>
  <p>The box below is empty in the HTML file. The script fills it in:</p>
  <div id="out"></div>
  <script>
    // An inline script runs as soon as the parser reaches it.
    console.log("inline script ran");   // visible in DevTools > Console
  </script>
</body>
```

```
// code/hello.js — a browser script (uses the DOM, so it can't run in Node)
const out = document.getElementById("out");
const now = new Date(2026, 0, 15, 9, 30); // fixed date so the screenshot is reproducible
out.innerHTML =
  "<strong>Hello, World!</strong> This text was written by <code>hello.js</code>.<br>" +
  "2 + 3 = " + (2 + 3) + " &middot; page built at " + now.toLocaleTimeString("en-US");
console.log("hello.js ran");
```



Figure — `hello.html` in headless Chrome. The box was empty in the HTML; the script wrote its contents.

The Console shows `inline script ran` first, then `hello.js ran` : inline scripts run when the parser reaches them; deferred scripts wait until the document is parsed. A plain script **pauses HTML parsing** while it downloads and runs:

Form	Downloads	Runs	Order kept?	Use for
`<script src>` (in `<head>`)	blocks parsing	immediately	yes	rarely — it delays the page
`<script src>` at end of `<body>`	when reached	immediately	yes	the classic old fix
`<script src defer>`	in parallel	after HTML is parsed	**yes**	most page scripts
`<script src async>`	in parallel	as soon as downloaded	**no**	independent scripts (analytics)
`<script type="module" src>`	in parallel	after parsing (deferred by default)	yes	modern code using `import` (Ch. 10)

3. Node.js from the terminal

Install the **LTS** (long-term support) version from nodejs.org — even-numbered major versions (the first number, e.g. 24 in 24.1.0) become LTS; this book uses Node 24 (LTS since October 2025). `node --version` → `v24.x.x` ; run a file with `node hello.js` .

4. The Node REPL

`node` with no file gives a `>` prompt. `.help` lists commands, `.exit` (or `Ctrl+D`) quits, `_` holds the last result.

```
$ node
> 2 + 3
5
> _ * 10
50
> .exit
```

An editor

Visual Studio Code (free) is the common choice: highlighting, error squiggles, integrated terminal and a debugger (Chapter 11). CodePen, JSFiddle and StackBlitz are handy for sharing experiments.

Common mistakes

- Plain `<script src>` in `<head>` touching page elements → `null` errors (elements don't exist yet). Use `defer` .

- Looking for `console.log` output *on the page* — it goes to the DevTools Console.
- Typing `node hello.js` inside the REPL (`>`) instead of the shell (`$`).

TL;DR

Console/REPL for experiments; `<script defer>` (or `type="module"`) in pages; `node file.js` outside the browser; VS Code to edit.

1.6 Hello, World! — your first program and the basic rules

What it is

```
// code/01_hello_world.js — run with:  node 01_hello_world.js
console.log("Hello, World!");          // double quotes
console.log('Hello, JavaScript!');     // single quotes work the same way
console.log(`2 + 3 = ${2 + 3}`);       // backticks = template literal, can embed expressions
console.log("Many", "values", 42, true); // console.log accepts several arguments, separated by spaces
```

Output:

```
Hello, World!
Hello, JavaScript!
2 + 3 = 5
Many values 42 true
```

- **`console`** — a host-provided object (a bundle of named values and functions, Chapter 5) for diagnostic messages.
- **`.log(...)`** — a *method* (function on an object) printing its arguments: to the DevTools Console in browsers, to standard output (the terminal) in Node.
- **`"Hello, World!"`** — a **string** (text). `"…"` and `'…'` are identical; **backticks** create *template literals* that embed expressions in `${ }` (Chapter 6).
- `;` ends the statement.

Try it yourself: change the message to `"Hello, <your name>!"` and rerun. `console.error` , `console.warn` , `console.table` , `console.time` are in Chapter 11.

The basic rules of the syntax

Rule	Detail
Statements	A program is a list of statements run top to bottom (`console.log(...)` , `let x = 5`).
Expressions	Anything that produces a value: `2 + 3` , `"hi"` , `x > 1` , `max(a, b)` ; they appear inside statements.
Case-sensitive	`console` , `Console` , `CONSOLE` are different names; only the first exists.
Identifiers	Letters, digits, `_` , `$` , not starting with a digit (`userName` , `_count` , `$el` ; not `2fast`). Reserved words (`let` , `class` , `return` …) are not allowed. Convention: **camelCase** (`totalPrice`).
Blocks	`{ … }` groups statements (after `if` , `for` , function bodies).
Whitespace	Mostly ignored between **tokens** (smallest meaningful pieces); a formatter like Prettier handles layout (Chapter 10).

Semicolons and ASI

Automatic Semicolon Insertion (ASI): where a line break appears and the grammar can't continue, the parser inserts a semicolon — so semicolon-free code usually works. Two traps:

```
function f() {
  return          // ASI inserts ";" right here, because a newline follows "return"
    { ok: true }; // never reached — f() returns undefined
}

const a = 1
const b = a
[1, 2].forEach(n => console.log(n)) // parsed as  a[1, 2].forEach(...)  -> TypeError
```

Keep the returned value on the `return` line; if you skip semicolons, prefix lines starting with `(`, `[` or `` ` `` with `;`. This book always uses semicolons.

Self-check

1. Where does `console.log` output appear in a browser? → DevTools Console, not the page.
2. `'hi'` vs `"hi"`? → Same string.
3. Why does `return` + newline + value return `undefined`? → ASI inserts a semicolon after `return`.

TL;DR

`console.log(value)` prints; strings use `'…'`, `"…"` or `` `…` ``; code is case-sensitive; end statements with `;`, never break a line straight after `return`.

1.7 Comments

What it is

A **comment** is source text the engine ignores entirely — it exists only for human readers and has no effect on behaviour or speed.

Kind	Syntax	Ends at	Typical use
Single-line	`// text`	end of the line	Short notes; switching off one line
Multi-line (block)	`/* text */`	the closing `*/` (may span many lines)	Longer explanations; switching off several lines
JSDoc (a convention built on block comments)	`/** … */` with tags like `@param`, `@returns`	`*/`	Documenting functions; editors show it as hover help and can type-check with it (Chapter 16)

Example

```
// code/02_comments.js
// A single-line comment runs from // to the end of the line.
console.log("Line 1 runs");   // a comment can also follow code on the same line
// console.log("Line 2 is switched off, so nothing prints for it");

/*
  A multi-line (block) comment can span many lines.
  Handy for a longer explanation, or to switch off a whole block:
console.log("switched off too");
console.log("and this");
*/
console.log("Line 3 runs");

/**
 * A JSDoc comment: starts with slash-star-star. Editors and tools read the tags.
 * @param {number} celsius - temperature in degrees Celsius
 * @returns {number} the same temperature in Fahrenheit
 */
function toFahrenheit(celsius) {
  // Why, not what: 9/5 and 32 come from the Fahrenheit scale definition.
  return celsius * 9 / 5 + 32;
}
console.log("100 C =", toFahrenheit(100), "F");

const total = 10 /* inline block comments are legal too */ + 5;
console.log("total =", total);
```

Output:

```
Line 1 runs
Line 3 runs
100 C = 212 F
total = 15
```

"**Commenting out**" code disables it temporarily without deleting it; in VS Code `Ctrl+/` (`Cmd+/`) toggles `//` on selected lines.

Best practices

1. **Explain *why*, not *what*.** `i++ // add one` repeats the code; `// the API counts pages from 1` adds information.
2. **Keep comments in sync** — a wrong comment is worse than none.
3. **Don't over-comment** — prefer clear names (`isLoggedIn`) and small functions; comment intent, trade-offs and gotchas.
4. **Delete commented-out code before committing** — version control (git) remembers old code.
5. `// TODO:` / `// FIXME:` markers, sparingly — editors list them.

Common mistakes

- **Nesting block comments:** in `/* outer /* inner */ still outer? */` the first `*/` ends the comment → `SyntaxError`. Use `//` per line when commenting out code containing block comments.
- **Secrets in comments:** browser JS (comments included) is downloaded by every visitor unless a build step strips them.
- **HTML comments `<!-- -->` in JS:** still treated as single-line comments in classic browser scripts for historical reasons — never rely on it.

TL;DR

`//` for one line, `/* … */` for many, `/** … */` for JSDoc. Explain intent, keep comments current, prefer self-explanatory code.

1.8 Strict mode

What it is

Strict mode (ES5, 2009) is an opt-in stricter variant that turns silent mistakes into errors and removes confusing features. Enable it with the **directive** `"use strict";` as the very first statement of a script or function. Code without it is informally **sloppy mode**.

Modern code is usually strict automatically: every **ES module** (`import` / `export`, `<script type="module">`, `.mjs` in Node) and every **class** body is strict. The directive is needed only in classic scripts.

The problem it solves

Early JS never complained: a misspelt variable silently created a global; writing to a read-only **property** (named value on an object) silently did nothing. Since the web can't break (§1.2), strict mode was added as an opt-in fix.

Example — sloppy vs strict

```
// code/03_strict_mode.js — a classic CommonJS script, so it starts in sloppy mode
function sloppyTypo() {
  countr = 1;               // typo: meant "counter". Sloppy mode silently creates a global!
  return typeof globalThis.countr;
}
console.log("sloppy: accidental global created ->", sloppyTypo());

function strictTypo() {
  "use strict";
  try {
    countr2 = 1;            // strict mode refuses to invent a global
  } catch (e) {
    return `${e.name}: ${e.message}`;
  }
}
console.log("strict:", strictTypo());

function sloppyThis() { return this === globalThis; }
function strictThis() { "use strict"; return this; }
console.log("plain call, sloppy this is globalThis ->", sloppyThis());
console.log("plain call, strict this ->", strictThis());
// ... (frozen-object and duplicate-parameter checks, see the file)
```

Output:

```
sloppy: accidental global created -> number
strict: ReferenceError: countr2 is not defined
plain call, sloppy this is globalThis -> true
plain call, strict this -> undefined
write to frozen object: sloppy keeps 1 | strict throws TypeError
duplicate params in strict code -> SyntaxError
```

(**CommonJS** is Node's original module format, loading files with `require`. `try` / `catch` is in Chapter 11; `this` in Chapter 9.)

What strict mode changes

Situation	Sloppy mode	Strict mode
Assigning to an undeclared variable (`x = 1`)	creates a global variable	`ReferenceError`
`this` in a plain function call `f()`	the global object	`undefined`
Writing to a read-only or frozen (`Object.freeze`) property, adding to a non-extensible object	silently ignored	`TypeError`
Deleting a non-deletable property (`delete Object.prototype`)	returns `false`	`TypeError`
Duplicate parameter names `function f(a, a)`	allowed	`SyntaxError`
Legacy octal (base-8) literal `010`	means 8	`SyntaxError` (use `0o10`)
`with` statement (treats an object's properties as variables)	allowed	`SyntaxError`
`eval("var x = 1")` (runs a string as code)	leaks `x` into the surrounding scope	`x` stays inside the eval
`arguments` object	stays linked to the parameters	an independent copy (Chapter 4)
Future reserved words (`implements` , `interface` , `package` , `private` , `protected` , `public` , `static` , `yield` , `let`) as names	allowed	`SyntaxError`

How to use it

1. **Modules** (`.mjs` , `"type": "module"` in `package.json` (Chapter 10), `<script type="module">`): already strict.
2. **Classic scripts:** `"use strict";` as the first statement of the file (only comments may precede it).
3. **One function:** first statement of its body.

Common mistakes

- **Directive after other code** — then it's just an unused string.
- **Concatenating files** — a strict file glued before a sloppy one makes both strict (or vice versa). Bundlers handle this.
- **`"use strict"` inside a function with default, rest or destructured parameters** (Chapter 4) is a `SyntaxError` ; put it at file level.
- **Assuming Node `.js` files are strict** — they're CommonJS (sloppy) unless `package.json` has `"type": "module"` or the file uses `import` / `export` . This book's early chapters use plain `.js` files for that reason.

In the interview

30-second answer — "What is strict mode?" An opt-in restricted variant enabled with `"use strict"` at the top of a script or function, and automatic in ES modules and classes. It turns silent errors into thrown ones (undeclared assignments, writes to read-only properties), makes `this` `undefined` in plain function calls, and forbids `with` , duplicate parameters and legacy octals.

- *Still need to write it?* Not in ES modules or class bodies; only in classic scripts / CommonJS files.
- *Why not default everywhere?* It would break sites relying on sloppy behaviour; the web must stay backward compatible.
- *Why is `this` undefined?* So a function called without an object can't silently read or modify the global object.

Self-check

1. In a `.mjs` file, does `x = 5` (undeclared) work? → No; modules are strict → `ReferenceError`.
2. Where must `"use strict"` appear? → First statement of the script or function body.
3. `this` in `function f() { "use strict"; return this; }` called as `f()`? → `undefined`.

TL;DR

Strict mode turns silent failures into errors and removes risky features; automatic in modules and classes, otherwise start with `"use strict";`.

Chapter 1 TL;DR

- **JavaScript**: high-level, dynamically typed, garbage-collected, multi-paradigm; created 1995, standardised as **ECMAScript**, maintained by **TC39**, new edition every June (this book: ES2025).
- An **engine** (V8, SpiderMonkey, JavaScriptCore) parses to an AST, runs bytecode, JIT-compiles hot code. A **host** (browser, Node.js, Deno, Bun) adds APIs like the DOM or file system; `globalThis` is the global object everywhere.
- Run code in the **browser console**, a page with **`<script defer>`** (or `type="module"`), **`node file.js`**, or the **Node REPL**.
- `console.log` prints; strings take `'`, `"` or `` ` ``; case-sensitive; use semicolons, never break after `return`.
- Comments: `//`, `/* */`, `/** JSDoc */` — explain *why*, keep current.
- **Strict mode** (`"use strict"`, automatic in modules and classes) turns silent mistakes into errors.

Next: Chapter 2 — Variables, Types and Operators.

Chapter 2 — Variables, Types and Operators

Programs store values and compute new ones. This chapter covers **variables** (names for values), **types** (what kind of value something is) and **operators** (`+`, `===`, `&&`, `??` …), plus **type coercion** — the automatic conversion of one type into another — so that `"3" + 4`, `0 == ""` and `0.1 + 0.2` stop being mysteries.

What you'll learn: `let` / `const` / `var`; hoisting and the TDZ; the seven primitives, objects and `typeof`; value vs reference; arithmetic, comparison, logical, `??`, `?.`, bitwise and assignment operators and precedence; `==` vs `===`; numbers in depth (precision, `NaN`, `Infinity`, safe integers, `Number` API, `BigInt`); truthy/falsy and explicit conversions.

Code: `code/` — every "Output:" block is real Node 24 output. Files are plain `.js` (sloppy-mode scripts) unless noted (Chapter 1 §1.8).

2.1 Variables: `let`, `const` and `var`

What it is

A **variable** is a named storage slot for a value. **Declare** creates the name (`let score;`), **assign** stores a value (`score = 10;`), **initialise** does both (`let score = 10;`). A declared-but-unassigned variable holds `undefined`. A `const` is a label glued to one value: you can't swap it, but if the value is an object you can still change what's inside.

How it works — the three keywords

```
// code/01_declarations.js
let score;                    // declared, no value yet
console.log("score before assignment:", score);
score = 10;                   // assign later
score = score + 5;            // reassign (let allows it)
console.log("score:", score);

const PI_APPROX = 3.14;       // const must be initialised and can't be reassigned
try { eval("PI_APPROX = 3"); } catch (e) { console.log("reassign const ->", e.name + ":", e.message); }

// const protects the BINDING, not the object: the contents can still change.
const cart = { items: 1 };
cart.items = 2;               // allowed: we mutate the object, not the variable
console.log("const object mutated:", cart);

// var: the old way. Function-scoped, can be redeclared, hoisted with value undefined.
var legacy = "first";
var legacy = "second";        // redeclaring with var is silently allowed
console.log("var redeclared:", legacy);
try { eval("let dup = 1; let dup = 2;"); } catch (e) { console.log("let redeclared ->", e.name); }

let a = 1, b = 2, c = a + b;  // several declarations in one statement
console.log("a, b, c =", a, b, c);

const greeting = "Hello";
const who = "World";
console.log(greeting + " " + who);
```

Output:

```
score before assignment: undefined
score: 15
reassign const -> TypeError: Assignment to constant variable.
const object mutated: { items: 2 }
var redeclared: second
let redeclared -> SyntaxError
a, b, c = 1 2 3
Hello World
```

(`eval` runs a string as code — used here only to catch a deliberate error. Don't use it in real code.)

Comparison table

	`let`	`const`	`var`
Introduced	ES2015	ES2015	original (1995)
Scope (where the name is visible)	**block** `{ }`	**block** `{ }`	**function** (or global)
Reassign	yes	**no** (`TypeError`)	yes
Redeclare in same scope	no (`SyntaxError`)	no	**yes, silently**
Must initialise	no	**yes**	no
Use before the declaration line	`ReferenceError` (TDZ, §2.2)	`ReferenceError` (TDZ)	gives `undefined`
At top level of a classic script, becomes a property of `globalThis`	no	no	**yes**

Scope is covered in Chapter 4 §4.5; in short, `let` / `const` inside `{ }` exist only inside those braces, `var` leaks to the whole function.

Which one should I use?

1. `const` **by default** — tells the reader the name never re-points.
2. `let` **when the value must change** — counters, accumulators, branch assignments.
3. `var` **never in new code** — recognise it in old code; its function scope, silent redeclaration and hoisting cause the bugs `let` / `const` prevent.

Naming rules and conventions

- Letters, digits, `_`, `$`; not starting with a digit. Unicode letters work (`café`) but stick to ASCII in shared code. Case-sensitive. Reserved words (`let`, `class`, `return`, `typeof` …) are not allowed.
- **camelCase** for variables/functions (`totalPrice`), **PascalCase** for classes (`ShoppingCart`, Chapter 9), **UPPER_SNAKE_CASE** for fixed config constants (`MAX_RETRIES`). Booleans as questions: `isOpen`, `hasAccess`.

Common mistakes

- `const` **≠ read-only object** — it only blocks reassignment; use `Object.freeze` (Chapter 5).
- **Forgetting the keyword** — `total = 5` creates an accidental global in sloppy mode, throws in strict mode.
- **Using a block's** `let` **outside the block** → `ReferenceError`.

TL;DR

`const` by default, `let` to reassign, `var` only when reading old code. `const` freezes the **binding** (the name→value link), not the value.

2.2 Hoisting and the Temporal Dead Zone

What it is

Before running a scope, the engine creates all its declared names — **hoisting**. What differs is the value before the declaration line runs:

Declaration	Before its line runs
`var x`	exists, value `undefined`
`let x` , `const x` , `class X`	exists but **uninitialised** — touching it throws `ReferenceError`
`function f() {}`	exists and is **fully usable** (covered in Chapter 4)

The window between entering the scope and a `let` / `const` declaration line is the **Temporal Dead Zone (TDZ)**.

Example

```
// code/02_tdz_and_hoisting.js
console.log("var before its line:", typeof hoistedVar, hoistedVar); // declared + set to undefined at start
var hoistedVar = "ready";

try {
  console.log(notYet);        // let exists but is uninitialised until its line runs
} catch (e) {
  console.log("let before its line ->", e.name + ":", e.message);
}
let notYet = "ready";

try { typeof inZone; let inZone = 1; } catch (e) { console.log("typeof in TDZ ->", e.name); }
{
  // The TDZ is about TIME, not position: a function may mention a later let,
  // as long as it is called after the let line has run.
  const show = () => later;
  let later = "defined by the time show() is called";
  console.log(show());
}
```

Output:

```
var before its line: undefined undefined
let before its line -> ReferenceError: Cannot access 'notYet' before initialization
typeof in TDZ -> ReferenceError
defined by the time show() is called
```

Why it matters

With `var`, early reads silently give `undefined` and the bug surfaces elsewhere; `let` / `const` fail immediately with a named error. Even `typeof` — normally safe on unknown names — throws in the TDZ.

In the interview

"What is the Temporal Dead Zone?" `let`, `const` and `class` are hoisted — the scope knows them from the start — but not initialised until their declaration is evaluated; access in that window throws `ReferenceError`. `var` is hoisted *and* initialised to `undefined`.

- *Are `let` / `const` hoisted?* Yes — that's why an inner `let x` **shadows** (hides) an outer `x` even on lines above it: you get a TDZ error, not the outer value.
- *Lines or time?* Time: a closure (a function remembering surrounding variables — Chapter 4) referring to a later `let` is fine if it runs after the declaration.

TL;DR

All declarations are hoisted; `var` starts as `undefined`, `let` / `const` start in the TDZ and throw if touched early.

2.3 Data types and `typeof`

What it is

JavaScript has **eight** types: seven **primitives** and **object**. It is **dynamically typed**: values have types, variables don't.

Type	Example values	Notes
`number`	`42` , `3.14` , `-0` , `Infinity` , `NaN`	One type for integers and decimals: 64-bit floating point (§2.10)
`bigint`	`10n` , `2n ** 64n`	Whole numbers of any size (ES2020)
`string`	`"hi"` , `'hi'` , `` `hi ${x}` ``	Text, immutable (Chapter 6)
`boolean`	`true` , `false`	§2.11
`undefined`	`undefined`	"No value assigned yet" — what JS itself gives you
`null`	`null`	"Intentionally empty" — what *you* assign to mean "nothing"
`symbol`	`Symbol("id")`	Unique identifiers, mostly for object keys (Chapter 8)
`object`	`{ a: 1 }` , `[1, 2]` , functions, dates, maps …	Everything that isn't a primitive

`typeof` — asking a value its type

```
// code/03_types_typeof.js (abridged)
for (const [label, value] of samples) {   // samples: [["number", 42], ["null", null], ["array", [1, 2]], ...]
  console.log(label.padEnd(15), "typeof ->", typeof value);
}
```

Output:

```
number          typeof -> number
number (float)  typeof -> number
NaN             typeof -> number
bigint          typeof -> bigint
string          typeof -> string
boolean         typeof -> boolean
undefined       typeof -> undefined
null            typeof -> object
symbol          typeof -> symbol
object          typeof -> object
array           typeof -> object
function        typeof -> function
date            typeof -> object
Array.isArray([1,2]) -> true
value === null      -> true
box: number
box: string
```

Quirks to memorise:

1. **`typeof null === "object"`** — a first-implementation bug that can never be fixed. Test with `value === null`.
2. **`typeof NaN === "number"`** — NaN is a special number value.

3. **Functions give `"function"`** (though they're objects); arrays give `"object"` — use `Array.isArray`.

`typeof` always returns a string and is safe on undeclared names (`"undefined"`), except in a TDZ.

`undefined` vs `null`

	`undefined`	`null`
Meaning	value not set	deliberately empty
Who produces it	the language: unassigned variables, missing properties, missing arguments, functions without `return`	your code (and some APIs, e.g. `document.getElementById` when nothing matches)
`typeof`	`"undefined"`	`"object"` (quirk)
`Number(…)`	`NaN`	`0`
`== null`	`true`	`true`

Idiom: `value == null` is true for both `null` and `undefined` and nothing else — the one widely accepted use of `==`.

TL;DR

Seven primitives + objects. `typeof` identifies them, except `null` (→ `"object"`) and arrays (use `Array.isArray`).

2.4 Primitives vs objects

What it is

- A **primitive** is an **immutable** value; the variable holds the value itself; copying copies the value.
- An **object** is a mutable collection of properties; the variable holds a **reference** (an address). Copying copies the reference, so both variables point at the *same* object.

Example

```
// code/04_primitives_vs_objects.js
let x = 10;
let y = x;        // copies the value 10
y = 20;
console.log("x, y:", x, y);

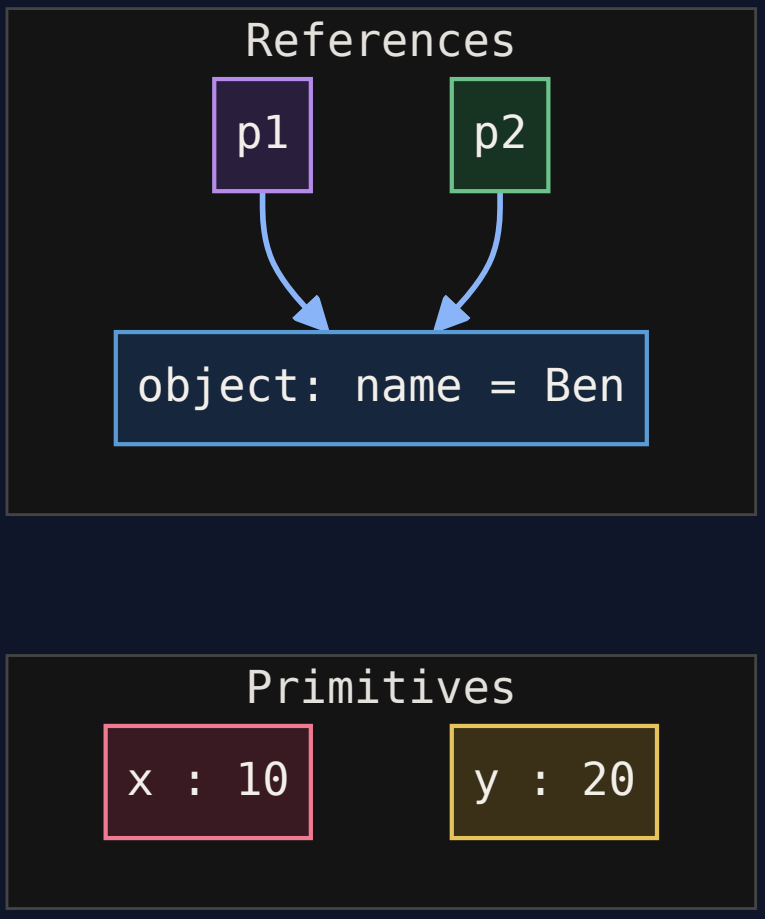
const p1 = { name: "Ana" };
const p2 = p1;    // copies the REFERENCE — both names point at one object
p2.name = "Ben";
console.log("p1.name:", p1.name);

console.log("{} === {}:", {} === {});          // two different objects
console.log("p1 === p2:", p1 === p2);          // same object
console.log("'a' === 'a':", "a" === "a");      // primitives compare by value

const s = "hello";
const upper = s.toUpperCase();                 // string methods return NEW strings
console.log(s, upper);
console.log("length of 'hello':", s.length, "| (255).toString(16):", (255).toString(16));
console.log("typeof new Number(5):", typeof new Number(5), "| typeof Number('5'):", typeof Number("5"));
```

Output:

```
x, y: 10 20
p1.name: Ben
{} === {}: false
p1 === p2: true
'a' === 'a': true
hello HELLO
length of 'hello': 5 | (255).toString(16): ff
typeof new Number(5): object | typeof Number('5'): number
```



Autoboxing and wrapper objects

Primitives have no properties, yet `"hello".length` works: the engine temporarily wraps the primitive in a **wrapper object** (`String` , `Number` , `Boolean` , `BigInt` , `Symbol`), reads the property and discards it — **autoboxing**.

Don't create wrappers with `new Number(5)` / `new String("x")` / `new Boolean(false)` : they're objects (`typeof` → `"object"` , `new Number(5) === 5` is `false` , `new Boolean(false)` is truthy). Call them **without** `new` — `Number("5")` , `String(5)` , `Boolean(0)` — to *convert* to primitives.

Why it matters

- A function receiving an object can change *your* object (Chapter 4).
- `===` on objects means "same object?", not "same contents?".
- React detects changes by checking for a *new* object — hence "don't mutate (change in place) state".

> **TL;DR**
>
> Primitives: immutable, copied/compared by value. Objects: mutable, copied/compared by reference. Convert with `Number()/String()/Boolean()` without `new` .

2.5 Arithmetic operators

What it is

An **operator** takes **operands** and produces a value: unary (`-x` , `!x`), binary (`a + b`), and one ternary (`c ? a : b` , Chapter 3).

Operator	Name	Example	Result
`+`	addition (or string concatenation)	`10 + 5`	`15`
`-`	subtraction	`10 - 5`	`5`
`*`	multiplication	`10 * 5`	`50`
`/`	division (always floating point)	`7 / 2`	`3.5`
`%`	remainder	`10 % 3`	`1`
`**`	exponentiation (ES2016)	`2 ** 10`	`1024`
`++`	increment by 1	`n++` / `++n`	see below
`--`	decrement by 1	`n--` / `--n`	see below
unary `+` / `-`	convert to number / negate	`+"42"`	`42`

Example

```
// code/05_arithmetic.js (abridged)
console.log("7 / 2 =", 7 / 2, "(no integer division)");
console.log("Math.trunc(7 / 2) =", Math.trunc(7 / 2));
console.log("-7 % 3 =", -7 % 3, "(sign follows the left operand)");
console.log("1 / 0 =", 1 / 0, "| -1 / 0 =", -1 / 0, "| 0 / 0 =", 0 / 0);

let n = 5;
console.log("n++ returns", n++, "then n is", n);   // returns old value
console.log("++n returns", ++n, "then n is", n);   // returns new value

console.log('"3" + 4 =', "3" + 4);
console.log('"3" - 4 =', "3" - 4);
console.log('"3" * "4" =', "3" * "4");
console.log('1 + 2 + "3" =', 1 + 2 + "3");
console.log('"1" + 2 + 3 =', "1" + 2 + 3);
console.log('+"42" =', +"42", '| -"5" =', -"5", '| +"" =', +"", '| +"abc" =', +"abc");
```

Output:

```
7 / 2 = 3.5 (no integer division)
Math.trunc(7 / 2) = 3
-7 % 3 = -1 (sign follows the left operand)
1 / 0 = Infinity | -1 / 0 = -Infinity | 0 / 0 = NaN
n++ returns 5 then n is 6
++n returns 7 then n is 7
"3" + 4 = 34
"3" - 4 = -1
"3" * "4" = 12
1 + 2 + "3" = 33
"1" + 2 + 3 = 123
+"42" = 42 | -"5" = -5 | +"" = 0 | +"abc" = NaN
```

How it works — the rules behind the surprises

1. **Division never throws:** `±Infinity` for division by zero; `0 / 0` is `NaN` .
2. `%` **is remainder, not modulo:** sign of the left operand (`-7 % 3` is `-1`). Always-positive modulo: `((a % n) + n) % n` .

3. **Prefix vs postfix:** both change `n`; postfix `n++` *returns* the old value, prefix `++n` the new one. Alone on a line, no difference.
4. `+` **is overloaded:** if either operand (after converting objects to primitives) is a string, it concatenates; otherwise adds. Left to right: `1 + 2 + "3"` → `"33"`, `"1" + 2 + 3` → `"123"`.
5. **Other arithmetic operators always convert to numbers:** `"3" - 4` → `-1`.
6. **Unary** `+` is the shortest number conversion; `""` → `0`, non-numeric text → `NaN`.
7. `-2 ** 2` **is a** `SyntaxError` — write `(-2) ** 2` or `-(2 ** 2)`. `**` is right-associative: `2 ** 3 ** 2` = `2 ** 9` = `512`.

Common mistakes

- Form values are strings: `"2" + "3"` → `"23"`. Convert first: `Number(input.value)`.
- Expecting `7 / 2` to be `3` — use `Math.trunc` / `floor` / `round` (Chapter 5).

2.6 Comparison operators and equality

What it is

Comparisons always produce a **boolean**.

Operator	Meaning	Converts types?
`===`	strictly equal: same type **and** same value	no
`!==`	strictly not equal	no
`==`	loosely equal: equal after converting types	**yes**
`!=`	loosely not equal	**yes**
`>` `<` `>=` `<=`	greater / less (or equal)	yes (to number, unless both are strings)

Example

```js
// code/06_comparison_equality.js (abridged)
const x = 5, y = "5";
console.log("x == y  :", x == y);
console.log("x === y :", x === y);
console.log("x != y  :", x != y);
console.log("x !== y :", x !== y);
console.log("x > 2, x < 10, x >= 5, x <= 5:", x > 2, x < 10, x >= 5, x <= 5);
// ... loops over a table of pairs, printing == and === for each
```

Output:

```
x == y  : true
x === y : false
x != y  : false
x !== y : true
x > 2, x < 10, x >= 5, x <= 5: true true true true
--- loose equality surprises ---
0         == ""        -> true  | === -> false
0         == "0"       -> true  | === -> false
""        == "0"       -> false | === -> false
false     == "0"       -> true  | === -> false
false     == ""        -> true  | === -> false
null      == undefined -> true  | === -> false
null      == 0         -> false | === -> false
undefined == 0         -> false | === -> false
NaN       == NaN       -> false | === -> false
[1]       == 1         -> true  | === -> false
[]        == ""        -> true  | === -> false
[1,2]     == "1,2"     -> true  | === -> false
--- relational comparisons ---
"10" < "9"  : true (both strings: compared character by character)
"10" < 9    : false (one number: string converted to number)
"apple" < "banana": true | "Z" < "a": true
null >= 0   : true | null == 0: false
NaN < 1, NaN >= 1: false false
--- Object.is ---
NaN === NaN: false | Object.is(NaN, NaN): true
0 === -0: true | Object.is(0, -0): false
```

`0 == ""` and `0 == "0"` are true, yet `"" == "0"` is false: loose equality isn't even **transitive** — the main reason to avoid it.

How it works — the `==` algorithm (simplified from the spec)

For `a == b`:

1. **Same type** → compare like `===`.
2. `null` **and** `undefined` equal each other and **nothing else** (`null == 0` is false).
3. **Number vs string** → string to number (`""` → `0`).
4. **Boolean vs anything** → boolean to number first. So `false == "0"` is true, and `if (x == true)` is a trap: `"2" == true` → `2 == 1` → false.
5. **Object vs primitive** → object to primitive (`valueOf` / `toString`; arrays join: `[1,2]` → `"1,2"`, `[]` → `""`), compare again.
6. **BigInt vs number** → compare mathematical values (`1n == 1` is true).
7. `NaN` equals nothing, including itself.

`===` skips all that: different types → `false`. Remaining oddities: `NaN === NaN` is `false`, `0 === -0` is `true`; `Object.is(a, b)` is `===` with those two fixed.

Relational comparisons (`<`, `>` …)

1. Objects → primitives.
2. **Both strings** → compared by UTF-16 code unit (JS stores text as 16-bit **code units**, one or two per character) — dictionary order by character code: `"10" < "9"`, and uppercase before lowercase (`"Z" < "a"`). For human sorting use `localeCompare` (Chapter 6).
3. Otherwise both → numbers. `null` → `0`, so `null >= 0` is true though `null == 0` is false. Anything vs `NaN` is false.

Trade-offs — `==` vs `===`

	`===`	`==`
Predictable	yes	needs the rules above
Catches type bugs	yes (`"5" === 5` false)	hides them
Useful idiom	—	`x == null` (null **or** undefined)
Linters (tools that scan code for likely bugs; ESLint rule `eqeqeq`)	recommended	flagged, except `== null` if configured

In the interview

30-second answer — "Difference between `==` and `===`?" `===` compares type and value with no conversion. `==` converts first: null/undefined equal only each other; strings and booleans become numbers; objects become primitives. That makes `==` non-transitive (`0 == ""` , `0 == "0"` , but `"" != "0"`), so I use `===` everywhere except `x == null` .

- *Check for `NaN`?* `Number.isNaN(x)` (or `Object.is(x, NaN)`); `x === NaN` is always false.
- *`[] == ![]`?* `true` : `![]` is `false` , then `[] == false` → `"" == 0` → `0 == 0` .
- *`null >= 0` true but `null == 0` false?* Relational operators convert `null` to `0` ; `==` has the special null/undefined rule.

TL;DR

Use `===` / `!==` . String-vs-string compares by character code, everything else numerically; `NaN` compares false with everything.

2.7 Logical operators, `??` and `?.`

What it is

Operator	Name	Returns
`a && b`	AND	`a` if `a` is falsy, otherwise `b`
`a || b`	OR	`a` if `a` is truthy, otherwise `b`
`!a`	NOT	`true` if `a` is falsy, else `false` (always a boolean)
`a ?? b`	nullish coalescing (ES2020)	`a` unless it is `null` / `undefined` , then `b`
`a?.b` , `a?.[k]` , `a?.()`	optional chaining (ES2020)	`undefined` if `a` is `null` / `undefined` , otherwise the normal access/call

`&&` and `||` return one of their operands, not necessarily a boolean (*truthy/falsy*: §2.11).

How it works — short-circuit evaluation

- `&&` returns the **first falsy** operand (or the last), `||` the **first truthy** (or the last) — and stop evaluating there.
- So the right side may never run: `user && user.logout()` , `isDebug && console.log(...)` .

Example

```
// code/07_logical_nullish_optional.js (abridged — each output line labels its expression)
const settings = { volume: 0, title: "", theme: null };
console.log("volume || 50:", settings.volume || 50, "| volume ?? 50:", settings.volume ?? 50);

const cfg = { retries: 0, name: "", port: undefined };
cfg.retries ||= 3;    // 0 is falsy -> replaced
cfg.name ??= "app";   // "" is not nullish -> kept
cfg.port ??= 8080;    // undefined -> replaced
let token = "abc"; token &&= token.toUpperCase();
console.log(cfg, token);
```

Output (full run):

```
t && f: false | t || f: true | !f: true
"cat" && "dog": dog
"" && "dog": ""
0 || "default": default
"" || null || 0: 0
!!'text': true | !!0: false
right side evaluated 0 times
volume || 50: 50 | volume ?? 50: 0
title || "Untitled": Untitled | title ?? "Untitled": ""
theme ?? "light": light
user.address?.city: undefined
user.profile?.photo?.url: undefined
user.greet?.(): hi | user.wave?.(): undefined
user.tags?.[0]: undefined
without ?. -> TypeError
{ retries: 3, name: '', port: 8080 } ABC
true || false && false: true
mixing ?? and || without () -> SyntaxError
(null || undefined) ?? 'x': x
```

`||` vs `??` for defaults

`||` falls back on **any falsy** value, discarding legitimate `0`, `""`, `false` (`volume || 50` loses a real volume of 0). `??` falls back **only on** `null` / `undefined`. Use `??` for defaults unless you really mean "any falsy".

Optional chaining `?.`

Reading a property of `null` / `undefined` throws `TypeError`. With `?.`, if the left side is `null` / `undefined` the **whole chain** short-circuits to `undefined`. Forms: `obj?.prop`, `obj?.[expr]`, `fn?.(args)`. You can't assign through it (`obj?.x = 1` is a `SyntaxError`).

Logical assignment (ES2021)

Operator	Equivalent	Assigns when `a` is…
`a ||= b`	`a || (a = b)`	falsy
`a &&= b`	`a && (a = b)`	truthy
`a ??= b`	`a ?? (a = b)`	`null` / `undefined`

They short-circuit: if no assignment is needed, `b` isn't evaluated.

Common mistakes

- `if (x = 5)` assigns (and `5` is truthy); you meant `===`.
- **Overusing** `?.` on values that *must* exist hides bugs behind `undefined`.
- **Mixing** `??` **with** `||` / `&&` **without parentheses** → `SyntaxError`.
- `&&` binds tighter than `||`: `a || b && c` is `a || (b && c)`.

TL;DR

`&&` / `||` short-circuit and return an operand. `??` for defaults (keeps `0`, `""`), `?.` for possibly-missing data, `||=` / `&&=` / `??=` to assign conditionally.

2.8 Bitwise and assignment operators

Bitwise operators

Bitwise operators treat numbers as 32-bit signed integers and work per binary digit (**bit**). Used in low-level code (graphics, compression, hashing, permission flags) and interview puzzles.

Operator	Name	Rule per bit
`&`	AND	1 if both bits are 1
`|`	OR	1 when either bit (or both) is 1
`^`	XOR	1 if the bits differ
`~`	NOT	flips every bit (`~x` equals `-x - 1`)
`<<`	left shift	moves bits left, filling with 0 (×2 per step)
`>>`	sign-propagating right shift	moves bits right, copying the sign bit (÷2, rounding down)
`>>>`	zero-fill right shift	moves bits right, filling with 0 (result is unsigned)

```
// code/08_bitwise_assignment.js (first half)
let c = 5, d = 2;               // 5 = 101, 2 = 010
console.log("c & d  =", c & d, bin(c & d));
console.log("c | d  =", c | d, bin(c | d));
console.log("c ^ d  =", c ^ d, bin(c ^ d));
console.log("~d     =", ~d);
console.log("c << 1 =", c << 1);
console.log("c >> 1 =", c >> 1, "| -5 >> 1 =", -5 >> 1, "(keeps sign)");
console.log("c >>> 1 =", c >>> 1, "| -5 >>> 1 =", -5 >>> 1, "(fills with 0)");
const READ = 1, WRITE = 2, EXEC = 4;
let perms = READ | WRITE;
console.log("can write?", (perms & WRITE) !== 0, "| can exec?", (perms & EXEC) !== 0);
```

Output:

```
c & d  = 0 000
c | d  = 7 111
c ^ d  = 7 111
~d     = -3
c << 1 = 10
c >> 1 = 2 | -5 >> 1 = -3 (keeps sign)
c >>> 1 = 2 | -5 >>> 1 = 2147483645 (fills with 0)
can write? true | can exec? false
```

`101 & 010` shares no 1-bit → 0; OR and XOR both give `111` = 7. `~2` = `-3` (**two's complement**: negatives stored as "flip all bits, add 1"). The **flags** idiom packs yes/no settings into one number: each flag a power of two, `|` combines, `&` tests (as in Unix `chmod 755`).

Caveat: operands are truncated to 32-bit integers, so bitwise ops break beyond $\pm 2^{31}$ (~2.1 billion). Prefer `Math.trunc` over the old `x | 0` trick.

Assignment operators

Compound assignment combines an operation with `=`: `x += y` means `x = x + y`.

Operator	Same as		Operator	Same as
`x = y`	assign		`x <<= y`	`x = x << y`
`x += y`	`x = x + y`		`x >>= y`	`x = x >> y`
`x -= y`	`x = x - y`		`x >>>= y`	`x = x >>> y`
`x *= y`	`x = x * y`		`x &= y`	`x = x & y`
`x /= y`	`x = x / y`		`x |= y`	`x = x | y`
`x %= y`	`x = x % y`		`x ^= y`	`x = x ^ y`
`x **= y`	`x = x ** y`		`||=` `&&=` `??=`	§2.7

```
// code/08_bitwise_assignment.js (second half, abridged — the file applies every operator in turn)
let p, q;
p = q = 7;                      // right to left: q = 7, then p = (q = 7) which is 7
console.log("p, q:", p, q);     // p, q: 7 7
```

An assignment is an expression whose value is the assigned value — which makes `p = q = 7` work and `if (x = 5)` a silent bug.

2.9 Operator precedence

Precedence decides which operator applies first (`2 + 3 * 4` is `14`); **associativity** orders equal-precedence operators (mostly left-to-right; `**`, assignment and `?:` right-to-left). Highest to lowest (simplified):

Level	Operators
1	`( … )` grouping
2	`.` `[]` `?.` function call `()` `new` with arguments
3	postfix `x++` `x--`
4	`!` `~` unary `+` `-` prefix `++x` `--x` `typeof` `void` `delete` `await`
5	`**` (right-to-left)
6	`*` `/` `%`
7	`+` `-`
8	`<<` `>>` `>>>`
9	`<` `<=` `>` `>=` `in` `instanceof`
10	`==` `!=` `===` `!==`
11–13	`&`, then `^`, then `|`
14	`&&`
15	`||` and `??` (cannot be mixed without parentheses)
16	`? :` (right-to-left)
17	`=` and all compound assignments, `=>` (right-to-left)
18	`,` comma

Don't memorise it — **when in doubt, add parentheses**.

2.10 Numbers in depth

What it is

All ordinary numbers are **64-bit IEEE-754 double-precision floating-point** values ("doubles", like `double` in Java/C). **Floating point** stores significant digits plus an exponent (like 1.23×10^5), giving huge range but approximating most decimal fractions. Consequences:

- about 15–17 significant decimal digits of precision;
- integers exact only up to $2^{53} - 1$ (9,007,199,254,740,991);
- `0.1` can't be stored exactly in binary (like 1/3 in decimal);
- special values `Infinity`, `-Infinity`, `NaN` and a separate `-0`.

The global `Number` holds constants and helpers; `Number(value)` converts (never use `new Number`, §2.4).

Example

`code/09_numbers.js` prints (each line labels the expression it evaluates):

```
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 === 0.3 ? false
close enough? true
money in cents: 10 + 20 = 0.3
MAX_VALUE         1.7976931348623157e+308
MIN_VALUE         5e-324
MAX_SAFE_INTEGER  9007199254740991
MIN_SAFE_INTEGER  -9007199254740991
EPSILON           2.220446049250313e-16
POSITIVE_INFINITY Infinity | NEGATIVE_INFINITY -Infinity
MAX_VALUE * 2     Infinity
2**53 + 1         9007199254740992 (not representable!)
isSafeInteger(2**53): false
literals: 1000000 255 15 10 1500 0.002 0.5
toFixed(2): 123.46 | toFixed(0): 123
toPrecision(4): 123.5 | toPrecision(2): 1.2e+2
(255).toString(2): 11111111 | (255).toString(16): ff
(1.005).toFixed(2): 1.00 (binary rounding!)
toLocaleString: 1,234,567.891 | 1.234.567,891
valueOf of wrapper: 7
parseInt('123px'): 123 | parseInt('px123'): NaN
parseInt('ff', 16): 255 | parseInt('101', 2): 5
parseFloat('3.14abc'): 3.14
Number('123px'): NaN | Number(''): 0 | Number(' 42 '): 42
Number(null): 0 | Number(undefined): NaN | Number(true): 1
isInteger(5.0): true | isInteger(5.5): false
isFinite('5'): true | Number.isFinite('5'): false
isNaN('abc'): true | Number.isNaN('abc'): false | Number.isNaN(NaN): true
2n ** 64n = 18446744073709551616n | typeof: bigint
9007199254740993n + 1n = 9007199254740994n
7n / 2n = 3n (truncates)
1n + 1 -> TypeError: Cannot mix BigInt and other types, use explicit conversions
BigInt(10) + 1n = 11n | Number(5n) + 1 = 6
```

Number literals

Form	Example	Value
decimal	`42`, `3.14`, `.5`	
exponent	`1.5e3`, `2e-3`	1500, 0.002
hex / octal / binary	`0xff`, `0o17`, `0b1010`	255, 15, 10
numeric separators (ES2021)	`1_000_000`	1000000 (underscores are ignored, just for readability)
BigInt	`123n`	a `bigint`

`Number` properties

Property	Value	Meaning
`Number.MAX_VALUE`	≈ 1.797e308	largest finite number; beyond it you get `Infinity`
`Number.MIN_VALUE`	5e-324	smallest **positive** number (closest to zero) — *not* the most negative number
`Number.MAX_SAFE_INTEGER`	$2^{53} - 1$	largest integer `n` such that `n` and `n + 1` are both exactly representable
`Number.MIN_SAFE_INTEGER`	$-(2^{53} - 1)$	the negative counterpart
`Number.POSITIVE_INFINITY` / `NEGATIVE_INFINITY`	`Infinity` / `-Infinity`	results of overflow or division by zero
`Number.NaN`	`NaN`	same as the global `NaN`

`Number` methods

Instance methods (on a number value; return strings except `valueOf`):

Method	What it does	Example → result
`toFixed(d)`	round to `d` digits after the decimal point	`(123.456).toFixed(2)` → `"123.46"`
`toPrecision(p)`	round to `p` significant digits (may switch to exponent form)	`(123.456).toPrecision(4)` → `"123.5"`
`toString(radix)`	write in base 2–36	`(255).toString(16)` → `"ff"`
`toLocaleString(locale)`	format for humans (separators, currency) using a locale (e.g. `"en-US"`)	`(1234.5).toLocaleString("en-US")` → `"1,234.5"`
`valueOf()`	the primitive inside a `Number` wrapper	`new Number(7).valueOf()` → `7`

On an integer literal, wrap it: `(255).toString(16)` — `255.toString()` is a syntax error (the dot reads as a decimal point).

Static methods (on `Number`):

Method	What it does	Watch out
`Number.isFinite(v)`	`true` only for finite *numbers*	the global `isFinite("5")` converts first and says `true`
`Number.isInteger(v)`	integer-valued number?	`5.0` is an integer (it's the same value as `5`)
`Number.isSafeInteger(v)`	integer within $\pm(2^{53} - 1)$?	use before trusting large IDs
`Number.isNaN(v)`	is it exactly `NaN`?	the global `isNaN("abc")` converts first and says `true`
`Number.parseFloat(s)`	read a decimal from the *start* of a string	stops at the first invalid character
`Number.parseInt(s, radix)`	read an integer from the *start* of a string in base `radix`	**always pass the radix** (usually 10)

`parseInt` / `parseFloat` read a leading number and ignore the rest (`"123px"` → `123`, good for CSS values); `Number()` needs the *whole* trimmed string numeric, else `NaN`, and treats `""` as `0`.

`NaN` and `Infinity`

- `NaN` results from failed numeric operations (`Number("abc")` , `0 / 0` , `Math.sqrt(-1)`), spreads through arithmetic, and is the only value not equal to itself. Use `Number.isNaN` .
- `Infinity` results from overflow or division by zero. `Number.isFinite(x)` rejects `±Infinity` and `NaN` at once.

Floating-point precision — what to do about it

1. **Never compare decimals with** `===` : `Math.abs(a - b) < 1e-9` (or `Number.EPSILON` near 1).
2. **Money in integer cents** (store 1999, show 19.99) or a decimal library — payment APIs like Stripe take the smallest currency unit for this reason.
3. **Round only for display** (`toFixed` , `Intl.NumberFormat` / `toLocaleString`). `toFixed` rounds the *binary* value: `(1.005).toFixed(2)` is `"1.00"` (stored as 1.00499999…).
4. **Integers beyond 2^{53}** (DB IDs, X post IDs, nanosecond timestamps) lose precision silently — keep them as **strings** or `BigInt` .

`BigInt`

`bigint` (ES2020): whole numbers of any size, written `10n` or `BigInt(10)` . **No mixing** with `number` in arithmetic (`TypeError`) — convert explicitly; division truncates (`7n / 2n` → `3n`); `Math` rejects BigInts; `JSON.stringify` can't serialise them without help. Use for big IDs, cryptography, exact large-integer maths; `number` otherwise (faster).

In the interview

"Why is `0.1 + 0.2 !== 0.3` ?" Numbers are IEEE-754 binary doubles; 0.1 and 0.2 have no exact binary form, so each is the nearest double and their sum is the double just above 0.3 (0.30000000000000004). Compare with a tolerance or use integer units like cents.

- *`Number.MAX_SAFE_INTEGER` ?* $2^{53} - 1$; above it not every integer is representable (`2**53 + 1 === 2**53`). Big IDs → strings or BigInt.
- *`isNaN` vs `Number.isNaN` ?* The global converts first (`isNaN("abc")` true); `Number.isNaN` is true only for `NaN` .
- *`parseInt("08")` ?* `8` in modern engines — still pass the radix. `parseInt(0.0000005)` is `5` (the number becomes the string `"5e-7"` first).

Self-check

1. `Number("")` vs `parseInt("")` ? → `0` vs `NaN` .
2. `Number.MIN_VALUE` ? → Smallest *positive* double, 5e-324.
3. `typeof NaN` ? → `"number"` .
4. `10n + 5` ? → `TypeError` .

TL;DR

One `number` type (64-bit float): exact integers to $2^{53} - 1$, inexact decimals, `NaN` , `±Infinity` . `Number.isNaN/isFinite/isInteger` ; `toFixed` for display; cents for money; `BigInt` for huge integers.

2.11 Booleans, truthy and falsy

What it is

boolean has two values, `true` and `false` ; conditions (`if` , `while` , `? :` , Chapter 3) consume them. Any value can be a condition: **falsy** values become `false` , **everything else is truthy** .

Falsy values (the complete list)
`false`
`0`, `-0` and `0n` (BigInt zero)
`""` — the empty string, whichever quotes are used
`null`
`undefined`
`NaN`

(Plus the legacy browser oddity `document.all`.) Surprising **truthy** values: `"0"`, `"false"`, `" "`, `[]`, `{}`, `-1`, `Infinity`, every function, every wrapper object — including `new Boolean(false)`.

Example

```
// code/10_boolean_truthiness.js (abridged — each output line labels its expression)
if (false || 0 || "" || null || undefined || NaN) {
  console.log("never printed");
} else {
  console.log("All values are falsy.");
}
const wrapped = new Boolean(false);
console.log("if (new Boolean(false)) runs?", wrapped ? "yes (object is truthy)" : "no");
```

Output:

```
falsy: false->false  0->false  -0->false  0n->false  ""->false  null->false  undefined->false  NaN->false
truthy: true
All values are falsy.
Boolean("hi"): true | !!"": false | !!42: true
typeof wrapped: object | valueOf(): false
if (new Boolean(false)) runs? yes (object is truthy)
new Boolean(true).valueOf(): true | toString(): true
isAdult: true boolean
true + true = 2 | [true, false, true].filter(Boolean).length = 2
```

(All six `||` operands are falsy, so it returns the last, `NaN`, and the `else` runs.)

Converting to boolean

- **`Boolean(value)`** — explicit; **`!!value`** — double NOT shorthand; **`array.filter(Boolean)`** — drops falsy entries.

The `Boolean` wrapper object

`new Boolean(x)` is an object (methods: `valueOf()`, `toString()`) and therefore *always truthy*, even `new Boolean(false)`. Use `Boolean(x)` without `new`.

Common mistakes

- **`if (arr)` for "non-empty"** — `[]` is truthy; use `arr.length > 0`.
- **`if (count)` when `0` is valid** — use `count !== undefined` or `??`.
- **`if (x == true)`** — `true` becomes `1`, so `"yes" == true` is false. Use `if (x)` or `x === true`.
- **`"false"` from a form/URL** is truthy — compare `value === "true"`.

In the interview

"List the falsy values." `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN` (plus legacy `document.all`). Everything else — including `"0"`, `[]`, `{}`, `new Boolean(false)` — is truthy.

- *`!!"false"` ?* `true` — non-empty string.
- *Why is `new Boolean(false)` truthy?* All objects are truthy.

TL;DR

Eight falsy values; everything else truthy. Convert with `Boolean(x)` or `!!x`; never `new Boolean`.

2.12 Type conversion summary

Type conversion (**coercion**) turns a value of one type into another — **explicit** when you ask (`Number(x)`, `String(x)`, `Boolean(x)`), **implicit** when an operator does it (`"3" * 2`, `if (x)`, `x == y`).

```
// code/11_conversions.js (the table loop)
const values = ["42", "", " ", "4px", "0x1f", null, undefined, true, false, [], [7], [1, 2], {}];
for (const v of values) {
  console.log(show(v).padEnd(10), String(Number(v)).padEnd(10), JSON.stringify(String(v)).padEnd(18), Boolean(v));
}
```

Output:

```
value      Number()   String()           Boolean()
"42"       42         "42"               true
""         0          ""                 false
" "        0          " "                true
"4px"      NaN        "4px"              true
"0x1f"     31         "0x1f"             true
null       0          "null"             false
undefined  NaN        "undefined"        false
true       1          "true"             true
false      0          "false"            false
[]         0          ""                 true
[7]        7          "7"                true
[1,2]      NaN        "1,2"              true
{}         NaN        "[object Object]"  true
parseInt vs Number on '08.5kg': 8 NaN
String(123) + 1 = 1231 | `${123}`: 123
```

1. **To number:** whitespace-only → `0`; numeric strings (incl. `0x…`, `0b…`, `0o…`) → the number; other text → `NaN`; `null` → `0`, `undefined` → `NaN`, booleans → `1` / `0`; objects → primitive first (arrays → joined string).
2. **To string:** arrays join with commas (`[]` → `""`); plain objects → `"[object Object]"` (use `JSON.stringify` to see contents).
3. **To boolean:** the §2.11 falsy list → `false`; else `true`.

Context	Converts to
`if`, `while`, `? :`, `!`, `&&`, `||`	boolean
`-` `*` `/` `%` `**`, unary `+` / `-`, `<` `>` (unless both strings)	number
`+` when either side is a string; template literals `` `${x}` ``	string
`==`	follows the §2.6 algorithm

Prefer **explicit** conversions (`Number(input.value)`) and `===`.

Chapter 2 TL;DR

- **Declare** with `const` (default) or `let`; avoid `var` (function-scoped, redeclarable, hoisted as `undefined`). `let` / `const` are block-scoped and throw in the **TDZ**.
- **Types:** seven primitives (`number`, `bigint`, `string`, `boolean`, `undefined`, `null`, `symbol`) + objects. `typeof null` is `"object"`; use `Array.isArray`.
- **Primitives** are immutable, copied by value; **objects** copied and compared by reference.

- **Arithmetic:** `/` is floating point; `%` keeps the left sign; `+` concatenates if either side is a string; postfix `++` returns the old value.
- **Equality:** `===` ; `==` coerces non-transitively (`x == null` is the exception). `Object.is` fixes `NaN` and `-0` .
- **Logic:** `&&` / `||` short-circuit and return operands; `??` defaults only on `null` / `undefined` ; `?.` reads possibly-missing data; `||=` , `&&=` , `??=` assign conditionally.
- **Numbers:** IEEE-754 doubles — `0.1 + 0.2 !== 0.3` , safe integers to $2^{53} - 1$, `NaN` / `Infinity` ; `Number.isNaN` , `toFixed` for display, cents for money, `BigInt` for huge integers; `parseInt(s, 10)` vs `Number(s)` differ on trailing text.
- **Truthiness:** falsy = `false, 0, -0, 0n, "", null, undefined, NaN` ; everything else (incl. `[]` , `"0"` , `new Boolean(false)`) truthy.

Next: Chapter 3 — Control Flow and Loops.

Chapter 3 — Control Flow and Loops

By default a program runs each statement once, top to bottom. **Control flow** changes that: *choosing* which statements run (conditionals) and *repeating* them (loops). With the values and operators of Chapter 2, these are enough to express any **algorithm** (a precise step-by-step recipe).

What you'll learn: `if` / `else if` / `else`, guard clauses and truthiness in conditions; the ternary `? :`; `switch` (cases, `default`, `break`, grouping, strict comparison, fall-through); the five loops `for`, `while`, `do…while`, `for…in`, `for…of`; nested loops; `break`, `continue`, labels; choosing a loop (or an array method).

Code: `code/`. Every "Output:" block is real Node 24 output.

3.1 Conditional statements — making decisions

What it is

A **conditional statement** evaluates a **condition** and, depending on whether it is truthy or falsy, runs one block or another.

Tool	Best for
`if … else if … else`	any conditions, including ranges and combinations (`age >= 18 && hasId`)
`switch`	comparing **one** value against many fixed options (`"Mon"`, `"Tue"` …)
`condition ? a : b`	choosing between two **values** inside an expression

Where you meet them in real applications

- **Form validation** — enable Submit only when the email looks valid.
- **Authentication and permissions** (who the user is, what they may do) — show the admin panel only if logged in *and* admin.
- **UI** (user interface) — toggle a menu; apply dark mode if preferred.
- **Filtering** — show products inside the selected price range.
- **Error handling** — HTTP status 404 → "Not found", 500 → "Try again later" (2xx success, 4xx client error, 5xx server error — System Design Fundamentals).

TL;DR

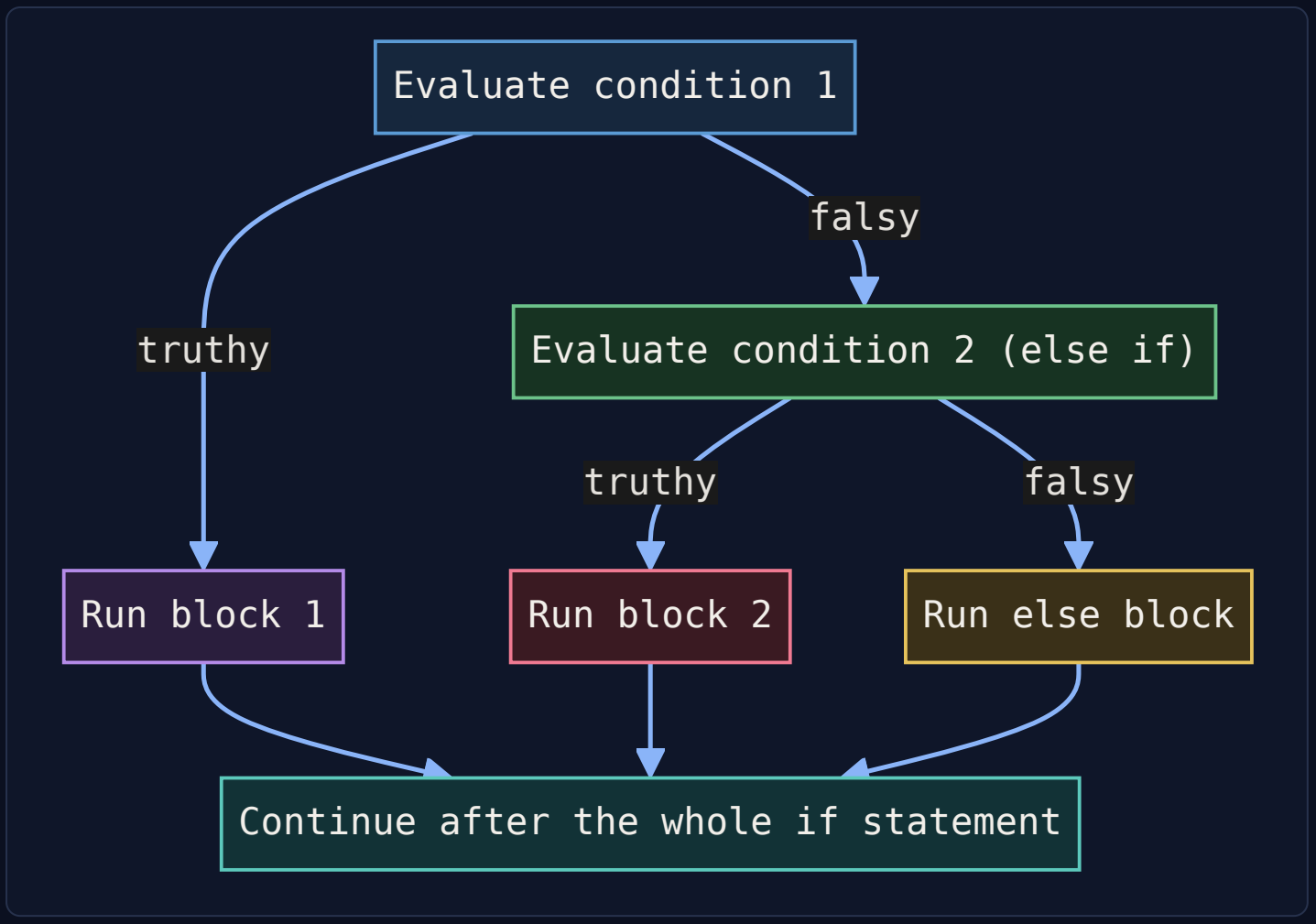
`if` handles anything, `switch` one-value-many-options, the ternary chooses between two values.

3.2 `if`, `else` and `else if`

What it is

- `if (condition) { … }` runs the block only when the condition is truthy.
- `else { … }` (optional) runs when it is falsy.
- `else if (condition) { … }` (any number) adds conditions tested **in order**; the first truthy one wins, the rest are skipped.

How it works



Conditions are converted with the truthy/falsy rules (Chapter 2 §2.11); at most **one** block of the chain runs.

Example

```
// code/01_if_else.js (part 1)
function votingMessage(age) {
  if (age >= 18) {
    return "You can vote.";
  } else {
    return "Not old enough to vote yet.";
  }
}
console.log("age 18 ->", votingMessage(18));
console.log("age 16 ->", votingMessage(16));

function letterGrade(score) {
  if (score >= 90) {
    return "A";
  } else if (score >= 80) {
    return "B";
  } else if (score >= 70) {
    return "C";
  } else if (score >= 60) {
    return "D";
  } else {
    return "F";
  }
}
for (const s of [95, 85, 75, 65, 40]) console.log(`score ${s} -> grade ${letterGrade(s)}`);

function badGrade(score) {
  if (score >= 70) return "C";   // checked first, so 95 also lands here
  if (score >= 90) return "A";
  return "F";
}
console.log("wrong order: 95 ->", badGrade(95));
```

Output:

```
age 18 -> You can vote.
age 16 -> Not old enough to vote yet.
score 95 -> grade A
score 85 -> grade B
score 75 -> grade C
score 65 -> grade D
score 40 -> grade F
wrong order: 95 -> C
```

(Functions are covered in Chapter 4; `for…of` in §3.10.) Order checks **from most specific to least specific**: in `badGrade` the broad `>= 70` catches 95 before `>= 90` is reached.

Conditions are truthy/falsy, not just `true` / `false`

```
// code/01_if_else.js (part 2)
const inputs = ["", "Ana", 0, 3, null, []];
for (const v of inputs) {
  console.log(JSON.stringify(v).padEnd(6), v ? "truthy -> if-branch" : "falsy  -> else-branch");
}
```

Output:

```
""     falsy  -> else-branch
"Ana"  truthy -> if-branch
0      falsy  -> else-branch
3      truthy -> if-branch
null   falsy  -> else-branch
[]     truthy -> if-branch
```

`if (name)` reads well for "non-empty string", but `0` / `""` are falsy and `[]` truthy — compare explicitly when it matters (`items.length > 0`).

Guard clauses — flatter code

A **guard clause** handles a special case at the top of a function and returns immediately, avoiding deep nesting:

```
// code/01_if_else.js (part 3)
function shippingCost(order) {
  if (!order) return "no order";
  if (order.items === 0) return "empty cart";
  if (order.total >= 50) return "free shipping";
  return "shipping: $4.99";
}
const user = { loggedIn: true, role: "editor" };
if (user.loggedIn && (user.role === "admin" || user.role === "editor")) {
  console.log("can publish");
}
```

Output:

```
no order | empty cart | free shipping | shipping: $4.99
can publish
```

Combine conditions with `&&` , `||` , `!` , using parentheses for explicit grouping.

Braces

Braces are optional around a single statement, but this classic bug follows:

```
if (isAdmin)
  showPanel();
  logAccess();   // looks conditional, but ALWAYS runs — only the first line belongs to the if
```

A formatter (e.g. Prettier) re-indents to expose it; many teams always use braces.

Common mistakes

- `=` **instead of** `===` : `if (role = "admin")` assigns and is always truthy.
- **Wrong ladder order** (see `badGrade`).
- `if (arr)` **for emptiness** — always truthy.
- **Overlapping or gapped conditions** (`< 60` and `> 60` miss 60) — test the boundaries.

Self-check

1. In `if (a) {…} else if (b) {…}` , can both run? → No, at most one.
2. `if ("0")` ? → Runs — non-empty string is truthy.
3. Why check `>= 90` before `>= 80` ? → The first truthy branch wins.

TL;DR

First truthy branch runs. Order specific → general, keep braces, prefer guard clauses to nesting.

3.3 The conditional (ternary) operator

What it is

`condition ? valueIfTruthy : valueIfFalsy` is an **expression** picking one of two values — JavaScript's only three-operand (*ternary*) operator.

Example

```
// code/02_ternary.js
const age = 20;
const label = age >= 18 ? "adult" : "minor";
console.log("label:", label);

const count = 1;
console.log(`You have ${count} new ${count === 1 ? "message" : "messages"}`);

const temp = 12;
const feel =
  temp < 0 ? "freezing" :
  temp < 15 ? "cold" :
  temp < 25 ? "pleasant" :
  "hot";
console.log("feel:", feel);

const price = 100, isMember = true;
console.log("to pay:", price * (isMember ? 0.9 : 1));
```

Output:

```
label: adult
You have 1 new message
feel: cold
to pay: 90
```

When to use it (and when not)

Use the ternary when…	Use `if` when…
choosing between two **values**	running **statements** (side effects such as printing or updating variables, several lines)
it fits on one line and reads naturally	the logic needs explanation or more than two outcomes
you need an expression: inside a template literal, an argument, a `const` initialiser, JSX in React (HTML-like syntax inside JavaScript)	—

Chained ternaries are fine one condition per line (as above); nesting inside branches gets unreadable.

TL;DR

`cond ? a : b` returns `a` or `b` — for choosing values, not running code.

3.4 `switch`

What it is

`switch` evaluates one expression once and compares it with a list of `case` values, running code from the first match — a tidy replacement for an `else if` chain testing *the same value*.

```
switch (expression) {
  case value1:
    // runs when expression === value1
    break;
  case value2:
    // runs when expression === value2
    break;
  default:
    // runs when no case matched (optional)
}
```

Part	Role
`expression`	evaluated **once**; its value is compared with each case
`case value:`	a candidate match, compared with `===` (strict equality)
`break`	leaves the switch; without it, execution **falls through** into the next case's code
`default:`	optional catch-all when nothing matched — the switch equivalent of `else`

How it works

Evaluate the expression; compare top-down with `===`; at the first match start executing *from there* and continue until `break` / `return` or the end — **through following cases' code**. No match → start at `default` if present.

Examples

```
// code/03_switch.js (selected parts)
function describeType(value) {
  switch (typeof value) {
    case "number":  return "a number";
    case "string":  return "a string";
    case "boolean": return "a boolean";
    default:        return "something else";
  }
}

function menu(option) {
  let result;
  switch (option) {
    case 1: result = "Home"; break;
    case 2: result = "Profile"; break;
    case 3: result = "Settings"; break;
    default: result = "Invalid selection";
  }
  return result;
}

function openStatus(day) {          // grouped cases share one block
  switch (day) {
    case "Mon": case "Tue": case "Wed": case "Thu": case "Fri":
      return "open 9-17";
    case "Sat": return "open 10-14";
    case "Sun": return "closed";
    default:    return "unknown day";
  }
}

function ageCase(age) {             // strict comparison: "18" is not 18
  switch (age) {
    case 18:   return "matched the NUMBER 18";
    case "18": return "matched the STRING '18'";
    default:   return "no match";
  }
}
console.log(describeType(42), "|", describeType("hi"), "|", describeType(null));
console.log([1, 2, 3, 4].map(menu).join(", "));
console.log(["Tue", "Sat", "Sun", "Funday"].map(openStatus).join(" | "));
console.log(ageCase("18"), "|", ageCase(18), "|", ageCase(18.0));
```

Output:

```
a number | a string | something else
Home, Profile, Settings, Invalid selection
open 9-17 | open 10-14 | closed | unknown day
matched the STRING '18' | matched the NUMBER 18 | matched the NUMBER 18
```

- `typeof null` is `"object"` → no case → `default`.
- `return` leaves the switch too, so no `break` is needed in functions.
- **Grouped cases**: empty cases fall through to the next non-empty one.
- **Strict comparison**: `"18"` matches only `case "18"`; `18.0` is `18`. Convert string input first: `switch (Number(age))`.

Fall-through — feature and trap

```
// code/03_switch.js — no breaks on purpose: each level also gets the lower levels' rights
function withoutBreak(level) {
  const granted = [];
  switch (level) {
    case "admin":  granted.push("delete");   // falls through
    case "editor": granted.push("edit");     // falls through
    case "viewer": granted.push("read");
  }
  return granted;
}
console.log("admin:", withoutBreak("admin"), "viewer:", withoutBreak("viewer"));
```

Output:

```
admin: [ 'delete', 'edit', 'read' ] viewer: [ 'read' ]
```

Intentional fall-through is rare; a forgotten `break` is a classic bug. Mark intentional cases with `// falls through` — ESLint's `no-fallthrough` rule recognises it.

Two useful patterns

```
// code/03_switch.js
function bmiCategory(bmi) {          // switch (true): each case is a boolean condition
  switch (true) {
    case bmi < 18.5: return "underweight";
    case bmi < 25:   return "normal";
    case bmi < 30:   return "overweight";
    default:         return "obese";
  }
}
const statusText = { 200: "OK", 404: "Not Found", 500: "Server Error" };  // lookup object
console.log([17, 22, 27, 33].map(bmiCategory).join(", "));
console.log(statusText[404], "|", statusText[418] ?? "Unknown status");
```

Output:

```
underweight, normal, overweight, obese
Not Found | Unknown status
```

- **`switch (true)`** matches the first case evaluating to `true` (ranges) — works, but `if … else if` is often clearer.
- **A lookup object** (or `Map`, Chapter 8) beats a switch that only maps values; add a fallback with `??`.

Scope inside cases

The switch body is **one block**, so `const r` in two cases collides. Wrap case bodies in braces:

```
case "circle": {
  const r = shape.r;
  return +(Math.PI * r * r).toFixed(2);
}
```

Trade-offs

	`switch`	`if … else if`	lookup object/Map
Compares	one value, `===` only	any condition	exact keys
Ranges (`< 18`)	only via `switch (true)`	natural	no
Risk	forgotten `break`	long chains	keys are strings (objects)
Best when	many discrete values, some sharing code	mixed or range conditions	pure value-to-value mapping

In the interview

"How does `switch` compare values?" With `===`, the expression evaluated once and checked against cases top-down; execution starts at the first match and continues until `break` / `return` or the end (fall-through). `default` runs when nothing matches and may appear anywhere (conventionally last).

- *`switch("1")` vs `case 1`?* Different types under `===`.
- *`let` in two cases?* One block → redeclaration `SyntaxError`; wrap case bodies in braces.

TL;DR

One value vs many `===` cases; `break` / `return` each case, group to share code, `default` as catch-all, `{ }` for per-case variables.

3.5 Loops — why and which

What it is

A **loop** repeats its **body** while a condition holds or once per item in a collection; one run is an **iteration**. You write the body once instead of copying it — for rendering every cart item, summing an order, retrying a request, reading file lines, walking a game grid, **polling** (checking repeatedly at intervals).

Loop	Repeats…	Checks condition	Typical use
`for`	a counted number of times	before each iteration	index-based loops (positions from 0) over arrays, counting
`while`	while a condition is truthy	before each iteration (may run 0 times)	unknown number of iterations
`do…while`	while a condition is truthy	**after** each iteration (runs at least once)	"do it, then decide whether to repeat"
`for…in`	once per enumerable **property name** of an object (§3.9)	—	inspecting an object's keys
`for…of`	once per **value** of an iterable (§3.10: array, string, Map, Set …)	—	the everyday way to walk a collection

---

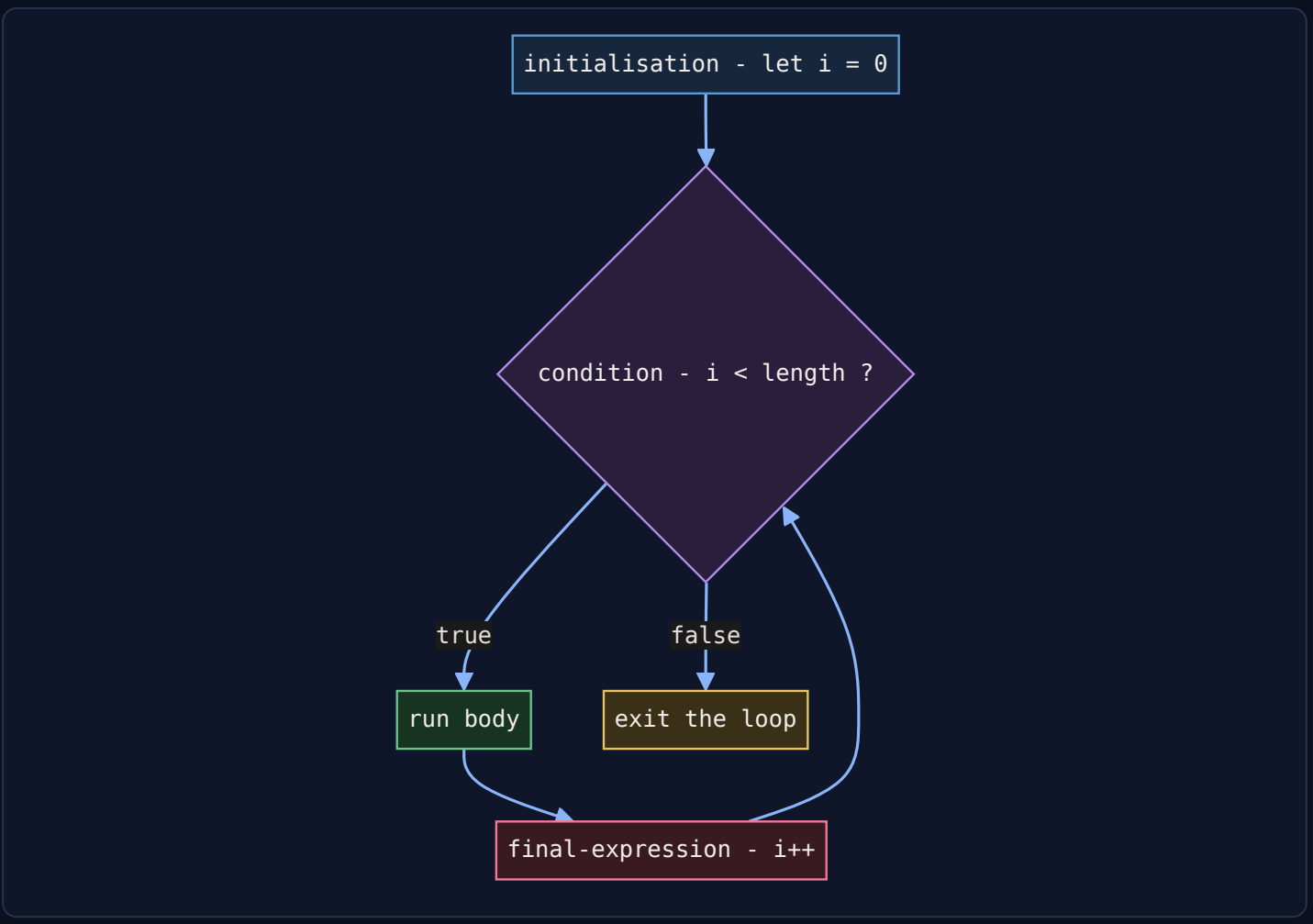
3.6 The `for` loop

What it is

```
for (initialisation; condition; final-expression) {
  // body
}
```

Part	When it runs	Typical content
initialisation	once, before the loop starts	declare a counter: `let i = 0`
condition	before **every** iteration; falsy → stop	`i < items.length`
final-expression	after every iteration	update the counter: `i++`

How it works



Example

```
// code/04_for_loop.js (part 1)
const fruits = ["Apple", "Banana", "Cherry"];
for (let i = 0; i < fruits.length; i++) {
  console.log(i, fruits[i]);
}

const prices = [12, 5, 30, 8];
let total = 0;
for (let i = 0; i < prices.length; i++) {
  total += prices[i];
}
console.log("total:", total);

let down = "";
for (let i = 5; i > 0; i--) down += i + " ";
console.log("countdown:", down.trim());
let evens = [];
for (let i = 0; i <= 10; i += 2) evens.push(i);
console.log("evens:", evens.join(","));
for (let lo = 0, hi = 4; lo < hi; lo++, hi--) console.log(`pair lo=${lo} hi=${hi}`);
```

Output:

```
0 Apple
1 Banana
2 Cherry
total: 55
countdown: 5 4 3 2 1
evens: 0,2,4,6,8,10
pair lo=0 hi=4
pair lo=1 hi=3
```

Trace: `i = 0` → `0 < 3` true → print → `i = 1` → … → `i = 3` → `3 < 3` false → exit. Indexes start at 0, so the last is `length - 1` — hence `<`, not `<=`. The header can count down (`i--`), step (`i += 2`) or manage two comma-separated variables (the "two pointers" interview pattern — Chapter 17).

Nested loops

For **each outer iteration, the inner loop runs all of its iterations**; *m* outer × *n* inner = *m* × *n* inner-body runs.

```
// code/04_for_loop.js (part 2)
const grid = [
  [1, 2, 3],
  [4, 5, 6],
];
for (let row = 0; row < grid.length; row++) {
  let line = "";
  for (let col = 0; col < grid[row].length; col++) {
    line += `[${row},${col}]=${grid[row][col]} `;
  }
  console.log(line.trim());
}
let iterations = 0;
for (let a = 1; a <= 4; a++) {
  let row = "";
  for (let b = 1; b <= 4; b++) { row += String(a * b).padStart(3); iterations++; }
  console.log(row);
}
console.log("inner body ran", iterations, "times");
```

Output:

```
[0,0]=1 [0,1]=2 [0,2]=3
[1,0]=4 [1,1]=5 [1,2]=6
  1  2  3  4
  2  4  6  8
  3  6  9 12
  4  8 12 16
inner body ran 16 times
```

Nested loops walk 2-D data (matrices, spreadsheets, game boards, pixels) and compare every item with every other. Their *m* × *n* cost is the first suspect when code is slow (10,000 × 10,000 = 100 million iterations); interviews often ask to replace such an **O(n²)** loop (Big-O: how work grows with input size *n*; O(n²) — double the input, ~4× the work) with one loop plus a `Map` / `Set` (Chapter 8).

Edge cases

```
// code/04_for_loop.js (part 3)
let n = 0;
for (;;) { if (++n === 3) break; }       // all three parts are optional
console.log("stopped at", n);

const letters = ["a", "b"];
for (let i = 0; i <= letters.length; i++) console.log("i =", i, "->", letters[i]);
```

Output:

```
stopped at 3
i = 0 -> a
i = 1 -> b
i = 2 -> undefined
```

- `for (;;)` is an intentional infinite loop — something inside must `break`.
- **Off-by-one**: `<=` reads a missing index, which quietly gives `undefined` (no error).

`let` in the loop header

With `let` each iteration gets a **fresh copy** of `i` — important when a closure created inside the loop remembers it; with `var` all iterations share one variable (Chapter 4 §4.7).

Common mistakes

- `i <= arr.length` (off-by-one).
- **Removing items while looping by index** — later items shift and get skipped. Loop backwards or use `filter`.
- **Not updating the counter** (or `i++` in the inner loop by mistake) → infinite loop.

TL;DR

`for (init; condition; update)` — init once, check before each iteration, update after. `<` with `length`. Nested loops multiply iteration counts.

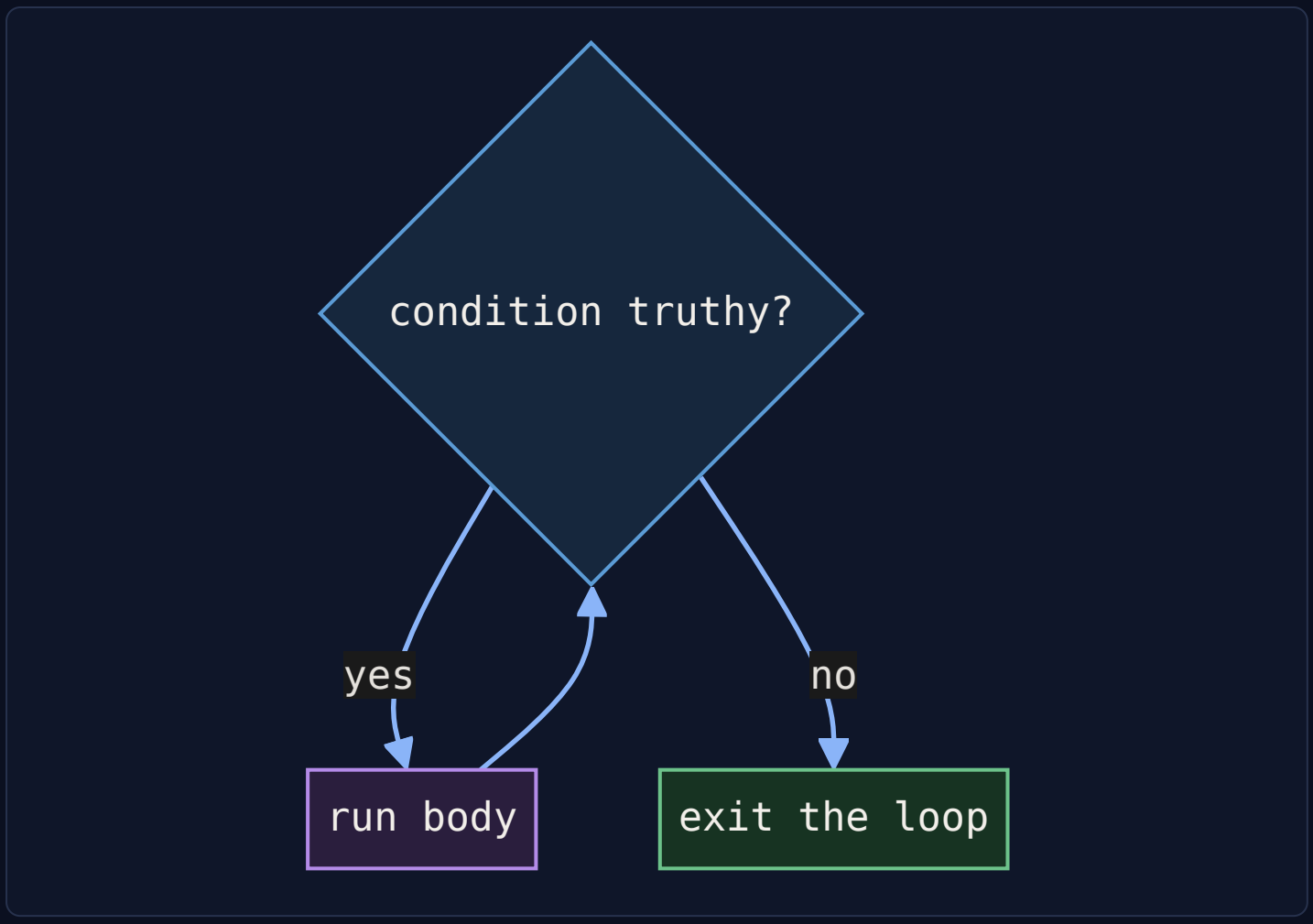
3.7 The `while` loop

What it is

`while` runs its body *as long as* the condition is truthy, checking **before** each iteration (so possibly zero times). Use it when the number of iterations is unknown (read until input ends, retry until success, halve until small). A `for` loop is a `while` with its bookkeeping gathered in the header.

```
while (condition) {
  // body — must eventually make the condition falsy (or break)
}
```

How it works



Example

```
// code/05_while_do_while.js (part 1)
let count = 1;
let out = [];
while (count <= 5) {
  out.push(count);
  count++;
}
console.log("while 1..5:", out.join(" "));

const readings = [5, 3, 7, -1, 2];
let sum = 0, i = 0;
while (i < readings.length && readings[i] >= 0) {
  sum += readings[i];
  i++;
}
console.log(`sum before first negative: ${sum} (stopped at index ${i})`);

let value = 1000, steps = 0;
while (value >= 1) { value /= 2; steps++; }
console.log("halvings:", steps);
```

Output:

```
while 1..5: 1 2 3 4 5
sum before first negative: 15 (stopped at index 3)
halvings: 10
```

In the compound condition, `i < readings.length` comes first, so short-circuiting (Chapter 2 §2.7) prevents reading past the end.

Common mistakes

- **Infinite loop** — nothing in the body changes the condition (no `count++`). Browser tab freezes; in Node press `Ctrl+C`.
- `while` **for a simple count** — `for` keeps counter logic visible in one place.
- **Busy-waiting** — `while (!dataArrived) {}` never works: the single thread is stuck in your loop, so the response can't be processed. Wait with callbacks, promises and `await` (Chapter 12).

TL;DR

`while (cond) { … }` checks first, may run zero times, for unknown counts. The body must move toward ending the loop.

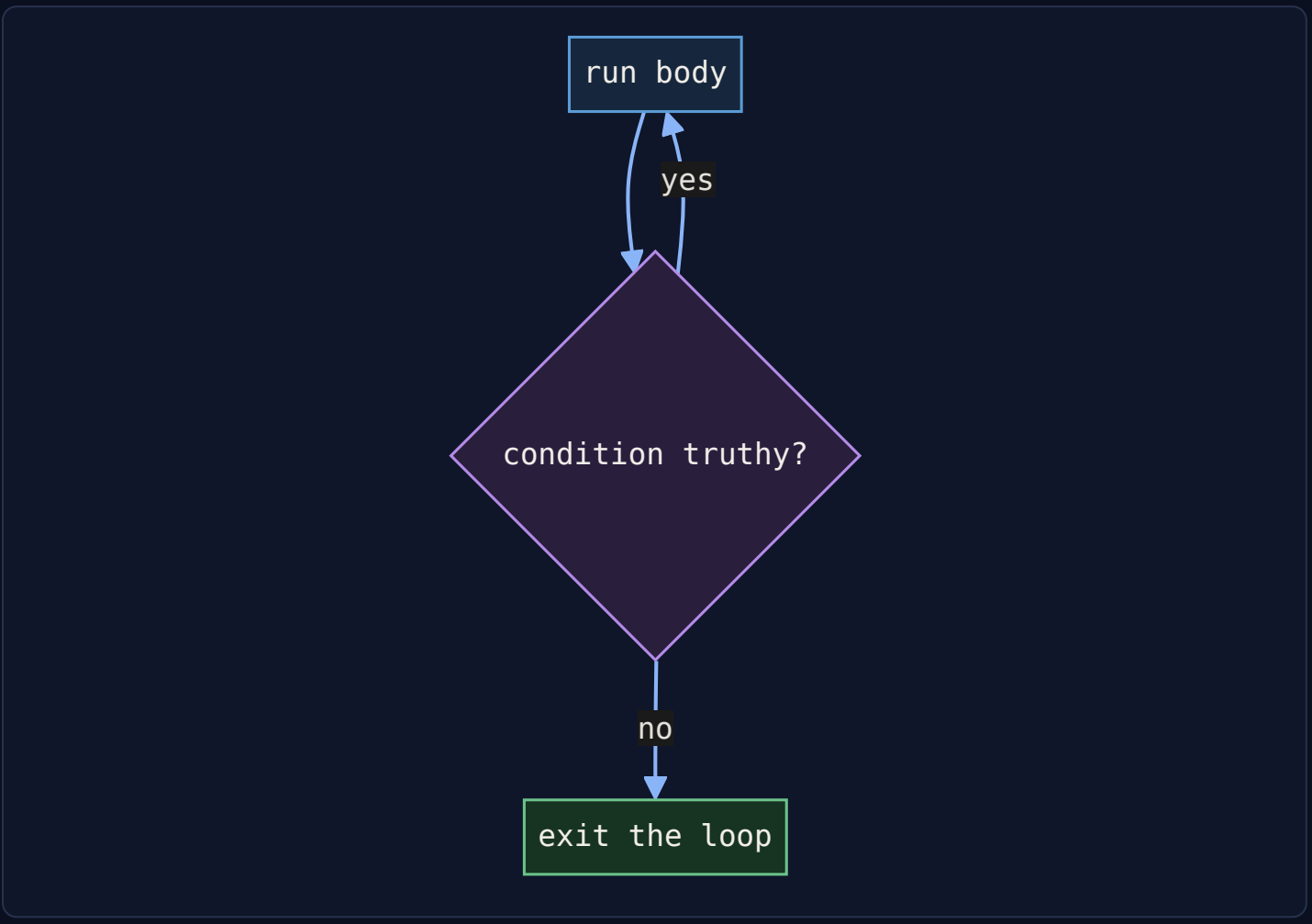
3.8 The `do…while` loop

What it is

`do…while` runs the body **first** and checks afterwards — so **at least once**, even if the condition was already false.

```
do {
  // body — always runs once
} while (condition);   // note the semicolon
```

How it works



Example

```
// code/05_while_do_while.js (part 2)
let x = 10;
while (x < 5) { console.log("while body (never)"); }
do { console.log("do-while body runs once, x =", x); } while (x < 5);

let c = 1; out = [];
do {
  out.push(c);
  c++;
} while (c <= 5);
console.log("do-while 1..5:", out.join(" "));

function makeRandom(seed) {            // tiny deterministic random generator, so the output is repeatable
  return () => (seed = (seed * 1103515245 + 12345) % 2 ** 31) / 2 ** 31;
}
const random = makeRandom(42);
let roll, rolls = [];
do {
  roll = Math.floor(random() * 6) + 1;   // 1..6
  rolls.push(roll);
} while (roll !== 6);
console.log("rolled until a 6:", rolls.join(" "));

let attempt = 0, ok = false;
do {
  attempt++;
  ok = attempt === 3;                    // pretend the 3rd try succeeds
  console.log(`attempt ${attempt}: ${ok ? "success" : "failed"}`);
} while (!ok && attempt < 5);
```

Output:

```
do-while body runs once, x = 10
do-while 1..5: 1 2 3 4 5
rolled until a 6: 4 4 6
attempt 1: failed
attempt 2: failed
attempt 3: success
```

Same false condition: `while` ran zero times, `do…while` once. Dice rolling is the natural shape — you can't test a roll before rolling. (Real code uses `Math.random()`, Chapter 5; the **seeded** generator repeats the same sequence so output is stable.) The retry loop adds a safety cap on attempts.

Trade-offs — `while` vs `do…while`

	`while`	`do…while`
Condition checked	before each iteration	after each iteration
Minimum runs	0	1
Typical use	process while data remains	prompt/generate/try first, then decide
How common	very	rare — a `while` with a flag or `break` is often clearer

TL;DR

`do { … } while (cond);` runs once before testing — use only when the first run is unconditional.

3.9 The `for…in` loop

What it is

`for…in` loops over the **property names (keys)** of an object — all **enumerable string-keyed** properties, **including inherited ones** (from an object it is linked to). An **enumerable** property shows up in such loops: ones you create are, built-ins like `toString` aren't (attributes: Chapter 5).

```
for (const key in object) {
  // key is a string: the current property's name
}
```

Example

```
// code/06_for_in.js
const person = { firstName: "Grace", lastName: "Hopper", born: 1906 };
for (const key in person) {
  console.log(`${key}: ${person[key]}`);
}

const laptop = { brand: "Acme", ram: 16, ssd: 512, weight: 1.3 };
let propertyCount = 0;
for (const key in laptop) propertyCount++;
console.log("laptop has", propertyCount, "properties | Object.keys:", Object.keys(laptop).length);

const base = { kind: "vehicle" };
const bike = Object.create(base);   // bike's prototype is base (see Chapter 9)
bike.wheels = 2;
const all = [], own = [];
for (const key in bike) {
  all.push(key);
  if (Object.hasOwn(bike, key)) own.push(key);   // filter to own properties (ES2022)
}
console.log("all keys:", all, "| own keys:", own);

const colors = ["red", "green", "blue"];
colors.note = "primary-ish";
for (const index in colors) {
  console.log(typeof index, index, "->", colors[index]);
}

const mixed = { b: 1, 2: "two", a: 2, 1: "one" };
const order = [];
for (const k in mixed) order.push(k);
console.log("order:", order.join(", "));
```

Output:

```
firstName: Grace
lastName: Hopper
born: 1906
laptop has 4 properties | Object.keys: 4
all keys: [ 'wheels', 'kind' ] | own keys: [ 'wheels' ]
string 0 -> red
string 1 -> green
string 2 -> blue
string note -> primary-ish
order: 1, 2, b, a
```

1. **The loop variable is the key** — read values with `person[key]` (`person.key` looks for a property named "key").
2. **Inherited properties appear** — `kind` comes from `bike`'s prototype (Chapter 9). Filter with `Object.hasOwn(obj, key)` (ES2022; older: `obj.hasOwnProperty(key)`) or use `Object.keys`.
3. **Arrays are a trap** — you get index **strings** (`"0"`), plus extra properties (`note`) or anything enumerable added to `Array.prototype`. Use `for…of`, `for` or array methods.

4. **Order:** integer-like keys ascending, then other string keys in creation order.

Modern alternatives

Need	Use
own keys	`Object.keys(obj)`
own values	`Object.values(obj)`
own `[key, value]` pairs	`for (const [k, v] of Object.entries(obj))`
number of own properties	`Object.keys(obj).length`

`for…in` is mostly seen in older code or when you want inherited properties.

TL;DR

`for…in` iterates enumerable property **names** (strings), inherited included. Fine for plain objects (with `Object.hasOwn`); not for arrays.

3.10 The `for…of` loop

What it is

`for…of` (ES2015) loops over the **values** of an **iterable** — an object that hands out values one at a time: arrays, strings, `Map`, `Set`, typed arrays, `arguments`, DOM collections like `NodeList`, generators. (The `Symbol.iterator` protocol behind it: Chapter 8.) No index bookkeeping — the everyday choice for collections.

```
for (const value of iterable) {
  // value is the current element
}
```

Example

```
// code/07_for_of.js
const numbers = [10, 20, 30];
for (const n of numbers) console.log("number", n);

for (const ch of "Hi!") console.log("char", ch);

const word = "a😀b";
console.log("length:", word.length, "| for...of count:", [...word].length);
for (const ch of word) console.log("code point:", ch);

const tasks = ["write", "test", "ship"];
for (const [i, task] of tasks.entries()) console.log(`${i + 1}. ${task}`);

const stock = new Map([["apples", 5], ["pears", 0]]);
for (const [item, qty] of stock) console.log(item, "=", qty);
for (const tag of new Set(["js", "css", "js"])) console.log("tag", tag);

const scores = { ana: 90, ben: 72 };
try { for (const x of scores) {} } catch (e) { console.log("for...of on object ->", e.name + ":", e.message); }
for (const [name, score] of Object.entries(scores)) console.log(name, score);

const arr = ["x", "y"];
const ins = [], ofs = [];
for (const k in arr) ins.push(k);
for (const v of arr) ofs.push(v);
console.log("for...in ->", ins, "| for...of ->", ofs);

const nums = [1, 2, 3];
for (let n of nums) n *= 10;
console.log("unchanged:", nums);
```

Output:

```
number 10
number 20
number 30
char H
char i
char !
length: 4 | for...of count: 3
code point: a
code point: 😀
code point: b
1. write
2. test
3. ship
apples = 5
pears = 0
tag js
tag css
for...of on object -> TypeError: scores is not iterable
ana 90
ben 72
for...in -> [ '0', '1' ] | for...of -> [ 'x', 'y' ]
unchanged: [ 1, 2, 3 ]
```

1. **Strings** iterate by Unicode **code point** (the number Unicode assigns each character): the emoji is one character for `for…of`, but `.length` is 4 because it occupies two UTF-16 units (Chapter 6).
2. **Index too?** Iterate `array.entries()` (`[index, value]` pairs) with destructuring `[i, task]` (Chapter 5).
3. **Maps** yield `[key, value]`; **Sets** each unique value once.
4. **Plain objects aren't iterable** → `TypeError`; use `Object.keys/values/entries`.
5. The loop variable is a **copy of each value** — reassigning it doesn't change the array (object values can still be mutated, Chapter 2 §2.4). Use `const` unless reassigning.

Trade-offs — `for…in` vs `for…of`

	`for…in`	`for…of`
Gives you	keys (always strings)	values
Works on	any object	iterables only (arrays, strings, Map, Set …)
Includes inherited properties	yes (enumerable ones)	n/a
Arrays	avoid	✓
Plain objects	✓	✗ — use `Object.entries(obj)`
Supports `break` / `continue`	yes	yes

TL;DR

`for…of` walks values of iterables — the default for collections. `.entries()` for indexes, `Object.entries()` for plain objects.

3.11 `break`, `continue` and labels

What it is

Statement	Effect
`break`	exits the innermost loop (or `switch`) immediately; execution continues after it
`continue`	skips the rest of the current iteration and starts the next one (in a `for` loop, the final-expression still runs first)
`label:`	names a statement (usually an outer loop) so `break label` / `continue label` can target it from inside a nested loop

A label is an identifier plus colon before a statement: `outer: for (…) { … }`.

Example

```
// code/08_loop_control.js
for (let i = 0; i < 5; i++) {
  if (i === 3) break;           // leave the loop entirely
  console.log("break demo i =", i);
}

for (let i = 0; i < 5; i++) {
  if (i === 2) continue;        // skip the rest of this iteration only
  console.log("continue demo i =", i);
}

const orders = [{ id: 1, paid: true }, { id: 2, paid: false }, { id: 3, paid: false }];
let firstUnpaid = null;
for (const order of orders) {
  if (!order.paid) { firstUnpaid = order; break; }
}
console.log("first unpaid order:", firstUnpaid.id);

outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (i === 1 && j === 1) break outer;
    console.log(`labelled break: i=${i}, j=${j}`);
  }
}

const matrix = [[1, 2], [-3, 4], [5, 6]];
const rowsWithoutNegatives = [];
rows: for (const row of matrix) {
  for (const v of row) {
    if (v < 0) continue rows;
  }
  rowsWithoutNegatives.push(row);
}
console.log("rows without negatives:", JSON.stringify(rowsWithoutNegatives));

let k = 0; const odd = [];
while (k < 6) {
  k++;                          // advance BEFORE continue, or the loop never ends
  if (k % 2 === 0) continue;
  odd.push(k);
}
console.log("odd numbers:", odd.join(","));

console.log("some unpaid?", orders.some((o) => !o.paid), "| find:", orders.find((o) => !o.paid).id);
```

Output:

```
break demo i = 0
break demo i = 1
break demo i = 2
continue demo i = 0
continue demo i = 1
continue demo i = 3
continue demo i = 4
first unpaid order: 2
labelled break: i=0, j=0
labelled break: i=0, j=1
labelled break: i=0, j=2
labelled break: i=1, j=0
rows without negatives: [[1,2],[5,6]]
odd numbers: 1,3,5
some unpaid? true | find: 2
```

- **Search-and-stop**: `break` once the first unpaid order is found.
- **Labelled `break`**: a plain inner `break` would let the outer loop continue with `i = 2`; `break outer` ends both at `i=1, j=1`.

- **Labelled `continue`**: a negative abandons the row and advances the *outer* loop, skipping the push.
- **`continue` in `while`** jumps straight to the condition — an update placed after it would be skipped (infinite loop).

When not to use them

- Search loops are often clearer as `find`, `findIndex`, `some`, `every` (Chapter 7), which stop early themselves.
- Labels are rare; alternatively move nested loops into a function and `return`.
- `break` / `continue` don't work inside `forEach` callbacks (separate functions, not a loop statement) — use `for…of`.

In the interview

"How do you exit a nested loop from the inner loop?" Label the outer loop and `break outerLabel`, or put the loops in a function and `return`. Plain `break` exits only the innermost loop.

- *`break` out of `forEach`?* No — `SyntaxError` inside the callback; throwing works but is hacky. Use `for…of`, `some` or `find`.
- *Does `continue` skip a `for` loop's update?* No — `i++` still runs before the next check.

TL;DR

`break` exits, `continue` skips to the next iteration, labels target an outer loop. Prefer `find` / `some` for search-and-stop.

3.12 Choosing the right loop

Situation	Best choice
Visit every element of an array/string/Map/Set	`for…of`
Need the index too	`for…of` over `arr.entries()`, or a classic `for`
Iterate backwards, skip by steps, two pointers	classic `for`
Repeat until something happens (unknown count)	`while`
Must run once before checking	`do…while`
Keys of a plain object	`Object.keys/entries` + `for…of` (or `for…in` + `Object.hasOwn`)
Transform every element into a new array	`arr.map(...)` (Ch. 7)
Keep only some elements	`arr.filter(...)`
Combine all elements into one value	`arr.reduce(...)` or a `for…of` with an accumulator
Find the first match / check any / check all	`find` / `some` / `every`
Wait for asynchronous work (work that finishes later, e.g. a network request)	`for await…of`, `await` in a loop (Ch. 12) — never a busy `while`

Engines optimise all loops; choose the one that states intent most clearly.

Chapter 3 TL;DR

- **Conditionals:** `if / else if / else` (first truthy branch wins — specific to general), ternary `c ? a : b` for values, `switch` for one value vs many `===` cases (`break` against fall-through, `default` catch-all, braces for per-case variables).

- **Loops:** `for` (init; condition; update) for counting, `while` (check first, maybe 0 runs) for unknown counts, `do…while` (at least once), `for…in` (property **names**, inherited included — not arrays), `for…of` (**values** of iterables — default for collections).
- **Nested loops** run the inner loop fully per outer iteration (m × n).
- `break` exits, `continue` skips, **labels** target an outer loop; `forEach` can't be broken — use `for…of`, `find`, `some`.
- Watch off-by-one (`<` vs `<=`), infinite loops (update before `continue`), truthiness surprises.

Next: Chapter 4 — Functions, Scope and Closures.

Chapter 4 — Functions, Scope and Closures

Functions let you name behaviour, reuse it, pass it to other code and return it from other functions. Mastering them means mastering **scope** (which variables code can see) and **closures** (functions that remember the variables around them) — the ideas behind private state, event handlers (functions run when something happens, like a click), React hooks (functions like `useState` that give components memory between screen updates) and many interview questions.

What you'll learn: defining/calling functions (parameters vs arguments, `return`, missing/extra arguments, passing values); declarations vs expressions and hoisting; arrow functions (every form and difference); defaults (evaluated at call time), rest, `arguments`, spread, options objects; global/function/block scope, lexical scoping, the scope chain, shadowing; hoisting in full; closures (counters, module pattern, factories, `once`, `memoize`, the loop puzzle); higher-order functions, callbacks, composition, currying; IIFEs, recursion, pure functions.

Code: `code/` — every "Output:" is real Node 24 output. `this` has its own chapter: Chapter 9 — `this`, Prototypes and Classes.

4.1 What a function is

What it is

A **function** is a named, reusable block of code. You **define** it once and **call** (**invoke**) it as often as you like. It takes inputs (**parameters**) and can hand back one output (a **return value**).

Why it matters

- **Reuse** — write logic once.
- **Readability** — `calculateShipping(order)` explains itself.
- **Modularity** — small functions with clear inputs/outputs are easy to test (Chapter 16) and change safely.
- **Abstraction** — callers need *what*, not *how*.

How it works — anatomy of a function declaration

```
function functionName(parameter1, parameter2) {
  // body: statements that run when the function is called
  return someValue;   // optional
}
```

Part	Meaning
`function`	keyword that starts a function **declaration**
`functionName`	the name you'll call it by; by convention a verb in camelCase (`getUser` , `isValid`)
`(parameter1, parameter2)`	**parameters**: local variables that receive the inputs; zero or more
`{ … }`	the **body**
`return value`	ends the function and sends `value` back to the caller

Calling `functionName(a, b)`:

1. **Argument** expressions are evaluated left to right.
2. A new **execution context** (private workspace with its own locals) is pushed onto the **call stack** (the engine's list of running functions).
3. Parameters are bound to the argument values in order.
4. The body runs until `return` or its closing brace.
5. The context is popped; the call evaluates to the returned value, or `undefined`.

Parameter vs argument: the *parameter* is the name in the definition (`name`); the *argument* is the value supplied in a call (`"Alice"`).

Example

```
// code/01_function_basics.js (abridged — each output line labels what it shows)
function square(number) {              // return sends a value back to the caller
  return number * number;
}
function describe(a, b) { return `a=${a}, b=${b}`; }
console.log(describe(1), "|", describe(1, 2, 3));   // missing -> undefined, extra ignored

function tryToChange(num, obj) {
  num = 100;                 // changes only the local copy
  obj.changed = true;        // changes the shared object
  obj = { replaced: true };  // rebinding the local name doesn't affect the caller
}
let n = 1; const o = {};
tryToChange(n, o);
console.log("after call: n =", n, "| o =", o);
```

Output (full run):

```
Hello, world!
Hello, world!
Hello, Alice!
square(4) = 16 | square(square(2)) = 16
function without return gives: undefined
negative non-negative
a=1, b=undefined | a=1, b=2
isEven(4): true | isEven(5): false
typeof isEven: function | name: isEven | length: 2
square without () is the function itself: function | with (): 9
after call: n = 1 | o = { changed: true }
```

1. Results can feed other calls: `square(square(2))` .
2. **No** `return` → `undefined` ; `return` exits immediately.
3. **Argument counts are never checked:** missing → `undefined` , extras ignored (still reachable, §4.4).
4. **Functions are values** (**first-class**): stored, passed, given properties. `typeof` → `"function"` ; `.name` is the name; `.length` the number of declared parameters.
5. `square` **vs** `square()` : the function itself vs calling it. Passing `handleClick()` where a function is expected passes its *result*.
6. **Pass by value, where an object's value is a reference** (Chapter 2 §2.4) — "call by sharing": the function can mutate the caller's object, but reassigning the parameter only changes the local name.

Common mistakes

- **Forgetting** `return` → caller gets `undefined` .
- **Return value on the next line** → `undefined` (ASI, Chapter 1 §1.6).
- **Calling instead of passing**: `addEventListener("click", save())` runs `save` immediately.

TL;DR

Define once, call many times; missing arguments and missing `return` give `undefined` ; functions are values.

4.2 Declarations vs expressions (and hoisting)

What it is

	Function declaration	Function expression
Looks like	`function add(a, b) { … }` as a statement on its own	`const add = function (a, b) { … };` — a function used as a value
Name	required	optional (anonymous, or named for internal use)
Hoisting	**fully hoisted** — callable before its line	follows the variable: `const` / `let` → TDZ (Temporal Dead Zone) error, `var` → `undefined` (so calling it gives `TypeError`)
Typical use	top-level helpers, named utilities	callbacks (functions handed to other code to call later), conditional definitions, values stored in objects

The **arrow function** (`const add = (a, b) => a + b;`, §4.3) is also an expression.

Example

```
// code/02_declarations_expressions_hoisting.js
console.log("declared later, called early:", add(2, 3));
function add(a, b) { return a + b; }

try {
  multiply(2, 3);
} catch (e) {
  console.log("const expression called early ->", e.name + ":", e.message);
}
const multiply = function (a, b) { return a * b; };

try {
  divide(6, 3);
} catch (e) {
  console.log("var expression called early ->", e.name + ":", e.message);
}
var divide = function (a, b) { return a / b; };

const factorial = function fact(n) { return n <= 1 ? 1 : n * fact(n - 1); };
console.log("factorial(5):", factorial(5), "| typeof fact outside:", typeof fact);

const anon = function () {};
const arrowFn = () => {};
console.log("inferred names:", anon.name, arrowFn.name);

(function () {
  "use strict";
  if (true) { function insideBlock() { return "hi"; } }
  console.log("strict: block function visible outside block?", typeof insideBlock);
})();
```

Output:

```
declared later, called early: 5
const expression called early -> ReferenceError: Cannot access 'multiply' before initialization
var expression called early -> TypeError: divide is not a function
factorial(5): 120 | typeof fact outside: undefined
inferred names: anon arrowFn
strict: block function visible outside block? undefined
```

1. A **declaration** is created whole on scope entry, so it's callable above its line — letting a file's "story" sit on top with helpers below.
2. A `const` **expression** is in the TDZ until its line (Chapter 2 §2.2); a `var` **expression** holds `undefined` → "is not a function".

3. A **named function expression** (`function fact`) can call itself by name — for recursion (§4.10) and clearer **stack traces** (the list of nested calls printed with an error) — but the name isn't visible outside.
4. Anonymous functions assigned to variables get an **inferred** `name` (shown in stack traces).
5. In strict code, a function *declared inside a block* is block-scoped (sloppy mode has murky legacy rules — avoid it there).

Which to use?

Both are fine; be consistent. Common guidance: declarations (or `const` + arrow) for named top-level functions, arrows for callbacks; never rely on calling `const` functions before their definition.

TL;DR

Declarations are hoisted whole; expressions behave like their variable. Named function expressions can call themselves.

4.3 Arrow functions

What it is

Arrow functions (ES2015) are compact function expressions, `(params) => body`, that also differ in behaviour — chiefly, no own `this` (the object a function is working on — Chapter 9).

Syntax forms

Form	Example	Notes
one parameter	`n => n * n`	parentheses optional for exactly one simple parameter
several parameters	`(a, b) => a + b`	parentheses required
no parameters	`() => "hello"`	empty parentheses **required**
default / rest / destructured parameter	`(name = "Guest") => …`, `(...xs) => …`, `({ id }) => …`	parentheses required
expression body	`x => x * 2`	the expression's value is returned automatically (**implicit return**)
block body	`x => { const y = x * 2; return y; }`	braces → you must write `return`
returning an object literal (an object written directly in `{ }`)	`(x, y) => ({ x, y })`	wrap in parentheses, or the braces are read as a block

Example

```
// code/03_arrow_functions.js (abridged — the file first exercises every syntax form in the table)
"use strict"; // strict: a plain function call gets this === undefined
const makePointWrong = (x, y) => { x: x, y };    // parsed as a block with a label -> returns undefined
const makePoint = (x, y) => ({ x, y });

const timer = {
  seconds: 0,
  tickRegular() {
    return [1, 2, 3].map(function () { return this === undefined ? "undefined" : typeof this; });
  },
  tickArrow() {
    return [1, 2, 3].map(() => { this.seconds++; return this.seconds; });
  },
};

const counter = {
  count: 5,
  regular() { return this.count; },
  arrow: () => this?.count,                       // `this` comes from the surrounding (file-level) code
};

function outer() {
  const inner = () => arguments.length;          // uses outer's arguments
  return inner();
}
try { new square(2); } catch (e) { console.log("new on arrow ->", e.name + ":", e.message); }
```

Output (full run):

```
25 5 hello Hello, Guest! Hello, Ana! 2 word(s)
without (): undefined | with (): { x: 1, y: 2 }
with tax: [ 6, 14.4, 36 ] | over 10: [ 12, 30 ]
regular callback this: undefined | arrow callback: [ 1, 2, 3 ]
method regular: 5 | method arrow: undefined
arrow sees outer arguments.length: 3
new on arrow -> TypeError: square is not a constructor
```

How arrows differ from regular functions

Feature	Regular `function`	Arrow `=>`
`this`	set by **how** the function is called (Ch. 9)	**lexical**: taken from the surrounding code, fixed forever (even `call` / `bind` can't change it)
`arguments` object	yes	no — uses the enclosing function's, if any
Can be called with `new`	yes (declarations/expressions)	no — `TypeError`; no `prototype` property
`super` (reaches the parent class), `new.target` (tells whether `new` was used)	own	lexical
Can be a generator (a function that pauses and hands out values one at a time with `yield`, Ch. 8)	yes	no
Hoisted	declarations yes	no (it's an expression)
Good as object method	yes	**no** — `this` won't be the object
Good as callback inside a method	needs `bind` / `self = this`	**yes** — sees the method's `this`

In `tickRegular`, `map` calls the inner `function` plainly, so in strict mode its `this` is `undefined` — the object is lost. In `tickArrow` the arrow borrows `this` from the method (`timer`), so `this.seconds++` works — replacing the pre-ES2015 `var self = this;` / `.bind(this)` workarounds. Conversely `counter.arrow`, an arrow *as* a method, gets the `this` of where the literal was written: at the top of a CommonJS file (Node's original `require` / `module.exports` format) Node sets `this` to `module.exports` (an empty object), so `this.count` is `undefined`; in an ES module top-level `this` is `undefined`, same result.

Why it matters / benefits

- **Concise** callbacks: `items.filter(i => i.inStock).map(i => i.price)`.
- **Predictable** `this` in callbacks, timers and event handlers inside class methods.
- Ubiquitous in React (a UI library built from reusable *components*) and functional-style code.

When not to use arrows

- **Object methods** using `this` → method shorthand `name() { … }`.
- **Constructors** (called with `new`) → a `class` (Ch. 9).
- **DOM handlers needing `this` as the element** → regular function or `event.currentTarget` (Ch. 13).
- **Long multi-step logic** — a named declaration may read better.

In the interview

30-second answer — "Arrow vs regular functions?" Arrows are shorter with implicit return for expression bodies, but the real difference: they don't bind their own `this`, `arguments`, `super` or `new.target` — they take them from the enclosing scope. Great as callbacks inside methods, wrong as object methods or constructors; no `new`, no `prototype`.

- *Change an arrow's `this` with `bind`?* No — `bind` / `call` / `apply` pass arguments but the `this` argument is ignored.
- *Return an object literal?* `() => ({ a: 1 })`.
- *All arguments in an arrow?* Rest parameter: `(...args) => args.length`.

TL;DR

`=> expression` returns it; `=> { … }` needs `return`. Lexical `this`, no `arguments` / `new` — perfect callbacks, poor methods.

4.4 Parameters: defaults, rest, `arguments` and spread

Default parameters — what problem they solve

Pre-ES2015 code patched missing arguments with `name = name || "Guest";` — extra code with a bug: `||` overwrites *every falsy value*, including deliberate `""` or `0`. **Default parameters** put the fallback in the **signature** (name + parameter list) — `function greet(name = "Guest")` — and replace **only** `undefined`.

How defaults work

1. Parameters initialise **left to right**.
2. If an argument is `undefined` (missing or explicit), the default is used; `null`, `0`, `""`, `false` are kept.
3. The default is an **expression evaluated at call time**, each time needed — it can call functions (`new Date()`, `nextId()`) and use **earlier** parameters.
4. Parameters behave like `let`: a default can't reference a *later* parameter (TDZ).

Example

```
// code/04_parameters.js (part 1, abridged — each output line labels what it shows)
function oldGreet(name) { name = name || "Guest"; return "Hello, " + name + "!"; }
function greet(name = "Guest") { return "Hello, " + name + "!"; }

function orderPizza(type = "Cheese", size = "Medium") {
  return `Ordering a ${size} ${type} pizza.`;
}
console.log(orderPizza(undefined, "Large"));   // skip the first by passing undefined

let counter = 0;
const nextId = () => ++counter;
function createUser(name, id = nextId(), tag = `${name.toLowerCase()}-${id}`) {
  return { name, id, tag };
}

function addItem(item, list = []) { list.push(item); return list; }

const clock = { now: () => "09:30:00" };
function logMessage(message, timestamp = clock.now()) { return `${timestamp}: ${message}`; }

function required(name) { throw new Error(`Missing parameter: ${name}`); }
function connect(url = required("url")) { return "connecting to " + url; }
```

Output (full run):

```
Hello, Guest! | Hello, Guest! | Hello, Ana!
empty string -> old: Hello, Guest! | new: "Hello, !"
undefined triggers the default: Hello, Guest! | null does not: Hello, null!
Ordering a Medium Cheese pizza.
Ordering a Medium Pepperoni pizza.
Ordering a Large Cheese pizza.
{ name: 'Ana', id: 1, tag: 'ana-1' } { name: 'Ben', id: 2, tag: 'ben-2' } { name: 'Cy', id: 99, tag: 'cy-99' }
[ 'a' ] [ 'b' ]
09:30:00: Server started | 10:00:00: Custom
Missing parameter: url
```

- **Several defaults**: to default an *earlier* parameter while supplying a later one, pass `undefined` — or use an options object.
- **Call-time evaluation**: `nextId()` ran per call; `tag` used earlier parameters. A real `timestamp = new Date().toLocaleTimeString()` gives each call its own time (a fixed clock keeps output reproducible).
- **Fresh default per call**: `list = []` is new each call — no Python-style shared mutable default.
- **Required parameters**: a throwing default turns a forgotten argument into a clear error.

Rest parameters, `arguments` and spread

```
// code/04_parameters.js (part 2)
function sum(...numbers) {
  let total = 0;
  for (const n of numbers) total += n;
  return total;
}
console.log("sum():", sum(), "| sum(1,2,3):", sum(1, 2, 3));
function tagAll(tag, ...words) { return words.map(w => `${tag}:${w}`); }
console.log(tagAll("js", "let", "const"));

function legacySum() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) total += arguments[i];
  return { total, isArray: Array.isArray(arguments), length: arguments.length };
}
console.log(legacySum(4, 5, 6));

const nums = [3, 9, 1];
console.log("Math.max(...nums):", Math.max(...nums), "| sum(...nums, 10):", sum(...nums, 10));
console.log("lengths:", orderPizza.length, sum.length, tagAll.length, createUser.length);

function makeButton({ label = "OK", color = "blue", disabled = false } = {}) {
  return `${label} (${color}${disabled ? ", disabled" : ""})`;
}
console.log(makeButton(), "|", makeButton({ label: "Save", disabled: true }));
```

Output:

```
sum(): 0 | sum(1,2,3): 6
[ 'js:let', 'js:const' ]
{ total: 15, isArray: false, length: 3 }
Math.max(...nums): 9 | sum(...nums, 10): 23
lengths: 0 0 1 1
OK (blue) | Save (blue, disabled)
```

Tool	Where	What it does
Rest parameter `...args`	last parameter in a definition	collects all remaining arguments into a **real array** (empty if none)
`arguments`	inside any non-arrow function	legacy **array-like** object of all arguments: has `length` and indexes, but no array methods; absent in arrows
Spread `...array`	in a call (or array/object literal)	expands an iterable (arrays, strings …) into separate arguments — the mirror image of rest
Options object with destructuring (pulling properties out into variables)	`function f({ a = 1, b = 2 } = {})`	named, order-independent parameters with defaults; the `= {}` lets you call `f()` with no argument at all
`fn.length`	property	number of parameters **before** the first one with a default or rest

Prefer rest over `arguments`: real arrays, work in arrows, self-describing. Destructuring is in Chapter 5; options objects are how most library APIs and React props (the settings object passed into a component) take many optional settings.

Common mistakes

- Expecting `null` to trigger a default — only `undefined` does; use `??` in the body for `null`.
- Required parameters after optional ones (`f(a = 1, b)`) — put defaults last or use an options object.
- In sloppy mode, mutating `arguments` also changes named parameters (and vice versa); in strict mode they're independent.

In the interview

- *When is a default evaluated?* At call time, each time the argument is `undefined`, left to right, with access to earlier parameters.
- *Rest vs spread?* Same `...`, opposite jobs: rest *collects* values into an array (parameter lists, destructuring); spread *expands* an iterable (calls, literals).

TL;DR

Defaults replace only `undefined`, per call, may use earlier parameters. `...rest` collects; spread expands; options objects give named parameters.

4.5 Scope: global, function and block

What it is

Scope is the region where a variable is visible. A name is looked up in the current scope, then each enclosing one outward.

Scope	Created by	Variables that live there	Visible
Global	top level of a script	top-level `var`, `let`, `const`, functions	everywhere
Function	every function body	`var`, `let`, `const` and parameters declared in the function	inside that function (and functions nested in it)
Block	any `{ … }`: `if`, `for`, `while`, `switch`, a bare block	`let`, `const`, `class` (and functions in strict mode)	only inside those braces

`var` **ignores blocks** (scoped to the nearest function or global); `let` / `const` **respect blocks**. ES modules add **module scope**: top-level names visible in that file only (Chapter 10). Inner scopes see outward; outer scopes can't see in.

Example

```
// code/05_scope.js (abridged — each output line labels what it shows)
function testScope() {
  if (true) {
    var functionScoped = "var: visible in the whole function";
    let blockLet = "let: only in this block";
  }
  console.log("after block:", functionScoped);
  try { blockLet; } catch (e) { console.log("after block, let ->", e.name + ":", e.message); }
}

const level = "global";
function showLevel() { return level; }
function caller() {
  const level = "caller";
  return showLevel();              // showLevel looks up `level` where it was DEFINED
}
console.log("lexical scope result:", caller());

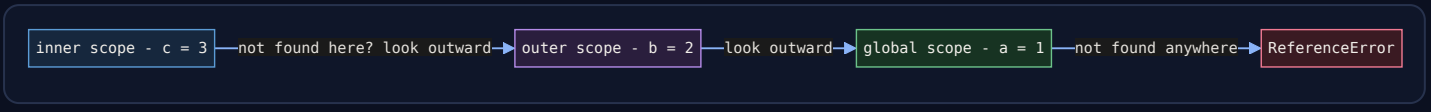
const a = 1;
function outer() {
  const b = 2;
  function inner() {
    const c = 3;
    return a + b + c;              // c local, b from outer, a from global
  }
  return inner();
}
console.log("scope chain sum:", outer());
```

Output (full run):

```
inside function: global var | global let | global const
inside block: let: only in this block / const: only in this block
after block: var: visible in the whole function
after block, let -> ReferenceError: blockLet is not defined
outside function, var -> ReferenceError
var loop counter after loop: 3 | let counter exists? undefined
shadowed: blue | outer still: red
lexical scope result: global
scope chain sum: 6
classic script globals -> var on global object: true | let: false | const: false
this CommonJS file's top-level var on globalThis? false
```

How it works — lexical scope and the scope chain

Lexical (static) scoping: visible scopes are decided by **where the function is written**, not where it's called. `showLevel` was written at top level, so `caller()` returns `"global"` despite its own `level`.



The **scope chain**: current scope → lexically enclosing scope → … → global → `ReferenceError: x is not defined` (or, when *assigning* in sloppy mode, an accidental global).

Shadowing: an inner declaration with the same name hides the outer one inside the inner scope (`paint` 's `color`); the outer is untouched. Legal but confusing — linters (e.g. ESLint) can flag it.

The global object and scripts vs modules

In a **classic browser script**, top-level `var` and function declarations become properties of the global object (`window` / `globalThis`); top-level `let` / `const` are global variables but not properties (the example shows this via Node's `vm` module). In **ES modules** and **Node CommonJS files**, top-level declarations are file-private — hence `globalVar` is not on `globalThis` in the last line.

Why it matters

- **Fewer collisions** — many functions can use `i` or `result`.
- **Encapsulation** — globals are shared by *every* script on the page, including third-party ads/analytics; keep the global scope clean.
- **Garbage collection** — once no code can reach a scope's variables, their memory can be freed.

Common mistakes

- `var` in an `if` / `for` visible after the block (`vi`).
- Expecting a variable from the *calling* function — scope is lexical, not dynamic.
- Forgetting `let` / `const` → accidental global (an error in strict mode, Chapter 1 §1.8).

In the interview

"Explain scope in JavaScript." Scope decides where a name is visible: global, function, block (plus module). `var` is function-scoped, `let` / `const` block-scoped. Scoping is lexical: a function sees the variables where it's *defined*, walking outward along the scope chain — which is what makes closures possible.

- *Shadowing?* An inner declaration hiding a same-named outer variable within the inner scope.
- *Does top-level `let` create a `window` property?* No; only top-level `var` and function declarations in classic scripts.

TL;DR

Global → function → block. `var` function-scoped, `let` / `const` block-scoped. Lookups walk the lexical scope chain; unresolved names throw `ReferenceError`.

4.6 Hoisting — the complete picture

Extending Chapter 2 §2.2: every declaration kind observed *before* its line runs.

```
// code/06_hoisting.js (the report part)
function demo() {
  const report = [];
  report.push(["function declaration", typeof hoistedFn === "function" ? hoistedFn() : "missing"]);
  report.push(["var", String(v)]);
  try { l; } catch (e) { report.push(["let", e.name]); }
  try { c; } catch (e) { report.push(["const", e.name]); }
  try { new K(); } catch (e) { report.push(["class", e.name]); }
  try { fe(); } catch (e) { report.push(["var function expression", e.name + " (" + typeof fe + ")"]); }

  function hoistedFn() { return "callable"; }
  var v = 1;
  let l = 2;
  const c = 3;
  class K {}
  var fe = function () {};
  return report;
}
```

Output (full run):

```
function declaration     -> callable
var                      -> undefined
let                      -> ReferenceError
const                    -> ReferenceError
class                    -> ReferenceError
var function expression  -> TypeError (undefined)
ReferenceError: inner x is in its TDZ
var shadow hoisting: [ undefined, 'grumpy' ]
```

Declaration	Hoisted?	Value before its line
`function f() {}`	yes, with its body	the function (callable)
`var x`	yes	`undefined`
`let x`, `const x`	yes	uninitialised → `ReferenceError` (TDZ)
`class X {}`	yes	uninitialised → `ReferenceError` (TDZ)
`var f = function () {}` / `const f = () => {}`	as the variable	`undefined` (→ `TypeError` if called) / TDZ

The last two output lines:

- `let` **shadowing with TDZ**: a function with a later `let x` can't read an *outer* `x` above that line — the inner `x` already exists (in its TDZ) for the whole block.
- `var` **shadowing**: in `moodSwing`, `mood` reads `undefined` instead of the global `"happy"` because the function's own `var mood` is hoisted.

Mental model: the engine first registers every declaration in a scope, then runs the code. Nothing physically moves.

4.7 Closures

What it is

A **closure** is a function together with the variables of the scope where it was created. Through lexical scoping an inner function uses outer variables — and **keeps using them after the outer function has returned**; they stay alive as long as the inner function exists. Think of a backpack of (references to) surrounding variables the function carries everywhere.

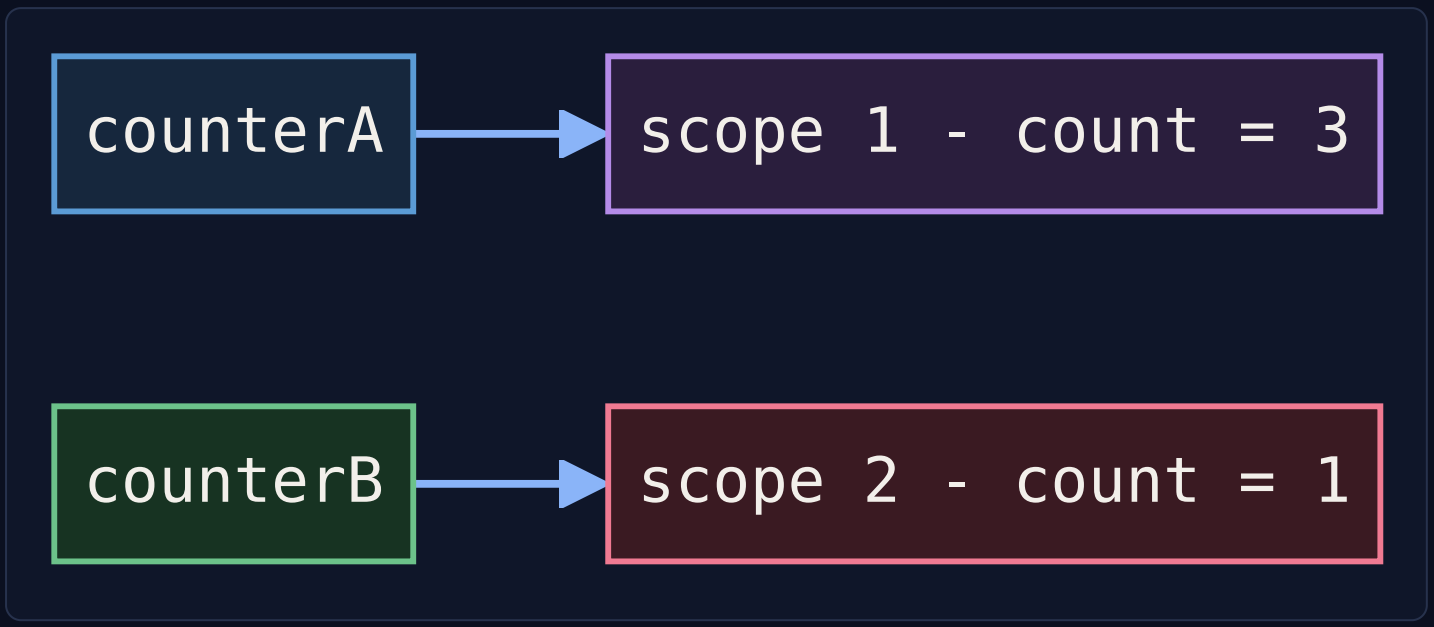
Example — a counter with private state

```
// code/07_closures.js (part 1)
function makeCounter() {
  let count = 0;                        // private: only the returned function can reach it
  return function () {
    count++;
    return count;
  };
}
const counterA = makeCounter();
const counterB = makeCounter();         // an independent closure with its own count
console.log("A:", counterA(), counterA(), counterA(), "| B:", counterB());
```

Output:

```
A: 1 2 3 | B: 1
```

1. `makeCounter()` creates a fresh scope with `count = 0`.
2. The inner function captures a reference to that scope and is returned.
3. `makeCounter` ends, but the scope survives because the function references it.
4. Each `counterA()` increments *that* `count`; nothing else can touch it.
5. Another `makeCounter()` call creates a **new** scope — `counterB` is independent.



The module pattern — several functions sharing private state

```
// code/07_closures.js (part 2)
function createBankAccount(initial) {
  let balance = initial;
  const history = [];
  function record(type, amount) { history.push(`${type} ${amount}`); }
  return {
    deposit(amount) { balance += amount; record("deposit", amount); return balance; },
    withdraw(amount) {
      if (amount > balance) throw new Error("Insufficient funds");
      balance -= amount; record("withdraw", amount); return balance;
    },
    get balance() { return balance; },
    history: () => [...history],
  };
}
const acct = createBankAccount(100);
acct.deposit(50);
acct.withdraw(30);
console.log("balance:", acct.balance, "| direct access:", acct.balanceVar, "| history:", acct.history());
try { acct.withdraw(1000); } catch (e) { console.log("withdraw 1000 ->", e.message); }
```

Output:

```
balance: 120 | direct access: undefined | history: [ 'deposit 50', 'withdraw 30' ]
withdraw 1000 -> Insufficient funds
```

All methods close over the same `balance` / `history`; the balance changes only through rule-enforcing `deposit` / `withdraw`, and `history()` returns a copy. This is **encapsulation** without classes — the pre-ES2015 **module pattern** (§4.9). Classes now offer `#private` fields (Chapter 9).

More closure patterns

```
// code/07_closures.js (part 3)
function makeMultiplier(factor) { return (x) => x * factor; }
const double = makeMultiplier(2), triple = makeMultiplier(3);
console.log("double(5):", double(5), "| triple(5):", triple(5));

function makeReader() {
  let message = "first";
  const read = () => message;
  message = "changed later";
  return read;
}
console.log("closure sees latest value:", makeReader()());

function once(fn) {
  let done = false, result;
  return (...args) => {
    if (!done) { done = true; result = fn(...args); }
    return result;
  };
}
const init = once(() => { console.log("  initialising..."); return 42; });
console.log("init:", init(), init(), init());

function memoize(fn) {
  const cache = new Map();
  return (n) => {
    if (cache.has(n)) return cache.get(n);
    const value = fn(n);
    cache.set(n, value);
    return value;
  };
}
let slowCalls = 0;
const slowSquare = (n) => { slowCalls++; return n * n; };
const fastSquare = memoize(slowSquare);
fastSquare(9); fastSquare(9); fastSquare(9);
console.log("memoized: underlying function ran", slowCalls, "time(s)");
```

Output:

```
double(5): 10 | triple(5): 15
closure sees latest value: changed later
  initialising...
init: 42 42 42
memoized: underlying function ran 1 time(s)
```

- **Function factories** — configure once (`factor`), get a specialised function.
- **Closures capture variables, not values** — `read` sees the value at *call* time.
- `once` — `done` flag in the closure; one-time init, "submit only once" buttons.
- `memoize` (**memoization** = remembering results of earlier calls) — the **cache** lives in the closure, so repeated inputs skip the work. (Real memoizers must handle cache size and multi-argument keys.)

The classic loop puzzle

```
// code/07_closures.js (part 4)
const withVar = [], withLet = [];
for (var i = 0; i < 3; i++) withVar.push(() => i);
for (let j = 0; j < 3; j++) withLet.push(() => j);
console.log("var closures:", withVar.map(f => f()), "| let closures:", withLet.map(f => f()));

for (var t = 0; t < 3; t++) setTimeout(() => process.stdout.write(`var-timer ${t}  `), 0);
for (let u = 0; u < 3; u++) setTimeout(() => process.stdout.write(`let-timer ${u}  `), 0);
setTimeout(() => console.log(), 0);
```

Output:

```
var closures: [ 3, 3, 3 ] | let closures: [ 0, 1, 2 ]
var-timer 3  var-timer 3  var-timer 3  let-timer 0  let-timer 1  let-timer 2
```

With `var` there is **one** function-scoped `i`; all closures share it, and when they run the loop has finished (`i` is 3). A `let` in a `for` header creates a **new binding** (fresh name-to-value slot) **per iteration**, copying the current value, so each closure has its own `j`. The timers are the same story with async callbacks (Chapter 12). Pre-ES2015 fix: an IIFE per iteration (§4.9); today, `let`.

Why closures matter — real-world examples

- **Event handlers** remember their data: `addEventListener("click", () => deleteItem(item.id))` (Ch. 13).
- **Debounce / throttle** (debounce = run once after calls stop for a moment; throttle = at most once per time window) keep timer IDs in a closure (Ch. 17).
- **React hooks**: each render's handlers close over that render's props and state — the source of "stale closure" bugs.
- **Callbacks with context**: `setTimeout(() => notify(user), 1000)` keeps `user` alive.
- **Private state** and factories.

Common mistakes and costs

- **Stale closures** — reading a variable from an old scope (an old React render) when you expected the latest.
- **Memory** — anything a live closure references can't be garbage-collected; a long-lived event listener capturing a huge array keeps it alive. Remove unneeded listeners.
- **The loop puzzle** — use `let`.

In the interview

30-second answer — "What is a closure?" A function bundled with references to the variables of the scope where it was defined. Because scope is lexical, an inner function accesses outer variables — and keeps that access after the outer function returns, so they stay alive. Closures give private state (counters, module pattern), factories, memoization, and underpin event handlers and callbacks.

- *`for (var i…) setTimeout(() => log(i))` prints the final value thrice?* One shared `i`; callbacks run after the loop. `let` gives one binding per iteration.
- *Do closures copy values?* No — live references; later changes show up.
- *Memory leaks (memory never freed because something still points at it)?* Long-lived closures (listeners, timers, caches) holding large objects keep them reachable.

Self-check

1. `const c = makeCounter(); c(); c();` — next `c()`? → `3`.
2. Do `makeCounter()` calls share a count? → No; new scope each call.
3. Variable only two functions can modify? → Declare it in an outer function, return both (module pattern).

TL;DR

Closure = function + the variables around its definition, alive as long as the function. Private state, factories, `once` / `memoize`; `let` in loops; beware stale values and retained memory.

4.8 Higher-order functions and callbacks

What it is

Because functions are **first-class**:

- A **callback** is a function passed to another function so it can call it ("call back") at the right moment.
- A **higher-order function** takes a function as argument, returns one, or both.

Example

```
// code/08_higher_order_callbacks.js (abridged — each output line labels what it shows)
const paidTotals = orders.filter((o) => o.status === "paid").map((o) => o.total);
const revenue = paidTotals.reduce((sum, t) => sum + t, 0);

function greaterThan(limit) { return (n) => n > limit; }      // returns a function

const compose = (...fns) => (x) => fns.reduceRight((acc, fn) => fn(acc), x);
const pipe = (...fns) => (x) => fns.reduce((acc, fn) => fn(acc), x);
const slugify = pipe(trim, lower, dashes);                     // trim, then lower, then dashes

const curriedAdd = (a) => (b) => (c) => a + b + c;
const baseTen = volume.bind(null, 10);                         // partial application

console.log("A: before");
[1].forEach(() => console.log("B: forEach callback runs immediately (synchronous)"));
setTimeout(() => console.log("D: setTimeout callback runs later (asynchronous)"), 0);
console.log("C: after");

function hits() { hits.count = (hits.count ?? 0) + 1; }      // functions can carry properties
```

Output (full run):

```
callback call 0
callback call 1
callback call 2
paid totals: [ 25, 40 ] | revenue: 65
sorted by total desc: [ 2, 3, 1 ]
[5, 15, 25] > 10: [ 15, 25 ]
slug: hello-big-world | compose order: a-b
curried: 6 20
partial: 60
A: before
B: forEach callback runs immediately (synchronous)
C: after
function property count: 2
D: setTimeout callback runs later (asynchronous)
```

The patterns

Pattern	Idea	Example above
Callback	pass behaviour in	`repeat(3, action)`
Array HOFs (higher-order functions)	`map`, `filter`, `reduce`, `sort`, `find`, `some` … take callbacks (Ch. 7)	revenue of paid orders
Returning a function	build a predicate (true/false function)/handler from settings	`greaterThan(10)`
Composition	combine small functions into a pipeline; `pipe` runs left→right, `compose` right→left	`slugify`
Currying	turn `f(a, b, c)` into `f(a)(b)(c)` so arguments can be supplied one at a time	`curriedAdd`
Partial application	fix some arguments now, supply the rest later (`bind` can do it)	`baseTen`

Synchronous vs asynchronous callbacks

Order **A, B, C … D**: `forEach` calls its callback *immediately* (**synchronous**); `setTimeout` only *registers* it to run after the current code finishes (**asynchronous**). Event listeners, timers and network responses use asynchronous callbacks, scheduled by the event loop (Chapter 12, which also covers "callback hell" and promises). The last line shows functions are objects that can carry properties (hence `.name`, `.length`, `.bind`).

Why it matters

- Separates **what** from **how/when**: `filter` loops; you supply the rule.
- **Declarative** (describe *what*, not each step), reusable code — the base of functional programming (small functions returning new values instead of mutating) and every event-driven API.

When not to use

- Long chains of tiny anonymous callbacks hurt readability and debugging — name them.
- In **hot code** (runs very often) a plain loop can be marginally faster than chained `map/filter` — measure first.

TL;DR

Callbacks are passed-in functions; higher-order functions take or return functions — enabling array methods, composition, currying and every event/async API.

4.9 IIFE — Immediately Invoked Function Expressions

What it is

An **IIFE** ("iffy") is a function expression called where it's defined: `(function () { … })();` or `(() => { … })();`. The parentheses make `function` an *expression*; a statement starting with `function` is a declaration and can't be called immediately (`SyntaxError`).

The problem it solves

Before `let` / `const` and modules, a function was the only way to create private scope. Wrapping a script in an IIFE kept variables out of the global scope; an IIFE returning an object of public functions was *the* **module pattern** of the 2000s (jQuery — then the dominant web-page library — plugins, early libraries).

Example

```
// code/09_iife.js
(function () {
  const secret = "only visible in here";
  console.log("IIFE ran:", secret);
})();
console.log("secret outside?", typeof secret);

const config = ((env) => {
  const base = { retries: 3 };
  return env === "prod" ? { ...base, debug: false } : { ...base, debug: true };
})("dev");
console.log("config:", config);

const Cart = (function () {
  const items = [];                          // private
  function total() { return items.reduce((s, i) => s + i.price, 0); }
  return {
    add(name, price) { items.push({ name, price }); return this; },
    total,
    count: () => items.length,
  };
})();
Cart.add("pen", 2).add("book", 12);
console.log("Cart:", Cart.count(), "items, total", Cart.total(), "| items exposed?", "items" in Cart);

(async () => {
  const value = await Promise.resolve("async IIFE done");
  console.log(value);
})();

try {
  eval("function () {}()");
} catch (e) {
  console.log("function statement called directly ->", e.name);
}
```

Output:

```
IIFE ran: only visible in here
secret outside? undefined
config: { retries: 3, debug: true }
Cart: 2 items, total 14 | items exposed? false
function statement called directly -> SyntaxError
async IIFE done
```

(The async IIFE prints last because `await` always resumes asynchronously — Chapter 12.)

Today

Old need	Modern replacement
private scope for a script	ES modules (each file has its own scope) or a plain `{ }` block with `let` / `const`
module pattern	ES modules with `export` (Ch. 10) or classes with `#private`
capture loop variable	`let` in the loop header
`await` at the top level	top-level `await` in modules

IIFEs still appear in bundler output, in snippets that must not leak globals (analytics tags), and as async IIFEs in CommonJS scripts. If a line starts with `(` and the previous lacks a semicolon, ASI (Automatic Semicolon Insertion) won't insert one — hence `;(function () { … })()`.

TL;DR

`(function () { … })()` creates and runs a private scope; modules and block scope have mostly replaced it.

4.10 Recursion

What it is

A **recursive** function calls itself on a smaller version of the problem. It needs a **base case** (answered directly, stops recursion) and a **recursive case** (calls itself on a *smaller* input, moving toward the base case).

Example

```
// code/10_recursion.js (abridged — the file also has totalSize over a folder tree, flatten, and fibLoop)
function factorial(n) {
  if (n <= 1) return 1;          // base case: stops the recursion
  return n * factorial(n - 1);   // recursive case: smaller input
}

function countdown(n, depth = 0) {
  const pad = "  ".repeat(depth);
  if (n === 0) { console.log(pad + "liftoff (base case)"); return; }
  console.log(pad + "countdown(" + n + ")");
  countdown(n - 1, depth + 1);
  console.log(pad + "back in countdown(" + n + ")");
}

function forever(n) { return forever(n + 1); }              // no base case

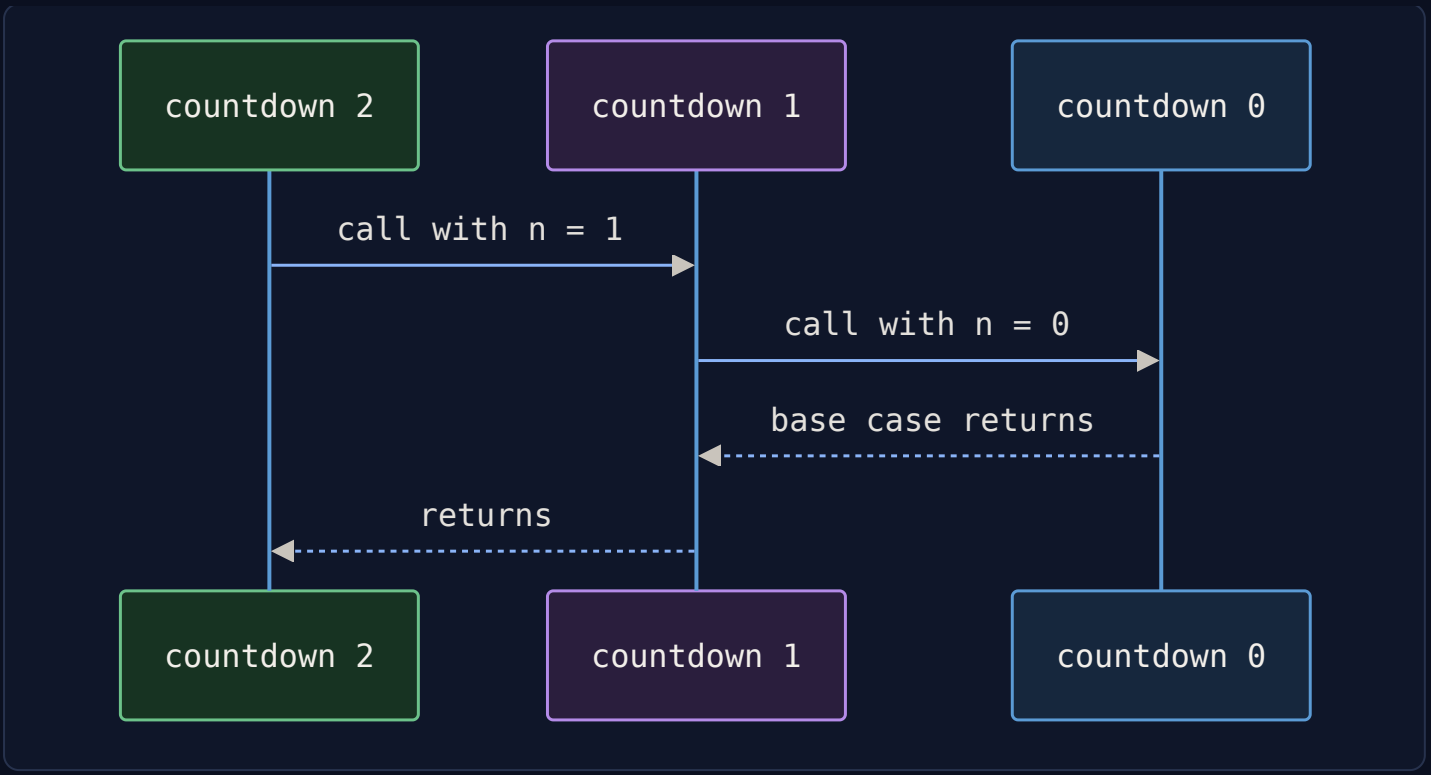
let calls = 0;
function fib(n) { calls++; return n < 2 ? n : fib(n - 1) + fib(n - 2); }
```

Output (full run):

```
factorial(5): 120
countdown(2)
  countdown(1)
    liftoff (base case)
  back in countdown(1)
back in countdown(2)
total size: 15
flatten: [ 1, 2, 3, 4, 5 ]
no base case -> RangeError: Maximum call stack size exceeded
naive fib(25) calls: 242785
fibLoop(25): 75025 | fibLoop(50): 12586269025
```

How it works — the call stack

Each call gets its own execution context on the **call stack**: calls go *down* (each waiting), the base case returns, then waiting calls resume in reverse order.



The stack is limited (roughly ten thousand **frames** — one per active call — depending on engine and frame size). Too deep, or no base case, throws `RangeError: Maximum call stack size exceeded` ("stack overflow"). Engines other than Safari's do **not** implement **tail-call optimisation** (reusing the frame when the last action is a self-call), so tail-call form doesn't make deep recursion safe.

When to use it (and when not)

Good fit	Better as a loop
tree-shaped data: file systems, the DOM, JSON, org charts, comments with replies	simple counting / summing over a flat list
divide-and-conquer algorithms (split the problem into halves, solve each, combine): merge sort, quicksort, binary search	inputs that could nest tens of thousands of calls deep
backtracking (try a choice, undo it if it leads nowhere): permutations, sudoku	naive recursions that repeat work (Fibonacci) unless memoized

Naive `fib(25)` made **242,785** calls by recomputing values — exponential time. Memoization (§4.7) or a loop makes it linear. `fibLoop` stops at 50 on purpose: `fib(79)` and up exceed `Number.MAX_SAFE_INTEGER` and silently lose precision (Chapter 2 §2.10) — use `BigInt`.

TL;DR

Base case + smaller recursive case. Natural for trees and divide-and-conquer; watch stack depth (`RangeError`) and repeated work (memoize).

4.11 Pure functions

What it is

A **pure function** (1) returns the **same output for the same inputs** and (2) has **no side effects** — no mutating arguments or globals, no I/O (input/output: files, keyboard, network), no DOM changes, no logging. Otherwise it's **impure**: `Math.max` is pure; `Math.random`, `Date.now`, `console.log`, `array.push` are not.

Example

```
// code/11_pure_functions.js (abridged)
const addTax = (price, rate) => +(price * (1 + rate)).toFixed(2);   // pure
console.log("pure:", addTax(100, 0.2), addTax(100, 0.2));

let rate = 0.2;
const addTaxImpure = (price) => +(price * (1 + rate)).toFixed(2);   // reads outside state
console.log("impure before:", addTaxImpure(100));
rate = 0.25;
console.log("impure after someone changed rate:", addTaxImpure(100));

function addItemMutating(cart, item) { cart.push(item); return cart; }  // side effect
function addItemPure(cart, item) { return [...cart, item]; }          // returns a new array
const cart = ["apple"];
const newCart = addItemPure(cart, "pear");
console.log("pure add -> original:", cart, "| new:", newCart);
addItemMutating(cart, "plum");
console.log("mutating add -> original changed:", cart);

const assert = require("node:assert");
assert.strictEqual(addTax(10, 0.1), 11);
assert.deepStrictEqual(addItemPure([], "x"), ["x"]);
console.log("tests passed");
```

Output:

```
pure: 120 120
impure before: 120
impure after someone changed rate: 125
pure add -> original: [ 'apple' ] | new: [ 'apple', 'pear' ]
mutating add -> original changed: [ 'apple', 'plum' ]
Date.now() is impure: number
tests passed
```

Why it matters

- **Testable** — no setup or **mocks** (fake stand-ins for dependencies): call and compare.
- **Safe to cache** (memoize) and run in any order or in parallel.
- **Frameworks rely on it**: React components must be pure functions of props and state while rendering; Redux **reducers** (compute next state from current state + action) must be pure. Returning a new array (`[...cart, item]`) lets them detect changes by reference (Chapter 2 §2.4).

Trade-offs

A program with no side effects does nothing visible. Aim for a **pure core, impure shell**: pure calculations, side effects at the edges (event handlers, API calls). Copying instead of mutating costs some memory/time — negligible for most UI-sized data.

TL;DR

Same inputs → same output, no side effects. Keep business logic (app-specific rules like prices and discounts) pure; isolate I/O and mutation.

Chapter 4 TL;DR

- **Functions** are first-class values: missing arguments and missing `return` give `undefined`; objects are passed as shared references.
- **Declarations** are hoisted whole; **expressions** (incl. arrows) follow their variable (`const` → TDZ).
- **Arrows**: concise (`x => x * 2`, `() => ({…})`), lexical `this`, no `arguments`, no `new` — ideal callbacks, wrong as methods.

- **Parameters**: defaults replace only `undefined`, per call, may use earlier params; `...rest` collects; spread expands; options objects give named parameters.
- **Scope**: global, function (`var`), block (`let`/`const`); lookups walk the **lexical** scope chain; inner names shadow outer.
- **Closures**: live access to the variables where a function was defined — private state, factories, `once`, `memoize`, handlers. `let` in loops = one binding per iteration.
- **Higher-order functions & callbacks**: `map/filter/reduce`, composition, currying; callbacks sync or async.
- **IIFE**: `(() => { … })()` — legacy private scope/module pattern, replaced by modules.
- **Recursion**: base case + smaller case; beware `RangeError` and repeated work.
- **Pure functions**: same input → same output, no side effects — easy to test, required by React and Redux.

Next: Chapter 5 — Objects, Math, Date and JSON. For `this`, `call/apply/bind` and classes see Chapter 9; for `try/catch`, Chapter 11.

Chapter 5 — Objects, Math, Date and JSON

Almost everything interesting in JavaScript is an **object**: a user profile, an HTTP response (a web server's reply — System Design Fundamentals), a config file, a DOM element (one piece of a web page as the browser exposes it), even arrays and functions. This chapter covers the object toolkit — creating, reading, changing, copying, merging and locking objects — then three built-ins you'll use constantly: `Math`, `Date` and `JSON`.

What you'll learn: the object toolkit (§5.1–5.8), the built-in objects (§5.9), `Math` (§5.10), `Date` and **Temporal** (§5.11), and JSON (§5.12).

Before you start: variables, types, operators (Chapter 2); functions and arrows (Chapter 4). Arrays: Chapter 7; `Map` / `Set` (often better lookup tables): Chapter 8.

Code is in `code/`. Every file is an ES module (`.mjs`, the `import` / `export` system — Chapter 10), so it runs in **strict mode** (silent failures become errors). Outputs are real Node 24 output.

5.1 What an object is

What it is

An **object** is a collection of **properties**: each a **key** (normally a string) paired with a **value** (any value — number, string, object, function). A property whose value is a function is a **method**. A car has properties (colour, brand) and methods (drive, brake). Unlike a **primitive** (string, number, boolean, `null`, `undefined`, bigint, symbol) — a single unchangeable value — an object groups related values and behaviour under one name.

Why it matters

Real data is structured: a user is a name, email, roles, signup date. Objects keep those together and attach behaviour. JSON (§5.12), the format of nearly every web API (a service other programs call over the internet), is a text version of objects and arrays.

How it works — creating objects with a literal

An **object literal** — curly braces with comma-separated `key: value` pairs — is the most common way:

```
const car = {
  color: 'red',
  brand: 'Toyota',
  model: 'Corolla',
  drive() {                       // a method
    console.log('The car is moving');
  },
};
```

- `{ ... }` creates a new object each time the line runs.
- `drive() { ... }` is **method shorthand** (ES2015) for `drive: function () { ... }`.
- A trailing comma is allowed and keeps diffs (version-control "what changed" views) clean.

Alternatives — `new Object()` (never needed), `Object.create(proto)` (§5.6), constructors and classes (Chapter 9) — but literals cover ~90% of code.

5.2 Reading, adding, updating and deleting properties

Dot notation vs bracket notation

	Dot notation `obj.key`	Bracket notation `obj['key']` / `obj[expr]`
Key must be a valid identifier?	Yes (letters, digits, `_`, `$`; cannot start with a digit; no spaces or dashes)	No — any string (or symbol, a unique key type from Chapter 8) works
Key known while writing the code?	Yes — the name is typed literally	Not necessary — the key can come from a variable or expression
Readability	Cleaner; the default choice	A little noisier
Typical use	`user.name`, `config.port`	`headers['content-type']`, `row[columnName]`, `scores[2024]`

Use dot notation unless you can't — special characters, or a key known only at **run time** (while the program runs).

Computed keys and shorthand properties

- **Computed keys** (ES2015) — `[expression]: value` uses the expression's *result* as the key.
- **Shorthand properties** — `{ name, age }` means `{ name: name, age: age }`.

`code/01_object_basics.mjs` (first part):

```
const field = 'price';

const laptop = {
  brand: 'Lenovo',            // plain property
  'model name': 'ThinkPad',   // key that is not a valid identifier -> must be quoted
  [field]: 1299,              // computed key: the value of `field` becomes the key
  [`${field}WithTax`]: 1299 * 1.2,
  open() {                    // method shorthand
    return `${this.brand} ${this['model name']} is open`;
  },
};

console.log(laptop.brand);                // known, valid identifier
console.log(laptop['model name']);        // spaces -> brackets required
const key = 'price';
console.log(laptop[key], laptop.priceWithTax);
console.log(laptop.color);                // missing property -> undefined, not an error

const name = 'Ada', age = 36;
const user = { name, age };
console.log(user);
```

Output:

```
Lenovo
ThinkPad
1299 1558.8
undefined
{ name: 'Ada', age: 36 }
```

A missing property reads as `undefined`; reading a property *of* `undefined` (`laptop.color.length`) throws `TypeError` — §5.8's `?.` avoids it.

Adding, updating and deleting

Objects are **mutable**: assigning to an existing key updates it, to a new key adds it; `delete obj.key` removes it.

```
laptop.brand = 'Dell';          // update (dot)
laptop['ram'] = '16GB';         // add (bracket)
delete laptop.priceWithTax;     // remove
console.log(laptop.open(), '|', Object.keys(laptop).join(','));
```

Output:

```
Dell ThinkPad is open | brand,model name,price,open,ram
```

Does the object have this key?

```
console.log('ram' in laptop, Object.hasOwn(laptop, 'ram'), laptop.ram !== undefined);
const withUndefined = { x: undefined };
console.log('x' in withUndefined, withUndefined.x !== undefined);
```

Output:

```
true true true
true false
```

Check	Looks at inherited (prototype) keys?	Works when the value is `undefined`?
`'key' in obj`	Yes (so `'toString' in {}` is `true`)	Yes
`Object.hasOwn(obj, 'key')` (ES2022)	No — own properties only	Yes
`obj.hasOwnProperty('key')`	No	Yes, but fails on objects without a prototype; prefer `Object.hasOwn`
`obj.key !== undefined`	Yes	**No** — a key explicitly set to `undefined` looks missing

(*Inherited* keys come from the object's **prototype** — the fallback object checked when a key isn't found on the object itself; §5.6.)

Key order

Order is fully specified since ES2020 for `Object.keys`, `for...in`, `JSON.stringify` etc.:

1. **Integer-like keys** (`'0'`, `'42'`) ascending;
2. other **string keys** in insertion order;
3. **symbol keys** in insertion order (only reported by symbol-aware functions).

```
const ordered = { b: 1, 2: 'two', a: 2, 1: 'one', [Symbol('s')]: 3 };
console.log(Object.keys(ordered));
console.log(Reflect.ownKeys(ordered));
```

Output:

```
[ '1', '2', 'b', 'a' ]
[ '1', '2', 'b', 'a', Symbol(s) ]
```

Deterministic, but not pure insertion order for number-like keys. For insertion order always (or non-string keys) use a `Map` (Chapter 8).

Objects are references

A variable holds a **reference** to the object; copying the variable copies the reference.

```
const a = { n: 1 };
const b = a;           // same object, second name
b.n = 99;
console.log(a.n, a === b, { n: 1 } === { n: 1 });

const cfg = { debug: false };
cfg.debug = true;      // allowed
console.log(cfg);
try { cfg = {}; } catch (e) { console.log(e.constructor.name + ':', e.message); }
```

Output:

```
99 true false
{ debug: true }
TypeError: Assignment to constant variable.
```

- `===` on objects means "same object?", not "same contents?".
- `const` freezes the **binding**, not the object — use `Object.freeze` (§5.6).

Common mistakes

- `obj.field` when the key is in variable `field` — looks for a property literally named "field"; use `obj[field]`.
- Expecting insertion order for number-like keys.
- Believing `const` makes an object immutable.
- `===` for "equal contents" — compare fields, or deliberately serialize both (e.g. to JSON) and compare strings.

Self-check

1. Read a property named `first-name`? → `obj['first-name']`.
2. `const k = 'x'; ({ [k]: 1 })`? → `{ x: 1 }`.
3. `'toString' in {}` vs `Object.hasOwn({}, 'toString')`? → `true` vs `false`.
4. `Object.keys({ z: 1, 10: 1, 2: 1 })`? → `['2', '10', 'z']`.

5.3 Methods and `this`

What it is

Calling `obj.method()` sets `this` inside the method to `obj` (the object left of the dot), so a method can read the object's other properties.

```
const person = {
  firstName: 'Grace',
  lastName: 'Hopper',
  greet() {
    console.log(`Hi, I'm ${this.firstName} ${this.lastName}`);
  },
};
person.greet();   // Hi, I'm Grace Hopper
```

Brackets work too: `person['greet']()`.

Common mistakes

- **Arrow functions as methods** — arrows borrow `this` from surrounding code, so they don't see the object:

```
const counter = {
  count: 0,
  incArrow: () => typeof this,          // `this` is the module's this (undefined)
  inc() { this.count++; return this.count; },
};
console.log(counter.incArrow(), counter.inc());
```

Output (from code/05_getters_setters_optional.mjs):

```
undefined 1
```

- **Detaching a method** — `const f = person.greet; f();` loses `this` (`undefined` in strict mode). Full rules (default, implicit, explicit `call` / `apply` / `bind`, `new`): Chapter 9, section 9.1.

Returning `this` for chaining

Returning `this` enables chained calls — the style of jQuery, query builders (step-by-step database query libraries) and test libraries:

```
deposit(amount) { this._balance += amount; return this; }
// account.deposit(10).deposit(5)
```

5.4 Spread, rest and copying objects

What it is

- **Spread** `{ ...source }` (ES2018) copies `source`'s *own* (not inherited), *enumerable* (visible to loops and listing methods) properties into a new literal.
- **Rest** in destructuring, `const { a, ...others } = obj`, gathers the un-named properties into a new object.

How it works

Properties copy left to right; for duplicate keys the **later one wins** (overrides on top of defaults); only the top level is copied — nested objects are shared (a **shallow copy**).

From `code/02_object_static_methods.mjs`:

```
const defaults = { theme: 'light', lang: 'en', fontSize: 14 };
const prefs = { theme: 'dark' };
const merged = Object.assign({}, defaults, prefs); // later sources win
console.log(merged);

console.log({ ...defaults, ...prefs, fontSize: 16 });

const { password, ...safeUser } = { id: 7, name: 'Lin', password: 'x' };
console.log(safeUser);

const original = { name: 'Kit', address: { city: 'Pune' } };
const shallow = { ...original };
const deep = structuredClone(original);
shallow.address.city = 'Delhi';
console.log(original.address.city, deep.address.city);
```

Output:

```
{ theme: 'dark', lang: 'en', fontSize: 14 }
{ theme: 'dark', lang: 'en', fontSize: 16 }
{ id: 7, name: 'Lin' }
Delhi Pune
```

The last line is the classic bug: `shallow` and `original` share the inner address object. `structuredClone(value)` (global in browsers and Node 17+) makes a **deep copy** — nested objects, arrays,

`Date`, `Map`, `Set`, typed arrays, even circular references (objects pointing back to themselves). It can't copy functions, DOM nodes or class identity (class instances come back as plain objects).

Trade-offs: ways to copy or merge

(*Getter/setter*: a property that runs a function when read/written — §5.7.)

Technique	Depth	Mutates target?	Copies getters as getters?	Notes
`{ ...a, ...b }`	Shallow	No (new object)	No — getters are *called*, values copied	Most common; readable
`Object.assign(target, a, b)`	Shallow	**Yes** — writes into `target`	No — getters called, setters on target triggered	Use `Object.assign({}, ...)` for a fresh object
`structuredClone(obj)`	Deep	No	No	Handles Date/Map/Set/cycles; throws on functions
`JSON.parse(JSON.stringify(obj))`	Deep	No	No	Lossy: Dates → strings, drops `undefined`/functions, fails on cycles and BigInt (§5.12)

Real-world examples

- Redux reducers and React state updates: `return { ...state, loading: false }` — new object, no mutation.
- Layered config: `{ ...defaultConfig, ...fileConfig, ...envConfig }`.
- Stripping sensitive fields: `const { password, ...safe } = user`.

In the interview

> **30-second answer:** "Spread copies own enumerable properties into a new literal; later keys override. It's shallow — nested objects are shared. For deep copies I use `structuredClone` (Dates, Maps, Sets, cycles); the JSON round-trip is lossy."

- *`Object.assign` vs spread?* `assign` mutates its first argument and triggers its setters; spread always makes a new object.
- *Deep-merge two configs?* Recurse: if both values are plain objects merge them, else take the right-hand value — or lodash `merge` (a popular utility library).

Self-check

1. `{ ...{ a: 1 }, a: 2 }` → `{ a: 2 }`.
2. After `const c = { ...o }`, does `c.list.push(1)` change `o.list`? → Yes (shallow).
3. `structuredClone` of a method? → Throws `DataCloneError`.

5.5 Destructuring objects

What it is

Destructuring (ES2015) unpacks properties into variables: `const { a, b } = obj` ≈ `const a = obj.a, b = obj.b`.

How it works

From `code/03_destructuring.mjs`:

```
const order = {
  id: 'A-17',
  customer: { name: 'Mira', address: { city: 'Lyon' } },
  items: 3,
};

const { id, items } = order;                      // basic
const { id: orderId } = order;                    // rename: take `id`, call it orderId
const { status = 'pending' } = order;             // default when the value is undefined
const { customer: { name, address: { city } } } = order; // nested
console.log(id, items, orderId, status, name, city);

const { a = 1, b = 2 } = { a: undefined, b: null };
console.log(a, b);

function createUser({ name, role = 'viewer', active = true } = {}) {
  return `${name ?? 'anonymous'} / ${role} / ${active}`;
}
console.log(createUser({ name: 'Jo', role: 'admin' }));
console.log(createUser());

let x, y;
({ x, y } = { x: 10, y: 20 });
console.log(x, y);

try { const { q } = null; } catch (e) { console.log(e.constructor.name); }
```

Output:

```
A-17 3 A-17 pending Mira Lyon
1 null
Jo / admin / true
anonymous / viewer / true
10 20
TypeError
```

- `{ id: orderId }` — "take `id`, store in `orderId`": the **right** of the colon is the new variable.
- Defaults apply only for `undefined`, **not** `null`.
- In `customer: { name }`, `customer` is only a path — no variable is created.
- **Parameter destructuring** gives "named arguments": order-independent options with defaults; trailing `= {}` makes a no-argument call safe.
- Assigning to *existing* variables needs parentheses (else `{` starts a block).
- Destructuring `null` / `undefined` throws `TypeError`.

Array destructuring (`const [first, second] = arr`): Chapter 7.

Self-check

1. `const { a: { b } } = { a: { b: 5 } }` — variables? → Only `b` (5).
2. `const { n = 10 } = { n: 0 }` → `n` is `0`.
3. Why `function f({ x } = {})`? → So `f()` doesn't throw.

5.6 `Object.*` static methods, descriptors and locking

What it is

`Object` holds **static methods** called as `Object.method(obj)`:

Group	Methods	What they do
Listing	`Object.keys(o)`, `Object.values(o)`, `Object.entries(o)`, `Object.fromEntries(pairs)`	Own enumerable string-keyed names / values / `[key, value]` pairs; `fromEntries` rebuilds an object from pairs
Copying & creating	`Object.assign(target, ...sources)`, `Object.create(proto, descriptors?)`	Copy properties into a target; make a new object with a chosen prototype
Reflection (code inspecting the structure of objects at run time)	`Object.getOwnPropertyNames(o)` (includes non-enumerable string keys), `Object.getOwnPropertySymbols(o)`, `Object.getOwnPropertyDescriptor(o, k)`, `Object.getOwnPropertyDescriptors(o)`, `Object.getPrototypeOf(o)`, `Object.setPrototypeOf(o, p)`, `Object.hasOwn(o, k)`, `Object.is(a, b)`	Inspect keys, flags and the prototype; compare values
Defining & locking	`Object.defineProperty(o, k, desc)`, `Object.defineProperties(o, descs)`, `Object.preventExtensions`, `Object.seal`, `Object.freeze`, and the tests `isExtensible`, `isSealed`, `isFrozen`	Control exactly how a property behaves; stop an object from changing

Listing: keys, values, entries, fromEntries

From `code/02_object_static_methods.mjs`:

```
const stock = { apples: 5, pears: 0, plums: 12 };

console.log(Object.keys(stock));
console.log(Object.values(stock));
console.log(Object.entries(stock));

const inStock = Object.fromEntries(
  Object.entries(stock).filter(([, qty]) => qty > 0).map(([k, qty]) => [k.toUpperCase(), qty]),
);
console.log(inStock);

for (const [fruit, qty] of Object.entries(stock)) console.log(`${fruit.padEnd(7)}${qty}`);
```

Output:

```
[ 'apples', 'pears', 'plums' ]
[ 5, 0, 12 ]
[ [ 'apples', 5 ], [ 'pears', 0 ], [ 'plums', 12 ] ]
{ APPLES: 5, PLUMS: 12 }
apples 5
pears  0
plums  12
```

entries → array methods → `fromEntries` is the idiomatic way to map/filter an object (objects have no `.map`). The listing methods return only **own, enumerable, string-keyed** properties in §5.2's order — unlike `for...in`, which also walks inherited keys (Chapter 3).

`Object.create` and `Object.is`

```
const animal = { speak() { return `${this.name} makes a sound`; } };
const dog = Object.create(animal);
dog.name = 'Rex';
console.log(dog.speak(), Object.getPrototypeOf(dog) === animal, Object.keys(dog));
const dict = Object.create(null);    // no prototype at all: a "pure" dictionary
console.log('toString' in dict, 'toString' in {});

console.log(NaN === NaN, Object.is(NaN, NaN), 0 === -0, Object.is(0, -0));
```

Output:

```
Rex makes a sound true [ 'name' ]
false true
false true true false
```

- `Object.create(animal)` makes an empty object whose **prototype** is `animal`; `speak` is found there (Chapter 9).
- `Object.create(null)` has *no* prototype — no inherited `toString`, `constructor`, `__proto__` — a safe dictionary for user-supplied keys.
- `Object.is` = `===` except `NaN` equals itself and `+0` / `-0` differ. `Map` / `Set` use nearly the same rule (but treat `+0` and `-0` as equal).

Property descriptors

Every property has hidden **attributes**, collected in a **property descriptor**:

Attribute	Meaning	Default for normal assignment	Default via defineProperty
`value`	The stored value	—	`undefined`
`writable`	Can the value be changed?	`true`	`false`
`enumerable`	Shows up in `Object.keys`, `for...in`, spread, `JSON.stringify`?	`true`	`false`
`configurable`	Can the property be deleted or its attributes changed?	`true`	`false`
`get` / `set`	Accessor functions (instead of `value` / `writable`) — §5.7	—	`undefined`

From `code/04_descriptors_freeze.mjs`:

```
const tryIt = (label, fn) => {
  try { fn(); console.log(label, '-> ok'); } catch (e) { console.log(label, '->', e.constructor.name); }
};

const product = { name: 'Pen' };
console.log(Object.getOwnPropertyDescriptor(product, 'name'));

Object.defineProperty(product, 'sku', { value: 'P-001' });
console.log(Object.getOwnPropertyDescriptor(product, 'sku'));
console.log(Object.keys(product), Object.getOwnPropertyNames(product), product.sku);
tryIt('write read-only sku', () => { product.sku = 'X'; });
tryIt('delete non-configurable sku', () => { delete product.sku; });

Object.defineProperties(product, {
  price: { value: 2.5, writable: true, enumerable: true, configurable: true },
});
console.log(JSON.stringify(product)); // non-enumerable sku is skipped

const secret = Symbol('secret');
product[secret] = 42;
console.log(Object.keys(product), Object.getOwnPropertySymbols(product));
```

Output:

```
{ value: 'Pen', writable: true, enumerable: true, configurable: true }
{
  value: 'P-001',
  writable: false,
  enumerable: false,
  configurable: false
}
[ 'name' ] [ 'name', 'sku' ] P-001
write read-only sku -> TypeError
delete non-configurable sku -> TypeError
{"name":"Pen","price":2.5}
[ 'name', 'price' ] [ Symbol(secret) ]
```

Plain assignment sets every flag `true`; `defineProperty` sets every omitted flag `false`. Writing a read-only property throws in **strict mode** but is silently ignored in **sloppy mode** ("why didn't my change stick?").

Locking an object: preventExtensions, seal, freeze

```
const ext = { a: 1 };  Object.preventExtensions(ext);
const sealed = { a: 1 }; Object.seal(sealed);
const frozen = { a: 1, inner: { b: 1 } }; Object.freeze(frozen);
tryIt('seal: delete             ', () => { delete sealed.a; });
tryIt('freeze: nested change    ', () => { frozen.inner.b = 2; }); // freeze is shallow!
// ... the file tries add / change / delete on each of the three objects
console.log(Object.isExtensible(ext), Object.isSealed(sealed), Object.isFrozen(frozen),
Object.isFrozen(frozen.inner));
```

Output (full run — one `tryIt` per operation):

```
preventExtensions: add    -> TypeError
preventExtensions: change -> ok
preventExtensions: delete -> ok
seal: add                 -> TypeError
seal: change              -> ok
seal: delete              -> TypeError
freeze: change            -> TypeError
freeze: nested change     -> ok
false true true false
```

Level	Add new keys	Delete keys	Change values	Implemented as
`Object.preventExtensions`	✗	✓	✓	object marked non-extensible
`Object.seal`	✗	✗	✓	+ every property `configurable: false`
`Object.freeze`	✗	✗	✗	+ every data property `writable: false`

All three are **shallow**. A recursive helper freezes everything:

```
function deepFreeze(obj) {
  for (const value of Object.values(obj)) {
    if (value && typeof value === 'object') deepFreeze(value);
  }
  return Object.freeze(obj);
}
const config = deepFreeze({ db: { host: 'localhost', port: 5432 } });
tryIt('deepFreeze: nested change', () => { config.db.port = 1; });   // -> TypeError
```

`Object.setPrototypeOf(obj, proto)` works, but engines optimise by object **shape** (internal record of which properties exist, in which order), and swapping prototypes later can slow code — use `Object.create` up front.

When not to use it

- Don't `freeze` large **hot-path** data (speed-critical, run very often) "just in case"; freeze exported constants and config, or use TypeScript's `readonly` for compile-time safety (checked before running — Chapter 16).
- Don't `defineProperty` ordinary data; use it for library-style hidden (non-enumerable) helpers, read-only constants, lazy getters (computed on first access).

Common mistakes

- Assuming `Object.freeze` is deep.
- Omitted `defineProperty` flags default to `false` — the property silently becomes hidden and read-only.
- Expecting `Object.keys` to include symbol or non-enumerable keys — use `Reflect.ownKeys` for "everything own".

In the interview

> **30-second answer:** "Every property has a descriptor: value, writable, enumerable, configurable — or get/set for accessors. `preventExtensions` stops additions, `seal` also deletions, `freeze` also writes. All shallow; violations throw only in strict mode."

- *Truly immutable object?* Deep-freeze, or immutable data structures (every update yields a new version — e.g. Immer, persistent collections). `const` does nothing for contents.
- *`Object.keys` vs `Object.getOwnPropertyNames`?* The latter includes non-enumerable string keys (e.g. an array's `length`).
- *`Object.is` vs `===`?* `NaN` / `NaN` (true) and `0` / `-0` (false).

Self-check

1. After `Object.seal(o)`, can you assign an existing `o.x`? → Yes.
2. `Object.defineProperty(o, 'k', { value: 1 })` — does `Object.keys(o)` show `k`? → No (non-enumerable).
3. Which lists symbol keys? → `Object.getOwnPropertySymbols` (or `Reflect.ownKeys`).
4. `Object.isFrozen({})`? → No; `Object.isFrozen(Object.freeze({}))` is.

5.7 Getters and setters

What it is

An **accessor property** looks like a normal property but runs a function when read (**getter**, `get`) or written (**setter**, `set`) — computing on demand or validating assignments without changing how callers use the object.

How it works

From `code/05_getters_setters_optional.mjs`:

```
const account = {
  owner: 'Sam',
  _balance: 100,                    // underscore = "please treat as internal" (convention only)
  get balance() { return `$${this._balance.toFixed(2)}`; },
  set balance(v) {
    if (typeof v !== 'number' || v < 0) throw new RangeError('balance must be a non-negative number');
    this._balance = v;
  },
  deposit(amount) { this._balance += amount; return this; }, // returning this enables chaining
};

console.log(account.balance);           // getter runs; no parentheses
account.balance = 250;                  // setter runs
console.log(account.deposit(10).deposit(5).balance);
try { account.balance = -1; } catch (e) { console.log(e.name + ':', e.message); }
console.log(Object.getOwnPropertyDescriptor(account, 'balance'));

const rect = { w: 3, h: 4, get area() { return this.w * this.h; } };
rect.w = 10;
console.log(rect.area);
```

Output:

```
$100.00
$265.00
RangeError: balance must be a non-negative number
{
  get: [Function: get balance],
  set: [Function: set balance],
  enumerable: true,
  configurable: true
}
40
```

- `account.balance` (no parentheses) calls the getter; assignment calls the setter, which validates and stores into the **backing** property `_balance`.
- The descriptor has `get` / `set` instead of `value` / `writable`.
- `rect.area` is **derived** — always current, never stored.
- Getter and backing property **must have different names** — `this.balance = v` inside the setter would recurse forever. The underscore is only convention; true privacy uses closures (Chapter 4) or `#private` fields (Chapter 9).

When not to use it

Readers expect property reads to be instant and side-effect free — slow work (network, heavy computation) belongs in a method like `loadBalance()`. A getter without a setter makes the property read-only (writes throw in strict mode).

Real-world examples

Vue 2's **reactivity** (screen updates automatically when data changes) was built on `defineProperty` accessors; MobX and many ORMs (Object-Relational Mappers, mapping database rows to objects) use them for computed fields; DOM properties like `innerHTML` and `input.value` are accessors on prototypes.

5.8 Optional chaining `?.`

The problem it solves

API data is often incomplete: `res.data.user.name` throws `TypeError: Cannot read properties of undefined` if any link is missing. Pre-ES2020: `res && res.data && res.data.user && res.data.user.name`.

How it works

`a?.b`: if `a` is `null` / `undefined`, stop and return `undefined`; else read `a.b`. Forms: `obj?.prop`, `obj?.[expr]`, `fn?.(args)`; the rest of the chain is skipped once it short-circuits.

```
const res = { data: { user: { name: 'Ana', tags: ['a'] } } };
const empty = {};
console.log(res.data?.user?.name, empty.data?.user?.name);
console.log(res.data?.user?.tags?.[0], empty.data?.user?.tags?.[0]);   // with brackets
console.log(res.logout?.(), typeof empty.save?.());                    // calling a maybe-missing method
console.log(empty.data?.user?.name ?? 'Guest');                        // pair with ?? for a default
```

Output:

```
Ana undefined
a undefined
undefined undefined
Guest
```

Pair with `??` (Chapter 2) for a default only on `null` / `undefined` (`||` also replaces `0`, `''`, `false`).

Common mistakes

- `?.` everywhere — if a value *must* exist, fail loudly instead of hiding a bug.
- `obj?.x = 1` is a `SyntaxError`.
- `?.` guards only `null` / `undefined`: `(5)?.toFixed` works, `''?.length` is `0`.

5.9 JavaScript's built-in (native) objects

Built-in objects exist in every environment (browser, Node, Deno, Bun) without import. Some are **constructors** (called with `new` or as converters), some **namespaces** (plain objects of static functions):

Built-in	Kind	Used for	Where in this book
`Object`	constructor + static helpers	Base of all objects; `Object.keys`, etc.	this chapter
`String`, `Number`, `Boolean`, `BigInt`, `Symbol`	wrappers / converters	Methods on primitives; conversions	Ch 2, Ch 6
`Array`	constructor	Ordered lists	Ch 7
`Function`	constructor	Every function is an object of this type	Ch 4
`Date`	constructor	Points in time	5.11
`RegExp`	constructor	Pattern matching on text	Ch 6
`Map`, `Set`, `WeakMap`, `WeakSet`, `WeakRef`	constructors	Keyed collections, unique values	Ch 8
`Promise`	constructor	Results that arrive later	Ch 12
`Error` (+ `TypeError`, `RangeError`, …)	constructors	Exceptions (error objects thrown to stop normal flow)	Ch 11
`Math`	namespace	Mathematical constants and functions	5.10
`JSON`	namespace	Converting to/from JSON text	5.12
`Intl`	namespace	Locale-aware (language + region, e.g. `en-US`) formatting (dates, numbers, lists)	5.11, Ch 6
`Reflect`, `Proxy`	namespace / constructor	Meta-programming: `Reflect` exposes object operations as functions (`Reflect.ownKeys`, 5.2); a `Proxy` wraps an object and intercepts reads and writes (how Vue 3 reactivity and Immer work)	overview only
`ArrayBuffer`, typed arrays	constructors	Raw binary data	Ch 7
`globalThis`	the global object	Access to globals in any environment	Ch 1

`document`, `window`, `fetch`, Node's `process` are **not** language built-ins — they come from the host (browser or Node; Chapters 13–15).

5.10 `Math`

What it is

`Math` is a **namespace object** of constants and functions — not a constructor (`new Math()` throws `TypeError`); call `Math.floor(x)` directly. All members are **static**.

Constants

Property	Value (approx.)	Meaning
`Math.PI`	3.14159	Circle circumference ÷ diameter
`Math.E`	2.71828	Euler's number, base of natural logarithms
`Math.SQRT2` / `Math.SQRT1_2`	1.41421 / 0.70711	√2 and √½
`Math.LN2` / `Math.LN10`	0.69315 / 2.30259	Natural log of 2 and of 10
`Math.LOG2E` / `Math.LOG10E`	1.44270 / 0.43429	$\log_2(e)$ and $\log_{10}(e)$

Methods by family

Family	Methods	Notes
Rounding	`round`, `floor`, `ceil`, `trunc`, `fround`	See the table below for negatives
Sign & size	`abs`, `sign`	`sign` returns -1, 0, 1 (or -0/NaN)
Extremes	`max(...xs)`, `min(...xs)`	Take separate arguments — spread arrays in
Powers & roots	`pow(x, y)` (same as `x ** y`), `sqrt`, `cbrt`, `hypot(...xs)` (√ of sum of squares)	
Exp & logs	`exp`, `expm1`, `log` (natural), `log10`, `log2`, `log1p`	`log(0)` is `-Infinity`; negative → `NaN`
Trigonometry (radians — an angle unit where a full circle is 2π)	`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2(y, x)`, and hyperbolic `sinh`, `cosh`, `tanh`, …	`atan2` gives the angle of point (x, y), full −π…π range
Random	`random()`	Float in **[0, 1)** — 0 possible, 1 never
Integer tricks	`clz32`, `imul`	Low-level 32-bit helpers

Newer: `Math.f16round` (ES2025, rounds to half-precision float; in Node 24) and `Math.sumPrecise` (ES2026, sums without accumulated rounding error — not in Node 24; check support).

How it works — examples

From `code/06_math.mjs`:

```
const r = 5;
console.log('area', Math.PI * r ** 2, Math.PI * Math.pow(r, 2));

const rows = [3.56, 3.5, -3.5, -3.56, 2.4].map((x) => ({
  x, round: Math.round(x), floor: Math.floor(x), ceil: Math.ceil(x), trunc: Math.trunc(x),
}));
console.table(rows);

console.log(Math.round(1.2345 * 100) / 100, (1.005).toFixed(2), (1.2345).toFixed(2));

const temps = [18, 25, 21, 30, 17];
console.log(Math.max(10, 20, 30, 40, 50), Math.min(...temps), Math.max(...temps));
console.log(Math.max(), Math.min(), Math.max(1, 'x'));

const clamp = (v, lo, hi) => Math.min(Math.max(v, lo), hi);
console.log(clamp(120, 0, 100), clamp(-5, 0, 100), clamp(42, 0, 100));
```

Output:

```
area 78.53981633974483 78.53981633974483
┌─────────┬─────────┬─────────┬─────────┬─────────┬─────────┐
│ (index) │ x       │ round   │ floor   │ ceil    │ trunc   │
├─────────┼─────────┼─────────┼─────────┼─────────┼─────────┤
│ 0       │ 3.56    │ 4       │ 3       │ 4       │ 3       │
│ 1       │ 3.5     │ 4       │ 3       │ 4       │ 3       │
│ 2       │ -3.5    │ -3      │ -4      │ -3      │ -3      │
│ 3       │ -3.56   │ -4      │ -4      │ -3      │ -3      │
│ 4       │ 2.4     │ 2       │ 2       │ 3       │ 2       │
└─────────┴─────────┴─────────┴─────────┴─────────┴─────────┘
1.23 1.00 1.23
50 17 30
-Infinity Infinity NaN
100 0 42
```

- `round` **rounds halves toward +∞**: `Math.round(-3.5)` is `-3` (not "half away from zero" as in many languages).
- `floor` **vs** `trunc` differ only for negatives: −3.5 → −4 vs −3.
- `toFixed` **returns a string** and inherits binary-float error: 1.005 is stored as 1.00499999…, so `(1.005).toFixed(2)` is `"1.00"`. For money use integer cents, display with `Intl.NumberFormat` (Chapter 2).
- `Math.max()` with no arguments is `-Infinity` (the identity for max); any argument converting to `NaN` (`'x'`) gives `NaN` (numeric strings like `'5'` work). Spreading a *huge* array can exceed the engine's argument limit — use `reduce` for hundreds of thousands of items.

```
console.log(Math.abs(-7), Math.sign(-7), Math.sqrt(81), Math.cbrt(27), Math.hypot(3, 4));
console.log(Math.pow(2, 10), 2 ** 10, Math.exp(1), Math.log(Math.E), Math.log10(1000), Math.log2(8));

const toRad = (deg) => (deg * Math.PI) / 180;
console.log(Math.sin(toRad(30)).toFixed(4), Math.cos(toRad(60)).toFixed(4), Math.sin(Math.PI));
console.log('angle of (1,1):', (Math.atan2(1, 1) * 180) / Math.PI, 'deg');
```

Output:

```
7 -1 9 3 5
1024 1024 2.718281828459045 1 3 3
0.5000 0.5000 1.2246467991473532e-16
angle of (1,1): 45 deg
```

Trig uses **radians** (2π = 360°); `Math.sin(Math.PI)` isn't exactly 0 because π can't be stored exactly.

Random integers

`Math.random()` gives `0 ≤ x < 1`. For an integer in `[min, max]` **inclusive**: multiply by the count `max - min + 1` (float in `[0, count)`), `Math.floor` (integer `0 … count-1`), add `min` (integer `min … max`):

```
const randInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;
```

The demo draws 30,000 values from `randInt(1, 3)` ; all land in `{1, 2, 3}` roughly evenly. Using `Math.round` instead of `floor` is a classic bug: the end values get half the probability.

`Math.random` **is not cryptographically secure** (its output can be predicted by attackers). For tokens, IDs, passwords use the Web Crypto API: `crypto.randomUUID()` (a **UUID** is a random 128-bit, practically unique identifier) or `crypto.getRandomValues(new Uint32Array(1))` (global in browsers and Node).

In the interview

- *`0.1 + 0.2 !== 0.3` ; round to two decimals?* IEEE-754 binary floats can't store 0.1 exactly. Compare with a tolerance (`Math.abs(a - b) < Number.EPSILON`), round with `Math.round(x * 100) / 100` , or use cents.
- *Random integer in [a, b]?* `Math.floor(Math.random() * (b - a + 1)) + a` .
- *Max of an array?* `Math.max(...arr)` ; `arr.reduce((m, x) => x > m ? x : m)` for huge arrays.

Self-check

1. `Math.round(-2.5)` ? → `-2`.
2. `Math.trunc(-4.7)` vs `Math.floor(-4.7)` ? → `-4` vs `-5`.
3. Can `Math.random()` return 1? → No.
4. 180° in radians? → `Math.PI`.

5.11 `Date`

What it is

A `Date` is **one instant in time**, stored as milliseconds since the **Unix epoch**, 1 January 1970 00:00:00 **UTC** (Coordinated Universal Time). Year, month, hour, weekday are *computed* from that number in **local time** (the running machine's zone) or UTC. A `Date` has no time zone of its own — the zone appears only when you read or print it.

Why it matters

Calendars, bookings, logs, subscriptions, "3 hours ago" all need time. `Date` is infamous for pitfalls — 0-based months, mutability, parsing quirks, DST (daylight-saving time, clocks jumping an hour) — hence the new **Temporal** API (end of section).

Creating dates

Form	Creates	Example
`new Date()`	The current instant	`new Date()`
`new Date(ms)`	Instant `ms` milliseconds after the epoch (negative = before 1970)	`new Date(0)` → 1970-01-01T00:00:00Z
`new Date(string)`	Parsed from text; reliable only for **ISO 8601** (`YYYY-MM-DDTHH:mm:ss.sssZ`)	`new Date('2024-03-10T14:30:00Z')`
`new Date(y, m, d?, h?, min?, s?, ms?)`	Built from **local-time** parts; **month is 0-based** (0 = January, 11 = December); omitted day = 1, omitted time parts = 0	`new Date(2024, 0, 31, 9, 15)`
`Date.now()`	Not a Date — just the current ms number	handy for timing
`Date.UTC(y, m, …)`	ms number for those parts **in UTC**	`new Date(Date.UTC(2024, 0, 1))`

`Date()` *without* `new` returns a string — always use `new`.

From `code/07_date.mjs` (it pins `process.env.TZ = 'America/New_York'` — `process.env` holds environment variables — for repeatable output):

```
const now = new Date();                                  // current moment
console.log('now is a Date:', now instanceof Date, typeof now.getTime());
const epoch = new Date(0);                               // ms since 1970-01-01T00:00:00Z
const fromMs = new Date(1_700_000_000_000);
const fromIso = new Date('2024-03-10T14:30:00Z');        // ISO 8601 string (Z = UTC)
const fromParts = new Date(2024, 0, 31, 9, 15);          // local time; month is 0-based (0 = Jan)
console.log(epoch.toISOString(), fromMs.toISOString());
console.log(fromIso.toISOString(), fromParts.toString());
```

Output:

```
now is a Date: true number
1970-01-01T00:00:00.000Z 2023-11-14T22:13:20.000Z
2024-03-10T14:30:00.000Z Wed Jan 31 2024 09:15:00 GMT-0500 (Eastern Standard Time)
```

Parsing pitfalls

```
console.log(new Date('2023-01-01').toString());
console.log(new Date('2023-01-01T00:00').toString());
console.log(new Date('not a date').getTime(), String(new Date('nope')));
```

Output:

```
Sat Dec 31 2022 19:00:00 GMT-0500 (Eastern Standard Time)
Sun Jan 01 2023 00:00:00 GMT-0500 (Eastern Standard Time)
NaN Invalid Date
```

- A **date-only** ISO string is **UTC midnight** (still 31 December in New York); a date-*time* string without `Z`/offset is **local** — a historical inconsistency in the spec.
- Non-ISO strings (`'March 25, 2015'`, `'03/25/2015'`) are parsed implementation-specifically (each engine may differ) — prefer ISO or numeric parts.
- Unparseable input gives an **Invalid Date** (time value `NaN`): test `Number.isNaN(d.getTime())`.

Reading components — getters

Local getter	UTC twin	Returns
`getFullYear()`	`getUTCFullYear()`	4-digit year (never use the obsolete `getYear`)
`getMonth()`	`getUTCMonth()`	**0–11**
`getDate()`	`getUTCDate()`	Day of month **1–31**
`getDay()`	`getUTCDay()`	Day of week **0–6**, 0 = Sunday
`getHours()` / `getMinutes()` / `getSeconds()` / `getMilliseconds()`	`getUTC…`	0–23 / 0–59 / 0–59 / 0–999
`getTime()` / `valueOf()`	—	ms since the epoch (same instant in any zone)
`getTimezoneOffset()`	—	Minutes to add to local time to get UTC (positive west of Greenwich, e.g. 240 for New York in summer)

```
const d = new Date('2024-07-04T23:30:00Z');
console.log('local:', d.getFullYear(), d.getMonth(), d.getDate(), d.getDay(), d.getHours(), d.getMinutes());
console.log('UTC  :', d.getUTCFullYear(), d.getUTCMonth(), d.getUTCDate(), d.getUTCDay(), d.getUTCHours());
console.log('offset minutes (UTC - local):', d.getTimezoneOffset());
```

Output:

```
local: 2024 6 4 4 19 30
UTC  : 2024 6 4 4 23
offset minutes (UTC - local): 240
```

Month `6` = **July**, weekday `4` = **Thursday**; the instant is 19:30 in New York, 23:30 UTC.

Changing components — setters

Setters (`setFullYear`, `setMonth`, `setDate`, `setHours`, `setMinutes`, `setSeconds`, `setMilliseconds`, `setTime`, plus `setUTC…`) **mutate** in place. Some take several parts (`setFullYear(year, month?, day?)`, `setHours(h, m?, s?, ms?)`); out-of-range values **roll over** (Jan 32 → Feb 1; day 0 → last day of previous month).

```
const b = new Date(2024, 5, 15);
b.setFullYear(1990, 0, 1);                  // year, month, day in one call
console.log(b.toDateString());
const roll = new Date(2024, 0, 31);
roll.setMonth(1);                           // Feb 31 does not exist -> Mar 2 (2024 is a leap year)
console.log(roll.toDateString());
const addDays = (date, n) => { const c = new Date(date); c.setDate(c.getDate() + n); return c; };
console.log(addDays(new Date(2024, 11, 30), 5).toDateString());
const lastDayOfFeb = new Date(2024, 2, 0).getDate();   // day 0 of March = last day of Feb
console.log('days in Feb 2024:', lastDayOfFeb);
```

Output:

```
Mon Jan 01 1990
Sat Mar 02 2024
Sat Jan 04 2025
days in Feb 2024: 29
```

`addDays` copies first so the caller's Date isn't mutated, and rollover crosses the year. "Month-end overflow" (`Jan 31 + 1 month = Mar 2`) is a real billing bug — decide explicitly if you want the month's last day.

Differences and comparisons

Subtracting Dates converts both to ms (via `valueOf`).

```
const start = new Date('2023-01-01');
const end = new Date('2023-12-31');
const MS_PER_DAY = 1000 * 60 * 60 * 24;
console.log((end - start) / MS_PER_DAY + ' days');
const beforeDst = new Date(2024, 2, 9), afterDst = new Date(2024, 2, 11);
console.log((afterDst - beforeDst) / MS_PER_DAY, Math.round((afterDst - beforeDst) / MS_PER_DAY));

const x = new Date(2024, 0, 1), y = new Date(2024, 0, 1);
console.log(x === y, x.getTime() === y.getTime(), x < addDays(y, 1));
```

Output:

```
364 days
1.9583333333333333 2
false true true
```

- 1 Jan → 31 Dec is 364 days (add 1 to count both ends).
- Across the March DST switch two local midnights are 47 hours apart (1.958 "days") — round, or compute in UTC.
- `<` / `>` compare the numbers; `===` compares identity — compare `getTime()` for equality.

Formatting

Method	Output (for the instant 2024-03-10 14:30 UTC, local zone New York)	Use for
`toISOString()`	`2024-03-10T14:30:00.000Z`	Storage, APIs, logs — always UTC, sortable
`toJSON()`	same as ISO	Used automatically by `JSON.stringify`
`toString()`	`Sun Mar 10 2024 10:30:00 GMT-0400 (Eastern Daylight Time)`	Debugging
`toDateString()` / `toTimeString()`	`Sun Mar 10 2024` / `10:30:00 GMT-0400 (…)`	Debugging
`toUTCString()`	`Sun, 10 Mar 2024 14:30:00 GMT`	HTTP headers (name/value lines sent with a request or response)
`toLocaleDateString(locale, opts)` / `toLocaleTimeString` / `toLocaleString`	`3/10/2024` (en-US), `10/03/2024` (en-GB), `10.3.2024` (de-DE)	Showing dates to people
`Intl.DateTimeFormat(locale, opts).format(d)`	reusable formatter	Formatting many dates
`Intl.RelativeTimeFormat`	`yesterday`, `in 3 weeks`	"time ago" labels

```
console.log(t.toLocaleString('en-US', { timeZone: 'Asia/Kolkata', dateStyle: 'full', timeStyle: 'short' }));
const fmt = new Intl.DateTimeFormat('en', { month: 'short', day: 'numeric', year: 'numeric', timeZone: 'UTC' });
console.log(fmt.format(t));
const rtf = new Intl.RelativeTimeFormat('en', { numeric: 'auto' });
console.log(rtf.format(-1, 'day'), '|', rtf.format(3, 'week'));
```

Output:

```
Sunday, March 10, 2024 at 8:00 PM
Mar 10, 2024
yesterday | in 3 weeks
```

`timeZone` *displays* an instant in any IANA zone (official time-zone database names: `'Asia/Kolkata'`, `'Europe/Berlin'`), but `Date` can't *do arithmetic* in a chosen zone — the gap Temporal fills.

Temporal — the modern replacement (status in 2026)

Temporal is a new global namespace of immutable date/time types. Verified status (September 2026):

- Reached **Stage 4** at the TC39 meeting of **March 2026**; TC39's finished-proposals list places it in **ES2027** (some articles say "ES2026"; that snapshot was already cut).
- Shipped by default in **Firefox 139+**, **Chrome/Edge 144+** and **Node.js 26+**. **Safari** stable hasn't shipped it yet, and **Node 24** (this book's runtime) lacks it — our script prints `Temporal available here: false`. Use `@js-temporal/polyfill` or `temporal-polyfill` (a **polyfill** adds a missing built-in to older environments).

Type	Represents	Example
`Temporal.Instant`	An exact moment (like a Date's number, in nanoseconds)	a log timestamp
`Temporal.ZonedDateTime`	Instant + time zone + calendar; DST-aware arithmetic	a meeting in Berlin
`Temporal.PlainDate` / `PlainTime` / `PlainDateTime`	Wall-clock values with **no** time zone	a birthday; a store's opening time
`Temporal.PlainYearMonth` / `PlainMonthDay`	Partial dates	a credit-card expiry; an anniversary
`Temporal.Duration`	A length of time	"2 hours 30 minutes"
`Temporal.Now`	Current time in various forms	`Temporal.Now.plainDateISO()`

A taste (Node 26 / Chrome 144 / Firefox 139 — not run on Node 24):

```
const due = Temporal.PlainDate.from('2024-01-31').add({ months: 1 });
due.toString();                                  // '2024-02-29' — clamps to month end, months are 1-based
const meeting = Temporal.ZonedDateTime.from('2026-03-28T09:00[Europe/Berlin]');
meeting.add({ days: 1 }).toString();             // DST-aware: still 09:00 local the next day
```

Better because: **immutable** values, **1-based** months, strict ISO parsing, first-class time zones and calendars, and separate types for "instant" vs "wall-calendar date". Until it's everywhere you target, use **date-fns**, **Luxon** or **Day.js**.

Common mistakes

- 0-based months (`new Date(2024, 1, 1)` is **February** 1).
- `getDate()` (day of month) vs `getDay()` (weekday).
- `'2023-01-01'` showing 31 December in the Americas.
- Mutating a shared Date — copy with `new Date(d)` first.
- Storing local-time strings — store UTC (ISO or epoch ms), convert for display.
- "Days between" by raw division across DST.

Real-world examples

- APIs send ISO 8601 (`"2024-05-01T08:00:00Z"`); clients parse with `new Date(str)` and display with `Intl.DateTimeFormat` .
- Timing uses `Date.now()` or better `performance.now()` — **monotonic** (never goes backward, even if the clock changes) and sub-millisecond.
- JWTs (JSON Web Tokens — signed login tokens) store expiry in **seconds** since the epoch — multiply by 1000 for `new Date` .

In the interview

30-second answer: "A Date wraps one number — ms since 1970 UTC; getters compute local or UTC parts. Months are 0-based, setters mutate and roll over, date-only ISO strings parse as UTC. I display with `Intl.DateTimeFormat` ; for serious date math I use Temporal (standard, shipping in Chrome, Firefox and Node 26) or a library."

- *Days in a month?* `new Date(year, month + 1, 0).getDate()` .
- *Valid Date?* `!Number.isNaN(d.getTime())` .
- *Why does `new Date('2024-01-01')` show the wrong day?* Parsed as UTC midnight, shown in a zone west of UTC.

Self-check

1. `new Date(2025, 11, 25).getMonth()` ? → `11` .

2. `getDay()` for Sunday? → `0`.
3. `new Date(2024, 0, 32)`? → 1 February 2024.
4. ms in a day? → 86,400,000 (ignoring DST days).
5. Store which format? → UTC — ISO 8601 or epoch ms.

5.12 JSON

What it is

JSON (JavaScript Object Notation) is a *text* format for structured data, standardised as ECMA-404 / RFC 8259 (an RFC is a numbered internet standards document). It borrows JS literal syntax but is stricter, and every language and database speaks it.

- `JSON.stringify(value, replacer?, space?)` → JSON **string** (serialisation).
- `JSON.parse(text, reviver?)` → values (deserialisation).

JSON syntax rules (vs JS literals)

Allowed in JSON	Not allowed in JSON (but fine in JS)
Objects `{}` with **double-quoted** keys	Unquoted or single-quoted keys: `{a: 1}`, `{'a': 1}`
Strings in **double quotes**	Single quotes, template literals
Numbers (decimal), `true`, `false`, `null`	`undefined`, `NaN`, `Infinity`, hex `0xFF`, BigInt
Arrays `[]`	Trailing commas `[1, 2,]`
Nesting to any depth	Comments, functions, Dates, Maps, Sets, symbols

How `stringify` works

From `code/08_json.mjs`:

```
const order = {
  id: 42, customer: 'Ravi', total: 19.99, paid: true, coupon: null,
  items: [{ sku: 'A1', qty: 2 }],
  placedAt: new Date('2024-05-01T08:00:00Z'),
};

const text = JSON.stringify(order);
console.log(text);
console.log(typeof text, text.length);
console.log(JSON.stringify({ a: 1, b: [1, 2] }, null, 2));
console.log(JSON.stringify(order, ['id', 'total']));
console.log(JSON.stringify(order, (key, value) => (key === 'customer' ? undefined : value)).slice(0, 60) + '...');
```

Output:

```
{"id":42,"customer":"Ravi","total":19.99,"paid":true,"coupon":null,"items":
[{"sku":"A1","qty":2}],"placedAt":"2024-05-01T08:00:00.000Z"}
string 136
{
  "a": 1,
  "b": [
    1,
    2
  ]
}
{"id":42,"total":19.99}
{"id":42,"total":19.99,"paid":true,"coupon":null,"items":[{"...
```

1. The result is a **string**; the Date became ISO via `Date.prototype.toJSON`.
2. `space` (up to 10 spaces, or a string like `'\t'`) pretty-prints.

3. The **replacer** is an **array** of allowed keys or a **function** `(key, value) => newValue` per property (`undefined` drops it).

What JSON silently changes or refuses

```
const weird = {
  f() {}, u: undefined, s: Symbol('x'), nan: NaN, inf: Infinity, negZero: -0,
  map: new Map([['k', 1]]), set: new Set([1]), arr: [undefined, () => 1, NaN],
};
console.log(JSON.stringify(weird));
try { JSON.stringify({ big: 10n }); } catch (e) { console.log('BigInt ->', e.constructor.name); }
const loop = { name: 'me' }; loop.self = loop;
try { JSON.stringify(loop); } catch (e) { console.log('circular ->', e.constructor.name); }

const money = { cents: 1999, toJSON() { return (this.cents / 100).toFixed(2); } };
console.log(JSON.stringify({ price: money }));
```

Output:

```
{"nan":null,"inf":null,"negZero":0,"map":{},"set":{},"arr":[null,null,null]}
BigInt -> TypeError
circular -> TypeError
{"price":"19.99"}
```

Value	In an object property	In an array	At top level
`undefined`, function, symbol	**property dropped**	becomes `null`	`JSON.stringify` returns `undefined`
`NaN`, `Infinity`	`null`	`null`	`"null"`
`-0`	`0`	`0`	`"0"`
`Date`	ISO string (via `toJSON`)	same	same
`Map`, `Set`	`{}` — data lost!	same	same
BigInt	**throws** `TypeError`	throws	throws
Circular reference	**throws** `TypeError`	throws	—
Non-enumerable or symbol keys	skipped	—	—

Any object can define **`toJSON()`** to control its representation. Convert a `Map` first: `Object.fromEntries(map)` or `[...map]`.

How `parse` works

```
const parsed = JSON.parse(text);
console.log(parsed.items[0].qty, typeof parsed.placedAt);          // dates come back as strings
const revived = JSON.parse(text, (key, value) => (key === 'placedAt' ? new Date(value) : value));
console.log(revived.placedAt instanceof Date, revived.placedAt.getUTCHours());

for (const bad of ["{'a': 1}", '{a: 1}', '{"a": 1,}', 'undefined', '']) {
  try { JSON.parse(bad); console.log('ok', bad); } catch (e) { console.log('SyntaxError for',
JSON.stringify(bad)); }
}
console.log(JSON.parse('"just a string"'), JSON.parse('3.5'), JSON.parse('[true,null]'));
console.log(JSON.parse('{"id": 9007199254740993}').id);
```

Output:

```
2 string
true 8
SyntaxError for "{'a': 1}"
SyntaxError for "{a: 1}"
SyntaxError for "{\"a\": 1,}"
SyntaxError for "undefined"
SyntaxError for ""
just a string 3.5 [ true, null ]
9007199254740992
```

- No date type: dates return as **strings**; a **reviver** `(key, value) => newValue` runs on every value (innermost first) to restore them.
- Invalid JSON throws `SyntaxError` — wrap `JSON.parse` of external input in `try/catch`.
- Any JSON value can be top level.
- Integers above `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$) lose precision (`…993` → `…992`) — why APIs like Twitter/X send big IDs as strings. ES2026's **"JSON.parse source text access"** passes the reviver a third argument whose `source` is the original text, so you can rebuild an exact BigInt (Node 24 supports it):

```
const exact = JSON.parse('{"id": 9007199254740993}', (key, value, ctx) =>
  key === 'id' ? BigInt(ctx.source) : value);
console.log(exact.id);   // 9007199254740993n
```

Deep copy via JSON vs `structuredClone`

```
const src = { when: new Date(0), tags: new Set(['x']), n: 1 };
console.log(JSON.parse(JSON.stringify(src)), structuredClone(src));
```

Output:

```
{ when: '1970-01-01T00:00:00.000Z', tags: {}, n: 1 } { when: 1970-01-01T00:00:00.000Z, tags: Set(1) { 'x' }, n: 1
}
```

JSON turned the Date into a string and emptied the Set; `structuredClone` kept both. Use `structuredClone` for copying, JSON for **transport** — HTTP bodies, files, `localStorage` (the browser's small key-value store), message queues (services holding messages until another program takes them — System Design Fundamentals).

Real-world examples

- `fetch` : `await response.json()` parses the body; send with `body: JSON.stringify(data)` and `Content-Type: application/json` (Chapter 14).
- `package.json` (project settings and dependencies), `tsconfig.json` (TypeScript settings — actually "JSONC", allowing comments) and most config files.
- `localStorage` stores only strings → `JSON.stringify` / `JSON.parse` .
- Log pipelines emit one JSON object per line ("NDJSON") so tools like Elasticsearch (a search engine for logs) can index fields.

In the interview

30-second answer: " `JSON.stringify` serialises to a string, dropping functions/undefined/symbols in objects, turning NaN/Infinity into null and Dates into ISO strings via `toJSON` , and throwing on BigInt and cycles. `JSON.parse` is strict — double quotes, no trailing commas — and takes a reviver to restore types like Dates."

- *Circular structures?* A replacer tracking seen objects in a `WeakSet` (doesn't keep them alive, Chapter 8) to drop/mark repeats, or restructure; `structuredClone` copies cycles but doesn't help sending JSON.
- *`JSON.stringify({a: undefined})` → `{}` ?* `undefined` properties are omitted.

- *Is JSON a subset of JavaScript?* Since ES2019 ("JSON superset"), yes.

Self-check

1. `JSON.stringify([undefined])`? → `"[null]"`.
2. `JSON.stringify({ d: new Date(0) })`? → `{"d":"1970-01-01T00:00:00.000Z"}`.
3. `{"a":1,}` valid? → No — trailing comma.
4. Reviver's job? → Transform parsed values (e.g. strings → Dates).

Chapter TL;DR

- **Objects** are key → value collections; literals `{}`; **dot** for fixed identifier keys, **bracket** for dynamic/unusual keys; computed keys `[expr]` and shorthand `{ name }`.
- Key order: integer-like ascending, then strings by insertion, then symbols.
- Variables hold **references**; `===` compares identity; `const` doesn't freeze contents.
- **Spread** and `Object.assign` are **shallow** (later keys win); `structuredClone` deep-copies.
- **Destructuring**: renames (`{ id: orderId }`), defaults (only `undefined`), nesting, parameters.
- `Object.keys/values/entries/fromEntries`; `hasOwn` for own keys; `Object.is` fixes `NaN`/`-0`; `Object.create(proto | null)` sets the prototype.
- **Descriptors** (`writable`, `enumerable`, `configurable`, `get`/`set`); `preventExtensions` < `seal` < `freeze`, all shallow.
- **Getters/setters** compute or validate on access; keep them cheap.
- `?.` short-circuits on `null`/`undefined`; pair with `??`.
- `Math`: `round` (halves toward +∞), `floor`/`ceil`/`trunc`, `max/min(...arr)`, radians for trig; random int = `Math.floor(Math.random() * (max - min + 1)) + min`; `crypto` for security.
- `Date` = ms since 1970 UTC; months 0-based, `getDay` = weekday, setters mutate and roll over, date-only ISO parses as UTC; format with `Intl`. **Temporal**: Stage 4 (March 2026), shipping in Chrome, Firefox, Node 26 — not Node 24/Safari.
- `JSON.stringify` drops/changes non-JSON values, throws on BigInt/cycles; `JSON.parse` is strict and takes a reviver.

Chapter 6 — Strings and Regular Expressions

Text is everywhere: user names, URLs (web addresses), log lines, CSV files (comma-separated values — spreadsheet-like plain text), chat messages, HTML. This chapter covers **strings** (how JavaScript represents text), the everyday methods to search, cut, change and compare it, the Unicode details that trip up experienced developers (why `'😀'.length` is 2), and **regular expressions** — a compact pattern language for finding and transforming text.

What you'll learn: quotes, escapes, template literals (interpolation, multi-line, tags, `String.raw`); indexing, `length`, `at()`, immutability, coercion; primitives vs `String` objects; the methods table; Unicode (code units, code points, graphemes, normalization, `Intl.Segmenter`, well-formed strings); regex syntax, flags, lookaround; regex methods and the `lastIndex` trap; modern features (named groups, `d` / `u` / `v` flags, `RegExp.escape`, modifiers, duplicate named groups).

Before you start: variables and types (Chapter 2), functions (Chapter 4); array methods like `map` / `join` are in Chapter 7.

Code is in `code/`; files are ES modules (always strict mode — Chapter 1); outputs are real Node 24 output.

6.1 What a string is

What it is

A **string** is an ordered, **immutable** sequence of characters — a **primitive** (`typeof 'hi'` is `'string'`). No operation changes a string in place; every method returns a *new* string. Precisely, it is a sequence of **UTF-16 code units** (16-bit numbers); for everyday English one unit = one character (§6.5 covers when that breaks).

Why it matters

Most input arrives as text — form fields, query strings (the `?key=value&…` part of a URL), JSON, files — and must be cleaned, validated, split and reformatted. Trailing spaces, case differences and emoji breaking length limits are among the most common web-app bugs.

6.2 Creating strings

Three kinds of quotes

Delimiter	Example	Interpolation `${}`	Real line breaks allowed	Typical use
Single quotes `'…'`	`'It works'`	No	No	Plain strings (common default in style guides — shared formatting rulebooks — e.g. Airbnb/StandardJS)
Double quotes `"…"`	`"It works"`	No	No	Plain strings (Prettier's default); identical behaviour to single quotes
Backticks `` `…` ``	`` `Hi ${name}` ``	**Yes**	**Yes**	Anything built from values, multi-line text, tagged templates

Single and double quotes are equivalent; a formatter (Prettier — Chapter 10) enforces one.

Quotes inside strings

Use a *different* outer quote, or **escape** the inner quote with a backslash. From `code/01_string_basics.mjs`:

```
console.log("She said 'hi'", '|', 'He said "bye"', '|', 'It\'s escaped', '|', `mix 'both' "kinds"`);
```

Output:

```
She said 'hi' | He said "bye" | It's escaped | mix 'both' "kinds"
```

Escape sequences

A backslash starts an **escape sequence** — source characters standing for one special character.

Escape	Produces	Notes
`\'` `\"` `\``	The quote character itself	Needed only when it matches the delimiter
`\\`	One backslash	Windows paths: `'C:\\temp'`
`\n`	Line feed (new line)	The normal line break on Linux/macOS and in most text protocols (message formats like HTTP)
`\r`	Carriage return	Windows line endings are `\r\n`
`\t`	Horizontal tab	
`\b`	Backspace (U+0008)	Rare. Careful: inside a **regex** (§6.7), `\b` means "word boundary"
`\f`	Form feed (U+000C)	Rare (old printers)
`\v`	Vertical tab (U+000B)	Rare; supported by all modern engines
`\0`	Null character (U+0000)	Must not be followed by a digit (`'\01'` is a legacy octal — base-8 — escape, a `SyntaxError` in strict mode)
`\xHH`	Character with hex (base-16) code HH (2 digits)	`'\x41'` is `'A'`
`\uXXXX`	UTF-16 code unit with hex code XXXX (4 digits)	`'\u00e9'` is `'é'`
`\u{X…}`	Any Unicode code point (§6.5; 1–6 hex digits), ES2015	`'\u{1F600}'` is `'😀'`
`\` + line break	Line continuation (no character added)	Old way to split a long literal across lines

```
console.log('Tab:\t|  Backslash: \\  Unicode: \u00e9 \u{1F600}  Hex: \x41');
console.log('Line 1\nLine 2');
console.log(JSON.stringify('a\r\nb\0c'), 'a\r\nb'.length);
```

Output:

```
Tab:    |  Backslash: \  Unicode: é 😀  Hex: A
Line 1
Line 2
"a\r\nb\u0000c" 4
```

Each escape is one character in the value: `'a\r\nb'.length` is 4.

6.3 Template literals

What it is

A **template literal** (ES2015, backticks) adds:

1. **Interpolation** — `${expression}` inserts any expression's value, converted to string.
2. **Multi-line text** — real line breaks are kept.
3. **Tags** — a function before the backtick receives the pieces and values and builds any result.

How it works

```
const item = 'coffee', price = 3.5, qty = 2;
console.log(`${qty} x ${item} = $${(price * qty).toFixed(2)} ${qty > 1 ? '(bulk)' : ''}`);
const multi = `Dear user,
  your order has shipped.`;
console.log(multi);
```

Output:

```
2 x coffee = $7.00 (bulk)
Dear user,
  your order has shipped.
```

- In `$${…}` the first `$` is literal; `${` starts interpolation.
- Any expression works inside `${}` — arithmetic, calls, ternaries, nested templates.
- **Indentation inside a multi-line template is part of the string.**

Templates replaced the pre-ES2015 `'line 1\n' + 'line 2'` style.

Tagged templates and `String.raw`

A **tag** is called with (a) an array of literal text pieces and (b) the interpolated values. Libraries use tags to escape HTML, build SQL (database query language) safely, or define CSS: styled-components' `styled.div`…`` , GraphQL's (an API query language) `gql`…`` , lit-html's `html`…`` .

```
function highlight(strings, ...values) {
  return strings.reduce((out, s, i) => out + s + (i < values.length ? `[${values[i]}]` : ''), '');
}
console.log(highlight`Hello ${'Ana'}, you have ${3} messages`);
console.log(String.raw`C:\new\table`, '|', 'C:\new\table'.length, String.raw`C:\new\table`.length);
```

Output:

```
Hello [Ana], you have [3] messages
C:\new\table | 10 12
```

`strings` is `['Hello ', ', you have ', ' messages']` , `values` `['Ana', 3]` — always one more piece than values. `String.raw` returns text *without* processing escapes (handy for Windows paths and regex sources); the normal `'C:\new\table'` has 10 characters because `\n` and `\t` became newline and tab.

Common mistakes

- `'${name}'` in normal quotes — only backticks interpolate.
- HTML from user input via templates without escaping → **XSS** (cross-site scripting). Escape or use `textContent` (Chapter 14).
- SQL by interpolation → **SQL injection** (attacker input runs as part of your command). Use parameterised queries (command and values sent separately, so values are never code).

6.4 Working with strings: indexing, immutability, coercion, String objects

Indexing, `length` and `at()`

Characters are numbered from **0**: `str[i]` , `str.charAt(i)` , or `str.at(i)` (ES2022), which accepts **negative** indexes from the end.

```
const greeting = 'Hello, World!';
console.log(greeting.length, greeting[0], greeting[7], greeting.at(-1), greeting[50], greeting.charAt(50) === '');
```

Output:

```
13 H W ! undefined true
```

`length` counts code units (spaces and punctuation included); last index is `length - 1` (or `at(-1)`); past the end, brackets give `undefined`, `charAt` gives `''`.

Strings are immutable

```
const s = 'cat';
try { s[0] = 'b'; } catch (e) { console.log('assigning s[0] ->', e.constructor.name); }
console.log(s, 'b' + s.slice(1));
```

Output:

```
assigning s[0] -> TypeError
cat bat
```

Index assignment is silently ignored in **sloppy** scripts and throws `TypeError` in **strict** mode. Build a new string instead.

Concatenation and coercion

`+` concatenates once either side is a string (`'5' + 3` → `'53'`; left to right, so `1 + 2 + '3'` is `3 + '3'` → `'33'` but `'1' + 2 + 3` is `'12' + 3` → `'123'`); `-`, `*`, `/` convert to numbers (`'5' - 3` → `2`) — full rules in Chapter 2 §2.5. Convert **to** string with `String(x)` (safe for `null` / `undefined`; `String([1, 2])` → `'1,2'`, `String({})` → `'[object Object]'`) or a template; `x.toString()` throws on `null` / `undefined`.

String primitives vs `String` objects

`new String('…')` creates a **wrapper object** — essentially never wanted.

```
const obj = new String('Hello');
const prim = 'Hello';
console.log(typeof obj, typeof prim, obj === prim, obj == prim, obj.valueOf() === prim);
console.log(new String('a') === new String('a'), typeof String(123));
console.log(prim.toUpperCase(), 'methods work on primitives via temporary wrapper objects');
```

Output:

```
object string false true true
false string
HELLO methods work on primitives via temporary wrapper objects
```

	`String` primitive `'Hello'`	`String` object `new String('Hello')`
`typeof`	`'string'`	`'object'`
Equality	By value — two equal primitives are `===`	By reference — two wrappers are never `===`, even with the same text
Truthiness	`''` is falsy	Every object is truthy — `new String('')` is truthy!
Cost	Cheap; engines optimise heavily	Extra allocation (memory for a new object)
Methods	Available through **auto-boxing**	Available directly

Auto-boxing: for `'Hello'.toUpperCase()` the engine briefly wraps the primitive to find the method on `String.prototype` (where all strings' methods live — Chapter 9), then discards the wrapper. `String(123)` **without** `new` converts and returns a primitive.

Comparing strings

```
console.log('apple' < 'banana', 'Zebra' < 'apple', 'a'.localeCompare('B'), ['é', 'z', 'e'].sort(), ['é', 'z', 'e'].sort((a, b) => a.localeCompare(b)));
```

Output:

```
true true -1 [ 'e', 'z', 'é' ] [ 'e', 'é', 'z' ]
```

`<`, `>` and default `sort()` compare **code unit by code unit**: all uppercase before lowercase (`'Z'` 90, `'a'` 97), accented letters after `z`. For human ordering use **`localeCompare`** or `Intl.Collator` (§6.6; `Intl` is the built-in internationalisation namespace).

Self-check

1. `'abc'[-1]` vs `'abc'.at(-1)`? → `undefined` vs `'c'`.
2. `'3' + 4 + 5`? → `'345'`; `3 + 4 + '5'`? → `'75'`.
3. `new String('') ? 'y' : 'n'`? → `'y'` (objects are truthy).
4. Why is `'B' < 'a'`? → Code unit 66 < 97.

6.5 Unicode: code units, code points and graphemes

The problem it solves

Why is `'😀'.length` 2? Why can reversing a string produce garbage? Why are `'café'` and `'café'` sometimes unequal? All about encoding.

Three levels of "character"

Level	What it is	Example	JavaScript view
Code unit	One 16-bit number in UTF-16	`'😀'` = two units `D83D DE00`	`length`, `str[i]`, `charCodeAt`, `slice` indexes
Code point	One Unicode character number (U+0000 … U+10FFFF)	`'😀'` = U+1F600	`codePointAt`, `String.fromCodePoint`, `for…of`, `[...str]`
Grapheme cluster	What a person sees as one character	`'👨‍👩‍👧'` = 5 code points, 1 grapheme	`Intl.Segmenter`

Code points up to U+FFFF (the **Basic Multilingual Plane**, most living scripts) fit one code unit. Above it — most emoji, rare CJK (Chinese/Japanese/Korean) characters, music and math symbols — use a **surrogate pair**: a "high" unit D800–DBFF then a "low" DC00–DFFF. A lone half is a **lone surrogate**; a string containing one is **not well-formed**.

From `code/03_unicode.mjs`:

```
const accented = 'café';                    // é as one code point U+00E9
const decomposed = 'cafe\u0301';            // e + combining acute accent U+0301
console.log(accented, decomposed, accented === decomposed, accented.length, decomposed.length);
console.log(accented === decomposed.normalize('NFC'), accented.normalize('NFD').length);

const smile = '😀';
console.log(smile.length, smile.charCodeAt(0).toString(16), smile.charCodeAt(1).toString(16));
console.log(smile.codePointAt(0).toString(16), String.fromCodePoint(0x1f600) === smile, '\u{1F600}' === smile);
console.log(JSON.stringify(smile[0]), smile[0].isWellFormed(), smile.isWellFormed());

const word = 'hi😀!';
console.log(word.length, [...word].length, [...word]);
console.log('naive reverse :', word.split('').reverse().join('').isWellFormed());
console.log('safer reverse :', [...word].reverse().join(''));
```

Output:

```
café café false 4 5
true 5
2 d83d de00
1f600 true true
"\ud83d" false true
5 4 [ 'h', 'i', '😀', '!' ]
naive reverse : false
safer reverse : !😀ih
```

1. **Normalization:** "é" can be one code point (U+00E9) or `e` + combining accent (U+0301) — identical-looking, not `===`. `normalize('NFC')` composes, `'NFD'` decomposes; normalize user input (e.g. usernames) before comparing/storing. `'NFKC'` / `'NFKD'` also fold "compatibility" variants (`fi` → `fi`).
2. **Surrogate pairs:** `smile[0]` is a lone surrogate; `codePointAt` / `fromCodePoint` / `\u{…}` handle whole code points.
3. **Iteration by code point:** `for…of`, `[...str]`, `Array.from(str)` split correctly; `split('')` splits code units and can cut pairs, breaking a reversed string.

Grapheme clusters and `Intl.Segmenter`

Some visible characters are several code points: family emoji (joined by the invisible **zero-width joiner** U+200D), flags (two "regional indicator" letters), skin tones, letters plus combining accents. `Intl.Segmenter` (modern browsers and Node) splits text into graphemes, words or sentences by Unicode rules:

```
const family = '👨‍👩‍👧';
const flag = '🇮🇳';
console.log(family.length, [...family].length, flag.length, [...flag].length);
const seg = new Intl.Segmenter('en', { granularity: 'grapheme' });
const graphemes = (s) => [...seg.segment(s)].map((x) => x.segment);
console.log(graphemes(`a${family}b${flag}`).length, graphemes('cafe\u0301').length);

const words = [...new Intl.Segmenter('en', { granularity: 'word' }).segment('Hello, big world!')]
  .filter((x) => x.isWordLike).map((x) => x.segment);
console.log(words);
```

Output:

```
8 5 4 2
4 4
[ 'Hello', 'big', 'world' ]
```

The family emoji: 8 code units, 5 code points, **1** grapheme. Use graphemes for user-facing limits ("max 280 characters"), cursor movement and truncation.

Well-formed strings and byte length

```
const broken = 'ab\uD800';
console.log(broken.isWellFormed(), broken.toWellFormed().codePointAt(2).toString(16));
console.log(encodeURIComponent(broken.toWellFormed()));
try { encodeURIComponent(broken); } catch (e) { console.log('encodeURIComponent(lone surrogate) ->',
e.constructor.name); }

const enc = new TextEncoder();
for (const s of ['abc', 'café', '😀']) console.log(s, 'length', s.length, 'utf8 bytes', enc.encode(s).length);
```

Output:

```
false fffd
ab%EF%BF%BD
encodeURIComponent(lone surrogate) -> URIError
abc length 3 utf8 bytes 3
café length 4 utf8 bytes 5
😀 length 2 utf8 bytes 4
```

- `isWellFormed()` / `toWellFormed()` (ES2024) detect and repair lone surrogates (→ U+FFFD, the replacement character `�`). Encoding APIs — `encodeURIComponent` (text → URL-safe `%XX` form), `TextEncoder`, `fetch` bodies — throw or silently substitute on broken strings.
- Files, network **payloads** (data in a message) and DB columns are measured in **UTF-8 bytes** (a **byte** is 8 bits; **UTF-8**, the web's common encoding, uses 1–4 bytes per character), not `length`. Use `new TextEncoder().encode(s).length` (browser) or `Buffer.byteLength(s)` (Node).

In the interview

30-second answer: "JS strings are UTF-16 code units; `length` and indexing count units, so characters beyond U+FFFF like most emoji count as 2. `for…of` and spread iterate code points; `Intl.Segmenter` gives user-perceived characters. Normalize with `normalize('NFC')` before comparing."

- *Reverse safely?* `[...s].reverse().join('')` for code points; reverse `Intl.Segmenter` graphemes for combining marks and ZWJ emoji.
- *Count characters for a tweet-like limit?* Graphemes, not `length`.

Self-check

1. `'😀'.length`? → 2. 2. `[...'😀'].length`? → 1.
2. Identical-looking strings unequal? → Different normalization forms.
3. Word list in any language? → `Intl.Segmenter` with `granularity: 'word'`.

6.6 String methods

How to read this section

Every method **returns a new value**. Index arguments are code-unit positions; an `end` index is **exclusive**.

Job	Method	Returns	Notes
Size	`length` (property)	number of code units	not a method — no `()`
Read a char	`charAt(i)`	1-char string, or `''` if out of range	
	`at(i)` (ES2022)	1-char string or `undefined`; negative `i` counts from end	
	`charCodeAt(i)`	UTF-16 code unit number 0–65535	`codePointAt(i)` for full code points
	`String.fromCharCode(...codes)` / `String.fromCodePoint(...cps)`	new string	**static**: called on `String`, not on a string
Search	`indexOf(sub, from?)` / `lastIndexOf(sub, from?)`	index or **-1**	case-sensitive
	`includes(sub, from?)`	boolean	ES2015
	`startsWith(sub, pos?)` / `endsWith(sub, len?)`	boolean	ES2015
	`search(regex)`	index of first regex match or -1	
	`match(regex)` / `matchAll(regex)`	match info / iterator (hands out results one at a time, Chapter 8)	§6.8
Extract	`slice(start, end?)`	substring; **negative** indexes count from end	the one to use
	`substring(start, end?)`	substring; negatives become 0; swaps args if start > end	older
	`substr(start, length?)`	substring by **length**	legacy (Annex B, the spec's appendix of old web-compatibility features, "normative optional") — avoid
Case	`toUpperCase()` / `toLowerCase()`	new string	locale-independent Unicode rules
	`toLocaleUpperCase(loc?)` / `toLocaleLowerCase(loc?)`	new string	locale-specific rules (a locale is a language + region such as `tr-TR`; e.g. Turkish dotted İ)
Whitespace	`trim()` / `trimStart()` / `trimEnd()`	new string	removes spaces, tabs, newlines and other Unicode whitespace
Padding	`padStart(len, fill?)` / `padEnd(len, fill?)`	new string of at least `len`	ES2017
Build	`repeat(n)`	`n` copies joined	
	`concat(...parts)`	joined string	`+` or templates are more common
Split / join	`split(sep?, limit?)`	**array** of pieces	`sep` can be a string or regex; `arr.join(sep)` reverses it
Replace	`replace(pattern, replacement)`	new string	string pattern → first occurrence only
	`replaceAll(pattern, replacement)` (ES2021)	new string	every occurrence; a regex must have `g`

Job	Method	Returns	Notes
Compare	`localeCompare(other, locale?, options?)`	negative / 0 / positive	language-aware ordering
Unicode	`normalize(form?)` , `isWellFormed()` , `toWellFormed()`	string / boolean	§6.5
Convert	`toString()` / `valueOf()`	the primitive string	mainly for `String` objects

Searching

From `code/02_string_methods.mjs`:

```
const log = 'ERROR 42: disk full; ERROR 7: fan';
console.log(log.indexOf('ERROR'), log.lastIndexOf('ERROR'), log.indexOf('WARN'), log.indexOf('ERROR', 1));
console.log(log.includes('disk'), log.startsWith('ERROR'), log.endsWith('fan'), log.startsWith('42', 6));
console.log(log.search(/\d+/));
```

Output:

```
0 21 -1 21
true true true true
6
```

Use `includes` for yes/no (`indexOf(...) !== -1` is the older idiom). Bug: `if (s.indexOf(x))` is falsy when found **at index 0** and truthy (−1) when missing.

Extracting: `slice` vs `substring` vs `substr`

```
const text = 'Hello, World!';
console.log(text.slice(0, 5), '|', text.slice(7), '|', text.slice(-6, -1));
console.log(text.substring(0, 5), '|', text.substring(5, 0), '|', text.substring(-3, 2));
console.log(text.slice(5, 0) === '', text.substr(7, 5), '(substr: legacy, avoid)');
console.log(text.charAt(4), text.at(-1), text.charCodeAt(0), String.fromCharCode(72, 105));
```

Output:

```
Hello | World! | World
Hello | Hello | He
true World (substr: legacy, avoid)
o ! 72 Hi
```

- `slice(0, 5)` → indexes 0–4 (end excluded); omit `end` to go to the end.
- `slice` accepts negatives; `substring` turns them into 0 and swaps reversed arguments; `slice` with start > end gives `''` .
- `substr(7, 5)` = "5 characters from 7" — legacy; use `slice` .

Case, whitespace and padding

```
console.log(text.toUpperCase(), text.toLowerCase());
console.log('istanbul'.toLocaleUpperCase('tr-TR'), 'istanbul'.toUpperCase());
console.log('Straße'.toUpperCase(), 'Dž'.toLowerCase());

const raw = '   padded text \n';
console.log(JSON.stringify(raw.trim()), JSON.stringify(raw.trimStart()), JSON.stringify(raw.trimEnd()));
console.log('7'.padStart(3, '0'), 'ab'.padEnd(5, '.') + '|', '4111222233334444'.slice(-4).padStart(16, '*'));
```

Output:

```
HELLO, WORLD! hello, world!
İSTANBUL ISTANBUL
STRASSE ẞ
"padded text" "padded text \n" "   padded text"
007 ab...| ************4444
```

- Case conversion is Unicode-aware and can change length (`ß` → `SS`); only the locale-aware version produces Turkish `İ` .
- **Case-insensitive comparison**: lowercase both for simple text; for accents/other languages `a.localeCompare(b, undefined, { sensitivity: 'base' }) === 0` .
- `padStart` for zero-padding and masking card numbers.

Building, splitting and replacing

```
console.log('ab'.repeat(3), 'a'.concat('b', 'c', 1), ['x', 'y', 'z'].join('-'));
console.log('a,b,,c'.split(','), 'hello'.split(''), 'a b  c'.split(' '), 'a b  c'.split(/\s+/), 'x'.split(','),
'a,b,c'.split(',', 2));
const s = 'cat, cat, cat';
console.log(s.replace('cat', 'dog'), '|', s.replaceAll('cat', 'dog'), '|', s.replace(/cat/g, 'cow'));
```

Output:

```
ababab abc1 x-y-z
[ 'a', 'b', '', 'c' ] [ 'h', 'e', 'l', 'l', 'o' ] [ 'a', 'b', '', 'c' ] [ 'a', 'b', 'c' ] [ 'x' ] [ 'a', 'b' ]
dog, cat, cat | dog, dog, dog | cow, cow, cow
```

- `split` keeps empty strings between consecutive separators; a regex like `/\s+/` collapses runs. No match → one-element array; the second argument limits pieces.
- **`replace` with a string pattern changes only the first occurrence** — use `replaceAll` or a `/g` regex.

Comparing with `localeCompare` and `Intl.Collator`

```
console.log('a'.localeCompare('b'), 'b'.localeCompare('a'), 'a'.localeCompare('a'));
console.log('résumé'.localeCompare('resume', 'en', { sensitivity: 'base' }), 'ä'.localeCompare('z', 'de'),
'ä'.localeCompare('z', 'sv'));
const collator = new Intl.Collator('en', { numeric: true });
console.log(['file10', 'file2', 'file1'].sort(), ['file10', 'file2', 'file1'].sort(collator.compare));
```

Output:

```
-1 1 0
0 -1 1
[ 'file1', 'file10', 'file2' ] [ 'file1', 'file2', 'file10' ]
```

`localeCompare` returns negative/0/positive — exactly what `sort` expects from a **comparator** (function deciding which item comes first). Order depends on **language**: German sorts `ä` with `a` , Swedish after `z` . `sensitivity: 'base'` ignores accents and case; `numeric: true` gives **natural sort** (`file2` before `file10`). For many strings reuse one `Intl.Collator` 's `compare` — much faster than repeated `localeCompare` .

Converting strings to numbers

`Number(str)` is strict (`Number('42px')` → `NaN` ; whole trimmed string must be numeric; `''` → `0`); `parseInt('42px', 10)` → `42` and `parseFloat('3.14abc')` → `3.14` read leading digits and stop. Always pass the **radix** (number base) to `parseInt` . Details: Chapter 2 §2.10. (`toString()` / `valueOf()` on a `String` wrapper return the primitive.)

Two real tasks

```
const slugify = (title) => title.trim().toLowerCase()
  .normalize('NFKD').replace(/[\u0300-\u036f]/g, '')   // strip accents
  .replace(/[^a-z0-9]+/g, '-').replace(/^-+|-+$/g, '');
console.log(slugify('  Crème Brûlée: 10 Tips & Tricks!  '));

const titleCase = (str) => str.split(' ').map((w) => w.charAt(0).toUpperCase() + w.slice(1).toLowerCase()).join('
');
console.log(titleCase('the qUICK brown fox'));
```

Output:

```
creme-brulee-10-tips-tricks
The Quick Brown Fox
```

`slugify` builds URL **slugs** (readable address endings like `my-first-post`) as blogs and CMSs (content management systems like WordPress) do: trim, lowercase, decompose accents and delete combining marks (U+0300–U+036F), turn non-alphanumeric runs into `-`, strip edge dashes.

Performance note

`s += piece` in a loop conceptually creates a new string each time, but engines optimise it with internal "rope" structures (trees of pieces joined only when needed). For very large output, pushing pieces to an array and `join('')` once is a predictable alternative.

Common mistakes

- `replace('a', 'b')` expecting every `a` to change.
- `if (str.indexOf(x))` instead of `includes`.
- Comparing user text without `trim()` and case/accent normalization.
- `substr` in new code; confusing `substring(start, end)` with `substr(start, length)`.
- Sorting names with default `sort()`.

In the interview

30-second answer: "Strings are immutable primitives; methods return new strings. `slice` for extraction (negatives OK), `includes` / `startsWith` for tests, `split` / `join` to convert to arrays, `replaceAll` or a `/g` regex to replace everywhere, `padStart` for formatting, `localeCompare` / `Intl.Collator` for human sorting."

- *Palindrome check* (reads the same backwards, like "level"): `const t = s.toLowerCase().replace(/[^a-z0-9]/g, ''); t === [...t].reverse().join('')`.
- *Count occurrences:* `s.split(sub).length - 1`, or `s.match(/sub/g)?.length ?? 0` (escape input patterns — `RegExp.escape`, §6.9).
- *`slice` vs `substring`?* Negative indexes and argument swapping.

Self-check

1. `'JavaScript'.slice(-6)`? → `'Script'`.
2. `'a-b-c'.split('-', 2)`? → `['a', 'b']`.
3. `'5'.padStart(3, '0')`? → `'005'`.
4. Case-insensitive sort of `['b', 'A', 'a']`? → `.sort((x, y) => x.localeCompare(y, undefined, { sensitivity: 'base' }))`.

6.7 Regular expressions: the pattern language

What it is

A **regular expression** (regex, regexp) is a pattern describing a set of strings ("five digits", "anything between `<` and `>`"); in JS a `RegExp` object, run by the regex engine against text from left to right until the pattern fits. Two forms:

- **Literal** `/pattern/flags` — compiled (into the engine's internal matcher) once at load; for fixed patterns.
- **Constructor** `new RegExp('pattern', 'flags')` — built at run time from data; double the backslashes: `new RegExp('\\d+')`.

Why it matters

Form validation, log extraction, search-and-replace, URL routing (`/users/:id` becomes a regex inside **routers**, which pick the handler for each address), syntax highlighting, **tokenizers** (chop text into words, numbers, symbols) — all use regexes, which are easy to get wrong.

Syntax cheat sheet

Category	Syntax	Matches
Literal characters	`abc`	the text `abc`
Any character	`.`	any char except line terminators (unless the `s` flag)
Escaping	`\.` `\*` `\\` `\/`	the special character itself
Character class	`[abc]`, `[a-z0-9]`	one char from the set / range
Negated class	`[^abc]`	one char **not** in the set
Shorthand classes	`\d` `\w` `\s`	digit `[0-9]`; word char `[A-Za-z0-9_]`; whitespace
Negated shorthands	`\D` `\W` `\S`	the complements
Unicode property	`\p{L}`, `\p{Nd}`, `\p{Script=Greek}` (needs `u` or `v`)	any letter / decimal digit / Greek char
Quantifiers	`*` `+` `?`	0 or more / 1 or more / 0 or 1
Counted	`{n}` `{n,}` `{n,m}`	exactly n / at least n / n to m
Lazy quantifiers	`*?` `+?` `??` `{n,m}?`	as **few** as possible
Anchors	`^` `$`	start / end of input (of each line with `m`)
Word boundary	`\b` `\B`	position between `\w` and `\W` / not a boundary
Alternation	`a	b`	
Capturing group	`( … )`	groups and remembers the match as `$1`, `m[1]`, …
Named group	`(?<name> … )`	remembered as `groups.name` / `$<name>`
Non-capturing group	`(?: … )`	groups without remembering
Backreference	`\1`, `\k<name>`	the same text a group matched earlier
Lookahead	`(?= … )` / `(?! … )`	position followed / not followed by …
Lookbehind (ES2018)	`(?<= … )` / `(?<! … )`	position preceded / not preceded by …

Flags

Flag	Name	Effect
`g`	global	Find **all** matches (affects `match`, `replace`, `matchAll`; makes `test`/`exec` stateful — they remember something between calls)
`i`	ignoreCase	Case-insensitive
`m`	multiline	`^` and `$` match at each line start/end
`s`	dotAll (ES2018)	`.` also matches newlines
`u`	unicode (ES2015)	Work in code points; enables `\p{…}` and `\u{…}`; stricter syntax
`v`	unicodeSets (ES2024)	Superset of `u`: set operations in classes, string properties; use `u` **or** `v`, not both
`y`	sticky	Match only at exactly `lastIndex` (used by tokenizers)
`d`	hasIndices (ES2022)	Add start/end `indices` for each group

6.8 Regex methods and techniques

Which method to use

Goal	Use	Returns
Does it match?	`regex.test(str)`	`true` / `false`
First match with details	`str.match(regexWithoutG)` or `regex.exec(str)`	match array (`[0]` full match, `[1]…` groups, `.index`, `.groups`) or `null`
All matched strings	`str.match(regexWithG)`	array of strings or **`null`**
All matches with details	`str.matchAll(regexWithG)` (ES2020)	iterator of match arrays (throws without `g`)
Position of first match	`str.search(regex)`	index or -1
Replace	`str.replace(regex, replacement)` / `replaceAll`	new string
Split	`str.split(regex)`	array

`test`, `match`, `matchAll` and named groups

From code/04_regex_basics.mjs:

```
const literal = /colou?r/i;
const dynamic = new RegExp('colou?r', 'i');
console.log(literal.test('Nice COLOR'), dynamic.source, dynamic.flags, literal.toString());

const zip = /^\d{5}(-\d{4})?$/;
console.log(['12345', '12345-6789', '1234', '12345-'].map((z) => `${z}:${zip.test(z)}`).join(' '));

const text = 'Order #123 shipped, order #456 pending';
const first = text.match(/#(\d+)/);
console.log(first[0], first[1], first.index);
console.log(text.match(/#\d+/g), 'no match ->', text.match(/#x/g));

for (const m of text.matchAll(/#(?<id>\d+) (?<status>\w+)/g)) {
  console.log(m.index, m.groups.id, m.groups.status);
}

const { groups: { year, month, day } } = '2024-07-04'.match(/(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/);
console.log(year, month, day);
```

Output:

```
true colou?r i /colou?r/i
12345:true 12345-6789:true 1234:false 12345-:false
#123 123 6
[ '#123', '#456' ] no match -> null
6 123 shipped
26 456 pending
2024 07 04
```

- `u?` makes `u` optional; `i` ignores case.
- The ZIP pattern is **anchored** `^…$` — otherwise `test` accepts `'abc12345xyz'` (a match anywhere counts). Validation needs anchors.
- `match` without `g`: full match, groups, position; with `g`: just strings. **No match returns** `null`, not `[]` — use `str.match(re) ?? []` or `matchAll`.
- **Named groups** self-document and pair with destructuring.

`replace` with patterns and functions

```
console.log('2024-07-04'.replace(/(\d{4})-(\d{2})-(\d{2})/, '$3/$2/$1'));
console.log('2024-07-04'.replace(/(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})/, '$<d>.$<m>.$<y>'));
console.log('price: 10 USD, 25 USD'.replace(/(\d+) USD/g, (all, n) => `€${(n * 0.9).toFixed(2)}`));
console.log('snake_case_name'.replace(/_(\w)/g, (_, c) => c.toUpperCase()));
console.log('a.b.c'.replaceAll('.', '/'), 'x'.replace('x', '$&$&'));
console.log('one, two;three  four'.split(/[,;\s]+/), '1+2-3'.split(/([+-])/));
```

Output:

```
04/07/2024
04.07.2024
price: €9.00, €22.50
snakeCaseName
a/b/c xx
[ 'one', 'two', 'three', 'four' ] [ '1', '+', '2', '-', '3' ]
```

Replacement-**string** patterns: `$1` … `$99`, `$<name>`, `$&` (whole match), `$`` / `$'` (text before/after), `$$` (literal `$`) — applied even for plain-string search patterns, so pass user text through a **function** instead. A replacement function receives `(match, group1, group2, …, offset, fullString, groups)` and returns the insert — the most powerful form. Splitting by a regex with a **capturing group** keeps the captured separators.

Greedy vs lazy, anchors, flags in action

```
const html = '<b>bold</b> and <i>italic</i>';
console.log(html.match(/<.+>/)[0], '|', html.match(/<.+?>/)[0], '|', html.match(/<[^>]+>/g));

console.log(/\bcat\b/.test('concatenate'), /\bcat\b/.test('a cat sat'));
const lines = 'id: 1\nname: Bo\nid: 2';
console.log(lines.match(/^id: \d+$/g), lines.match(/^id: \d+$/gm));

console.log(/start.+end/.test('start\nend'), /start.+end/s.test('start\nend'));
```

Output:

```
<b>bold</b> and <i>italic</i> | <b> | [ '<b>', '</b>', '<i>', '</i>' ]
false true
null [ 'id: 1', 'id: 2' ]
false true
```

- Quantifiers are **greedy**: `.+` grabs all it can then backs off, so `<.+>` runs to the *last* `>`; `?` makes it **lazy**. A negated class (`[^>]+`) is clearer and faster.
- `\b` keeps `cat` from matching inside `concatenate`.
- Without `m`, `^` / `$` mean the whole input; with `m`, each line.
- `.` skips `\n` without the `s` flag.

(Real HTML needs a **parser** — reads a format into a structure — such as `DOMParser`; regexes are fragile.)

Lookaround and backreferences

```
console.log('$30 and €40 and $5'.match(/(?<=\$)\d+/g));        // numbers preceded by $
console.log('100px 2em 30px'.match(/\d+(?=px)/g));             // numbers followed by px
const strong = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/;
console.log(['weakpass', 'Str0ngPass', 'Short1A'].map((p) => `${p}:${strong.test(p)}`).join(' '));
console.log('this is is a test test'.match(/\b(\w+) \1\b/g));
```

Output:

```
[ '30', '5' ]
[ '100', '30' ]
weakpass:false Str0ngPass:true Short1A:false
[ 'is is', 'test test' ]
```

Lookarounds test what's before/after a position *without including it*. Stacked lookaheads (`(?=.*[a-z])(?=.*[A-Z])(?=.*\d)`) express "must contain each" — the classic password rule. Backreference `\1` = "same text as group 1" — finds doubled words.

The `lastIndex` trap and `exec` loops

```
const g = /a/g;
console.log(g.test('a'), g.lastIndex, g.test('a'), g.lastIndex, g.test('a'));

const re = /\d+/g; let m; const found = [];
while ((m = re.exec('a1b22c333')) !== null) found.push(`${m[0]}@${m.index}`);
console.log(found);

const sticky = /\d+/y; sticky.lastIndex = 1;
console.log(sticky.exec('a12b')?.[0], (sticky.lastIndex = 0, sticky.exec('a12b')));
```

Output:

```
true 1 false 0 true
[ '1@1', '22@3', '333@6' ]
12 null
```

A `g` (or `y`) regex stores where its last match ended in **`lastIndex`**, and `test` / `exec` resume from there — so reusing one global regex to validate several strings alternates `true` / `false`. Fixes: no `g` for `test`, reset `lastIndex = 0`, or `matchAll` (which copies the regex). The `while (exec)` loop is the pre-2020 way to walk matches; `matchAll` replaces it. **Sticky** `y` matches only exactly at `lastIndex` — a tokenizer building block.

6.9 Modern regex features and practical recipes

From `code/05_regex_modern.mjs`:

```
console.log(/^.$/.test('😀'), /^.$/u.test('😀'));
console.log('Ünïcödé 123 ñ'.match(/\p{L}+/gu));
console.log('Price: ৲৪ or 34'.match(/\p{Nd}+/gu));
console.log('Hi 👋 there 🎉'.replace(/\p{Extended_Pictographic}/gu, '*'));

console.log('abcdef'.match(/[\p{L}--[aeiou]]/gv));
console.log('I 🇮🇳 flags'.match(/\p{RGI_Emoji_Flag_Sequence}/v)?.[0]);

const userInput = 'a.b*c';
const safe = new RegExp(RegExp.escape(userInput), 'g');
console.log(RegExp.escape(userInput), 'xa.b*cx axbbc'.replace(safe, '[hit]'));
console.log('without escaping:', 'xa.b*cx axbbc'.replace(new RegExp(userInput, 'g'), '[hit]'));

const withIdx = /(?<key>\w+)=(?<val>\w+)/d.exec('  mode=dark');
console.log(withIdx.indices.groups.key, withIdx.indices.groups.val);

const dateRe = /(?<y>\d{4})-(?<m>\d{2})|(?<m>\d{2})\/(?<y>\d{4})/;
console.log('2024-07'.match(dateRe).groups, '07/2024'.match(dateRe).groups);

console.log(/^(?i:hello) World$/.test('HELLO World'), /^(?i:hello) World$/.test('HELLO WORLD'));
```

Output:

```
false true
[ 'Ünïcödé', 'ñ' ]
[ '৲৪', '34' ]
Hi * there *
[ 'b', 'c', 'd', 'f' ]
🇮🇳
\x61\.b\*c x[hit]x axbbc
without escaping: xa.b*cx [hit]
[ 2, 6 ] [ 7, 11 ]
[Object: null prototype] { y: '2024', m: '07' } [Object: null prototype] { y: '2024', m: '07' }
true false
```

Feature	Version	What it gives you
`u` flag + `\p{…}` property escapes	ES2015 / ES2018	Code-point matching; letters, digits, emoji in **any** script (`\w` and `\d` are ASCII-only)
`v` flag	ES2024	Class **set operations** — difference `--`, intersection `&&`, nested classes — and multi-code-point properties like `\p{RGI_Emoji_Flag_Sequence}`
`d` flag	ES2022	`indices` with `[start, end]` for the match and each group — for editors and highlighters
`RegExp.escape(str)`	**ES2025**	Escapes every regex-special character so user text is matched literally. Note it also hex-escapes a leading letter or digit (`\x61`) so the result is safe even when placed after another escape
Duplicate named groups	**ES2025**	The same group name may appear in different `
Pattern modifiers `(?i:…)`	**ES2025**	Turn `i`, `m` or `s` on/off for part of a pattern

All run in Node 24 and current evergreen (self-updating) browsers. Without `RegExp.escape`, the user's `.` and `*` acted as operators and matched the wrong text — an injection bug, and with patterns like `(a+)+` a potential **ReDoS** (regular-expression denial of service: exponential-time matching that freezes the thread).

Practical recipes

```
const logs = `127.0.0.1 - [10/Oct/2024:13:55:36] "GET /index.html HTTP/1.1" 200 2326
10.0.0.5 - [10/Oct/2024:13:56:01] "POST /api/login HTTP/1.1" 401 87`;
const lineRe = /^(?<ip>\S+) - \[(?<time>[^\]]+)\] "(?<method>[A-Z]+) (?<path>\S+) [^"]+" (?<status>\d{3}) (?
<bytes>\d+)$/gm;
console.table([...logs.matchAll(lineRe)].map((m) => ({ ...m.groups, status: Number(m.groups.status) })));

const email = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
console.log(['a@b.co', 'no-at.com', 'x@y', 'sp ace@a.io'].map((e) => `${e}:${email.test(e)}`).join(' '));

const n = 1234567.891;
console.log(String(Math.trunc(n)).replace(/\B(?=(\d{3})+(?!\d))/g, ','), n.toLocaleString('en-US'),
n.toLocaleString('de-DE'));

const tpl = 'Hi {name}, your code is {code}. {unknown}';
const data = { name: 'Ana', code: 'X9' };
console.log(tpl.replace(/\{(\w+)\}/g, (m, key) => data[key] ?? m));
```

Output:

```
┌─────────┬─────────────┬──────────────────────┬──────────┬──────────────┬──────────┬──────────┐
│ (index) │ ip          │ time                 │ method   │ path         │ status   │ bytes    │
├─────────┼─────────────┼──────────────────────┼──────────┼──────────────┼──────────┼──────────┤
│ 0       │ '127.0.0.1' │ '10/Oct/2024:13:55:36' │ 'GET'  │ '/index.html' │ 200     │ '2326'   │
│ 1       │ '10.0.0.5'  │ '10/Oct/2024:13:56:01' │ 'POST' │ '/api/login'  │ 401     │ '87'     │
└─────────┴─────────────┴──────────────────────┴──────────┴──────────────┴──────────┴──────────┘
a@b.co:true no-at.com:false x@y:false sp ace@a.io:false
1,234,567 1,234,567.891 1.234.567,891
Hi Ana, your code is X9. {unknown}
```

1. **Log parsing**: `gm` + named groups + `matchAll` turn lines into objects — the idea behind log pipelines (Logstash "grok" patterns, Fluent Bit parsers).
2. **Email**: a *pragmatic* check ("something@something.tld", no spaces; the **TLD** — top-level domain — is the last part like `.com`). Full RFC 5322 (the email-format standard) validation by regex is impractical; the real test is a confirmation email. `<input type="email">` uses a similar simplified rule.
3. **Thousands separators**: lookahead `(?=(\d{3})+(?!\d))` finds positions before groups of three digits — neat, but `toLocaleString` / `Intl.NumberFormat` is correct for every locale.
4. **Templating**: a replacer function looks up each `{key}`, leaving unknown keys.

Trade-offs: regex or string methods?

Situation	Prefer
Fixed substring test / replace	`includes`, `startsWith`, `replaceAll` — clearer and faster
Patterns (digits, optional parts, alternatives)	Regex
Structured formats (HTML, JSON, CSV with quotes, URLs)	A real parser: `DOMParser`, `JSON.parse`, a CSV library, `new URL()`
Human-language text (words, characters)	`Intl.Segmenter`, `\p{…}` with `u` / `v`

When not to use it

Avoid giant unreadable regexes — split into steps or write a small parser. Never run a user-supplied pattern without a timeout (or a linear-time engine like RE2, whose time grows only in proportion to

input): **catastrophic backtracking** can hang a Node server — the cause of real outages such as Cloudflare's July 2019 outage.

Common mistakes

- No anchors in validation (`/\d{5}/.test('abc123456')` is `true`).
- Forgetting `g` in `replace` ; `matchAll` / `replaceAll` with a non-global regex (both throw `TypeError`).
- Reusing a `/g` regex with `test` (`lastIndex`).
- `new RegExp(userText)` without `RegExp.escape` .
- `'\d'` instead of `'\\d'` in the constructor string.
- Expecting `\w` to match `é` / `ñ` — ASCII-only; use `\p{L}` with `u` .

Real-world examples

- **Routers**: Express (the most popular Node web framework) and front-end routers compile `/users/:id` into a regex with a capture group.
- **Linters/formatters** use regexes for simple rules; editors use `d` indices for highlighting.
- HTML `pattern` **attribute** validation is a regex (evaluated with `v` in modern browsers).
- **Search tools** (ripgrep, VS Code search) and log processing run on regex engines.

In the interview

> **30-second answer:** "A regex is a pattern object — `/…/flags` or `new RegExp` . `test` for yes/no, `matchAll` with named groups to extract, `replace` with a callback to transform. Gotchas: anchors for validation, `g` for global replace, stateful `lastIndex` with `g` , greedy vs lazy, and `RegExp.escape` for user input."

- *`match` vs `matchAll`?* `match` with `g` returns only strings (or `null`); `matchAll` an iterator of full matches with groups and indexes.
- *Match a word in any language?* `/\p{L}+/u` .
- *Catastrophic backtracking?* Nested quantifiers like `(a+)+$` try exponentially many splits on near-matches; avoid nested unbounded quantifiers.

Self-check

1. `/^\d{3}-\d{4}$/` matches? → Three digits, dash, four digits (`555-1234`).
2. `'aaa'.replace(/a/, 'b')` ? → `'baa'` .
3. Named group `year` in a replacement string? → `$<year>` .
4. Why is `/<.+>/` wrong for one tag? → Greedy; use `/<[^>]+>/` .
5. ES2025 method to escape user input? → `RegExp.escape` .

Chapter TL;DR

- Strings are **immutable UTF-16 primitives**: `'…'` / `"…"` (identical) or backticks (interpolation, multi-line, tags). Escapes: `\n` , `\t` , `\\` , `\'` , `\u{…}` .
- Index with `[i]` , `charAt` , `at(-1)` ; out-of-range brackets → `undefined` . Index assignment never changes a string (throws in strict mode).
- `+` concatenates if either side is a string; other arithmetic converts to numbers. Avoid `new String` ; primitives get methods via auto-boxing.
- Core methods: `includes` / `startsWith` / `endsWith` / `indexOf` , `slice` , `toUpperCase` / `toLowerCase` , `trim*` , `padStart` / `padEnd` , `repeat` , `split` / `join` , `replace` (first only)/ `replaceAll` , `localeCompare` / `Intl.Collator` .
- `length` counts code units (emoji = 2). `for…of` /spread for code points; `Intl.Segmenter` for visible characters; `normalize('NFC')` before comparing; `isWellFormed` / `toWellFormed` for lone surrogates.

- Regexes: classes, quantifiers (greedy; `?` lazy), anchors, groups (numbered, named, non-capturing), lookaround, backreferences; flags `g i m s u v y d`.
- Methods: `test`, `match` (null on no match), `matchAll` (needs `g`), `replace` with `$1` / `$<name>` or a callback, `split` by regex. Beware `lastIndex` with `/g`.
- ES2024–2025: `v` flag, `RegExp.escape`, pattern modifiers, duplicate named groups — all in Node 24. String methods for fixed text, real parsers for structured formats.

Chapter 7 — Arrays

An **array** is JavaScript's ordered list: cart items, search results, database rows, pixels, game moves. Arrays have about forty methods, and knowing which **change** the array and which **return a new one** is the single most useful thing to learn — it prevents a whole class of bugs, especially in React (a UI library) and Redux (a library managing an app's shared data, its *state*).

What you'll learn: why arrays exist and how to create them (literal, `new Array`, `Array.of`, `Array.from`, `fill`); index access, `at()`, `length`, holes, arrays as objects; `push` / `pop` / `unshift` / `shift` and their costs; the full methods table split into **mutating** vs **non-mutating** (incl. ES2023 `toSorted`, `toReversed`, `toSpliced`, `with`); `forEach`, `map`, `filter`, `reduce`, `find` / `findLast`, `some` / `every`, `flat` / `flatMap` and pipelines; sorting with comparators, stability, shuffling; spread/rest/destructuring, shallow vs deep copies, array-likes, recipes; arrays vs objects vs `Map` / `Set`; typed arrays.

Before you start: functions and arrows (Chapter 4) — **callbacks** (functions passed in to be called) are everywhere here; loops (Chapter 3); object spread/destructuring (Chapter 5); `Map`, `Set` and iterators (Chapter 8).

Code is in `code/`; outputs are real Node 24 output.

7.1 Why arrays?

What it is

An **array** stores many values in order under one name. Each value is an **element** with a numeric **index** starting at **0** — like a row of numbered lockers.

The problem it solves

```
const fruit1 = 'apple', fruit2 = 'banana', fruit3 = 'cherry';   // three separate variables...
const fruits = ['apple', 'banana', 'cherry'];                   // ...or one array
```

Separate variables can't be looped, counted, sorted or passed as a group, and a fourth item means new code; an array handles any number with the same code.

How it works

From `code/01_array_basics.mjs`:

```
console.log(fruits, fruits.length, typeof fruits, Array.isArray(fruits), Array.isArray({ length: 0 }));
const mixed = [1, 'two', { three: 3 }, [4], null, () => 5];   // any types, can nest
console.log(mixed.length, mixed[2].three, mixed[3][0], mixed[5]());
```

Output:

```
[ 'apple', 'banana', 'cherry' ] 3 object true false
6 3 4 5
```

- An **array literal** is a comma-separated list in `[ ]`.
- `length` = number of elements (strictly, highest index + 1).
- Arrays are **objects** (`typeof` → `'object'`); test with **`Array.isArray(x)`**.
- Arrays can mix types and nest (arrays of arrays make a grid/**matrix**). Prefer one type per array — clearer and better optimised.

7.2 Creating arrays

Way	Example	Result	When to use
Literal	`['red', 'green', 'blue']`	3 elements	Almost always
`new Array(n)` (one number)	`new Array(3)`	length 3, **no elements** (3 empty slots)	Rarely — prefer `Array.from`
`new Array(a, b, …)`	`new Array('a', 'b', 'c')`	those elements	Never needed — use a literal
`Array.of(...items)`	`Array.of(3)`	`[3]`	When a single number must be an element, not a length
`Array.from(iterableOrArrayLike, mapFn?)` (iterable = anything `for…of` can loop over; array-like = see section 7.7)	`Array.from({ length: 4 }, (_, i) => i * i)`	`[0, 1, 4, 9]`	Build from strings, Sets, Maps, NodeLists (lists of page elements from DOM queries), `arguments`, or generate by index
`Array(n).fill(v)`	`new Array(3).fill(0)`	`[0, 0, 0]`	Pre-filled with a **primitive**
Spread	`[...set]`, `[...'abc']`	copy of any iterable	Copy or convert
`str.split(sep)`	`'a,b'.split(',')`	`['a', 'b']`	From text

```
new Array(3);          // [ <3 empty items> ]  (length 3)
new Array(3, 4);       // [ 3, 4 ]
Array.of(3);           // [ 3 ]
Array.from('hey');     // [ 'h', 'e', 'y' ]
Array(2).fill().map(() => []);   // [ [], [] ]
```

The `new Array` **constructor** (called with `new` to build an object) is inconsistent: **one** number means length, two arguments mean elements — and `new Array(5)` holds five **empty slots** (holes, §7.3), not `undefined`s. `Array.of` / `Array.from` (ES2015) avoid both surprises.

7.3 Reading, changing, adding and removing

Index access, `at()` and `length`

```
const colors = ['red', 'green', 'blue'];
console.log(colors[0], colors[colors.length - 1], colors.at(-1), colors[5]);
colors[2] = 'purple';
console.log(colors);
```

Output:

```
red blue blue undefined
[ 'red', 'green', 'purple' ]
```

Last element: `colors[colors.length - 1]` or **`colors.at(-1)`** (ES2022, negatives from the end). Missing index → `undefined` (no error). Assigning to an index changes the array in place.

The four end-methods

```
const nums = [1, 2, 3];
console.log(nums.push(4), nums.unshift(0), nums);          // both return the NEW length
console.log(nums.pop(), nums.shift(), nums);               // both return the REMOVED element
```

Output:

```
4 5 [ 0, 1, 2, 3, 4 ]
4 0 [ 1, 2, 3 ]
```

Method	Where	Returns	Cost
`push(...items)`	adds at the **end**	new length	O(1) amortised (fast on average; the occasional slow resize is spread over many calls)
`pop()`	removes from the **end**	the removed item (`undefined` if empty)	O(1)
`unshift(...items)`	adds at the **start**	new length	**O(n)** — every element shifts up one index
`shift()`	removes from the **start**	the removed item	**O(n)**

(O(1): time independent of size; O(n): grows with element count — Chapter 17.) `push` / `pop` make a fast **stack** (last in, first out); `shift` as a **queue** (first in, first out) is slow on huge arrays — use an index pointer or a linked structure (each item points to the next, so removing the first moves nothing).

`length` is writable, and holes

```
const letters = ['a', 'b', 'c', 'd'];
letters.length = 2;
console.log(letters);
letters[5] = 'f';                                          // writing past the end creates holes
console.log(letters, letters.length, 3 in letters, letters[3]);

const holey = [1, , 3];
const withUndef = [1, undefined, 3];
let visited = 0; holey.forEach(() => visited++);
console.log(holey, visited, withUndef.map((x) => x ?? 'U'), holey.map((x) => x ?? 'U'));
```

Output:

```
[ 'a', 'b' ]
[ 'a', 'b', <3 empty items>, 'f' ] 6 false undefined
[ 1, <1 empty item>, 3 ] 2 [ 1, 'U', 3 ] [ 1, <1 empty item>, 3 ]
```

- Shrinking `length` **deletes** elements; `arr.length = 0` empties in place (all references see it).
- Writing past the end makes a **sparse array** with **holes** (`3 in letters` is `false`). Holes read as `undefined`; `forEach`, `map`, `filter`, `reduce`, `some`, `every` **skip** them, while `for…of`, `find`, `includes`, `Array.from`, `toSorted`, `with` treat them as `undefined`. Avoid holes — confusing, and engines de-optimise the array (switch to slower general storage).

Looping over an array

```
for (let i = 0; i < colors.length; i++) process.stdout.write(`${i}:${colors[i]} `);  // 0:red 1:green 2:purple
for (const c of colors) process.stdout.write(c + ' ');                               // red green purple
colors.forEach((c, i) => process.stdout.write(`[${i}]${c} `));                       // [0]red [1]green
[2]purple
for (const [i, c] of colors.entries()) process.stdout.write(`${i}=${c} `);           // 0=red 1=green 2=purple
```

Loop	Index?	break / continue ?	await inside works sequentially?	Notes
Classic `for (let i…)`	Yes	Yes	Yes	Most control (step by 2, go backwards)
`for (const x of arr)`	Via `arr.entries()`	Yes	Yes	Cleanest default
`arr.forEach(fn)`	Yes (2nd arg)	**No**	**No** (async callbacks aren't awaited — Chapter 12)	Fine for simple side effects (printing etc.)
`for (const i in arr)`	Yes, as **strings**	Yes	Yes	**Avoid** for arrays: iterates keys, includes inherited enumerable ones (loop-visible properties added to `Array.prototype`)

Arrays are references

```
const a = [1, 2]; const b = a; b.push(3);
console.log(typeof a, a, a === b, [1, 2] === [1, 2]);
```

Output:

```
object [ 1, 2, 3 ] true false
```

As with objects (Chapter 5): assignment copies the reference, `===` compares identity, `const` blocks reassignment but not `push`.

Self-check

1. `[1, 2, 3].push(4)` returns? → `4` (new length).
2. Last element without `length - 1`? → `arr.at(-1)`.
3. `new Array(2)` vs `Array.of(2)`? → two empty slots vs `[2]`.
4. Why avoid `for…in` on arrays? → String keys, visits extra/inherited enumerable properties, not meant for ordered iteration.

7.4 The methods table: mutating vs non-mutating

Why it matters

A **mutating** method changes the array it's called on — fine for a local array, but if it's shared (passed in, in React state, cached) everyone holding a reference sees the change. React compares state by reference, so in-place mutation may mean no **re-render** (redraw). ES2023 added **copying** versions of the classic mutators.

Complete reference

`length` is the only instance **property** you'll use. ✎ = mutates the original.

Method	✎	What it does / returns	Added
`push(...x)` / `pop()`	✎	Add to / remove from end; new length / removed item	ES1
`unshift(...x)` / `shift()`	✎	Add to / remove from start	ES1
`splice(start, deleteCount?, ...items)`	✎	Remove and/or insert anywhere; returns **removed** items	ES1
`sort(compareFn?)`	✎	Sorts in place; returns the same array	ES1
`reverse()`	✎	Reverses in place; returns the same array	ES1
`fill(value, start?, end?)`	✎	Overwrites a range with one value	ES2015
`copyWithin(target, start, end?)`	✎	Copies a range to another position inside the array	ES2015
`toSorted(compareFn?)`		**New** sorted array	**ES2023**
`toReversed()`		**New** reversed array	**ES2023**
`toSpliced(start, deleteCount?, ...items)`		**New** array with the splice applied	**ES2023**
`with(index, value)`		**New** array with one index replaced (negative OK; out of range → `RangeError`)	**ES2023**
`slice(start?, end?)`		Shallow copy of a range (negatives OK)	ES1
`concat(...arraysOrValues)`		New array joining this with others (flattens one level of array args)	ES1
`map(fn)`		New array of `fn` results, same length	ES5
`filter(fn)`		New array of elements where `fn` is truthy	ES5
`flat(depth = 1)`		New array with nested arrays flattened	ES2019
`flatMap(fn)`		`map` then `flat(1)`, in one pass	ES2019
`forEach(fn)`		Calls `fn` on each element; returns `undefined`	ES5
`reduce(fn, init?)` / `reduceRight(fn, init?)`		Folds (combines step by step) to a single value, left→right / right→left	ES5
`find(fn)` / `findIndex(fn)`		First matching element / its index (`undefined` / −1)	ES2015
`findLast(fn)` / `findLastIndex(fn)`		Same, searching from the end	ES2023
`some(fn)` / `every(fn)`		Does any / do all elements pass? (short-circuit)	ES5
`includes(x, from?)`		Boolean; finds `NaN`	ES2016
`indexOf(x, from?)` / `lastIndexOf(x, from?)`		Index using `===` (cannot find `NaN`), or −1	ES5
`at(i)`		Element at `i`; negatives from the end	ES2022
`join(sep = ',')`		String of elements; `null`/`undefined` → empty	ES1
`toString()` / `toLocaleString()`		Comma-joined string / locale-formatted	ES1
`keys()` / `values()` / `entries()`		Iterators (hand out values one at a time, Chapter 8) over indexes / values / `[index, value]` pairs	ES2015
Static: `Array.isArray(x)`, `Array.of(...)`, `Array.from(x, fn?)`, `Array.fromAsync(x)`		Test / create arrays (`fromAsync` awaits async iterables; ES2026, already in Node 24)	ES5–ES2026

(`toSource()` in old lists was a non-standard, now-removed Firefox method — ignore it.)

Mutators vs copying twins in action

From `code/02 mutating vs copying.mjs` — `show(label, returnValue, original)` prints the return value, then the original afterwards (the file also demos `reverse`, `fill`, `copyWithin`):

```
let arr = ['a', 'b', 'c', 'd', 'e'];
show('splice(1, 2)', arr.splice(1, 2), arr);                  // remove 2 at index 1
show("splice(1, 0, 'X', 'Y')", arr.splice(1, 0, 'X', 'Y'), arr);  // insert, remove nothing
show("splice(-1, 1, 'Z')", arr.splice(-1, 1, 'Z'), arr);         // replace the last one

arr = [3, 1, 2];
const sorted = arr.sort();
console.log(sorted === arr, arr);

const base = [3, 1, 2];
show('toSorted()', base.toSorted(), base);
show('toReversed()', base.toReversed(), base);
show("toSpliced(1, 1, 'x')", base.toSpliced(1, 1, 'x'), base);
show('with(0, 99)', base.with(0, 99), base);
show('with(-1, 7)', base.with(-1, 7), base);
try { base.with(10, 1); } catch (e) { console.log('with(10, 1) ->', e.constructor.name); }
show('slice(1)', base.slice(1), base);
show('concat([4], 5)', base.concat([4], 5), base);
show('[...base, 4]', [...base, 4], base);
```

Output:

```
splice(1, 2)              ["b","c"]              original: ["a","d","e"]
splice(1, 0, 'X', 'Y')    []                     original: ["a","X","Y","d","e"]
splice(-1, 1, 'Z')        ["e"]                  original: ["a","X","Y","d","Z"]
true [ 1, 2, 3 ]
toSorted()                [1,2,3]                original: [3,1,2]
toReversed()              [2,1,3]                original: [3,1,2]
toSpliced(1, 1, 'x')      [3,"x",2]              original: [3,1,2]
with(0, 99)               [99,1,2]               original: [3,1,2]
with(-1, 7)               [3,1,7]                original: [3,1,2]
with(10, 1) -> RangeError
slice(1)                  [1,2]                  original: [3,1,2]
concat([4], 5)            [3,1,2,4,5]            original: [3,1,2]
[...base, 4]              [3,1,2,4]              original: [3,1,2]
```

`splice` does everything: *where*, *how many* to delete, *what* to insert. `sort` returns **the same array** (`sorted === arr`). The four ES2023 "change array by copy" methods leave the original alone; supported in all current browsers and Node 20+.

A real bug: a helper that mutates its argument

```
function topTwoBad(scores) { return scores.sort((a, b) => b - a).slice(0, 2); }
function topTwoGood(scores) { return scores.toSorted((a, b) => b - a).slice(0, 2); }
const s1 = [50, 90, 70];
console.log(topTwoBad(s1), 'caller now sees', s1);
const s2 = [50, 90, 70];
console.log(topTwoGood(s2), 'caller still sees', s2);
```

Output:

```
[ 90, 70 ] caller now sees [ 90, 70, 50 ]
[ 90, 70 ] caller still sees [ 50, 90, 70 ]
```

The bad version silently reorders the caller's data. Pre-ES2023 fix: `[...scores].sort(...)` / `scores.slice().sort(...)`.

Immutable update cheat sheet (React/Redux style)

Goal	Mutating	Non-mutating
Add to end	`arr.push(x)`	`[...arr, x]`
Add to start	`arr.unshift(x)`	`[x, ...arr]`
Insert at `i`	`arr.splice(i, 0, x)`	`arr.toSpliced(i, 0, x)`
Remove at `i`	`arr.splice(i, 1)`	`arr.toSpliced(i, 1)` or `arr.filter((_, j) => j !== i)`
Remove by id	find index + `splice`	`arr.filter((t) => t.id !== id)`
Replace at `i`	`arr[i] = x`	`arr.with(i, x)`
Update item by id	`item.done = true`	`arr.map((t) => (t.id === id ? { ...t, done: true } : t))`
Sort / reverse	`arr.sort(fn)` / `arr.reverse()`	`arr.toSorted(fn)` / `arr.toReversed()`

Common mistakes

- `sort` / `reverse` / `splice` on React props (a component's inputs from its parent) or state — use the copying twins.
- Confusing `slice` (copy a range) with `splice` (edit in place).
- `arr = arr.push(x)` — `push` returns the new length.
- Thinking `const` prevents `push`; use `Object.freeze(arr)` (shallow) if needed.

7.5 Iteration methods: map, filter, reduce and friends

What it is

These methods call a **callback** per element with `(element, index, array)`, describing *what* to do (**declarative**) rather than *how to loop* (**imperative**).

How it works

From `code/03_iteration_methods.mjs`, with:

```
const orders = [
  { id: 1, customer: 'Ana', total: 120, status: 'paid' },
  { id: 2, customer: 'Ben', total: 35, status: 'pending' },
  { id: 3, customer: 'Ana', total: 60, status: 'paid' },
  { id: 4, customer: 'Cy', total: 250, status: 'refunded' },
];
```

`forEach`, `map`, `filter`, `reduce`:

```
console.log(orders.map((o) => o.total));
console.log(['1', '2', '3'].map(Number), ['1', '2', '3'].map(parseInt)); // classic trap
const paid = orders.filter((o) => o.status === 'paid');
console.log(paid.map((o) => o.id));
const revenue = paid.reduce((sum, o) => sum + o.total, 0);
const byCustomer = orders.reduce((acc, o) => { acc[o.customer] = (acc[o.customer] ?? 0) + o.total; return acc; },
{});
console.log(revenue, byCustomer);
try { [].reduce((a, b) => a + b); } catch (e) { console.log('empty reduce without initial ->',
e.constructor.name); }
console.log(['a', 'b', 'c'].reduceRight((acc, x) => acc + x, ''));
```

Output:

```
[ 120, 35, 60, 250 ]
[ 1, 2, 3 ] [ 1, NaN, NaN ]
[ 1, 3 ]
180 { Ana: 180, Ben: 35, Cy: 250 }
empty reduce without initial -> TypeError
cba
```

Method	Callback returns	Method returns	Think of it as
`forEach`	ignored	`undefined`	"do something with each"
`map`	the new element	new array, **same length**	"transform each"
`filter`	truthy/falsy	new array, **same or fewer** elements	"keep some"
`reduce`	the new accumulator (the running result carried from one step to the next)	the final accumulator	"combine all into one"

`reduce` summing `[120, 60]` from `0`: `0 + 120 → 120`, `120 + 60 → 180`, done → `180`. **Always pass the initial value** — without it the first element is the start, which throws on empty arrays and gives wrong types when building objects.

The `map(parseInt)` trap: `map` passes `(element, index, array)` and `parseInt` takes `(string, radix)`, so it runs `parseInt('2', 1)`, `parseInt('3', 2)` → `NaN`. Use `map((s) => parseInt(s, 10))` or `map(Number)`.

Chaining into a pipeline:

```
const report = orders
  .filter((o) => o.status !== 'refunded')
  .map((o) => ({ ...o, total: o.total * 1.1 }))
  .toSorted((a, b) => b.total - a.total)
  .map((o) => `${o.id}:${o.total.toFixed(2)}`)
  .join(', ');
console.log(report);
```

Output:

```
1:132.00, 3:66.00, 2:38.50
```

Each step returns a new array (originals untouched) and loops once; irrelevant for UI-sized lists, but for millions of rows a single loop or **lazy** iterator helpers (values computed only when asked, Chapter 8) avoid intermediate arrays.

Searching:

```
console.log(orders.find((o) => o.total > 100).id, orders.findIndex((o) => o.total > 100));
console.log(orders.findLast((o) => o.customer === 'Ana').id, orders.findLastIndex((o) => o.customer === 'Ana'));
console.log(orders.find((o) => o.total > 1000), orders.findIndex((o) => o.total > 1000));
console.log(orders.some((o) => o.status === 'refunded'), orders.every((o) => o.total > 0), [].every(() => false));
const vals = [1, 2, NaN, 2];
console.log(vals.includes(NaN), vals.indexOf(NaN), vals.indexOf(2), vals.lastIndexOf(2), vals.includes('2'));
```

Output:

```
1 0
3 2
undefined -1
true true true
true -1 1 3 false
```

Question	Method	Not found
Which element matches?	`find` / `findLast`	`undefined`
At what index?	`findIndex` / `findLastIndex`	`-1`
Is this exact value present?	`includes` (handles `NaN`)	`false`
Where is this exact value?	`indexOf` / `lastIndexOf` (strict `===`, never finds `NaN`)	`-1`
Does any element pass?	`some`	`false` (also for an empty array)
Do all elements pass?	`every`	`true` for an empty array ("vacuous truth")

`find`, `some`, `every` **stop early**. `includes` / `indexOf` use `===`-style equality — `includes('2')` misses the number 2, and objects match only by reference; use `some((o) => o.id === id)`.

Flattening:

```
const nested = [1, [2, [3, [4]]]];
console.log(nested.flat(), nested.flat(2), nested.flat(Infinity));
const sentences = ['hello world', 'good morning'];
console.log(sentences.map((s) => s.split(' ')), sentences.flatMap((s) => s.split(' ')));
console.log([1, 2, 3, 4].flatMap((n) => (n % 2 ? [] : [n, n * 10])));   // filter + map in one pass
```

Output:

```
[ 1, 2, [ 3, [ 4 ] ] ] [ 1, 2, 3, [ 4 ] ] [ 1, 2, 3, 4 ]
[ [ 'hello', 'world' ], [ 'good', 'morning' ] ] [ 'hello', 'world', 'good', 'morning' ]
[ 2, 20, 4, 40 ]
```

`flat(depth)` removes `depth` nesting levels (default 1; `Infinity` = all). `flatMap` maps to *arrays* and joins one level — `[]` drops an element, several items expand it — filter + map in one pass.

Iterators and joining: `[...['x', 'y'].entries()]` → `[ [ 0, 'x' ], [ 1, 'y' ] ]` (`keys()` / `values()` give indexes/values); `[1, [2, 3], null, undefined, 4].join('-')` → `'1-2,3---4'` — `join` makes `null` / `undefined` empty and nested arrays comma-joined. `forEach` can't stop early (`return` ends only that call) — use `for…of` + `break`, or `some` / `find`.

Trade-offs: methods vs loops

	Array methods	`for` / `for…of` loops
Readability	Name the intent (`filter`, `map`)	Generic; intent must be read from the body
Mutation risk	Encourage new arrays	Easy to mutate by accident
Early exit	Only `find` / `some` / `every`	`break` / `return` anywhere
`await` inside	Callbacks are not awaited sequentially	`for…of` + `await` runs in order
Performance	Slight overhead per call; intermediate arrays	Fastest; one pass

Methods by default; loops for early exit, sequential `await` (Chapter 12), or measured performance needs.

Real-world examples

- React lists: `items.map((item) => <Row key={item.id} {...item} />)` (React book).
- API cleanup: `rows.filter(isValid).map(normalize)`.
- Cart total: `cart.reduce((sum, i) => sum + i.price * i.qty, 0)`.
- Feature flags (per-user feature switches): `user.roles.some((r) => r === 'admin')`.

In the interview

> **30-second answer:** "`map` transforms, `filter` selects, `reduce` folds to one value, `find` returns the first match, `some` / `every` answer yes/no with short-circuiting. They take `(element, index, array)` callbacks and don't mutate. I always give `reduce` an initial value and avoid `forEach` when I need `break` or sequential `await`."

- *`map` via `reduce`:* `arr.reduce((acc, x, i) => { acc.push(fn(x, i, arr)); return acc; }, [])`.
- *`['1','2','3'].map(parseInt)` → `[1, NaN, NaN]`?* The index is passed as the radix.
- *`find` vs `filter`?* `find` returns the first element (or `undefined`) and stops; `filter` scans all and returns an array.

Self-check

1. `[1, 2, 3].map((x) => x * 2)`? → `[2, 4, 6]`.
2. `[5, 12, 8].find((x) => x > 6)`? → `12`.
3. `[].every(Boolean)`? → `true`.
4. Sum with `reduce`? → `arr.reduce((s, x) => s + x, 0)`.
5. `[[1], [2, [3]]].flat()`? → `[1, 2, [3]]`.

7.6 Sorting correctly

The problem it solves

`sort()` with no arguments converts elements to **strings** and orders by UTF-16 code units — fine for plain ASCII words, **wrong for numbers**.

From `code/04_sorting.mjs`:

```
console.log([10, 9, 1, 100, 25].sort());                    // default: compared as STRINGS
console.log([10, 9, 1, 100, 25].sort((a, b) => a - b));     // ascending numbers
console.log([10, 9, 1, 100, 25].sort((a, b) => b - a));     // descending numbers
console.log([3, undefined, 1, , 2].sort());                 // undefined and holes go last
```

Output:

```
[ 1, 10, 100, 25, 9 ]
[ 1, 9, 10, 25, 100 ]
[ 100, 25, 10, 9, 1 ]
[ 1, 2, 3, undefined, <1 empty item> ]
```

How it works — the comparator contract

A **comparator** `(a, b) => number` returns **negative** (`a` first), **positive** (`b` first) or **0** (keep relative order). `a - b` works for numbers and Dates (subtraction gives ms). It must be **consistent** — a boolean (`a > b`) is a bug: `false`→0, `true`→1, never negative, so results depend on the engine:

```
console.log('boolean comparator ->', [3, 1, 2].sort((a, b) => a > b));
```

Output:

```
boolean comparator -> [ 3, 1, 2 ]
```

Strings, multiple keys and stability

```
const people = [
  { name: 'Zoe', dept: 'eng', age: 31 },
  { name: 'adam', dept: 'ops', age: 25 },
  { name: 'Émile', dept: 'eng', age: 25 },
  { name: 'Bob', dept: 'ops', age: 31 },
];
console.log(people.toSorted((a, b) => a.age - b.age).map((p) => p.name));
console.log(people.map((p) => p.name).toSorted());                              // code-unit order
console.log(people.map((p) => p.name).toSorted((a, b) => a.localeCompare(b)));  // human order

const byDeptAgeName = (a, b) =>
  a.dept.localeCompare(b.dept) || b.age - a.age || a.name.localeCompare(b.name);
console.log(people.toSorted(byDeptAgeName).map((p) => `${p.dept}/${p.age}/${p.name}`));

const tasks = [['write', 2], ['test', 1], ['deploy', 2], ['plan', 1]];
console.log(tasks.toSorted((a, b) => a[1] - b[1]).map((t) => t[0]));
```

Output:

```
[ 'adam', 'Émile', 'Zoe', 'Bob' ]
[ 'Bob', 'Zoe', 'adam', 'Émile' ]
[ 'adam', 'Bob', 'Émile', 'Zoe' ]
[ 'eng/31/Zoe', 'eng/25/Émile', 'ops/31/Bob', 'ops/25/adam' ]
[ 'test', 'plan', 'write', 'deploy' ]
```

- Strings: `localeCompare` (or a reused `Intl.Collator().compare`) — default order puts capitals first, accents last (Chapter 6).
- **Multi-key sort**: chain with `||` — a tie (0) falls through to the next key; `b.age - a.age` sorts descending.
- **Stability**: since ES2019 `sort` is **stable** — equal elements keep their order (`test` before `plan`), so you can sort by a secondary key, then the primary.

Computed keys, booleans, dates

```
const words = ['banana', 'Kiwi', 'apple', 'fig'];
const sortedByLength = words
  .map((w) => ({ w, key: w.length }))
  .sort((a, b) => a.key - b.key || a.w.localeCompare(b.w))
  .map((x) => x.w);
console.log(sortedByLength);
console.log([true, false, true].toSorted((a, b) => Number(a) - Number(b)));
const dates = [new Date('2024-03-01'), new Date('2023-12-25'), new Date('2024-01-15')];
console.log(dates.toSorted((a, b) => a - b).map((d) => d.toISOString().slice(0, 10)));
```

Output:

```
[ 'fig', 'Kiwi', 'apple', 'banana' ]
[ false, true, true ]
[ '2023-12-25', '2024-01-15', '2024-03-01' ]
```

For expensive keys (parsing dates, normalizing text) compute each **once**, sort by it, strip it — "decorate–sort–undecorate" (Schwartzian transform) — instead of recomputing in the comparator O(n log n) times (the usual cost of a good sort).

Shuffling

```
function shuffle(arr, rand = Math.random) {
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(rand() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}
```

Fisher–Yates: from the end, swap each position with a random earlier-or-same one — every ordering equally likely, O(n). `arr.sort(() => Math.random() - 0.5)` is **biased** (inconsistent comparator) and slower. (The demo uses a **seeded** generator — same sequence from the same seed — so its output is reproducible: `[ 1, 2, 5, 6, 3, 4 ]`.)

Complexity

V8 uses TimSort; other engines use merge-sort variants (split, sort parts, merge): **O(n log n)**, stable, fast on partly-sorted data.

In the interview

30-second answer: "Default `sort` compares strings, so numbers need `(a, b) => a - b`. The comparator returns negative, zero or positive; chain keys with `||`; strings use `localeCompare`. `sort` mutates and is stable since ES2019; `toSorted` keeps the original."

- *By `lastName`, then `firstName`?* `(a, b) => a.lastName.localeCompare(b.lastName) || a.firstName.localeCompare(b.firstName)`.
- *Why not `sort(() => Math.random() - 0.5)`?* Biased and O(n log n); use Fisher–Yates.

Self-check

1. `[2, 10, 1].sort()`? → `[1, 10, 2]`.
2. Descending numeric comparator? → `(a, b) => b - a`.
3. "Stable"? → Equal elements keep their original relative order.

7.7 Spread, rest and destructuring with arrays

From `code/05_spread_destructuring.mjs`:

```
const a = [1, 2], b = [3, 4];
console.log([...a, ...b], [0, ...a, 2.5, ...b], Math.max(...b), [...'hi'], [...new Set([1, 1, 2])]);

const [first, second, third = 'default', ...rest] = ['x', 'y'];
console.log(first, second, third, rest);
const [, , keep] = ['skip', 'skip', 'take'];
console.log(keep);
let p = 1, q = 2;
[p, q] = [q, p];
console.log(p, q);
const [[r1c1, r1c2], [r2c1]] = [[1, 2], [3, 4]];
console.log(r1c1, r1c2, r2c1);
const { 0: head, length } = ['h', 'e', 'y'];
console.log(head, length);

const minMax = (arr) => [Math.min(...arr), Math.max(...arr)];
const [lo, hi] = minMax([5, 3, 9]);
console.log(lo, hi);
const sum = (...nums) => nums.reduce((s, n) => s + n, 0);
console.log(sum(1, 2, 3), sum(...[10, 20]));
```

Output:

```
[ 1, 2, 3, 4 ] [ 0, 1, 2, 2.5, 3, 4 ] 4 [ 'h', 'i' ] [ 1, 2 ]
x y default []
take
2 1
1 2 3
h 3
6 30
```

- **Spread** `...arr` expands any **iterable** (array, string, Set, Map, generator) in literals and calls.
- **Array destructuring** assigns by **position**; defaults for `undefined`; commas skip; `...rest` gathers the remainder (always an array).
- `[p, q] = [q, p]` swaps.
- Returning an array to destructure is a light multi-value return — React's `const [state, setState] = useState(0)` (a *hook* giving a component a remembered value plus a setter).
- A **rest parameter** `(...nums)` collects arguments into a real array (replacing `arguments` — Chapter 4).

Copies are shallow; the `fill([])` trap

```
const grid = [[0, 0], [0, 0]];
const shallow = [...grid];
const deep = structuredClone(grid);
shallow[0][0] = 9;
console.log(grid[0][0], deep[0][0]);

const bad = Array(3).fill([]);
bad[0].push('x');
const good = Array.from({ length: 3 }, () => []);
good[0].push('x');
console.log(bad, good);
```

Output:

```
9 0
[ [ 'x' ], [ 'x' ], [ 'x' ] ] [ [ 'x' ], [], [] ]
```

`[...arr]`, `slice()`, `concat()`, `Array.from(arr)` copy only the outer array; `structuredClone` copies deeply. `fill(obj)` puts the **same** object in every slot — `Array.from({ length }, () => newThing)` creates one per slot (the standard 2-D grid builder).

Array-likes

An **array-like** has numeric keys and `length` but no array methods — `arguments`, DOM `NodeList` / `HTMLCollection`, strings. Convert with `Array.from` (or spread if iterable), or borrow a method with `call`:

```
const arrayLike = { 0: 'a', 1: 'b', length: 2 };
console.log(Array.from(arrayLike), Array.prototype.map.call(arrayLike, (s) => s.toUpperCase()));
```

Output:

```
[ 'a', 'b' ] [ 'A', 'B' ]
```

Everyday recipes

```
const range = (start, end, step = 1) => Array.from({ length: Math.ceil((end - start) / step) }, (_, i) => start + i * step);
const unique = (arr) => [...new Set(arr)];
const chunk = (arr, size) => Array.from({ length: Math.ceil(arr.length / size) }, (_, i) => arr.slice(i * size, i * size + size));
const zip = (xs, ys) => xs.map((x, i) => [x, ys[i]]);
const last = (arr) => arr.at(-1);
const arraysEqual = (x, y) => x.length === y.length && x.every((v, i) => Object.is(v, y[i]));
```

Output of the demo calls:

```
[ 0, 1, 2, 3, 4 ] [ 10, 7, 4, 1 ]
[ 3, 1, 2 ]
[ [ 1, 2 ], [ 3, 4 ], [ 5 ] ]
[ [ 'a', 1 ], [ 'b', 2 ] ]
3 undefined
true true
```

The last `true` is `[1, 2] == '1,2'` — loose equality stringifies the array. Never compare arrays with `==` / `===`; compare element by element (`arraysEqual`) or, for simple data, `JSON.stringify` results. Grouping (`Object.groupBy`): Chapter 8.

7.8 Arrays vs objects (vs Map/Set)

When to pick which

Use…	When	Example
Array	Order matters; you access by position; you iterate, sort, filter	A playlist, search results, a queue of jobs
Object	A fixed set of named fields describing one thing	`{ name, email, age }` — a record
Array of objects	A list of records — the most common shape of real data	API response `[{ id, title }, …]`
Object / Map keyed by id	Frequent lookup by a key	`usersById[id]` — O(1) instead of `find` O(n)
Set	Uniqueness and fast "is it in?" checks	Visited URLs, selected ids

From `code/01_array_basics.mjs` :

```
const fruitList = ['Apple', 'Banana', 'Cherry'];                    // ordered, by position
const fruitDetails = {                                              // lookup by name
  apple: { color: 'red', taste: 'sweet' },
  cherry: { color: 'red', taste: 'tart' },
};
console.log(fruitList[1], fruitDetails.cherry.taste);
const fruitRecords = [                                              // most common in real apps
  { name: 'Apple', color: 'red' }, { name: 'Banana', color: 'yellow' },
];
console.log(fruitRecords.find((f) => f.name === 'Banana').color);
```

Output:

```
Banana tart
yellow
```

An ordered list is an array; details looked up by name suit an object keyed by name. Real apps hold an **array of objects** and, when id lookups get frequent, build an index: `Object.fromEntries(list.map((x) =>`

`[x.id, x]))` or `new Map(list.map((x) => [x.id, x]))` (Chapter 8).

Common mistakes

- `arr.includes(x)` / `arr.find(...)` inside a loop over another big array — O(n²); build a `Set` / `Map` first.
- Object with numeric keys as a list — no `length`, subtler ordering, no array methods.
- Array as a dictionary (`arr['name'] = …`) — works (arrays are objects) but `length` and methods ignore it.

7.9 Typed arrays (a short note)

What it is

A **typed array** is a fixed-length array of raw numbers of one machine type (8-bit unsigned integers, 32-bit floats, …) over a contiguous memory block, an `ArrayBuffer`; several typed-array **views** can read the same buffer differently. Used for binary data: files, images, audio, WebGL/WebGPU (GPU graphics APIs), WebSockets (always-open two-way browser–server connections), `fetch` bodies, WebAssembly memory (fast compiled code from C/Rust in the browser), and Node's `Buffer` (a `Uint8Array` subclass).

From `code/06_typed_arrays.mjs`:

```
const buf = new ArrayBuffer(8);                 // 8 raw bytes, all zero
const bytes = new Uint8Array(buf);              // view them as 8 unsigned bytes
const words = new Uint16Array(buf);             // ...or as 4 unsigned 16-bit ints (same memory)
bytes[0] = 0x34; bytes[1] = 0x12;
console.log(bytes, words[0].toString(16), '(little-endian on this machine)');

const u8 = new Uint8Array([255, 256, -1, 3.7]);
const clamped = new Uint8ClampedArray([255, 256, -1, 3.7]);
const i8 = new Int8Array([127, 128]);
console.log(u8, clamped, i8);

const f = new Float64Array([3, 1, 2]);
console.log(f.length, typeof f.push, f.map((x) => x * 2), f.toSorted(), Array.isArray(f));
console.log(Float32Array.of(0.1)[0], new Float16Array([0.1])[0]);

console.log(Array.from(f), [...new TextEncoder().encode('Hi')], new TextDecoder().decode(new Uint8Array([72,
105])));
const view = new DataView(buf);
view.setUint16(2, 0xabcd, false);                                   // big-endian write, e.g. a network protocol
console.log(bytes.slice(2, 4), view.getUint16(2, false).toString(16));
```

Output:

```
Uint8Array(8) [
  52, 18, 0, 0,
   0,  0, 0, 0
] 1234 (little-endian on this machine)
Uint8Array(4) [ 255, 0, 255, 3 ] Uint8ClampedArray(4) [ 255, 255, 0, 4 ] Int8Array(2) [ 127, -128 ]
3 undefined Float64Array(3) [ 6, 2, 4 ] Float64Array(3) [ 1, 2, 3 ] false
0.10000000149011612 0.0999755859375
[ 3, 1, 2 ] [ 72, 105 ] Hi
Uint8Array(2) [ 171, 205 ] abcd
```

Type	Element	Range / precision
`Int8Array` / `Uint8Array` / `Uint8ClampedArray`	1 byte	−128…127 / 0…255 (wraps) / 0…255 (clamps and rounds — used by `<canvas>`, the HTML drawing surface, for pixel data)
`Int16Array` / `Uint16Array`	2 bytes	
`Int32Array` / `Uint32Array`	4 bytes	
`Float16Array` (ES2025) / `Float32Array` / `Float64Array`	2 / 4 / 8 bytes	half / single / double precision
`BigInt64Array` / `BigUint64Array`	8 bytes	64-bit integers as BigInt

vs normal arrays: **fixed length** (no `push` / `pop` / `splice`), values **coerced** to the element type (`256` wraps to `0` in `Uint8Array`, `-1` → `255`), `Array.isArray` is `false`. Byte order (**endianness**) follows the machine; `DataView` chooses it explicitly, as binary formats and network protocols require.

Chapter TL;DR

- Arrays are ordered, zero-indexed lists — and objects (`typeof` → `'object'`; `Array.isArray`). Literals; `Array.from({ length }, fn)` / `Array.of` instead of quirky `new Array`.
- `arr[i]` or `arr.at(-1)`; missing index → `undefined`. `length` is writable; avoid holes.
- `push` / `pop` O(1) at the end; `unshift` / `shift` O(n) at the start; `push` returns the new length.
- **Mutators**: `push pop shift unshift splice sort reverse fill copyWithin`. **Copying twins (ES2023)**: `toSorted toReversed toSpliced with`. Non-mutating: `slice concat map filter flat flatMap`, spread. Never mutate shared state/props.
- `map` transforms, `filter` selects, `reduce` folds (pass the initial value), `find` / `findLast` element, `findIndex` index, `some` / `every` short-circuit, `includes` finds `NaN`, `flat` / `flatMap` flatten. `forEach` can't `break` or await.
- Default `sort` is by string; `(a, b) => a - b` for numbers, `localeCompare` for text, chain keys with `||`. Stable since ES2019. Shuffle with Fisher–Yates.
- Spread copies shallowly; `structuredClone` deep; `Array(n).fill([])` shares one object. Destructure by position; swap with `[a, b] = [b, a]`.
- Arrays for ordered lists, objects for records, Map/Set for lookups and uniqueness. Typed arrays: fixed-length binary numbers over an `ArrayBuffer`.

Chapter 8 — Maps, Sets, Iterators and Generators

Two jobs are so common JavaScript has dedicated collections for them: **looking things up by key** (`Map`) and **keeping values unique** (`Set`). This chapter covers both, their "weak" cousins that avoid memory leaks (memory never freed because something still points at data nobody needs), the **iteration protocols** that make `for…of` , spread and destructuring work, **generators** (the easiest way to make **lazy** sequences — values computed only when asked), **iterator helpers** (ES2025) and `Object.groupBy` / `Map.groupBy` (ES2024).

What you'll learn: `Map` (methods, iteration, conversion, vs objects); `Set` (methods, ES2025 set algebra); `WeakMap` , `WeakSet` , `WeakRef` , `FinalizationRegistry` ; the **iterable/iterator** protocols and `Symbol.iterator` ; **generators** (`function*` , `yield` , `yield*` , two-way communication, async generators); **iterator helpers** and `Iterator.from` ; **grouping**.

Before you start: objects (Chapter 5), arrays (Chapter 7), closures (Chapter 4). One example uses `#private` class fields (Chapter 9); async generators use promises (Chapter 12).

Code is in `code/` ; outputs are real Node 24 output.

8.1 `Map`

What it is

A `Map` (ES2015) is a collection of **key → value** pairs, like an object used as a dictionary (lookup table) but built for it:

- **Keys can be any value** — objects, functions, numbers, `NaN` .
- **Always remembers insertion order.**
- Knows its `size` .
- **Directly iterable** with `for…of` .

How it works — creating a Map

`new Map()` is empty; pass an **iterable of** `[key, value]` **pairs** to fill it. From `code/01_map.mjs` :

```
const stock = new Map([
  ['apples', 500],
  ['bananas', 300],
  ['oranges', 200],
]);
console.log(stock, stock.size);
```

Output:

```
Map(3) { 'apples' => 500, 'bananas' => 300, 'oranges' => 200 } 3
```

Properties and methods

Member	What it does	Returns
`size` (property)	Number of entries	number
`set(key, value)`	Adds a new entry or **updates** an existing key	the map itself (so calls chain)
`get(key)`	Looks up a key	the value, or `undefined` if absent
`has(key)`	Checks whether the key exists	boolean
`delete(key)`	Removes one entry	`true` if something was removed, else `false`
`clear()`	Removes all entries	`undefined`
`keys()` / `values()` / `entries()`	Iterators over keys / values / `[key, value]` pairs, in insertion order	iterator
`forEach((value, key, map) => …, thisArg?)`	Calls a function per entry, in insertion order	`undefined`
`[Symbol.iterator]()`	Same as `entries()` — why `for (const [k, v] of map)` works	iterator
`Map.groupBy(items, fn)` (static, ES2024)	Builds a Map of groups (§8.7)	Map

```
const books = new Map();
books.set('1984', 'George Orwell').set('Dune', 'Frank Herbert');   // set returns the map -> chainable
console.log(books.get('1984'), books.get('Missing'), books.has('Dune'));
books.set('Dune', 'F. Herbert');                                   // same key -> update
console.log(books.size, books.delete('Dune'), books.delete('Dune'), books.size);
```

Output:

```
George Orwell undefined true
2 true false 1
```

Keys of any type

Keys compare by **SameValueZero**: `===` except `NaN` equals `NaN` (and `+0` equals `-0`). Object keys match **by reference**.

```
const m = new Map();
const objKey = { id: 1 };
const fnKey = () => {};
m.set(objKey, 'object as key').set(fnKey, 'function as key').set(1, 'number 1').set('1', 'string 1').set(NaN, 'NaN works');
console.log(m.get(objKey), m.get(fnKey), m.get(1), m.get('1'), m.get(NaN), m.get({ id: 1 }));

const o = {};
o[objKey] = 'x'; o[1] = 'number'; o['1'] = 'string overwrites';
console.log(Object.keys(o), o);
```

Output:

```
object as key function as key number 1 string 1 NaN works undefined
[ '1', '[object Object]' ] { '1': 'string overwrites', '[object Object]': 'x' }
```

A plain object stringifies keys: every object key becomes `'[object Object]'` (one shared slot) and `1` / `'1'` collide. The Map keeps them distinct.

Iterating and converting

```
for (const [fruit, qty] of stock) console.log(fruit, qty);          // apples 500 / bananas 300 / oranges 200
console.log([...stock.keys()], [...stock.values()], [...stock.entries()][0]);
stock.forEach((value, key) => process.stdout.write(`${key}=${value} `));

const asObject = Object.fromEntries(stock);
const backToMap = new Map(Object.entries(asObject));
console.log(asObject, backToMap.get('apples'), JSON.stringify(stock), JSON.stringify([...stock]));
```

Output (after the three `for…of` lines):

```
[ 'apples', 'bananas', 'oranges' ] [ 500, 300, 200 ] [ 'apples', 500 ]
apples=500 bananas=300 oranges=200
{ apples: 500, bananas: 300, oranges: 200 } 500 {} [["apples",500],["bananas",300],["oranges",200]]
```

- `forEach` callback order: **value, then key** (like arrays' `(element, index)`).
- `Object.fromEntries(map)` / `new Map(Object.entries(obj))` convert (sensible for string keys).
- **`JSON.stringify(map)` gives `{}` — serialize** (turn into text for saving/sending) `[...map]` or `Object.fromEntries(map)`.

Map vs Object

Feature	Map	Plain object {}
Key types	**Any value** (objects, functions, numbers, `NaN`)	Strings and symbols only (others are converted to strings)
Order	Always insertion order	Defined, but integer-like keys come first in ascending order, then strings by insertion (Ch 5)
Size	`map.size`, O(1) (instant, whatever the size)	`Object.keys(obj).length`, O(n) (time grows with the number of keys)
Iteration	Directly iterable (`for…of`)	Via `Object.keys/values/entries`
Accidental keys	None — starts empty	Inherits from `Object.prototype` (`'toString' in {}`); a key named `__proto__` is special. `Object.create(null)` avoids this
Frequent add/delete	Optimised for it	`delete` can make objects slower in engines
JSON	Not directly	Native
Syntax	`map.get(k)` / `map.set(k, v)`	`obj.k` / `obj[k] = v`, literals, destructuring
Best for	Dynamic dictionaries, caches (stores of computed results for reuse), counting, non-string keys	Fixed-shape records (`{ id, name }`), config, JSON data

Real tasks

```
const text = 'the cat and the hat and the bat';
const freq = new Map();
for (const w of text.split(' ')) freq.set(w, (freq.get(w) ?? 0) + 1);
console.log(freq, [...freq].toSorted((a, b) => b[1] - a[1])[0]);

const users = [{ id: 'u1', name: 'Ana' }, { id: 'u2', name: 'Ben' }];
const byId = new Map(users.map((u) => [u.id, u]));
console.log(byId.get('u2').name);
```

Output:

```
Map(5) { 'the' => 3, 'cat' => 1, 'and' => 2, 'hat' => 1, 'bat' => 1 } [ 'the', 3 ]
Ben
```

The **count pattern** `map.set(k, (map.get(k) ?? 0) + 1)` and **index-by-id pattern** `new Map(list.map((x) => [x.id, x]))` turn O(n) searches into O(1) lookups — interview staples (Chapter 17).

Insertion order plus cheap delete/re-insert make a compact **LRU cache** (least-recently-used: when full, evict the entry unused longest — how browser caches, CDNs (servers worldwide keeping copies near users — System Design Fundamentals) and database buffer pools (in-memory copies of recent disk pages) decide what to drop):

```
class LRU {
  #max; #map = new Map();
  constructor(max) { this.#max = max; }
  get(k) {
    if (!this.#map.has(k)) return undefined;
    const v = this.#map.get(k); this.#map.delete(k); this.#map.set(k, v); return v;
  }
  set(k, v) {
    this.#map.delete(k); this.#map.set(k, v);
    if (this.#map.size > this.#max) this.#map.delete(this.#map.keys().next().value); // oldest
  }
  keys() { return [...this.#map.keys()]; }
}
const cache = new LRU(2);
cache.set('a', 1); cache.set('b', 2); cache.get('a'); cache.set('c', 3);
console.log(cache.keys());   // [ 'a', 'c' ]  — 'b' was least recently used
```

(The LLD (low-level design) handbook builds a full LRU with a doubly linked list — each item points to both neighbours: `../../../lld/`.)

Coming next — upsert (ES2026). `map.getOrInsert(key, default)` / `map.getOrInsertComputed(key, fn)` return the existing value or insert one in a single call (replacing `has` / `get` / `set`). In the ES2026 spec but **not in Node 24** — the demo feature-detects them and prints `false`.

Common mistakes

- `map['k'] = 1` sets an ordinary *property*; `get('k')` won't see it, `size` stays 0.
- Expecting `map.get({ id: 1 })` to find an entry stored under a different look-alike object.
- `JSON.stringify(map)` → `{}`.
- A Map for a small fixed record — an object is simpler and JSON/destructuring-friendly.

In the interview

> **30-second answer:** "Map for dynamic key–value data: any key type, insertion order, `size`, direct iteration, optimised for frequent adds/deletes. Objects for fixed-shape records and JSON. Keys compare by SameValueZero, so objects match by reference."

- *LRU cache in JS?* A Map: on `get` delete and re-insert; on overflow delete `map.keys().next().value`.
- *Why not an object as a hash map* (table finding entries by hashing the key)? String-only keys, inherited properties, `__proto__` pitfalls, no `size`, slower churn (frequent add/remove).

Self-check

1. `new Map([[1, 'a']]).get('1')`? → `undefined`.
2. `map.set` returns? → The map.
3. `map.forEach` callback: key or value first? → Value.
4. Map as JSON? → `JSON.stringify([...map])` or `Object.fromEntries(map)`.

8.2 `Set`

What it is

A `Set` (ES2015) holds values (primitives or object references), each **at most once**; re-adding does nothing. Like Map: insertion order, `size`, **SameValueZero** equality.

How it works

```
const colors = new Set(['red', 'green', 'blue', 'red']);
console.log(colors, colors.size);

const numbers = new Set();
numbers.add(1).add(2).add(2);                    // add is chainable; duplicates ignored
console.log(numbers.has(1), numbers.has(3), numbers.size, numbers.delete(1), numbers.delete(1), numbers);
numbers.clear();
console.log(numbers.size);
```

Output (from `code/02_set.mjs`):

```
Set(3) { 'red', 'green', 'blue' } 3
true false 2 true false Set(1) { 2 }
0
```

`new Set(iterable)` keeps the first occurrence of each value from any iterable.

Member	What it does	Returns
`size` (property)	Number of values	number
`add(value)`	Adds a value if not already present	the set (chainable)
`has(value)`	Membership test — **O(1)** on average	boolean
`delete(value)`	Removes a value	`true` if it was present
`clear()`	Removes everything	`undefined`
`values()` / `keys()`	Iterator over values (`keys` is an alias kept for symmetry with Map)	iterator
`entries()`	Iterator of `[value, value]` pairs (again, for Map symmetry)	iterator
`forEach((value, sameValue, set) => …)`	Calls a function per value, insertion order	`undefined`
`union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`, `isSupersetOf`, `isDisjointFrom`	Set algebra (**ES2025**)	new Set / boolean

Iteration and equality

```
const s = new Set([1, 2, 3]);
s.forEach((value) => process.stdout.write(value + ' '));   // 1 2 3
console.log([...s.values()], [...s.keys()], [...s.entries()]);

const mixed = new Set([NaN, NaN, 0, -0, '1', 1, { a: 1 }, { a: 1 }]);
console.log(mixed.size, mixed);
```

Output (after the `forEach` line):

```
[ 1, 2, 3 ] [ 1, 2, 3 ] [ [ 1, 1 ], [ 2, 2 ], [ 3, 3 ] ]
6 Set(6) { NaN, 0, '1', 1, { a: 1 }, { a: 1 } }
```

`NaN` stored once, `0` / `-0` one value, `'1'` ≠ `1`, and two separate `{ a: 1 }` objects are **both** kept (by reference). Dedupe objects by content by keying an id in a Map.

Classic uses

```
console.log([...new Set([3, 1, 3, 2, 1])]);           // [ 3, 1, 2 ] — dedupe, keeping first-seen order
const visited = new Set();
for (const url of ['/a', '/b', '/a', '/c', '/b']) {
  if (visited.has(url)) continue;
  visited.add(url);
}
console.log([...visited]);                            // [ '/a', '/b', '/c' ]
```

The "visited" set drives web crawlers (follow links to fetch pages), graph searches (BFS/DFS — breadth-first and depth-first exploration of connected nodes) and "don't process twice" logic.

Set algebra (ES2025)

```
const a = new Set([1, 2, 3, 4]);
const b = new Set([3, 4, 5]);
console.log('union              ', a.union(b));
console.log('intersection       ', a.intersection(b));
console.log('difference a-b     ', a.difference(b));
console.log('symmetricDifference', a.symmetricDifference(b));
console.log('isSubsetOf         ', new Set([3, 4]).isSubsetOf(a), a.isSupersetOf(new Set([1, 2])),
a.isDisjointFrom(new Set([9])));
```

Output:

```
union              Set(5) { 1, 2, 3, 4, 5 }
intersection       Set(2) { 3, 4 }
difference a-b     Set(2) { 1, 2 }
symmetricDifference Set(3) { 1, 2, 5 }
isSubsetOf         true true true
```

Method	Meaning	Returns
`a.union(b)`	In A **or** B	new Set
`a.intersection(b)`	In A **and** B	new Set
`a.difference(b)`	In A but **not** B	new Set
`a.symmetricDifference(b)`	In exactly one of them	new Set
`a.isSubsetOf(b)`	Every element of A is in B	boolean
`a.isSupersetOf(b)`	Every element of B is in A	boolean
`a.isDisjointFrom(b)`	No shared elements	boolean

Standardized in **ES2025**; in all current browsers and **Node 22+**. None mutates `a` or `b`. The argument may be any **set-like** (has `size`, `has()`, `keys()`), e.g. a Map (its keys):

```
const allowed = new Map([['read', true], ['write', false]]);
console.log(new Set(['read', 'delete']).intersection(allowed));   // Set(1) { 'read' }
```

Older runtimes: `new Set([...a].filter((x) => b.has(x)))` (intersection), `new Set([...a, ...b])` (union), `new Set([...a].filter((x) => !b.has(x)))` (difference).

Performance: `Set.has` vs `Array.includes`

`includes` scans — **O(n)**; `has` uses **hashing** (value → number pointing straight at its slot) — **O(1)** average. The demo (2,000 lookups in 200,000 numbers) confirms `Set.has` is faster (it prints only the comparison; timings vary). Repeated membership tests → build a Set once.

Common mistakes

- Expecting a Set to dedupe equal-content objects.
- `set[0]` — no indexes; spread or iterate.
- `new Set('hello')` iterates the string: `Set { 'h', 'e', 'l', 'o' }`.
- `JSON.stringify(set)` → `{}`; send `[...set]`.

In the interview

> **30-second answer:** "A Set stores unique values by SameValueZero, keeps insertion order and gives O(1) average `has`. I use it to dedupe (`[...new Set(arr)]`), track visited nodes, and — since ES2025 — for set algebra with `union`, `intersection`, `difference` and friends."

- *Dedupe objects by id?* `[...new Map(arr.map((o) => [o.id, o])).values()]` (keeps the last per id).
- *Intersection of two arrays in O(n + m)?* `const s = new Set(b); a.filter((x) => s.has(x))`.

Self-check

1. `new Set([1, '1', 1]).size`? → 2.
2. `set.add(x)` returns? → The set.
3. `new Set([1,2]).union(new Set([2,3]))`? → `Set {1, 2, 3}`.
4. `set.has` vs `arr.includes` in a loop? → O(1) vs O(n) per check.

8.3 `WeakMap`, `WeakSet`, `WeakRef` and `FinalizationRegistry`

What it is

The **garbage collector** (GC) frees an object once nothing **reachable** refers to it. A Map/Set holds **strong** references, keeping stored objects alive forever — a classic leak. **Weak** collections hold keys **weakly**: the entry doesn't prevent collection, and disappears when the object is collected.

How they work

	`WeakMap`	`WeakSet`	`WeakRef` (ES2021)	`FinalizationRegistry` (ES2021)
Holds	key (weakly) → value	values (weakly)	one target (weakly)	callbacks for targets
Allowed keys/targets	Objects, and non-registered symbols (ES2023)	Same	Same	Same
Methods	`set`, `get`, `has`, `delete`	`add`, `has`, `delete`	`deref()` → target or `undefined`	`register`, `unregister`
`size`, iteration, `clear`	**No**	**No**	—	—

No `size`/iteration because contents change whenever the (unpredictable) GC runs — exposing them would make runs differ.

From `code/03_weak_collections.mjs`:

```
// 03_weak_collections.mjs (abridged — each output line matches one use)
const metadata = new WeakMap();                // 1) side table keyed by an object
const button = { tag: 'button' };              // imagine a DOM element
metadata.set(button, { clicks: 0, firstSeen: 'now' });
metadata.get(button).clicks++;

const areaCache = new WeakMap();               // 2) per-object memo cache
function area(shape) {
  if (!areaCache.has(shape)) areaCache.set(shape, shape.w * shape.h);
  return areaCache.get(shape);
}

const processed = new WeakSet();               // 3) "seen" tagging
try { new WeakMap().set('str', 1); } catch (e) { console.log('primitive key ->', e.constructor.name); }
const sym = Symbol('local');
console.log(new WeakMap([[sym, 'symbol key ok (ES2023)']]).get(sym));
try { new WeakSet().add(Symbol.for('registered')); } catch (e) { console.log('registered symbol ->',
e.constructor.name); }
```

Output (full run):

```
{ clicks: 1, firstSeen: 'now' } true false
12 12 computed 1 time(s)
processed skipped
primitive key -> TypeError
false undefined undefined
symbol key ok (ES2023)
registered symbol -> TypeError
```

Three standard uses:

1. **Side tables** — attach data to objects you don't own (DOM elements, library objects) without adding properties; it goes away with the object.
2. **Per-object memoization caches** (remembering computed results) — no manual **eviction** (removing old entries).
3. **Tagging/branding** with a WeakSet — "have I already processed/validated/created this object?" (how some libraries did private state before `#private`).

Primitives are rejected because they're never collected; `Symbol.for` (global registry) symbols likewise.

`WeakRef` and `FinalizationRegistry`

```
let bigObject = { payload: 'x'.repeat(10) };
const ref = new WeakRef(bigObject);
console.log(ref.deref()?.payload.length);       // still reachable -> 10
bigObject = null;                               // after GC, deref() MAY return undefined

const registry = new FinalizationRegistry((heldValue) => console.log('cleaned up', heldValue));
registry.register({ temp: true }, 'temp-object');
```

A `WeakRef` points at an object *without* keeping it alive (`deref()` → object or `undefined`). A `FinalizationRegistry` asks for a callback some time after a registered object is collected. Both serve caches of large objects and releasing external resources, but timing is **unspecified** (late or never, e.g. on exit) — never put essential logic (saving data, closing files) there; the TC39 proposal itself advises avoiding them where possible.

When not to use it

- You need to list, count or iterate → `Map` / `Set` with manual removal.
- String/number keys → weak collections can't hold them.

In the interview

- *Map vs WeakMap?* WeakMap keys must be objects (or non-registered symbols), held weakly — entries vanish on GC — so no `size`, iteration or `clear`.

- *Real use?* Per-DOM-node or per-request caches; private per-instance data.

Self-check

1. String key in a WeakMap? → `TypeError`.
2. Why can't you iterate a WeakSet? → Contents depend on GC timing.
3. `weakRef.deref()` after collection? → `undefined`.

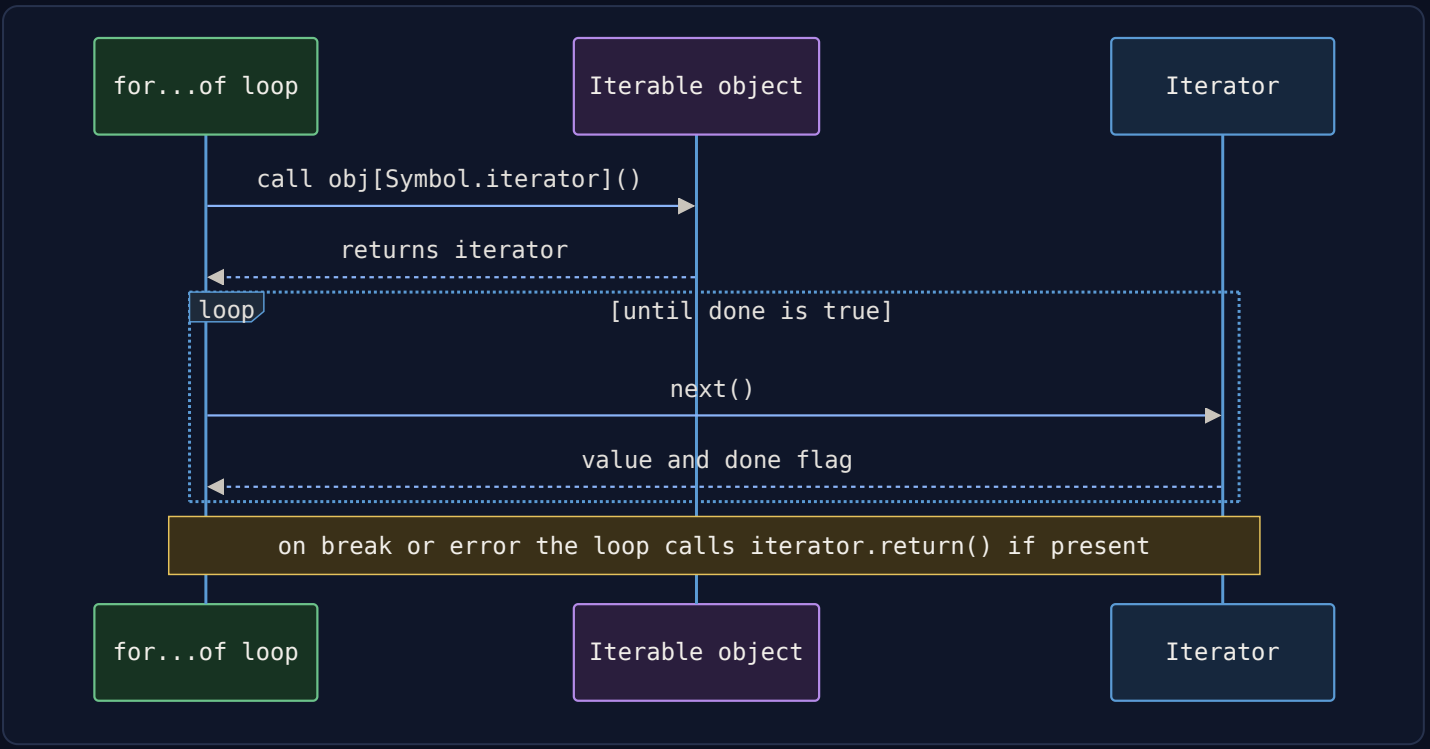
8.4 Iterators and the iterable protocol

What it is

Two **protocols** (agreed object shapes, ES2015) make `for…of`, spread, destructuring, `Array.from`, `new Map(…)`, `new Set(…)`, `Promise.all` etc. work on *any* collection:

- An **iterator** has `next()` returning `{ value, done }`; `done: true` ends the sequence.
- An **iterable** has a method under `Symbol.iterator` returning a fresh iterator.

`Symbol.iterator` is a **well-known symbol** — a built-in unique key the language looks for, which can't clash with ordinary property names. (Iterable = a book; iterator = a forward-only bookmark — open the book many times, each with a new bookmark.)



How it works — driving it by hand

From `code/04_iterators.mjs`:

```
const it = ['a', 'b'][Symbol.iterator]();
console.log(it.next(), it.next(), it.next());

const isIterable = (x) => x != null && typeof x[Symbol.iterator] === 'function';
console.log([['array', [1]], ['string', 'hi'], ['Map', new Map()], ['Set', new Set()], ['object', {}], ['number',
5]]
  .map(([n, v]) => `${n}:${isIterable(v)}`).join(' '));
```

Output:

```
{ value: 'a', done: false } { value: 'b', done: false } { value: undefined, done: true }
array:true string:true Map:true Set:true object:false number:false
```

Built-in iterables: **arrays, strings, Maps, Sets, typed arrays,** `arguments`, **DOM** `NodeList`**s, generator objects**, and the `keys()` / `values()` / `entries()` iterators. Plain objects are **not** — hence `for (const [k, v] of Object.entries(obj))`.

Making your own iterable

```
const countdown = {
  from: 3,
  [Symbol.iterator]() {
    let current = this.from;
    return {
      next: () => (current > 0 ? { value: current--, done: false } : { value: undefined, done: true }),
      [Symbol.iterator]() { return this; },           // iterators are usually iterable too
    };
  },
};
console.log([...countdown], Array.from(countdown), Math.max(...countdown));
for (const n of countdown) process.stdout.write(n + ' ');
console.log('liftoff');
const [first, ...others] = countdown;
console.log(first, others, new Set(countdown).size, new Map([[1, 'x']]).size);
```

Output:

```
[ 3, 2, 1 ] [ 3, 2, 1 ] 3
3 2 1 liftoff
3 [ 2, 1 ] 3 1
```

`[Symbol.iterator]()` is a **computed method name** (Chapter 5); each call creates a new iterator with its own `current` (closure), so the object is re-iterable; every consumer now works. A **generator** does the same in two lines (the demo's `Range` class uses `*[Symbol.iterator]() { … yield v; }` and prints `[ 0, 3, 6, 9 ]` — §8.5).

Early exit and `return()`

A consumer stopping before `done` (`break`, `return`, exception, partial destructuring) calls the iterator's optional `return()` — the place for cleanup (closing a file, releasing a **lock**, i.e. an exclusive-use flag):

```
function resource() {
  let i = 0;
  return {
    [Symbol.iterator]() { return this; },
    next() { i++; return { value: i, done: i > 5 }; },
    return() { console.log('  cleanup after', i, 'items'); return { done: true }; },
  };
}
for (const x of resource()) { if (x === 2) break; }
const [one] = resource();
console.log('first item was', one);
```

Output:

```
  cleanup after 2 items
  cleanup after 1 items
first item was 1
```

Iterators are single-use

```
const arr = [1, 2];
const once = arr.values();
console.log([...once], [...once], [...arr], [...arr]);   // [ 1, 2 ] [] [ 1, 2 ] [ 1, 2 ]
```

An **iterator** is exhausted after one pass; an **iterable** hands out a fresh one each time. Passing an iterator to code that loops twice is a subtle bug.

Everything that consumes iterables

Consumer	Example
`for…of`	`for (const x of set)`
Spread in arrays and calls	`[...map]` , `Math.max(...set)`
Array destructuring	`const [a, b] = iterable`
`Array.from(iterable, mapFn?)`	`Array.from(map.values())`
Collection constructors	`new Map(pairs)` , `new Set(values)` , `new WeakMap(pairs)`
`Object.fromEntries(pairs)`	`Object.fromEntries(map)`
Promise combinators (functions that combine many promises into one)	`Promise.all(iterable)` , `allSettled` , `race` , `any`
`yield*`	inside generators
Grouping	`Object.groupBy(iterable, fn)`

In the interview

30-second answer: "An iterable has `[Symbol.iterator]()` returning an iterator; an iterator has `next()` returning `{ value, done }` . `for…of` , spread, destructuring, `Array.from` and Map/Set constructors all use this protocol, so implementing it makes any object work with them. Iterators may define `return()` for cleanup on early exit."

- *Why isn't a plain object iterable?* Keys, values and entries are all plausible, so you choose via `Object.keys/values/entries` .
- *`for…in` vs `for…of` ?* `for…in` walks enumerable **keys** (incl. inherited); `for…of` walks **values** of an iterable (Chapter 3).

Self-check

1. `next()` returns? → `{ value, done }` .
2. Why does `[...{a: 1}]` throw? → Plain objects aren't iterable.
3. When is `return()` called? → On early stop (break, throw, partial destructuring).

8.5 Generators

What it is

A **generator function** (`function*` , ES2015) can **pause** and **resume**. Calling it runs nothing; it returns a **generator object** — both iterator and iterable. Each `next()` runs the body to the next `yield` , which emits a value and pauses there.

How it works

From `code/05_generators.mjs` :

```
function* threeSteps() {
  console.log('  start');
  yield 1;
  console.log('  after first yield');
  yield 2;
  console.log('  after second yield');
  return 3;                                   // return value is NOT included by for...of / spread
}
const gen = threeSteps();                     // nothing runs yet
console.log('created');
console.log(gen.next());
console.log(gen.next());
console.log(gen.next());
console.log(gen.next());
console.log([...threeSteps()]);
```

Output:

```
created
  start
{ value: 1, done: false }
  after first yield
{ value: 2, done: false }
  after second yield
{ value: 3, done: true }
{ value: undefined, done: true }
  start
  after first yield
  after second yield
[ 1, 2 ]
```

1. `threeSteps()` returns a generator; **no body code runs** (`created` prints first).
2. First `next()` runs to `yield 1` → `{ value: 1, done: false }`; each further `next()` resumes after the last `yield`.
3. `return 3` → `{ value: 3, done: true }`; afterwards always `{ value: undefined, done: true }`.
4. `for…of`/spread stop at `done: true`, **ignoring the return value** — `[1, 2]`.

Forms: `function* name() {}`, `const g = function* () {}`, methods `*name() {}`, `*[Symbol.iterator]() {}`. **No arrow generators.**

Lazy and infinite sequences

Values are produced on request, so **infinite** sequences are safe if the consumer stops.

```
function* naturals() { let n = 1; while (true) yield n++; }
function* take(iterable, n) {
  if (n <= 0) return;
  for (const x of iterable) { yield x; if (--n <= 0) return; }
}
console.log([...take(naturals(), 5)]);

function* fibonacci() { let [a, b] = [0, 1]; while (true) { yield a; [a, b] = [b, a + b]; } }
console.log([...take(fibonacci(), 10)]);

function* map(iter, fn) { for (const x of iter) yield fn(x); }
function* filter(iter, pred) { for (const x of iter) if (pred(x)) yield x; }
const firstSquaresOfOdds = take(map(filter(naturals(), (n) => n % 2), (n) => n * n), 4);
console.log([...firstSquaresOfOdds]);
```

Output:

```
[ 1, 2, 3, 4, 5 ]
[
  0, 1,  1,  2,  3,
  5, 8, 13, 21, 34
]
[ 1, 9, 25, 49 ]
```

The pipeline moves **one value at a time** through `filter → map → take` with no intermediate arrays — "lazy evaluation", like Python generators, Java streams or Unix pipes; how you'd process a huge log file line by line. §8.6 shows the built-in version.

Delegation with `yield*` and recursion

```
function* inner() { yield 'b'; yield 'c'; return 'inner-done'; }
function* outer() { yield 'a'; const r = yield* inner(); yield r; yield* [1, 2]; }
console.log([...outer()]);

const tree = { value: 1, children: [{ value: 2, children: [{ value: 4, children: [] }] }, { value: 3, children: [] }] };
function* depthFirst(node) { yield node.value; for (const c of node.children) yield* depthFirst(c); }
console.log([...depthFirst(tree)]);
```

Output:

```
[ 'a', 'b', 'c', 'inner-done', 1, 2 ]
[ 1, 2, 4, 3 ]
```

`yield* iterable` yields every value of another iterable as if yielded here, and evaluates to that generator's **return value**. With recursion, tree/graph **traversal** (visiting every node) reads like its definition.

Two-way communication: `next(value)`, `return()`, `throw()`

```
function* runningAverage() {
  let sum = 0, count = 0, avg;
  while (true) {
    const value = yield avg;
    sum += value; count++; avg = sum / count;
  }
}
const avg = runningAverage();
avg.next();                                  // prime: run to the first yield
console.log(avg.next(10).value, avg.next(20).value, avg.next(60).value);

function* guarded() {
  try { yield 1; yield 2; } finally { console.log('  finally ran'); }
}
const g1 = guarded(); g1.next();
console.log(g1.return('stopped'), g1.next());
const g2 = guarded(); g2.next();
try { g2.throw(new Error('boom')); } catch (e) { console.log('caught outside:', e.message); }
```

Output:

```
10 15 30
  finally ran
{ value: 'stopped', done: true } { value: undefined, done: true }
  finally ran
caught outside: boom
```

- **`gen.next(x)`** resumes and makes the paused `yield` evaluate to `x`. The first `next()` only runs to the first `yield` (its argument is ignored) — it "primes".
- **`gen.return(v)`** finishes as if `return v` happened at the paused `yield`; `finally` still runs.
- **`gen.throw(err)`** throws at the paused `yield`; uncaught, it propagates to the caller.

This channel drives **redux-saga** (a Redux add-on for side effects like network calls) and emulated `async` / `await` before ES2017 (the `co` library ran generators yielding promises — Chapter 12).

More uses: ID makers, state machines, async paging

```
function* idMaker(prefix) { let i = 0; while (true) yield `${prefix}-${++i}`; }
const ids = idMaker('order');
console.log(ids.next().value, ids.next().value);

function* trafficLight() { while (true) { yield 'green'; yield 'yellow'; yield 'red'; } }
const light = trafficLight();
console.log(Array.from({ length: 5 }, () => light.next().value));

async function* fetchPages(totalPages) {
  for (let page = 1; page <= totalPages; page++) {
    await new Promise((r) => setTimeout(r, 5));          // pretend network call
    yield { page, items: [`item${page * 2 - 1}`, `item${page * 2}`] };
  }
}
for await (const { page, items } of fetchPages(3)) console.log('page', page, items);
console.log(await Array.fromAsync(fetchPages(2), (p) => p.page));
```

Output:

```
order-1 order-2
[ 'green', 'yellow', 'red', 'green', 'yellow' ]
page 1 [ 'item1', 'item2' ]
page 2 [ 'item3', 'item4' ]
page 3 [ 'item5', 'item6' ]
[ 1, 2 ]
```

The traffic light is a tiny **state machine** (always in one of a fixed set of states, moving by fixed rules). An **async generator** (`async function*`, ES2018) can `await` and is consumed with `for await…of` — the natural shape for paginated APIs (results one page at a time), streams read chunk by chunk (Node readable streams are async iterables — Chapter 15), or polling. `Array.fromAsync` (ES2026, in Node 24) collects an async iterable.

Trade-offs

	Generator	Array
Memory	One value at a time	All values at once
Infinite sequences	Yes	No
Random access `[i]`, `length`	No	Yes
Reusable	No — create a new generator each time	Yes
Speed for small data	Slightly slower (pause/resume overhead)	Faster

When not to use it

Small, finite data you'll index, sort or reuse → an array. Generators shine for large, lazy, infinite or step-by-step data and for `[Symbol.iterator]`.

Common mistakes

- Expecting `gen()` alone to run the body.
- Expecting the `return` value in `for…of` /spread.
- Reusing an exhausted generator.
- Spreading an infinite generator without `take` → never ends (out of memory).
- Not priming before `next(x)`.

In the interview

> **30-second answer:** "A `function*` returns a generator object — iterator and iterable. `yield` pauses and emits; `next()` resumes. That gives lazy/infinite sequences, two-line `Symbol.iterator` implementations, and two-way communication via `next(x)`, `return`, `throw`. Async generators with `for await` handle paged or streamed data."

- *`range(start, end)` generator:* `function* range(s, e) { for (let i = s; i < e; i++) yield i; }`
- *`yield*`?* Delegation: yields all values of another iterable, evaluates to its return value.
- *Generators vs async/await?* Async functions behave like generators yielding promises, resumed when each **settles** (finishes, success or error).

Self-check

1. `[...(function* () { yield 1; return 2; })()]`? → `[1]`.
2. First `next()`? → Runs to the first `yield` (or end).
3. Arrow generators? → No.
4. Consume an async generator? → `for await (const x of gen())` or `Array.fromAsync`.

8.6 Iterator helpers (ES2025)

What it is

Iterator helpers add array-like methods — `map`, `filter`, `take`, `drop`, `flatMap`, `reduce`, `toArray`, `forEach`, `some`, `every`, `find` — to **iterators** (via a new global `Iterator` class all built-in iterators inherit from). Those returning iterators are **lazy**. **ES2025**; Node 22+ and all current browsers.

From `code/06_iterator_helpers.mjs`:

```
function* naturals() { let n = 1; while (true) yield n++; }

const result = naturals()
  .filter((n) => n % 3 === 0)
  .map((n) => n * n)
  .take(4)
  .toArray();
console.log(result);

let pulled = 0;
const lazy = naturals().map((n) => { pulled++; return n * 10; }).take(3);
console.log('pulled before use:', pulled);
console.log([...lazy], 'pulled after use:', pulled);

console.log(naturals().drop(5).take(3).toArray());
console.log(['a b', 'c'].values().flatMap((s) => s.split(' ')).toArray());
console.log(naturals().take(5).reduce((sum, n) => sum + n, 0));
console.log(naturals().some((n) => n > 1000), naturals().take(3).every((n) => n < 5), naturals().find((n) => n * n
> 50));

const prices = new Map([['tea', 3], ['cake', 5], ['pie', 7]]);
console.log(prices.entries().filter(([, p]) => p > 4).map(([k]) => k).toArray());

const plain = { i: 0, next() { return this.i < 3 ? { value: this.i++, done: false } : { value: undefined, done:
true }; } };
console.log(Iterator.from(plain).map((x) => x + 100).toArray());

const arr = [1, 2, 3, 4];
console.log(typeof arr.take, arr.values().take(2).toArray());
```

Output:

```
[ 9, 36, 81, 144 ]
pulled before use: 0
[ 10, 20, 30 ] pulled after use: 3
[ 6, 7, 8 ]
[ 'a', 'b', 'c' ]
15
true true 8
[ 'cake', 'pie' ]
[ 100, 101, 102 ]
undefined [ 1, 2 ]
```

Helper	Lazy?	Returns
`map(fn)`, `filter(fn)`, `flatMap(fn)`	Yes	iterator
`take(n)`, `drop(n)`	Yes	iterator
`toArray()`	No — consumes	array
`reduce(fn, init?)`, `forEach(fn)`	No — consumes	value / `undefined`
`some(fn)`, `every(fn)`, `find(fn)`	No — consumes, **short-circuits**	boolean / value
`Iterator.from(obj)` (static)	—	wraps any iterator or iterable so helpers are available

- Works on **infinite** generators because `take(4)` stops pulling; `pulled` shows `map` ran only 3 times.
- Map/Set iterators get the helpers too — no spreading first.
- **Arrays are iterables, not iterators**: `arr.take` is `undefined` — use `arr.values()`. For ordinary arrays, array methods (Chapter 7) are simpler; helpers suit large, lazy or infinite data.
- `Iterator.concat` (ES2026) isn't in Node 24 (the demo feature-detects it); **async** iterator helpers are still a separate proposal.

8.7 Grouping: `Object.groupBy` and `Map.groupBy` (ES2024)

What it is

Static methods taking an iterable and a **key function**, putting each item in the bucket named by its key. **ES2024**; Node 21+ and all current browsers. (First proposed as `Array.prototype.group`, which broke existing websites, so they became static.)

- `Object.groupBy(items, fn)` → a **null-prototype object** with string keys.
- `Map.groupBy(items, fn)` → a **Map**, keys keep their type.

From `code/07_groupby.mjs`:

```
const inventory = [
  { name: 'laptop', type: 'electronics', quantity: 4 },
  { name: 'desk', type: 'furniture', quantity: 0 },
  { name: 'monitor', type: 'electronics', quantity: 12 },
  { name: 'chair', type: 'furniture', quantity: 30 },
  { name: 'stapler', type: 'office', quantity: 5 },
];

const byType = Object.groupBy(inventory, (item) => item.type);
console.log(Object.keys(byType), byType.furniture.map((i) => i.name));
console.log(Object.getPrototypeOf(byType) === null);

const byStock = Object.groupBy(inventory, ({ quantity }) => (quantity > 5 ? 'ok' : 'restock'));
console.log(Object.fromEntries(Object.entries(byStock).map(([k, v]) => [k, v.map((i) => i.name)])));

const restock = { label: 'restock' }, sufficient = { label: 'sufficient' };
const byObjectKey = Map.groupBy(inventory, ({ quantity }) => (quantity < 6 ? restock : sufficient));
console.log(byObjectKey.get(restock).map((i) => i.name));
const byLength = Map.groupBy(['a', 'bb', 'cc', 'ddd'], (w) => w.length);
console.log(byLength, byLength.get(2));

console.log(Object.groupBy(new Set([1, 2, 3, 4, 5]), (n) => (n % 2 ? 'odd' : 'even')));

const legacy = inventory.reduce((acc, item) => { (acc[item.type] ??= []).push(item.name); return acc; }, {});
console.log(legacy);
```

Output:

```
[ 'electronics', 'furniture', 'office' ] [ 'desk', 'chair' ]
true
{
  restock: [ 'laptop', 'desk', 'stapler' ],
  ok: [ 'monitor', 'chair' ]
}
[ 'laptop', 'desk', 'stapler' ]
Map(3) { 1 => [ 'a' ], 2 => [ 'bb', 'cc' ], 3 => [ 'ddd' ] } [ 'bb', 'cc' ]
[Object: null prototype] { odd: [ 1, 3, 5 ], even: [ 2, 4 ] }
{
  electronics: [ 'laptop', 'monitor' ],
  furniture: [ 'desk', 'chair' ],
  office: [ 'stapler' ]
}
```

How it works: walk the iterable in order, call `fn(item, index)` for a key, append the item to that key's array (created on first sight). Group order = first appearance of each key; items keep their order. The **`null` prototype** makes keys like `toString` safe but removes inherited methods — use `Object.keys(result)`, not `result.hasOwnProperty(...)`. Groups hold the **original items**, not copies.

Pre-ES2024: the `reduce` version with `??=` (assign only if `null` / `undefined`), or lodash's `_.groupBy` (a widely used utility library).

Trade-offs

	`Object.groupBy`	`Map.groupBy`
Keys	Strings (and symbols)	Any value
Result	Null-prototype object	Map
JSON-friendly	Yes	Convert first
Destructuring `const { ok, restock } = …`	Yes	No
Use when	Grouping by a label/category	Grouping by object, number, or you need Map features

Real-world examples

- Dashboards: orders by status or day; chat apps: messages by date ("Today", "Yesterday"); cleanup: records by email to find duplicates.
- Counting per group: `Object.fromEntries(Object.entries(byType).map(([k, v]) => [k, v.length]))` → `{ electronics: 2, furniture: 2, office: 1 }`.

In the interview

- *Group an array by a property?* `Object.groupBy(arr, (x) => x.prop)`; otherwise `reduce` with `(acc[k] ??= []).push(x)`.
- *`Map.groupBy` over `Object.groupBy`?* Non-string keys or Map semantics (order, `size`, object keys).

Self-check

1. Is `groupBy` an array method? → No — static `Object.groupBy` / `Map.groupBy`.
2. `Object.groupBy` result's prototype? → `null`.
3. Even/odd with a Map? → `Map.groupBy(nums, (n) => n % 2 === 0)` (keys `true` / `false`).

Version summary (verified September 2026)

Feature	Standard	Node 24	Notes
`Map`, `Set`, `WeakMap`, `WeakSet`, iterators, generators	ES2015	✓	
Async generators, `for await…of`	ES2018	✓	
`WeakRef`, `FinalizationRegistry`	ES2021	✓	Timing unspecified
Symbols as WeakMap keys	ES2023	✓	Non-registered symbols only
`Object.groupBy`, `Map.groupBy`	ES2024	✓	Node 21+
Set methods (`union`, `intersection`, …)	ES2025	✓	Node 22+
Iterator helpers, `Iterator.from`	ES2025	✓	Node 22+
`Array.fromAsync`	ES2026	✓	
`Map/WeakMap.prototype.getOrInsert(Computed)` (upsert)	ES2026	✗	Feature-detect
`Iterator.concat`	ES2026	✗	Feature-detect

Chapter TL;DR

- **Map**: any-type keys (SameValueZero), insertion order, `size`, direct iteration; `set` (chainable), `get`, `has`, `delete`, `clear`, `keys/values/entries`, `forEach(value, key)`. For dynamic dictionaries, counting, LRU caches, non-string keys; objects for fixed records and JSON. `JSON.stringify(map)` is `{}`.
- **Set**: unique values, insertion order, O(1) `has`; `add` (chainable), `delete`, `clear`, `values`, `entries` (`[v, v]`), `forEach`. Dedupe with `[...new Set(arr)]`. ES2025: `union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`, `isSupersetOf`, `isDisjointFrom`.
- **WeakMap/WeakSet**: object (or non-registered symbol) keys held weakly — no leaks, no `size`/iteration; side tables, per-object caches, tagging. **WeakRef/FinalizationRegistry**: advanced; never rely on timing.
- **Iterable** = `[Symbol.iterator]()` returning an **iterator** with `next() → { value, done }` (optional `return()`). Arrays, strings, Maps, Sets, typed arrays, NodeLists, generators are iterable; plain objects aren't. Consumers: `for…of`, spread, destructuring, `Array.from`, Map/Set constructors, `Promise.all`.

- **Generators** (`function*`) pause at `yield` ; calling returns an iterator without running. Lazy/infinite sequences, `Symbol.iterator` , traversal (`yield*`), two-way **coroutines** (functions that pause and exchange values with the caller: `next(x)` , `return` , `throw`). **Async generators** + `for await…of` for paged/streamed data.
- **Iterator helpers** (ES2025): lazy `map/filter/take/drop/flatMap` , consuming `toArray/reduce/forEach/some/every/find` ; `Iterator.from` ; arrays need `.values()` .
- **`Object.groupBy`** / **`Map.groupBy`** (ES2024): bucket by a key function — null-prototype object (string keys) or Map (any keys).

Chapter 9 — `this`, Prototypes and Classes

JavaScript's object model surprises people from Java, C# or Python. There are no "real" classes underneath: objects link to other objects through a hidden **prototype** link, and a method finds its object at call time through **`this`**. `class` (ES2015) is cleaner syntax over that machinery plus a few genuinely new features (private `#fields`, static blocks). This chapter explains the machinery first, then the syntax.

What you'll learn: the four `this` rules (plus arrows) and their precedence; `call` / `apply` / `bind` (choosing `this`, borrowing, partial application); the prototype chain (lookup, shadowing, `Object.create`, `Object.create(null)`); constructor functions and what `new` does (re-implemented); classes (fields, `#private`, static members, getters/setters, static blocks, brand checks); `extends` / `super`, subclassing built-ins, initialization order; `instanceof` / `isPrototypeOf` and how they're fooled; mixins and composition vs inheritance.

Before you start: functions, arrows, closures (Chapter 4); object literals, descriptors, getters/setters (Chapter 5); symbols like `Symbol.iterator` (Chapter 8).

Code is in `code/`; outputs are real Node 24 output. Every file is an ES module (`.mjs`, Chapter 10), and **modules always run in strict mode** — which matters for `this`.

9.1 `this` — the call-time context

What it is

`this` is an automatically defined identifier in every non-arrow function whose value is decided **each time the function is called**, by *how* it's called — like the word "me", whose meaning depends on who's speaking. The **call site** is where the function is invoked (`user.greet()`); look there, never at the definition.

Why it matters

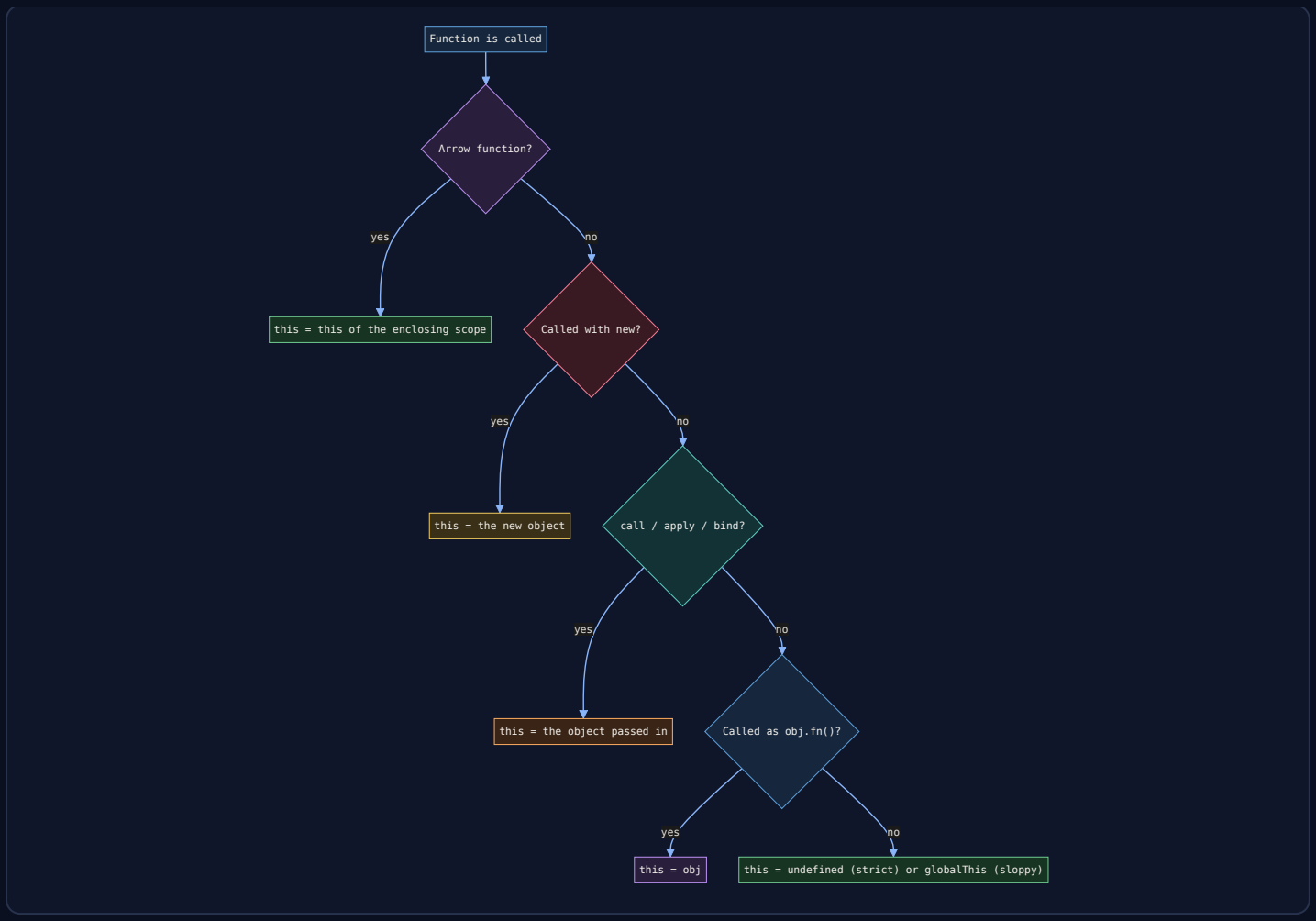
One function can serve many objects: `Array.prototype.push` is shared by every array, working because `this` is the array it's called on. The cost: the most common JS bug, the **lost** `this` — a method passed around as a plain function suddenly has `this === undefined`.

How it works — the four rules

For a normal (non-arrow) function, the highest matching rule wins:

#	Rule	Call-site shape	`this` is…
4	**`new` binding**	`new Fn(...)`	the brand-new object being constructed
3	**Explicit binding**	`fn.call(obj)`, `fn.apply(obj)`, `fn.bind(obj)()`	`obj`
2	**Implicit binding**	`obj.fn()` or `obj['fn']()`	`obj` — the object just before the dot
1	**Default binding**	plain `fn()`	`undefined` in strict mode; the global object (`globalThis`) in sloppy mode
—	**Arrow function**	any	no own `this`; it uses the `this` of the enclosing scope, *where it was written* (lexical `this` — "lexical" means decided by where code sits in the source)

Precedence: **`new` > explicit > implicit > default**. Arrows are outside the list — nothing (not `call`, not `new`) changes their `this`, and they can't be used with `new`.



(**Strict mode** — `'use strict'`, automatic in modules and class bodies — stops plain calls from getting the global object; Chapter 1. **`globalThis`** — the global object everywhere: `window` in browsers, `global` in Node.)

`code/01_this_rules.mjs` :

```js
function whoAmI() {
  return this === undefined ? 'undefined' : this.name;
}

// Rule 1 — default binding: a plain call. Strict mode -> this is undefined.
console.log('1 default :', whoAmI());

// Rule 2 — implicit binding: called as obj.method() -> this is obj.
const alice = { name: 'Alice', whoAmI };
console.log('2 implicit:', alice.whoAmI());

// Lost binding: take the method off the object, then call it plainly.
const detached = alice.whoAmI;
console.log('  lost    :', detached());

// Rule 3 — explicit binding: call / apply / bind choose this for you.
const bob = { name: 'Bob' };
console.log('3 call    :', whoAmI.call(bob));
console.log('  apply   :', whoAmI.apply(bob, []));
const boundToBob = whoAmI.bind(bob);
console.log('  bind    :', boundToBob(), '| still Bob as a method of alice?', { name: 'x', f: boundToBob }.f());

// Rule 4 — new binding: new creates a fresh object and uses it as this.
function Person(name) { this.name = name; }
const carol = new Person('Carol');
console.log('4 new     :', carol.name);

// Arrow functions have NO own this: they use this from the surrounding scope.
const timer = {
  name: 'Timer',
  regular() { return [1].map(function () { return this?.name; })[0]; },
  arrow()   { return [1].map(() => this.name)[0]; },
};
console.log('arrow     : regular callback ->', timer.regular(), '| arrow callback ->', timer.arrow());

// Arrow functions ignore call/apply/bind for this.
const arrowWho = () => (this === undefined ? 'undefined (module top level)' : this);
console.log('arrow+call:', arrowWho.call(bob));

// Precedence: new > explicit (bind) > implicit > default.
const Bound = Person.bind({ name: 'ignored' });
const dave = new Bound('Dave');
console.log('precedence: new beats bind ->', dave.name);
console.log('precedence: call beats implicit ->', alice.whoAmI.call(bob));

// Many array methods accept a thisArg.
const counter = { hits: 0 };
[1, 2, 3].forEach(function () { this.hits++; }, counter);
console.log('thisArg   :', counter.hits);

// Sloppy-mode default binding uses globalThis (Function-created code is never strict here).
const sloppy = new Function('return this === globalThis');
console.log('sloppy default this is globalThis:', sloppy());
```

Output:

```
1 default : undefined
2 implicit: Alice
  lost    : undefined
3 call    : Bob
  apply   : Bob
  bind    : Bob | still Bob as a method of alice? Bob
4 new     : Carol
arrow     : regular callback -> undefined | arrow callback -> Timer
arrow+call: undefined (module top level)
precedence: new beats bind -> Dave
precedence: call beats implicit -> Bob
thisArg   : 3
sloppy default this is globalThis: true
```

- `detached()` gives `undefined`: functions don't belong to objects; `alice.whoAmI` is just a property referencing a function.
- In `timer.regular`, `map` calls the inner `function` plainly (rule 1 → `undefined`); the arrow in `timer.arrow` sees `arrow()`'s `this` (`timer`) — *the* reason arrows were added.
- Top-level `this`: `undefined` in an ES module, `window` in a classic browser `<script>`, `module.exports` in a Node CommonJS file (older `require` format — Chapter 10).

`this` in event listeners

A regular function registered with `addEventListener` is called with `this` = the element the listener is attached to (`event.currentTarget`); an arrow keeps the outer `this`. So a class method passed directly gets the *element*, not the class **instance** (object created with `new`). `code/08_this_in_dom.mjs` runs this in jsdom (a browser-like DOM for Node):

```
class Form {
  constructor(el) {
    this.name = 'SignupForm';
    el.addEventListener('click', this.handle);                // loses this
    el.addEventListener('click', (e) => this.handle2(e));     // arrow keeps this
    el.addEventListener('click', this.handle3.bind(this));    // bind keeps this
  }
  handle() { console.log('method passed directly:', `this is the <${this.tagName.toLowerCase()}> element,
this.name =`, JSON.stringify(this.name)); }
  handle2() { console.log('wrapped in arrow      :', this.name); }
  handle3() { console.log('bound                 :', this.name); }
}
```

Output (including two simpler listeners defined earlier in the file):

```
function listener this: save | same as currentTarget? true
arrow listener this   : undefined
method passed directly: this is the <button> element, this.name = ""
wrapped in arrow      : SignupForm
bound                 : SignupForm
```

The third line is a silent bug: `this.name` read the button's empty `name` attribute instead of throwing. Events: Chapter 13.

Common mistakes

- **Passing a method as a callback** — `setTimeout(user.greet, 100)`, `promise.then(obj.handle)`, `arr.map(obj.transform)` call it plainly. Fix: arrow wrapper (`() => user.greet()`), `bind`, or an arrow-function class field (§9.5).
- **Arrow functions as object methods** — `{ name: 'x', hi: () => this.name }` uses the surrounding scope's `this` (module: `undefined`). Use `hi() { … }`.
- **Assuming `this` is the function itself** — never; name the function if you need it.
- **Nested regular functions in methods** lose `this` (`timer.regular`). Old fix `const self = this;`; today, an arrow.

In the interview

30-second answer: "`this` is decided at call time by the call site. By precedence: `new` makes it the new object; `call` / `apply` / `bind` set it explicitly; `obj.method()` makes it `obj`; a plain call gives `undefined` in strict mode or the global object in sloppy mode. Arrows have no own `this` — they capture it lexically and nothing can rebind it."

- *Q:* `const f = obj.m; f()` — `this`*?* A: `undefined` in strict code (global object in sloppy) — implicit binding lost.
- *Q:* `new` *an arrow?* A: `TypeError: … is not a constructor`; arrows have no `prototype` and no own `this`.
- *Q: Does* `bind` *beat* `new`*?* A: No — `new BoundFn()` creates a fresh object; the bound `this` is ignored (bound arguments kept).

Self-check

1. `const o = { v: 1, f() { return () => this.v; } }; o.f()()` ? → `1`.
2. `this` at the top level of an ES module? → `undefined`.
3. `[1,2].forEach(function(){ console.log(this) }, {a:1})` ? → `{ a: 1 }` twice (`thisArg`).
4. Why does `setTimeout(obj.m, 0)` lose `this` ? → It receives only the function and calls it plainly.

TL;DR

Call site decides: `new` > `call/apply/bind` > `obj.fn()` > plain call (`undefined` in strict). Arrows borrow `this` from where they're written.

9.2 `call` , `apply` and `bind`

What it is

Every function inherits from `Function.prototype` three methods to set `this` explicitly:

Method	Signature	Runs now?	Returns
`call`	`fn.call(thisArg, a, b, …)`	yes	the function's result
`apply`	`fn.apply(thisArg, [a, b, …])`	yes	the function's result
`bind`	`fn.bind(thisArg, a, …)`	no	a new **bound function** with `this` (and optionally leading arguments) fixed

Memory trick: **a**pply takes an **a**rray, **c**all takes **c**ommas.

Why it matters

- **Method borrowing** — use a method on an object lacking it, e.g. array methods on **array-likes** (numbered keys + `length`, no array methods: `arguments`, DOM `NodeList`).
- **Fixing lost `this`** — `bind` makes a function safe to hand to callback APIs.
- **Partial application** — `bind` pre-fills leading arguments (related to currying — turning a many-argument function into a chain of one-argument functions; Chapter 4).

`code/02_call_apply_bind.mjs` :

```
function greet(greeting, punctuation) {
  return `${greeting}, ${this.name}${punctuation}`;
}
const ann = { name: 'Ann' };

console.log(greet.call(ann, 'Hi', '!'));        // args one by one
console.log(greet.apply(ann, ['Hello', '?']));  // args as an array
const hiAnn = greet.bind(ann, 'Hey');           // this AND first argument fixed
console.log(hiAnn('.'));

// Borrowing a method: array methods work on any "array-like" (has length + indexes).
const arrayLike = { 0: 'a', 1: 'b', 2: 'c', length: 3 };
console.log(Array.prototype.join.call(arrayLike, '-'));
console.log(Array.prototype.map.call('abc', (ch) => ch.toUpperCase()));

// apply was the old way to spread an array into arguments; spread syntax replaced it.
const nums = [4, 9, 2];
console.log(Math.max.apply(null, nums), Math.max(...nums));

// A bound function cannot be re-bound.
const boundOnce = greet.bind({ name: 'First' });
const boundTwice = boundOnce.bind({ name: 'Second' });
console.log(boundTwice('Yo', '!'));
console.log('bound name:', boundOnce.name, '| length:', greet.length, '->', hiAnn.length);

// Classic fix for a lost this: bind in advance.
class Button {
  constructor(label) { this.label = label; this.onClick = this.onClick.bind(this); }
  onClick() { return `clicked ${this.label}`; }
}
const { onClick } = new Button('Save');   // detached, but still works
console.log(onClick());
```

Output:

```
Hi, Ann!
Hello, Ann?
Hey, Ann.
a-b-c
[ 'A', 'B', 'C' ]
9 9
Yo, First!
bound name: bound greet | length: 2 -> 1
clicked Save
```

How it works

1. `call` / `apply` invoke immediately with the given `this`.
2. `bind` returns an **exotic bound function object** (spec term for built-in special behaviour) remembering the target, `this` and pre-filled arguments.
3. Re-binding can't change `this` (`Yo, First!`), though extra arguments are appended.
4. A bound function's `name` is `"bound " + name`; `length` drops by the pre-filled count.
5. `null` / `undefined` as `thisArg`: kept as-is in strict code; `globalThis` in sloppy code.

Trade-offs

Need	Best tool today
Spread an array into arguments	spread syntax `f(...arr)` (clearer than `apply`)
Stable callback that uses `this`	arrow wrapper `() => obj.m()`, class arrow field, or `bind`
Pre-fill arguments without caring about `this`	arrow `(x) => f(1, x)` — easier to read than `f.bind(null, 1)`
Borrow a method for an array-like	`Array.from(arrayLike)` then use normal methods, or `Array.prototype.x.call`

Common mistakes

- `bind` in a render loop (code redrawing the screen repeatedly) or per event creates a new function each time — breaks identity: `removeEventListener('click', fn.bind(this))` removes nothing (a *different* function).
- Expecting `bind` to change an arrow's `this`.

Self-check

1. `f.call(o, 1, 2)` vs `f.apply(o, [1, 2])`? → Same effect; argument passing differs.
2. `f.bind(o)` returns? → A new function; doesn't call `f`.
3. `f.bind(a).bind(b)()` — `this`? → `a`.

TL;DR

`call` / `apply` run now with a chosen `this` (commas vs array); `bind` returns a function with `this` locked forever. Prefer spread and arrows where clearer.

9.3 Prototypes and the prototype chain

What it is

Every object has an internal `[[Prototype]]` slot pointing to another object or `null`. Reading a missing property follows that link, then the next, until found or `null` — the **prototype chain**, JavaScript's inheritance mechanism: **delegation**, not copying. (A help desk: the clerk asks the supervisor, who asks the manager; if nobody knows, `undefined`.)

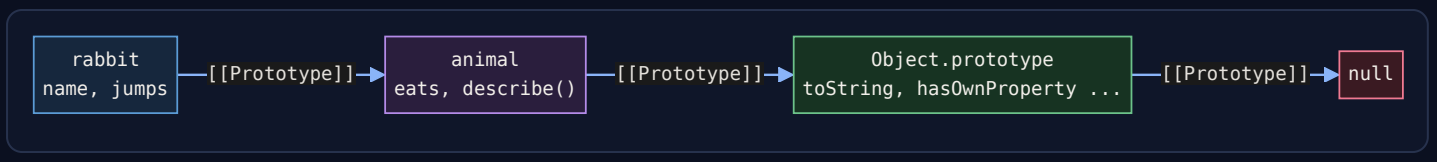
Term	Meaning
`[[Prototype]]`	The hidden link every object has. Read with `Object.getPrototypeOf(obj)`, set with `Object.setPrototypeOf` or at creation with `Object.create(proto)`.
`__proto__`	A legacy getter/setter on `Object.prototype` exposing `[[Prototype]]`. Avoid it in code; the **`__proto__:` key in an object literal** is a separate, standard way to set the prototype at creation.
`Fn.prototype`	An ordinary property that every normal function (and class) has. It is *not* the function's own prototype — it is the object that becomes the `[[Prototype]]` of instances created with `new Fn()`.
Own property	A property stored directly on the object. Check with `Object.hasOwn(obj, key)` (ES2022).
Shadowing	An own property hides a property of the same name further up the chain.

How it works

1. **Read** `obj.x`: own property, else repeat on the prototype; at `null` return `undefined`.
2. **Write** `obj.x = v`: creates/updates an **own** property — assignment never modifies the prototype. (Exceptions: a *setter* for `x` up the chain runs; a *non-writable* inherited `x` makes the write fail —

silently in sloppy, `TypeError` in strict.)

3. **Delete** removes only own properties, so an inherited value can "reappear".
4. **Methods run with `this` = the starting object**, not the prototype where found — so shared methods work on per-object data.



code/03_prototype_chain.mjs (key parts):

```
const animal = {
  eats: true,
  describe() { return `${this.name} eats: ${this.eats}`; },
};
const rabbit = Object.create(animal);   // rabbit.[[Prototype]] = animal
rabbit.name = 'Rabbit';
rabbit.jumps = true;

console.log(Object.getPrototypeOf(rabbit) === animal);         // true
console.log(rabbit.eats, rabbit.describe());                   // found on animal
console.log('own keys:', Object.keys(rabbit));
console.log('hasOwn eats?', Object.hasOwn(rabbit, 'eats'), '| "eats" in rabbit?', 'eats' in rabbit);

const seen = [];
for (const k in rabbit) seen.push(k);                           // for...in walks the chain
console.log('for...in:', seen);

rabbit.eats = false;                                            // creates an OWN property
console.log('rabbit.eats', rabbit.eats, '| animal.eats', animal.eats);
delete rabbit.eats;
console.log('after delete, rabbit.eats', rabbit.eats);

console.log('rabbit :', chain(rabbit));   // chain() walks getPrototypeOf until null
console.log('[]     :', chain([]));
console.log('fn     :', chain(function f() {}));
console.log('Map    :', chain(new Map()));

const a = [1], b = [2];
console.log('shared push?', a.push === b.push, a.push === Array.prototype.push);

const dict = Object.create(null);         // no prototype at all
dict.toString = 'just data';
console.log('proto of dict:', Object.getPrototypeOf(dict), '| "hasOwnProperty" in dict:', 'hasOwnProperty' in
dict);

const base = { hello() { return 'hi from base'; } };
const late = {};
Object.setPrototypeOf(late, base);        // works, but slow — prefer Object.create
console.log(late.hello());

const lit = { __proto__: base, own: 1 };  // literal __proto__ key sets the prototype
console.log(lit.hello(), Object.keys(lit));
console.log('missing ->', rabbit.nope);
```

Output:

```
true
true Rabbit eats: true
own keys: [ 'name', 'jumps' ]
hasOwn eats? false | "eats" in rabbit? true
for...in: [ 'name', 'jumps', 'eats', 'describe' ]
rabbit.eats false | animal.eats true
after delete, rabbit.eats true
rabbit : animal -> Object.prototype -> null
[]     : Array.prototype -> Object.prototype -> null
fn     : Function.prototype -> Object.prototype -> null
Map    : Map.prototype -> Object.prototype -> null
shared push? true true
proto of dict: null | "hasOwnProperty" in dict: false
hi from base
hi from base [ 'own' ]
missing -> undefined
```

Checking properties — which tool?

Check	Looks at own props	Looks up the chain	Notes
`Object.hasOwn(o, k)`	✓	✗	Preferred (ES2022); works on `Object.create(null)` objects
`o.hasOwnProperty(k)`	✓	✗	Fails on null-prototype objects, can be shadowed
`k in o`	✓	✓	Includes non-enumerable and inherited
`Object.keys(o)`	✓ (enumerable, string keys)	✗	
`for (k in o)`	✓	✓ (enumerable only)	Why `for…in` is risky for objects with prototypes — see Chapter 3

Why it matters

- **Memory**: a thousand arrays share one `push`.
- **Built-ins live on prototypes** (`Array.prototype`) — extendable, though you shouldn't.
- `Object.create(null)` gives a pure dictionary whose keys can't collide with `toString`, `constructor`, `__proto__` — useful for user-supplied keys (a `Map` is often better — Chapter 8).

Common mistakes

- **Modifying built-in prototypes** ("monkey-patching", `Array.prototype.last = …`) can clash with future standards — it really happened: planned `Array.prototype.contains` became `includes` and `flatten` became `flat` because the MooTools library (popular in the late 2000s) had patched incompatible versions onto millions of sites.
- `Object.setPrototypeOf` **on hot objects** (used very frequently) — engines optimise by object **shape** (internal record of an object's properties); changing the prototype later de-optimises access (MDN warns about this).
- **Confusing** `Fn.prototype` **with** `Object.getPrototypeOf(Fn)` — the former is for instances; the latter is `Function.prototype`.
- **Prototype pollution** — naively deep-merging (recursively copying) untrusted JSON like `{"__proto__": {"isAdmin": true}}` can modify `Object.prototype` program-wide. Use null-prototype objects, `Map`, or libraries skipping `__proto__` / `constructor` keys (Chapter 14).

In the interview

30-second answer: "Every object has a hidden `[[Prototype]]` link; a missing property is looked up along that chain until found or `null`. Writing creates an own property, shadowing the inherited one. `Object.create(proto)` sets the prototype; constructors and classes do it via `Fn.prototype`.

> Methods are shared through the chain and run with `this` bound to the original **receiver** (the object the method was called on)."

- *Q: `__proto__` vs `prototype`?* A: `__proto__` (legacy accessor) is an object's own prototype link; `prototype` is a function property naming what `new` links instances to.
- *Q: End of every chain?* A: `null` (usually after `Object.prototype`).
- *Q: Object with no prototype?* A: `Object.create(null)` or `{ __proto__: null }`.

Self-check

1. `const p = { x: 1 }; const c = Object.create(p); c.x = 2; p.x` → `1`.
2. Does `Object.keys` include inherited properties? → No.
3. Where does `[].map` live? → `Array.prototype.map`.
4. `Object.getPrototypeOf(Object.prototype)`? → `null`.

TL;DR

Missing reads delegate up `[[Prototype]]` links to `null`; writes stay on the object. Use `Object.create`, `Object.getPrototypeOf`, `Object.hasOwn`; don't patch built-ins.

9.4 Constructor functions and `new`

What it is

Before ES2015, many similar objects came from a **constructor function**: an ordinary, conventionally Capitalized function called with `new`, assigning per-instance data to `this`, with shared methods on `Constructor.prototype`. Classes (§9.5) are built on exactly this.

How `new` works

`new Fn(args)`:

1. Creates an empty object.
2. Sets its `[[Prototype]]` to `Fn.prototype`.
3. Calls `Fn` with `this` = that object.
4. If `Fn` returns an **object**, that's the result; otherwise (no return or a primitive) the new object is.

Every `Fn.prototype` starts with `constructor` pointing back to `Fn` — how `instance.constructor` finds its maker.

`code/04_constructor_functions.mjs`:

```
function Account(owner, balance = 0) {
  this.owner = owner;       // own, per-instance data
  this.balance = balance;
}
Account.prototype.deposit = function (amount) {  // one shared copy for all instances
  this.balance += amount;
  return this;
};
Account.prototype.toString = function () { return `${this.owner}: $${this.balance}`; };

const acc = new Account('Ravi', 100);
acc.deposit(50).deposit(25);
console.log(String(acc));
console.log('proto link  :', Object.getPrototypeOf(acc) === Account.prototype);
console.log('constructor :', acc.constructor === Account, Account.prototype.constructor === Account);
console.log('own keys    :', Object.keys(acc));

// What `new` does — written out by hand
function myNew(Ctor, ...args) {
  const obj = Object.create(Ctor.prototype);        // 1+2. new object linked to Ctor.prototype
  const result = Ctor.apply(obj, args);             // 3. run constructor with this = obj
  return result !== null && (typeof result === 'object' || typeof result === 'function')
    ? result                                        // 4a. if it returned an object, use that
    : obj;                                          // 4b. otherwise the new object
}
const acc2 = myNew(Account, 'Mei', 10);
console.log('myNew       :', String(acc2), acc2 instanceof Account);

// Forgetting `new` — in strict code this is undefined, so assigning throws
try {
  Account('Oops');
} catch (e) {
  console.log('no new      :', e.constructor.name, '-', e.message);
}

// Guard with new.target
function Safe(name) {
  if (!new.target) return new Safe(name);
  this.name = name;
}
console.log('new.target  :', Safe('guarded').name);

// Inheritance the old way
function Savings(owner, balance, rate) {
  Account.call(this, owner, balance);               // "super" call
  this.rate = rate;
}
Savings.prototype = Object.create(Account.prototype);  // link the prototypes
Savings.prototype.constructor = Savings;               // repair constructor
Savings.prototype.addInterest = function () { return this.deposit(this.balance * this.rate); };
const s = new Savings('Lena', 1000, 0.05);
console.log('old-style   :', String(s.addInterest()), s instanceof Savings, s instanceof Account);
```

Output:

```
Ravi: $175
proto link  : true
constructor : true true
own keys    : [ 'owner', 'balance' ]
myNew       : Mei: $10 true
no new      : TypeError - Cannot set properties of undefined (setting 'owner')
new.target  : guarded
old-style   : Lena: $1050 true true
```

`new.target` is a meta-property: the constructor `new` was applied to, or `undefined` in a plain call.

Common mistakes

- **Methods defined inside the constructor** (`this.deposit = function …`) — a copy per instance, wasting memory. Use the prototype (classes do automatically).
- **Forgetting** `new` — sloppy code silently creates globals; strict code throws `TypeError`. Classes refuse plain calls.
- **Replacing** `Fn.prototype` **without restoring** `constructor` — `s.constructor` would report `Account`.

Self-check

1. `[[Prototype]]` of `new Fn()`? → `Fn.prototype`.
2. Constructor ends `return 42;`? → The new object (primitives ignored).
3. Ends `return { a: 1 };`? → `{ a: 1 }`, **not** linked to `Fn.prototype`.

TL;DR

`new` = create → link to `Fn.prototype` → run `Fn` with that `this` → return it (unless `Fn` returns another object). Shared methods on `Fn.prototype`.

9.5 Classes

What it is

A **class** is a template for objects. `class` declares a special constructor function and fills its `prototype` with methods — §9.4's arrangement — with shorter syntax, safer defaults, and features impossible with plain functions: truly private members, static blocks, `super`. "Syntactic sugar" (nicer syntax for something already possible) is mostly right for methods and inheritance, not for private fields.

Class features at a glance

Feature	Syntax	Lives on	Added in
Constructor	`constructor(a) { … }`	the class function itself	ES2015
Method	`m() { … }`	`Class.prototype` (non-enumerable)	ES2015
Getter / setter	`get x() {}` / `set x(v) {}`	`Class.prototype` (accessor property)	ES2015
Static method	`static m() {}`	the class function	ES2015
Computed name	`[expr]() {}`	prototype or class	ES2015
Public instance field	`x = 0;`	each instance (own, enumerable)	ES2022
Static field	`static x = 0;`	the class function	ES2022
Private field / method	`#x = 0;` `#m() {}`	hidden per-instance slot	ES2022
Private static	`static #x`, `static #m()`	hidden slot on the class	ES2022
Static initialization block	`static { … }`	runs once at definition time	ES2022
Private "brand check"	`#x in obj`	—	ES2022

- **Field** — declared in the body, created on every instance (or the class, if `static`); its initializer runs per instance with `this` = the instance.
- **Private member** — `#name`, usable only by code *lexically inside* the class body. Not a property: absent from `Object.keys`, `JSON.stringify`, `for…in`, proxies (`Proxy` objects intercepting property access); outside access is a **syntax error** at parse time (before the code runs).
- **Accessor** — getter/setter pair that looks like a property but runs code (Chapter 5).

- **Static member** — belongs to the class: factory methods (build and return instances, e.g. `Temperature.fromFahrenheit`), counters, constants, caches.

`code/05_classes.mjs`:

```
class Temperature {
  unit = 'C';                          // public instance field
  #celsius;                            // private instance field
  static instances = 0;                // static field
  static #absoluteZero = -273.15;      // private static field

  static {                             // static initialization block — runs once
    Temperature.freezing = new Temperature(0);
    Temperature.instances = 0;         // don't count the helper instance
  }

  constructor(celsius) {
    Temperature.#check(celsius);
    this.#celsius = celsius;
    Temperature.instances++;
  }

  get fahrenheit() { return this.#celsius * 9 / 5 + 32; }
  set fahrenheit(f) {
    const c = (f - 32) * 5 / 9;
    Temperature.#check(c);
    this.#celsius = c;
  }
  get celsius() { return this.#celsius; }   // read-only from outside: no setter

  #format(v) { return v.toFixed(1); }       // private method
  toString() { return `${this.#format(this.#celsius)}°C`; }

  static #check(c) {
    if (c < Temperature.#absoluteZero) throw new RangeError(`below absolute zero: ${c}`);
  }
  static fromFahrenheit(f) { const t = new Temperature(0); t.fahrenheit = f; return t; }
  static isTemperature(obj) { return #celsius in obj; }   // brand check
}

const t = new Temperature(25);
console.log(String(t), '=', t.fahrenheit, '°F');
t.fahrenheit = 212;
console.log('after set  :', String(t), '| unit field:', t.unit);
console.log('static     :', Temperature.instances, String(Temperature.freezing),
String(Temperature.fromFahrenheit(-40)));
console.log('brand check:', Temperature.isTemperature(t), Temperature.isTemperature({ celsius: 1 }));
console.log('own keys   :', Object.keys(t), '(private fields are invisible)');
try { new Temperature(-300); } catch (e) { console.log('validation :', e.name, e.message); }
// t.#celsius  -> SyntaxError at parse time if written outside the class
```

Output (first part):

```
25.0°C = 77 °F
after set  : 100.0°C | unit field: C
static     : 1 0.0°C -40.0°C
brand check: true false
own keys   : [ 'unit' ] (private fields are invisible)
validation : RangeError below absolute zero: -300
```

Why `static` prints `1`: the static block created `freezing` then reset the counter; `new Temperature(25)` made it 1; arguments evaluate left to right, so `instances` was read (1) *before* `fromFahrenheit(-40)` bumped it to 2 — evaluation order matters with **side effects** (changes beyond producing a value).

Facts about classes that trip people up

The same file continues:

```
// 05_classes.mjs, second part (abridged — each output line labels its check)
class Clicker {
  count = 0;
  method() { return ++this.count; }
  arrow = () => ++this.count;   // one function per instance, this captured forever
}
const { method, arrow } = new Clicker();
try { method(); } catch (e) { console.log('detached method     :', e.message); }
console.log('detached arrow      :', arrow(), arrow());
// ... also: typeof, prototype method names, calling without new, use before the class line (TDZ),
//     a computed method name + static [Symbol.hasInstance], and the getter's descriptor
```

Output (full run):

```
typeof class        : function
methods on prototype: [ 'constructor', 'fahrenheit', 'celsius', 'toString' ]
methods enumerable? : false
call without new    : Class constructor Temperature cannot be invoked without 'new'
TDZ                 : ReferenceError - Cannot access 'Later' before initialization
detached method     : Cannot read properties of undefined (reading 'count')
detached arrow      : 1 2
arrow on prototype? : false | own? true
computed name       : HEY | 42 instanceof Loud: true
accessor descriptor : function function enumerable: false
```

Fact	Consequence
A class is a function (`typeof` → `'function'`)	It has `prototype` , `name` , `length` like any function
Class bodies are always strict mode	Detached methods get `this === undefined` , not the global object
Methods are non-enumerable	`for…in` over an instance shows only data fields — nicer than §9.4
Must be called with `new`	Calling it plainly throws `TypeError`
Declarations are hoisted into the **TDZ** (temporal dead zone, Chapter 2)	You can't use a class before its `class` line runs — unlike function declarations
Private methods (`#format`) and the `#check` static are not on the prototype	`getOwnPropertyNames` doesn't list them
Arrow-function **fields** are own properties created per instance	Great for callbacks (`this` is fixed), but each instance gets a new function and subclasses can't call them via `super`
`static [Symbol.hasInstance]` customizes `instanceof`	Rarely needed; shown only so you recognize it

Getters, setters and private fields together

`Temperature` shows **encapsulation** (hide internal state, expose a controlled interface): real value in a private field, a getter to expose it, validation in setter/constructor. `celsius` has no setter, so it's read-only — `t.celsius = 5` is silently ignored in sloppy code, `TypeError` in strict. Pre-ES2022 "private" meant `this._celsius` (convention, fully public) or closure/`WeakMap` tricks.

Static initialization blocks

`static { … }` runs once when the class is evaluated, with `this` = the class — for setup needing statements (loops, `try/catch` , several related statics) or private statics. Several blocks run in order with static field initializers.

When not to use a class

- A single object (config, a namespace of functions) → object literal or module (Chapter 10).
- A data-transforming function → a plain function.

- React function components replaced class components for new code (see the React handbook).

Common mistakes

- A field initializer using a *later* field — fields initialize top to bottom.
- Expecting `#private` fields to survive `structuredClone`, `JSON.stringify` or `{...spread}` — none copy them.
- `Object.keys(instance)` to inspect private data — use the DevTools/Node inspector (debuggers that pause code and show values), which displays private fields.
- `class A { method = function () {} }` is a per-instance field, not a method.

In the interview

> **30-second answer:** "A class is a constructor function plus a prototype with its methods, with nicer syntax. It's strict, `new`-only, has non-enumerable methods, and is in the TDZ until defined. ES2022 added public and private fields, `#` private methods, static blocks and `#x in obj` checks. Private fields are enforced by the language, unlike the `_underscore` convention."

- *Q: Are classes hoisted?* A: Hoisted but uninitialized (TDZ) → `ReferenceError` if used early.
- *Q: Method vs arrow field for event handlers?* A: Arrow field keeps `this` without `bind` but costs one function per instance and no `super`; a method is shared on the prototype.
- *Q: How private is `#x`?* A: Inaccessible outside the class body, invisible to every object API; class-body code (incl. static methods handling other instances) can read it.

Self-check

1. Where does a class method live? → `Class.prototype`.
2. A public field? → Each instance, own enumerable property.
3. `class A { #x = 1; static has(o) { return #x in o; } } A.has({})` → `false`.
4. `typeof class {}`? → `'function'`.

TL;DR

`class` = constructor + prototype methods with safer defaults (strict, `new`-only, non-enumerable methods). ES2022: fields, `#private`, `static {}`, brand checks. Getters/setters over private fields for encapsulation.

9.6 Inheritance with `extends` and `super`

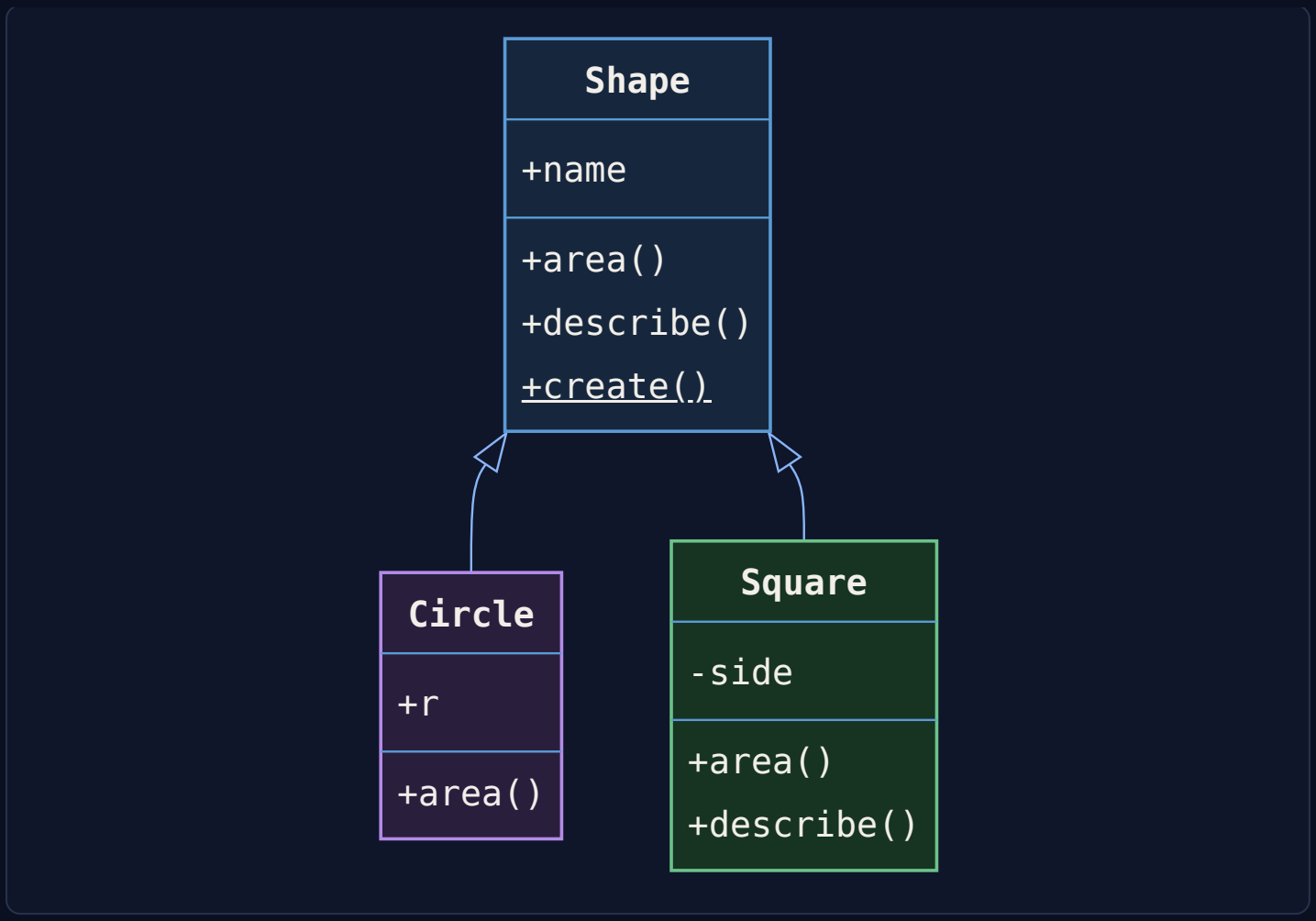
What it is

`class Child extends Parent` makes a specialized `Parent`: instances inherit `Parent`'s methods via the prototype chain, and `Child` can add or **override** (replace) them. `super(...)` calls the parent constructor; `super.method()` the parent's version of a method.

How it works

`extends` links **two** chains:

1. `Child.prototype.[[Prototype]] = Parent.prototype` — instance methods.
2. `Child.[[Prototype]] = Parent` — static methods.



Derived-constructor rules:

- **Call `super(...)` before using `this`** — in a derived class the *parent* constructor creates the object; using `this` first throws `ReferenceError`.
- No constructor written → default `constructor(...args) { super(...args); }`.
- **Initialization order**: `super()` runs parent field initializers and constructor; then child fields; then the rest of the child constructor.

code/06_inheritance.mjs :

```
class Shape {
  constructor(name) {
    if (new.target === Shape) throw new TypeError('Shape is abstract');
    this.name = name;
  }
  area() { throw new Error(`${this.constructor.name} must implement area()`); }
  describe() { return `${this.name} with area ${this.area().toFixed(2)}`; }
  static create(...args) { return new this(...args); } // `this` = the class it's called on
}

class Circle extends Shape {
  constructor(r) {
    super('circle');          // must run before touching `this`
    this.r = r;
  }
  area() { return Math.PI * this.r ** 2; }
}

class Square extends Shape {
  #side;
  constructor(side) { super('square'); this.#side = side; }
  area() { return this.#side ** 2; }
  describe() { return super.describe() + ' (all sides equal)'; } // extend parent behaviour
}

const shapes = [new Circle(1), Square.create(3)];
for (const s of shapes) console.log(s.describe());

try { new Shape('x'); } catch (e) { console.log('abstract   :', e.message); }
class Broken extends Shape { constructor() { super('broken'); } }
try { new Broken().describe(); } catch (e) { console.log('no area   :', e.message); }
class NoSuper extends Shape { constructor() { this.x = 1; super('x'); } }
try { new NoSuper(); } catch (e) { console.log('this first :', e.name, '-', e.message); }

console.log('instance chain:', Object.getPrototypeOf(Circle.prototype) === Shape.prototype);
console.log('static chain  :', Object.getPrototypeOf(Circle) === Shape, '-> Circle.create exists:', typeof
Circle.create);
```

Output:

```
circle with area 3.14
square with area 9.00 (all sides equal)
abstract   : Shape is abstract
no area   : Broken must implement area()
this first : ReferenceError - Must call super constructor in derived class before accessing 'this' or returning
from derived constructor
instance chain: true
static chain  : true -> Circle.create exists: function
```

- **Polymorphism** — `Shape.describe()` calls `this.area()`, which runs the *subclass's* `area`: **dynamic dispatch** (method chosen at call time by the actual object) via the prototype chain.
- **Abstract classes** — no `abstract` keyword; `new.target === Shape` plus a throwing `area()` imitate one (TypeScript adds real compile-time `abstract` — Chapter 16).
- `static create` uses `new this(...)`, so `Square.create(3)` builds a `Square` — statics are inherited with `this` = the calling class.
- **Private fields aren't accessible to subclasses** — not `Circle`, not even a `Square` subclass; private = "this class body only". Use a normal or "protected"-by-convention property (protected, in other languages, means subclass-visible only) if subclasses need it.

Subclassing built-ins and initialization order

```
class Stack extends Array {
  peek() { return this.at(-1); }
}
const st = Stack.from([1, 2, 3]);
st.push(4);
console.log('Stack         :', st.peek(), st.length, Array.isArray(st), st.map((x) => x * 2) instanceof Stack);

class HttpError extends Error {
  constructor(status, message) { super(message); this.name = 'HttpError'; this.status = status; }
}
const he = new HttpError(404, 'Not Found');
console.log('Error subclass:', he instanceof Error, he.name, he.status, he.message);

class Queue { get [Symbol.toStringTag]() { return 'Queue'; } }
console.log('toStringTag   :', Object.prototype.toString.call(new Queue()));

class A { a = console.log('  A field') ?? 1; constructor() { console.log('  A constructor'); } }
class B extends A { b = console.log('  B field') ?? 2; constructor() { console.log('  B before super'); super();
console.log('  B after super'); } }
console.log('init order:'); new B();
```

Output (the file prints the three §9.7 `instanceof` lines first):

```
Stack         : 4 4 true true
Error subclass: true HttpError 404 Not Found
toStringTag   : [object Queue]
init order:
  B before super
  A field
  A constructor
  B field
  B after super
```

- `extends Array` gives real arrays: auto-updating `length`, `Array.isArray` true, `map` returning a `Stack` (via `Symbol.species`) — impossible with old constructor functions.
- Subclassing `Error` is the standard way to make custom errors (Chapter 11).
- The init order shows why a parent constructor mustn't call an overridable method relying on child fields — they don't exist yet.

`super` in object literals

`super` works in **concise methods** of object literals, referring to the object's prototype: `const child = { __proto__: parent, hi() { return super.hi() + '!'; } }`. It's bound to the **home object** (where the method was *defined*), not `this` — copying the method elsewhere doesn't change `super`.

Common mistakes

- Forgetting `super()` or calling it after `this`.
- Deep hierarchies (`Animal → Mammal → Dog → ServiceDog → …`) couple you to (make you depend on) every level above (§9.9).
- Overriding without calling `super.method()` when the parent's behaviour (cleanup, validation) is still needed.
- Calling overridable methods from a parent constructor.

In the interview

30-second answer: "`extends` links `Child.prototype` to `Parent.prototype` for instance methods and `Child` to `Parent` for statics. A derived constructor must call `super()` before `this`, because the parent constructs the object. `super.method()` calls the parent's implementation. `Array`, `Error`, `Map` subclass properly with class syntax."

- *Q: Derived class without a constructor?* A: A default one forwards all arguments to `super(...)`.

- *Q: Extend `null` or an expression?* A: `extends` accepts any expression evaluating to a constructor (or `null`) — enabling mixin factories (§9.8).
- *Q: Initialization order?* A: Parent fields → parent constructor → child fields → rest of child constructor.

Self-check

1. Why `super()` before `this`? → The parent constructor creates the object.
2. How are statics inherited? → `Child.[[Prototype]]` is `Parent`.
3. Can a subclass read the parent's `#secret`? → No.

TL;DR

`extends` links instance and static chains; `super()` first, then `this`; `super.m()` reaches the parent. Built-ins subclass cleanly. Keep hierarchies shallow.

9.7 `instanceof` and type checks

What it is

`obj instanceof C` asks **"is `C.prototype` anywhere in `obj`'s prototype chain?"** — nothing about how the object was created. `C.prototype.isPrototypeOf(obj)` asks the same the other way round.

```
const c = shapes[0];
console.log('instanceof    :', c instanceof Circle, c instanceof Shape, c instanceof Object, c instanceof Square);
console.log('isPrototypeOf :', Shape.prototype.isPrototypeOf(c));
const fake = Object.create(Circle.prototype);
console.log('fooled?       :', fake instanceof Circle, '(instanceof only checks the chain)');
```

Output:

```
instanceof    : true true true false
isPrototypeOf : true
fooled?       : true (instanceof only checks the chain)
```

`fake` never ran `Circle`'s constructor (no `r`), yet is "a `Circle`".

Choosing a type check

Question	Use	Pitfall
Is it a primitive of type T?	`typeof x === 'string'` etc.	`typeof null === 'object'` (Chapter 2)
Is it an array?	`Array.isArray(x)`	`instanceof Array` fails for arrays from another realm (iframe, `vm` context)
Is it an error?	`Error.isError(x)` (ES2026; Node 24 has it) or `x instanceof Error`	Same cross-realm issue for `instanceof`
Was it made by my class, really?	`#brand in x` (private brand check)	Only works inside the class body
Is it "like" a type (has the methods I need)?	**duck typing**: `typeof x?.then === 'function'`	Can't distinguish unrelated objects with the same method names
Built-in tag	`Object.prototype.toString.call(x)` → `"[object Date]"`	Can be customized via `Symbol.toStringTag`

A **realm** is one global environment — each tab, iframe or Node `vm` context has its own `Array`, `Object`, `Error`. An iframe's array has the *iframe's* `Array.prototype`, so `instanceof Array` in the parent is `false`

— why `Array.isArray` exists.

Duck typing ("walks like a duck…") checks capabilities, not ancestry — how promises interoperate: anything with `then` (a *thenable*) is treated as a promise by `await` (Chapter 12).

Self-check

1. `instanceof` true without the constructor running? → Yes (`Object.create(C.prototype)`, `Object.setPrototypeOf`).
2. Most robust array check? → `Array.isArray`.
3. `Object.create(null) instanceof Object` ? → `false`.

TL;DR

`instanceof` = "is `C.prototype` in the chain?". Prefer `Array.isArray`, `Error.isError`, `#brand in obj` or duck typing when correctness matters.

9.8 Mixins

What it is

Classes have **single inheritance** (one `extends`). A **mixin** is a reusable bundle of methods mixed into a class that already has a parent, so unrelated classes share a capability (serializable — convertible to text like JSON; observable — others can subscribe to its changes; timestamped) without a common ancestor.

1. **Object mixins** — `Object.assign(Target.prototype, mixinA, mixinB)`.
2. **Subclass-factory mixins** — a function taking a base class and returning `class extends Base { … }`; stackable since `extends` takes any expression: `class Duck extends CanSwim(CanFly(Animal)) {}`.

`code/07_mixins_composition.mjs` (first part):

```
// 1) Object.assign mixin — copy methods onto a prototype
const Serializable = { toJSONString() { return JSON.stringify(this); } };
const Timestamped = { touch() { this.updatedAt = '2026-01-01T00:00:00Z'; return this; } };

class Note { constructor(text) { this.text = text; } }
Object.assign(Note.prototype, Serializable, Timestamped);
console.log('assign mixin :', new Note('hi').touch().toJSONString());

// 2) Subclass-factory mixins — functions that take a class and return a subclass
const CanFly = (Base) => class extends Base { fly() { return `${this.name} flies`; } };
const CanSwim = (Base) => class extends Base { swim() { return `${this.name} swims`; } };
class Animal { constructor(name) { this.name = name; } }
class Duck extends CanSwim(CanFly(Animal)) {}
const d = new Duck('Donald');
console.log('factory mixin:', d.fly(), '|', d.swim(), '| instanceof Animal:', d instanceof Animal);
```

Output:

```
assign mixin : {"text":"hi","updatedAt":"2026-01-01T00:00:00Z"}
factory mixin: Donald flies | Donald swims | instanceof Animal: true
```

Trade-offs

	`Object.assign` mixin	Subclass-factory mixin
Can use `super`	✗ (copied methods keep their original home object)	✓
Can use private `#` fields	✗	✓ (inside the factory class)
Shows up in `instanceof`	✗	only for the base class (each call creates a new anonymous class)
Name clashes	later mixin silently overwrites earlier	nearer subclass overrides farther
Copies getters correctly	✗ — `Object.assign` *invokes* getters and copies the value	✓

Common mistakes

- Two mixins with the same method name — last wins silently.
- State-heavy mixins assuming target fields (`this.items`) create hidden coupling — document the "requires" contract (what each side must provide).

Real-world examples

Lit (web components — custom HTML tags you define) uses subclass-factory mixins; Node's `EventEmitter` (lets objects emit named events to registered listeners) is often inherited or mixed in for "observable" objects (Chapter 15).

TL;DR

Mixins add capabilities across unrelated classes; prefer subclass factories (support `super` and private fields); watch name clashes.

9.9 Composition vs inheritance

What it is

- **Inheritance** = **is-a** (`Circle` *is a* `Shape`): behaviour received from a parent.
- **Composition** = **has-a** (`Car` *has an* `Engine`): behaviour from held objects/functions, with calls **delegated** to them.

"Favour composition over inheritance" (*Design Patterns*, 1994) applies strongly in JavaScript, where functions and literals make composition cheap. General principle: the LLD (low-level design — designing classes and their relationships) handbook's design principles chapter.

The problem it solves

Hierarchies break when categories overlap. If `Duck extends Bird` gives every duck `fly()`, a `RoboDuck` (flies, quacks, can't swim) and a `RubberDuck` (only floats) force overrides that throw "not supported", violating the parent's promise (Liskov substitution, LLD chapter 4). With composition each object gets just its capabilities — the **gorilla/banana problem**: you wanted a banana but got the gorilla holding it and the whole jungle.

`code/07_mixins_composition.mjs` (second part):

```
// 3) Composition — objects HAVE capabilities instead of BEING a subtype
const flyer = (state) => ({ fly: () => `${state.name} flies at ${state.speed} km/h` });
const quacker = (state) => ({ quack: () => `${state.name}: quack` });
function makeRobotDuck(name) {
  const state = { name, speed: 40 };
  return Object.freeze({ ...flyer(state), ...quacker(state) }); // no swim — robots sink
}
const robo = makeRobotDuck('RoboDuck');
console.log('composition  :', robo.fly(), '|', robo.quack(), '| can swim?', 'swim' in robo);

// 4) Composition with classes — delegate to a collaborator
class Engine { start() { return 'engine started'; } }
class Car {
  #engine;
  constructor(engine = new Engine()) { this.#engine = engine; }
  start() { return `car: ${this.#engine.start()}`; }   // delegation
}
class ElectricMotor { start() { return 'motor humming'; } }
console.log('delegation   :', new Car().start(), '|', new Car(new ElectricMotor()).start());

// 5) OLOO (objects linked to other objects) — pure prototype delegation, no classes
const Widget = { init(w) { this.width = w; return this; }, render() { return `widget ${this.width}px`; } };
const ButtonW = Object.create(Widget);
ButtonW.setup = function (w, label) { this.init(w); this.label = label; return this; };
ButtonW.render = function () { return `${Widget.render.call(this)} [${this.label}]`; };
console.log('OLOO         :', Object.create(ButtonW).setup(120, 'OK').render());
```

Output:

```
composition  : RoboDuck flies at 40 km/h | RoboDuck: quack | can swim? false
delegation   : car: engine started | car: motor humming
OLOO         : widget 120px [OK]
```

- **Factory functions** (`makeRobotDuck`) build objects from capability functions closing over shared `state` — private by closure, no `this`, nothing to lose.
- `Car` receives its collaborator via the constructor (**dependency injection**) — swap in `ElectricMotor` without subclassing; tests pass a fake.
- **OLOO** ("objects linked to other objects", from *You Don't Know JS*) uses `Object.create` directly — showing JS "inheritance" is really delegation along the chain.

Trade-offs

	Inheritance (extends)	Composition
Relationship	is-a	has-a / can-do
Reuse	automatic, all-or-nothing	pick exactly the parts you need
Coupling (how much one piece depends on another's details)	tight — child depends on parent internals and init order	loose — depends on a small interface (the set of methods it promises)
Changing behaviour at runtime	hard	easy (swap the collaborator)
Testing	need the whole hierarchy	inject fakes
Best when	a true, stable taxonomy (a classification tree of kinds and sub-kinds: `HttpError extends Error`, UI base classes a framework requires)	most application code

When not to use composition

When the platform expects a subclass (custom `Error`s, web components `extends HTMLElement`, `extends EventEmitter`) — one shallow level of inheritance.

In the interview

> **30-second answer:** "Inheritance is is-a and couples you to the parent's implementation; composition is has-a and assembles behaviour from small parts, adapting better when requirements cut across a hierarchy. I use `extends` for shallow, framework-mandated or truly taxonomic cases like custom errors, and composition — factories, injected collaborators, mixins — for most app code."

- *Q: Fragile base class problem?* A: A parent change (new method, changed call order) silently breaks subclasses depending on its internals.
- *Q: Share "logging" across unrelated classes?* A: Inject a logger (composition) or a mixin — not a `Loggable` base class.

Self-check

1. "A `Car` has an `Engine`"? → Composition.
2. Why are factory functions immune to lost-`this`? → Methods use closed-over variables, not `this`.
3. When is inheritance right in JS? → Shallow, stable is-a, or platform-required (errors, custom elements).

TL;DR

Default to composition (factories, delegation, injection); one shallow `extends` where the platform or a real taxonomy demands it.

Chapter 9 TL;DR

- **`this`** is set by the call site: `new` > `call/apply/bind` > `obj.m()` > plain call (`undefined` in strict). **Arrows** take `this` from where they're written; can't be rebound.
- **`call`** / **`apply`** invoke now with a chosen `this`; **`bind`** returns a permanently bound function (also partial application).
- Every object has **`[[Prototype]]`**; missing reads walk the **chain** to `null`; writes create own properties (shadowing). `Object.create`, `Object.getPrototypeOf`, `Object.hasOwn`.
- **`new Fn()`** creates an object linked to `Fn.prototype`, runs `Fn` with it as `this`, returns it.
- **Classes** wrap that: strict, `new`-only, non-enumerable methods, TDZ. ES2022: **fields**, **`#private`**, **static blocks**, **`#x in obj`**.
- **`extends`** / **`super`**: `super()` before `this`; instance and static methods inherited; `Array`, `Error` subclass properly.
- **`instanceof`** only checks the chain — prefer `Array.isArray`, `Error.isError`, brand checks or duck typing when it matters.
- **Mixins** share capabilities across unrelated classes; **composition** (has-a) usually beats deep inheritance (is-a).

Next: organize code into files and packages — Chapter 10: Modules and Tooling.

Chapter 10 — Modules and Tooling

A real program is hundreds of files plus thousands of third-party **packages** (bundles of reusable code downloaded into your project). **Modules** give each file a private scope that shares only what it exports. **Tooling** — npm, bundlers, transpilers, linters, formatters — turns files and packages into something a browser or server can run, and keeps code consistent. This chapter covers ES modules, how Node runs both module systems, then the tool chain.

What you'll learn: why modules exist and how module systems evolved; every `import` / `export` form, default vs named, barrels, side-effect imports; how ESM loads (three phases, hoisting, live bindings, singletons, cycles); `import()`, top-level `await`, `import.meta`, JSON modules; browser modules and import maps; CommonJS vs ESM in Node (`require(esm)`, `createRequire`, `"exports"`); npm and `package.json` (`npx`, `node --run`, `node_modules` resolution); semver, ranges, lockfiles; bundlers (Vite, esbuild, webpack, Rollup/Rolldown); transpilers, polyfills, ESLint, Prettier.

Before you start: functions and closures (Chapter 4) — the pre-module "module pattern" is a closure. Here we only need that `await` pauses until a promise (a result arriving later) settles; full details in Chapter 12.

Code in `code/` is a small multi-file ES module project; files starting `// tools_run: skip` are helpers imported by the numbered programs. Outputs are real (Node 24). **npm, bundler and linter commands are not-run snippets** (they need the network or a full project); their output is described.

10.1 Why modules?

What it is

A **module** is a file whose top-level variables are private; it **exports** what it shares and **imports** what it needs. Modules connected by imports form the **module graph**; the first one run is the **entry point**.

The problem it solves

Early pages loaded `<script>` files sharing **one global scope**, so every top-level `var`/function was global:

- **Name collisions** — two libraries defining `$` or `init` overwrote each other.
- **Hidden dependencies** — script order in HTML *was* the dependency system.
- **No encapsulation** (internals hidden behind a small public surface) — anything could change anything.

How the solutions evolved

Era	Approach	How it works	Still seen?
~2000s	Global scripts + namespaces (one global object holding everything, to reduce name clashes)	`var MyApp = MyApp || {}; MyApp.utils = …`	Legacy pages
~2008+	**IIFE / module pattern**	A function called immediately; a closure hides private state and returns a public object (Chapter 4)	Output of old bundlers
2009	**CommonJS** (Node)	`require()` / `module.exports`, synchronous (each file is fully loaded before the next line runs), one file = one module	Very common in Node packages
2011	AMD (RequireJS), later UMD	Asynchronous (loads in the background and calls back when ready) `define()` for browsers; UMD wraps a file to work as AMD, CommonJS or a global	Old libraries
ES2015	**ES modules (ESM)**	`import` / `export` built into the language; static; async-friendly	**The standard** — browsers and Node

ESM alone is part of the language, native in browsers and Node, and **statically analyzable** (understood without running). Use it for new code.

> **TL;DR**
>
> Modules give each file a private scope with explicit imports/exports. ESM (ES2015) is the standard; CommonJS remains common in Node.

10.2 `export` and `import` syntax

Export forms

Any number of **named exports**, at most one **default export**.

`code/math.mjs` — named exports:

```
// tools_run: skip — helper module imported by other examples
export const TAU = 2 * Math.PI;                // named export (declaration form)
export function add(a, b) { return a + b; }
function square(x) { return x * x; }
function cube(x) { return x ** 3; }
export { square, cube as toThePowerOfThree };   // export list, with renaming
console.log('[math.mjs] evaluated');            // runs once, however many importers
```

`code/logger.mjs` — default plus named:

```
export default function log(level, msg) {       // the default export
  return `[${level.toUpperCase()}] ${msg}`;
}
export const LEVELS = ['debug', 'info', 'warn', 'error'];
```

`code/barrel.mjs` — re-exports (a **barrel** gathers several modules' exports into one import point):

```
export * from './math.mjs';                          // re-export all named exports
export { default as log, LEVELS } from './logger.mjs'; // re-export a default under a name
export * as counterNS from './counter.mjs';          // re-export a namespace (ES2020)
```

Export form	Example
Declaration	`export const x = 1;` `export function f() {}` `export class C {}`
List (after declaring)	`export { a, b };`
Rename	`export { a as alpha };`
Default (expression)	`export default 42;` `export default { … };`
Default (declaration, may be anonymous)	`export default function () {}` `export default class {}`
Name something the default	`export { x as default };`
Re-export	`export { a } from './m.mjs';` `export * from './m.mjs';` `export * as ns from './m.mjs';`

`export *` does **not** re-export the default — re-export it by name (as `barrel.mjs` does).

Import forms

code/01_import_export.mjs :

```
// Every static import form. Imports are hoisted: they load before any code here runs.
import './side_effect.mjs';                     // side-effect only
import log, { LEVELS } from './logger.mjs';     // default + named in one statement
import { add, square as sq, TAU } from './math.mjs'; // named, with renaming
import * as MathNS from './math.mjs';           // namespace object
import { toThePowerOfThree, counterNS, log as log2 } from './barrel.mjs';

console.log('named   :', add(2, 3), sq(4), TAU.toFixed(4));
console.log('default :', log('info', 'server started'), '| levels:', LEVELS.join(','));
console.log('ns keys :', Object.keys(MathNS));
console.log('barrel  :', toThePowerOfThree(2), typeof counterNS.increment, log2 === log);
console.log('polyfill:', globalThis.polyfilled);
```

Output:

```
[side_effect.mjs] patched globalThis
[math.mjs] evaluated
named   : 5 16 6.2832
default : [INFO] server started | levels: debug,info,warn,error
ns keys : [ 'TAU', 'add', 'square', 'toThePowerOfThree' ]
barrel  : 8 function true
polyfill: true
```

Import form	What you get
`import x from './m.mjs'`	the default export, under any name you choose
`import { a, b } from './m.mjs'`	named exports (names must match)
`import { a as z } from './m.mjs'`	named export, renamed locally
`import * as ns from './m.mjs'`	a **namespace object** with every named export as a property (the default is `ns.default`)
`import x, { a } from './m.mjs'`	default + named together
`import './m.mjs'`	runs the module for its side effects (things it does on load besides exporting — e.g. polyfills, which add missing built-in features; CSS in bundlers; registering elements)

`side_effect.mjs` and `math.mjs` printed **before** the importer's body, and `math.mjs` ran once despite three importing statements: imports are processed first and each module is evaluated exactly once.

A **module specifier** is the string after `from`:

- **Relative** — `'./math.mjs'`, `'../lib/x.js'`. Browsers and Node ESM need the extension (no `.js` guessing; CommonJS and bundlers do guess).
- **Absolute URL** — `'https://cdn.example.com/lib.js'` (browsers; a CDN serves files from servers near the user — System Design Fundamentals).
- **Bare** — `'react'`: a package name, resolved by Node in `node_modules` (where installed packages live); browsers need an import map or bundler (§10.7).
- **Built-in** — `'node:fs'`, `'node:path'`; `node:` marks a core module unambiguously.

Default vs named exports

	Named exports	Default export
Importer's name	Must match (rename with `as`)	Anything — and different files may call it different things
Refactoring (restructuring code without changing behaviour) / IDE (code editor such as VS Code) auto-import	Rename-safe, auto-import finds it	Weaker: the name isn't tied to the export
Tree shaking (removing unused code from the final build, §10.11)	Fine-grained	Works, but a default *object* of many functions is all-or-nothing
Typical use	Utility modules, libraries	One main thing per file: a React component, a class, a config

Many teams prefer named exports (`eslint-plugin-import`'s `import/no-default-export` enforces it); Next.js (a React framework for full websites) requires default exports for page files. Be consistent.

Common mistakes

- `import { default } from …` — reserved word; use `import x from …` or `import { default as x } from …`.
- Missing extension (`'./math'`) in native ESM → `ERR_MODULE_NOT_FOUND`.
- Big barrels in apps: importing one function from `index.js` can make tools and test runners load every re-exported module — import the specific file in hot paths (startup or very frequent code).
- `export default const x = 1` is a syntax error (`export default` takes an expression or function/class declaration).

Self-check

1. Default exports per module? → At most one.
2. Does `export * from './m.mjs'` include the default? → No.
3. `import * as ns` contents? → All named exports as read-only properties, plus `default` if any.

TL;DR

Named: many, name must match. Default: one, importer names it. `import * as ns` → namespace; `import './x'` → side effects. Always write extensions in native ESM.

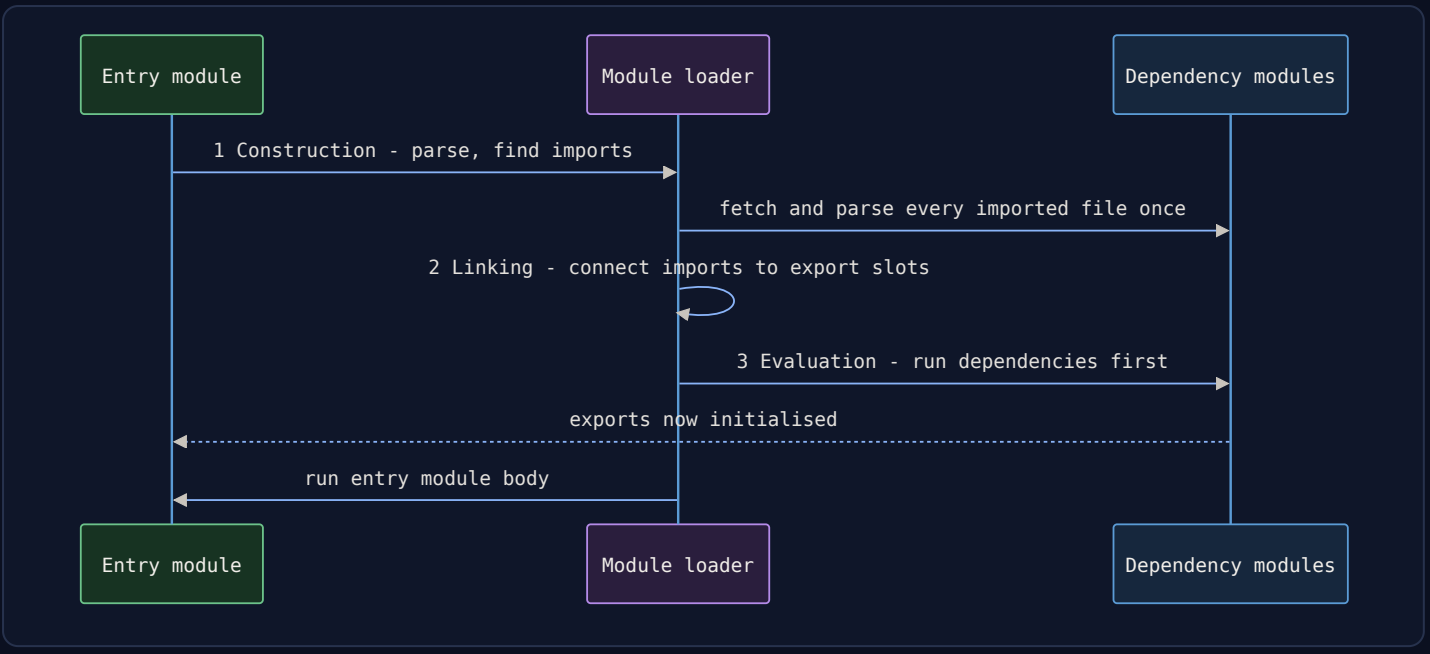
10.3 How ES modules load

What it is

ES modules are **static**: `import` / `export` only at a module's top level (not in `if`s or functions) with string-literal specifiers, so the engine knows the whole graph *before running any code*.

How it works — three phases

1. **Construction (parse)** — from the entry point, fetch and parse each file, follow its imports until the graph is complete. Nothing runs. A **module map** records each URL, so each module is fetched once.
2. **Linking (instantiation)** — create module environments and wire each import to the exporter's variable slot — *references, not copies*. Missing named exports fail here, before any code runs (`SyntaxError: The requested module … does not provide an export named …`).
3. **Evaluation** — run each module's top level exactly once, **dependencies first** (depth-first post-order: children before parents, siblings in import order).



Property	Meaning
Hoisting	Imports are resolved before the body runs, wherever the `import` line appears in the file.
Singletons	Each module is evaluated once per program (per realm — one separate global environment, such as a browser tab or iframe); every importer shares the same instance and state.
Live bindings	An import is a live, **read-only** view of the exporter's variable. If the exporter changes it, importers see the change.
Strict mode	Module code is always strict (no `'use strict'` needed).
Own scope	Top-level `const` / `let` / `function` are module-scoped, not globals. `this` at top level is `undefined`.
Deferred	In browsers, module scripts don't block HTML parsing (§10.7).

`code/counter.mjs` and `code/02_live_bindings.mjs`:

```
// counter.mjs
export let count = 0;
export function increment() { count++; }
```

```
// 02_live_bindings.mjs (abridged — each output line labels its check)
import { count, increment } from './counter.mjs';
import * as ns from './math.mjs';
import * as again from './math.mjs';     // same module -> evaluated once, same namespace object

console.log('before     :', count);
increment(); increment();
console.log('after x2   :', count, '(the import sees the new value)');
try { count = 100; } catch (e) { console.log('assign     :', e.name, '-', e.message); }   // imports are read-only
// ... then checks ns === again, the namespace's tag/sealing, top-level `this`, globals and strict mode
```

Output (full run):

```
[math.mjs] evaluated
before     : 0
after x2   : 2 (the import sees the new value)
assign     : TypeError - Assignment to constant variable.
same ns?   : true
ns tag     : [object Module]
ns sealed  : true | frozen: false
ns write   : TypeError - Cannot assign to read only property 'add' of object '[object Module]'
this       : undefined
global?    : false
strict     : ReferenceError - undeclared is not defined
```

- `count` went 0 → 2 in the importer; CommonJS `const { count } = require(...)` would have copied `0`.
- Assigning an import throws like a `const` even though the exporter used `let`; only the exporter can change it.
- The namespace is sealed with non-writable properties; "not frozen" only because its values stay live.

Circular imports

A **circular dependency**: A imports B and B (directly or indirectly) imports A. ESM allows it — one module runs before the other finishes. Rule: **top-level code must not use an imported `let` / `const` / `class` not yet initialized** (it's in its **TDZ**, temporal dead zone). Use inside later-running functions is fine, and function declarations are initialized at link time, so they're usable early.

`code/07_circular_imports.mjs` loads two tiny cycles:

```
// cyc_a.mjs                                  // cyc_b.mjs
import { b } from './cyc_b.mjs';              import { a } from './cyc_a.mjs';
export const a = 'A';                         export const b = `B knows ${a}`; // a in TDZ
console.log('[cyc_a] sees', b);
```

```
// cyc_safe_a.mjs                             // cyc_safe_b.mjs
import { describeB } from './cyc_safe_b.mjs'; import { nameA } from './cyc_safe_a.mjs';
export function nameA() { return 'A'; }       export function describeB() { return `B calls ${nameA()}`; }
export const result = describeB();
```

```
try {
  await import('./cyc_a.mjs');
} catch (e) {
  console.log('unsafe cycle:', e.name, '-', e.message);
}
const { result } = await import('./cyc_safe_a.mjs');
console.log('safe cycle  :', result);
```

Output:

```
unsafe cycle: ReferenceError - Cannot access 'a' before initialization
safe cycle  : B calls A
```

`cyc_b` evaluates first and reads `a` before `cyc_a` has run; in the safe version `nameA` is a hoisted function declaration. Best real fix: **break the cycle** by moving the shared piece into a third module.

In the interview

30-second answer: "ES modules are static: the loader parses the whole graph, links imports to exports, then evaluates each module once, dependencies first. Imports are hoisted, read-only live bindings, and modules are strict singletons with their own scope. That static structure enables tree shaking. Cycles work as long as top-level code doesn't touch a binding still in its TDZ."

- *Q: ESM live bindings vs CommonJS exports?* A: `require` returns `module.exports`; destructuring copies values at that moment. ESM imports stay connected to the exporter's variable.
- *Q: Why no `import` inside `if`?* A: Static imports must be known before execution; use `import()`.

Self-check

1. Two files import `config.mjs` — top level runs how often? → Once.
2. Can an importer reassign an import? → No — `TypeError`.
3. Phase that reports a missing named export? → Linking.

TL;DR

Parse → link → evaluate once, dependencies first. Imports: hoisted, live, read-only; modules: strict singletons. Avoid top-level use of bindings in cycles.

10.4 Dynamic `import()`

What it is

`import(specifier)` (ES2020) is special syntax that works anywhere (`if`, functions, handlers), accepts computed strings, and returns a **promise** of the module's namespace object.

Why it matters

- **Lazy loading / code splitting** — load a heavy chart library only when its tab opens; bundlers emit each `import()` as a separate on-demand file (**chunk**, §10.11).
- **Conditional loading** — by platform, feature flag (on/off switch for a feature) or locale (`import(\`./locales/${lang}.mjs`)`).
- **From CommonJS** — the only way for CJS to load an ES module that uses top-level `await`.

`code/03_dynamic_import.mjs`:

```
// 03_dynamic_import.mjs (abridged)
const verbose = process.env.VERBOSE !== '0';   // decide at runtime whether to load the logger
if (verbose) {
  const { default: log } = await import('./logger.mjs');  // default export is the "default" key
  console.log(log('debug', 'logger loaded lazily'));
}
const dyn = await import(`./${name}.mjs`);                // computed specifier
const [m, l] = await Promise.all([import('./math.mjs'), import('./logger.mjs')]);  // in parallel
try { await import('./does-not-exist.mjs'); } catch (e) { console.log('missing  :', e.code); }  // rejected promise
```

Output (full run — the file also compares `dyn` with a static import and loads `node:path`):

```
[math.mjs] evaluated
[DEBUG] logger loaded lazily
same instance as static import? true
parallel : 2 4
missing  : ERR_MODULE_NOT_FOUND
node:path: report.pdf
```

Trade-offs

	Static `import`	Dynamic `import()`
Where	top level only	anywhere
Specifier	string literal	any expression
Timing	before the module body runs	when that line runs (async)
Tree shaking / analysis	full	limited — tools can't know a fully computed path
Errors	whole graph fails to load	a catchable rejected promise

Common mistakes

- Forgetting the default export is `.default`: `(await import('./x.mjs')).default(...)`.
- Fully dynamic paths (`import(userInput)`) — un-prebuildable, and a security risk with untrusted input.

TL;DR

`import()` loads on demand and returns a promise of the namespace — for code splitting and conditional loading.

10.5 Top-level `await`

What it is

Since ES2022, `await` works at the **top level of an ES module** (not classic scripts or CommonJS). Such an **async module** makes its importers wait until it finishes evaluating.

`code/slow_config.mjs` and `code/04_top_level_await.mjs`:

```
// slow_config.mjs
console.log('[slow_config] start loading');
const loadSettings = () => new Promise((resolve) => setTimeout(() => resolve({ theme: 'dark', retries: 3 }), 50));
export const settings = await loadSettings();    // importers wait until this resolves
console.log('[slow_config] done');
```

```
// 04_top_level_await.mjs
console.log('main body starts (after all static imports finished)');
import { settings } from './slow_config.mjs';
console.log('settings:', settings);

const results = await Promise.all([1, 2, 3].map(async (n) => n * 10));
console.log('awaited at top level:', results);
```

Output:

```
[slow_config] start loading
[slow_config] done
main body starts (after all static imports finished)
settings: { theme: 'dark', retries: 3 }
awaited at top level: [ 10, 20, 30 ]
```

The `import` written after the first `console.log` still ran first (hoisting), and the importer waited for `slow_config`'s `await`.

Why it matters / real-world examples

- Loading config, a WebAssembly module (fast compiled code, often from C/Rust), or a database connection before exporting it.
- Scripts and CLIs (command-line programs) without `(async () => { … })()` wrappers.

- Fallbacks: `let lib; try { lib = await import('fast-lib'); } catch { lib = await import('./slow-fallback.mjs'); }`.

When not to use it

- **Widely imported library modules** — every importer waits; a slow `await` blocks startup. Export an async function (`export async function connect()`) and let the app decide.
- **Modules loaded via `require()`** — `ERR_REQUIRE_ASYNC_MODULE` (§10.8).

TL;DR

ES modules only; importers wait. Great for entry points and scripts, risky in shared libraries.

10.6 `import.meta` and JSON modules

What it is

- `import.meta` — host-provided **metadata** (data about the module, not its contents) for the current module. Standard: `import.meta.url` (full URL) and `import.meta.resolve(specifier)` (specifier → URL relative to this module).
- **Import attributes** (ES2025) — a `with { … }` clause telling the host how to interpret a file. First use: **JSON modules**, `import data from './data.json' with { type: 'json' }`. Required so a server can't trick a page into *executing* a file it expected to be data.

`code/05_import_meta_and_json.mjs` (with `code/data.json`):

```
import config from './data.json' with { type: 'json' };
import path from 'node:path';

console.log('json     :', config.app, config.version, config.features);
console.log('url      :', import.meta.url.startsWith('file://'), path.basename(import.meta.url));
console.log('filename :', path.basename(import.meta.filename));   // Node 20.11+
console.log('dirname  :', path.basename(import.meta.dirname));
console.log('resolve  :', path.basename(import.meta.resolve('./math.mjs')));
console.log('is main? :', import.meta.main);                     // Node 24.2+, early-stage feature
```

Output:

```
json     : notes 2.1.0 [ 'search', 'tags' ]
url      : true 05_import_meta_and_json.mjs
filename : 05_import_meta_and_json.mjs
dirname  : code
resolve  : math.mjs
is main? : true
```

(Base names only, so output doesn't depend on the book's location.)

Property	Where	Replaces
`import.meta.url`	browsers + Node	—
`import.meta.resolve()`	browsers + Node (20.6+ unflagged — usable without a special command-line switch)	`require.resolve`
`import.meta.filename` / `.dirname`	Node 20.11+	CommonJS `__filename` / `__dirname`
`import.meta.main`	Node 24.2+ (early development)	`require.main === module`

A JSON module's default export is the parsed value (no named exports), evaluated once and shared — treat it as read-only. For runtime data (uploads etc.) use `fs` or `fetch` + `JSON.parse` (Chapter 5, Chapter 15). The earlier `assert { type: 'json' }` syntax was replaced by `with` and removed from V8/Node — update old code.

TL;DR

`import.meta.url` identifies the module (Node adds `filename` / `dirname`); import JSON with `with { type: 'json' }` (ES2025).

10.7 Modules in the browser

How it works

```
<script type="module" src="/js/main.js"></script>
<script type="module">
  import { add } from './js/math.js';
  document.title = `2 + 3 = ${add(2, 3)}`;
</script>
```

(HTML — not run.) Versus a classic `<script>`:

	Classic `<script>`	`<script type="module">`
Scope	top-level `var`/functions become globals	module scope
Strict mode	opt-in	always
Top-level `this`	`window`	`undefined`
Loading	blocks HTML parsing unless `defer` / `async`	**deferred by default** (runs after parsing, in order); `async` also allowed
Executes	every time it's included	once per URL
Cross-origin (from a different domain)	loads without CORS	must be served with CORS (Cross-Origin Resource Sharing: headers in which a server says which other sites may use its files) headers
`file://` pages	work	blocked by browsers — use a local dev server (a small web server on your own machine for development)
`import` / `export` / top-level `await`	✗	✓

`<script nomodule>` runs only in browsers *without* module support — once used for fallback bundles, rarely needed now: all evergreen (self-updating) browsers have supported modules since about 2018.

Bare specifiers and import maps

Browsers can't resolve `import { h } from 'preact'` — there's no `node_modules` on the web. An **import map** (all major browsers since 2023) maps bare names:

```
<script type="importmap">
{
  "imports": {
    "preact": "https://cdn.jsdelivr.net/npm/preact@10/dist/preact.module.js",
    "utils/": "/js/utils/"
  }
}
</script>
<script type="module">
  import { h } from 'preact';
  import { slugify } from 'utils/strings.js';
</script>
```

(HTML — not run.) Good for small apps and prototypes without a **build step** (automated processing before release); larger apps still bundle (§10.11) for tree shaking, **minification** (removing spaces, shortening names) and fewer requests. Browser loading and CORS: Chapter 14.

TL;DR

`<script type="module">`: module scope, strict, deferred, once per URL, CORS, no `file://`. Import maps resolve bare names.

10.8 CommonJS vs ES modules in Node

What it is

CommonJS (CJS), Node's original system: `require(path)` synchronously loads and runs a file and returns its `module.exports`. Node wraps each CJS file in `(function (exports, require, module, __filename, __dirname) { … })` — the source of those five "globals". Still everywhere on npm, so you must read, write and mix both.

How Node decides which system a file uses

1. `.mjs` → ESM; `.cjs` → CommonJS.
2. `.js` → nearest `package.json`: `"type": "module"` → ESM; `"type": "commonjs"` → CJS.
3. `.js` with **no** `"type"` → CJS, *unless* it contains ESM-only syntax (`import`/`export`, top-level `await`); since Node 22.7 it's then re-run as ESM (**syntax detection**). Set `"type"` explicitly in real projects.

This book uses `.mjs`/`.cjs` so there's never doubt.

Side-by-side comparison

	CommonJS	ES modules
Syntax	`const x = require('./x')` / `module.exports = …` / `exports.a = …`	`import` / `export`
Loading	synchronous, at runtime, can be inside `if`	static graph, async-capable; `import()` for dynamic
Exports	a single object (copied values when destructured)	live, read-only bindings
Top-level `await`	✗	✓
Strict mode	opt-in	always
Top-level `this`	`module.exports`	`undefined`
`__dirname` / `__filename`	✓	`import.meta.dirname` / `.filename`
File extensions	optional (`require('./x')` finds `x.js`, `x.json`, `x/index.js`)	required for relative paths
JSON	`require('./data.json')`	`import d from './data.json' with { type: 'json' }`
Cache	`require.cache` (can be deleted to reload)	internal module map (no official reload)
Tree shaking	poor (dynamic)	good (static)
Browsers	✗ (needs a bundler)	✓ native

Interop

- **ESM importing CJS** — the **default import** is `module.exports`; **named imports** work when Node can detect them by statically scanning the CJS source (patterns like `exports.name = …`, `module.exports = { name, … }`).

- **CJS requiring ESM** — `require(esm)` is on by default since Node 22.12 / 20.19 (no warning since 23.5 / 22.13; officially stable in 25.4), returning the namespace object. It **fails with** `ERR_REQUIRE_ASYNC_MODULE` if the ESM graph has top-level `await` — use `await import()`.
- `createRequire(import.meta.url)` gives ESM a real `require` — for CJS-only files or `require.resolve`.

`code/legacy.cjs`, `code/uses_esm.cjs` and `code/06_commonjs_interop.mjs`:

```
// legacy.cjs — CommonJS
const path = require('node:path');
console.log('[legacy.cjs] this === module.exports:', this === module.exports, '| __filename:',
path.basename(__filename));
let calls = 0;
function greet(name) { calls++; return `hello ${name} (call ${calls})`; }
module.exports = { greet, get calls() { return calls; } };
module.exports.version = '1.0.0';
```

```
// uses_esm.cjs — CommonJS requiring an ES module
const math = require('./math.mjs');
module.exports = { sum: math.add(20, 22), keys: Object.keys(math) };
```

```
// 06_commonjs_interop.mjs
import legacy from './legacy.cjs';                    // default import = module.exports
import { greet } from './legacy.cjs';                 // named import (detected by static analysis)
import { createRequire } from 'node:module';

console.log('default  :', legacy.greet('ESM'), '| version', legacy.version);
console.log('named    :', greet('named'));

const require = createRequire(import.meta.url);     // a require() usable inside ESM
const again = require('./legacy.cjs');
console.log('cached   :', again === legacy, '| calls so far', again.calls);

const fromCjs = require('./uses_esm.cjs');          // CJS that itself require()s an ESM file
console.log('require(esm):', fromCjs);

console.log('cache key:', Object.keys(require.cache).some((k) => k.endsWith('legacy.cjs')));
```

Output:

```
[legacy.cjs] this === module.exports: true | __filename: legacy.cjs
default  : hello ESM (call 1) | version 1.0.0
named    : hello named (call 2)
cached   : true | calls so far 2
[math.mjs] evaluated
require(esm): { sum: 42, keys: [ 'TAU', 'add', 'square', 'toThePowerOfThree' ] }
cache key: true
```

- `legacy.cjs` ran once; ESM import and later `require` share the cached instance (counter 1 → 2).
- `version`, added *after* `module.exports = {…}`, isn't detected as a named export but is on the default import — when in doubt with CJS, use the default import.

Common mistakes

- `exports = { a }` exports nothing — only `module.exports = …` or `exports.a = …`.
- `ReferenceError: require is not defined in ES module scope` — use `import` or `createRequire`.
- `SyntaxError: Cannot use import statement outside a module` — Node treated the file as CJS (`"type": "commonjs"` up the tree, or `.cjs`); rename to `.mjs` or set `"type": "module"`.
- `__dirname is not defined` in ESM — `import.meta.dirname`.
- **Dual package hazard**: a package shipping CJS and ESM builds may load *twice* — two copies of state, and `instanceof` failing across them.

Package entry points: `"exports"`

`package.json` can declare which files outsiders may import and which file each system gets:

```
{
  "name": "tiny-date",
  "type": "module",
  "exports": {
    ".": { "import": "./dist/index.js", "require": "./dist/index.cjs" },
    "./format": "./dist/format.js",
    "./package.json": "./package.json"
  }
}
```

(Not run.) With `"exports"`, `import 'tiny-date/dist/secret.js'` fails with `ERR_PACKAGE_PATH_NOT_EXPORTED` — internals are encapsulated. The older `"main"` named one entry and exposed everything. `"imports"` (keys starting `#`) defines private in-package aliases, e.g. `import db from '#db'`.

In the interview

30-second answer: "CommonJS is Node's original synchronous `require` / `module.exports`; ES modules are the standard with static `import` / `export`, live bindings, top-level `await` and browser support. Node chooses by extension (`.mjs` / `.cjs`) or `package.json` `type`. ESM can import CJS (default = `module.exports`), and modern Node can `require()` ESM unless it uses top-level `await`. New code should be ESM."

- *Q: Why can't `require()` load ESM with top-level await?* A: `require` is synchronous and must return immediately; an async module can't finish synchronously.
- *Q: What does `"exports"` do?* A: Defines public entry points and conditional builds (`import` vs `require`), blocking deep imports.

Self-check

1. `.js` under `"type": "module"`? → ESM.
2. `__dirname` in ESM? → `import.meta.dirname` (Node 20.11+), or `path.dirname(fileURLToPath(import.meta.url))` earlier.
3. `import x from './legacy.cjs'` gives? → `module.exports`.

TL;DR

`.mjs` / `.cjs` or `"type"` decides. CJS: sync `require`, copied values, `__dirname`. ESM: static, live, strict, TLA. They interoperate; prefer ESM, declare entry points with `"exports"`.

10.9 npm and `package.json`

What it is

- **npm** = the **registry** (npmjs.com, millions of public packages) + the **CLI** (command-line interface, `npm`, installed with Node) + the company running them.
- A **package** is a folder with a `package.json`. What you depend on is a **dependency**; their dependencies are **transitive dependencies**.
- `package.json` is the project manifest: name, version, module type, scripts, dependencies.

Alternatives on the same registry: **pnpm** (content-addressed store — each file stored once, named by its contents — strict `node_modules`, saves disk), **Yarn**, **Bun** (runtime with built-in package manager). Pick one per project and commit its lockfile (exact installed versions, §10.10).

A typical `package.json`

```
{
  "name": "notes-app",
  "version": "2.1.0",
  "private": true,
  "type": "module",
  "engines": { "node": ">=22" },
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "node --test",
    "lint": "eslint .",
    "format": "prettier --write .",
    "check": "npm run lint && npm test"
  },
  "dependencies": {
    "date-fns": "^4.1.0"
  },
  "devDependencies": {
    "eslint": "^10.0.0",
    "prettier": "^3.3.0",
    "vite": "^8.0.0"
  }
}
```

(Not run; versions are examples.)

Field	Meaning
`name`, `version`	Identity. Required if you publish. `version` follows semver (§10.10).
`private: true`	Prevents accidental `npm publish`.
`type`	`"module"` → `.js` files are ESM.
`main` / `exports`	Entry points for people importing your package (§10.8).
`scripts`	Named commands: `npm run <name>`; `npm test` and `npm start` are shortcuts.
`dependencies`	Needed at runtime (installed for anyone who installs your package).
`devDependencies`	Needed only to develop/build/test (linters, bundlers, test runners).
`peerDependencies`	"I work with *your* copy of X" — plugins (add-ons that extend another tool) declare the host they plug into (e.g., a React component library declares `react`). npm 7+ installs them automatically.
`optionalDependencies`	Install failures are tolerated.
`engines`	Supported Node versions (advisory unless enforced).
`bin`	Command-line executables the package provides.
`files`	What to include when publishing.

Everyday commands (not run)

```
npm init -y                     # create package.json with defaults
npm install                     # install everything in package.json (and update the lockfile if needed)
npm install date-fns            # add a runtime dependency (saved as "^<latest>")
npm install -D eslint prettier  # add dev dependencies (--save-dev)
npm uninstall date-fns          # remove
npm ci                          # clean, exact install from package-lock.json (CI servers)
npm run build                   # run a script; npm test / npm start need no "run"
npm run test -- --watch         # pass extra args to the script after --
npx eslint .                    # run a package's binary (local first, else download temporarily)
npm outdated                    # which dependencies have newer versions
npm update                      # upgrade within the allowed ranges
npm audit                       # check for known vulnerabilities (npm audit fix to patch)
npm ls react                    # why / where is a package in the tree
npm version patch               # bump version 2.1.0 -> 2.1.1, commit and tag
node --run test                 # Node 22+: run a script without npm's startup cost
node --watch server.mjs         # restart on file change (stable since Node 22)
```

`npm run` and `node --run` put `./node_modules/.bin` on the `PATH` (folders the shell searches for commands), so scripts say `eslint .` without global installs. `npm run build` also runs `prebuild` / `postbuild` if present; `node --run` skips those hooks and package-manager environment variables for speed.

How `node_modules` resolution works

For `import 'date-fns'` in `/app/src/utils/x.mjs`, Node checks `node_modules/date-fns` in `/app/src/utils`, `/app/src`, `/app`, `/` — first match wins — then reads its `package.json` (`exports`, else `main`). npm **hoists** (flattens) most transitive dependencies to the top level to share them; incompatible versions get nested copies.

Consequence: **phantom dependencies** — importing an undeclared package that a dependency happened to hoist; it breaks when that dependency changes. pnpm's strict layout prevents this.

Common mistakes

- Global installs (`npm install -g`) make the project depend on your machine — use devDependencies + scripts/`npx`.
- Build tools in `dependencies` (bloat) or runtime libs in `devDependencies` (breaks production).
- Committing `node_modules` (never); *not* committing the lockfile (always commit it).
- `npx` with a typo may download and run a look-alike package.

TL;DR

`package.json` = identity + type + scripts + dependencies. `dependencies` runtime, `devDependencies` tooling; tools via scripts/`npx`; `node_modules` searched upward.

10.10 Semantic versioning, ranges and lockfiles

What it is

Semantic versioning (semver): `MAJOR.MINOR.PATCH`.

Bump	When	Example	Should upgrading break you?
MAJOR	incompatible API (the functions and options a package exposes) change	3.4.1 → 4.0.0	May — read the changelog (the list of changes in each release)
MINOR	new feature, backward compatible	3.4.1 → 3.5.0	No
PATCH	backward-compatible bug fix	3.4.1 → 3.4.2	No
Pre-release	unstable preview	4.0.0-beta.2	Anything goes

`0.x` = **initial development** — anything may change, so tools treat a `0.x` minor like a major.

Version ranges in `package.json`

Range	Means	Allows
`1.2.3`	exactly	1.2.3 only
`^1.2.3` (caret — npm's default)	same **major**	≥1.2.3 <2.0.0
`^0.2.3`	same **minor** when major is 0	≥0.2.3 <0.3.0
`^0.0.3`	exact patch when major and minor are 0	≥0.0.3 <0.0.4
`~1.2.3` (tilde)	same **minor**	≥1.2.3 <1.3.0
`1.x` / `1` / `1.*`	any 1.y.z	≥1.0.0 <2.0.0
`>=1.2 <1.5`	explicit comparators	as written
`1.2 - 1.4`	hyphen range	≥1.2.0 ≤1.4.x
`^1 || ^2`	either range	
`*` / `latest`	anything	avoid

Caret in one sentence: **don't change the left-most non-zero number.**

`code/08_semver.mjs` models these rules (ignoring pre-release tags and `||`/hyphen ranges; real projects use the `semver` package npm itself uses):

```
// code/08_semver.mjs (core idea): turn a range into [low, high) bounds, then test low <= version < high
// '^' -> bump the left-most non-zero part; '~' -> bump minor (or major if only major given); exact/x-range -> bump the last given part
```

Output:

```
^1.2.3  >=1.2.3 <2.0.0         matches: 1.2.3, 1.2.9, 1.3.0, 1.9.9
~1.2.3  >=1.2.3 <1.3.0         matches: 1.2.3, 1.2.9
1.2.3   >=1.2.3 <1.2.4         matches: 1.2.3
1.x     >=1.0.0 <2.0.0         matches: 1.2.3, 1.2.9, 1.3.0, 1.9.9
^0.2.3  >=0.2.3 <0.3.0         matches: 0.2.5
^0.0.3  >=0.0.3 <0.0.4         matches: 0.0.3
~1      >=1.0.0 <2.0.0         matches: 1.2.3, 1.2.9, 1.3.0, 1.9.9
```

Lockfiles

With `^4.1.0`, installs a month apart may get 4.1.0 vs 4.3.2 — and every transitive dependency may differ. A **lockfile** (`package-lock.json`, `pnpm-lock.yaml`, `yarn.lock`, `bun.lock`) records the **exact** version, download URL and **integrity hash** (content fingerprint detecting tampering) of *every* package in the tree.

	npm install	npm ci
Reads	`package.json` + lockfile	lockfile (must match `package.json`, else error)
May change the lockfile	yes	never
Existing `node_modules`	reused/updated	deleted first
Use for	development, adding packages	CI servers (continuous integration — machines that automatically build and test every change), deploy builds, reproducible installs

Commit the lockfile, use `npm ci` in automated builds, upgrade deliberately (`npm outdated` → `npm update` or `npm install pkg@latest` → test). Dependabot and Renovate open **pull requests** (proposed changes for review, e.g. on GitHub) for updates.

Real-world examples — why this matters

- **left-pad (2016)**: an 11-line package was unpublished and thousands of builds broke; npm then restricted unpublishing.
- **Supply-chain attacks** (malicious code smuggled into your dependencies): compromised maintainers published malicious patch versions (e.g. `event-stream` in 2018, several hijacked packages in 2025). Caret ranges without a lockfile install the bad version automatically; lockfiles, `npm ci`, `npm audit` and reviewing new dependencies reduce the risk.

In the interview

> **30-second answer:** "Semver is MAJOR.MINOR.PATCH — breaking, feature, fix. `^` allows anything not changing the left-most non-zero number, `~` allows patch updates. Ranges stay flexible, but the lockfile pins the exact tree so everyone and CI install the same thing — `npm ci` installs exactly from it."

- *Q: Does `^0.3.1` allow 0.4.0?* A: No — for `0.x` caret locks the minor: ≥0.3.1 <0.4.0.
- *Q: Should a library commit its lockfile?* A: Yes, for its own dev/CI reproducibility — consumers ignore it and resolve the library's ranges themselves.

Self-check

1. Largest version `~2.4.1` allows? → Anything <2.5.0.
2. Command that never modifies the lockfile? → `npm ci`.
3. Bug fix without API change bumps? → PATCH.

TL;DR

MAJOR.MINOR.PATCH; `^` = same left-most non-zero, `~` = same minor. Commit the lockfile, `npm ci` in CI, upgrade on purpose.

10.11 Bundlers

What it is

A **bundler** follows every `import` from your entry file (including CSS, images, `node_modules` packages) to build the module graph and writes a few optimized **bundles** for the browser, transpiling, minifying and splitting along the way.

The problem it solves

Shipping a large app as hundreds of raw native-ESM files means many requests, no dead-code removal, unminified code, and no bare-name imports from `node_modules` or CSS/image/TypeScript/JSX imports. Bundlers fix all of these.

What a bundler does

Feature	Meaning
Resolution	Turns `'react'` into a real file in `node_modules`; handles extensions, `exports`, aliases.
Tree shaking	Removes exports nobody imports (dead-code elimination). Depends on ESM's static structure; packages mark safe-to-drop files with `"sideEffects": false`.
Code splitting	Each dynamic `import()` becomes a separate **chunk** loaded on demand; shared code goes into common chunks.
Minification	Removes whitespace/comments, shortens local names — smaller downloads.
Transforms	TypeScript, JSX, modern syntax → target syntax (via a transpiler, §10.12); CSS modules; asset hashing (`app.3f9a1c.js`) for long-term caching (letting browsers keep files for a long time, because a changed file gets a new name — see System Design Fundamentals).
Source maps	`.map` files that map minified output back to your original source, so DevTools and error stacks show real file names and line numbers (Chapter 11).
Dev server + HMR	Serves the app locally and, with **Hot Module Replacement**, swaps changed modules in the running page without a full reload.



The main tools (2026)

Tool	What it is	Typical use
Vite	Dev server that serves native ESM on demand (very fast start) + production build. Vite 8 (March 2026) uses **Rolldown**, a Rust bundler, for builds; earlier versions used esbuild + Rollup.	Default choice for new front-end apps (React, Vue, Svelte templates)
esbuild	Extremely fast bundler/transpiler written in Go; minimal plugin API.	Libraries, scripts, tools built on top of it (this book's runner uses it)
webpack (v5)	The long-time standard; hugely configurable, loaders and plugins for everything.	Existing large apps, custom pipelines, Module Federation (loading parts of an app from separately deployed builds at runtime)
Rollup / Rolldown	Rollup pioneered ESM tree shaking; Rolldown is its Rust-based successor with a largely compatible plugin API.	Building libraries with clean ESM output
Turbopack	Rust bundler from Vercel; the default in Next.js since version 16.	Next.js apps
Parcel	Zero-configuration bundler.	Quick projects

A Vite workflow and an esbuild one-liner (not run):

```
npm create vite@latest my-app -- --template vanilla   # scaffold a project
cd my-app && npm install
npm run dev        # dev server with HMR, e.g. http://localhost:5173
npm run build      # production bundle in dist/
npm run preview    # serve dist/ locally to check the build
npx esbuild src/main.js --bundle --minify --sourcemap --format=esm --outdir=dist
```

Trade-offs

Option	Pros	Cons
No build (native ESM + import maps)	Zero tooling, instant	No tree shaking/minify, many requests, no JSX/TS
Vite	Fast, sensible defaults, big plugin ecosystem	Dev (unbundled) and prod (bundled) differ slightly
webpack	Maximum control	Slower, configuration-heavy
esbuild directly	Blazing fast, simple	Fewer features (no HMR dev server of its own, limited code-splitting options)

When not to use a bundler

Node servers and CLIs usually run source directly (Node runs ESM, and strips TypeScript types natively since 23.6/22.18). Bundle server code only for special cases: serverless **cold starts** (delay when the platform starts a fresh copy of your function), single-file distribution.

Common mistakes

- `import _ from 'lodash'` for one function — use ESM packages and named imports (`import { debounce } from 'lodash-es'`) so tree shaking works.
- Deploying without source maps (unreadable production errors) — or publishing them when source is secret (upload to an error tracker like Sentry, which collects users' crashes, instead).
- Assuming dev equals production — run `build` + `preview` before shipping.

TL;DR

Bundlers resolve, transform, tree shake, split, minify, source-map. Vite for apps, esbuild/Rollup-style tools for libraries, webpack where a project already has it.

10.12 Transpilers and polyfills

What it is

- A **transpiler** (source-to-source compiler) rewrites JavaScript: newer → older syntax, TypeScript → JavaScript (types removed), JSX → function calls. **Babel** (plugin-based), `tsc`, **SWC** (Rust), **esbuild** (Go).
- A **polyfill** *adds a missing API at runtime* (e.g. defines `Array.prototype.at` if absent). Library: **core-js**.

Syntax (`?.`, `class`, `#private`) can't be polyfilled — old parsers reject the file — so it must be transpiled; **APIs** (`findLast`, `Object.groupBy`, `structuredClone`) can be polyfilled.

```
// Input (modern)                    // Transpiled for an old target (roughly)
const city = user?.address?.city ?? 'n/a';
                                     var _user$address;
                                     var city = (_user$address = user == null ? void 0 : user.address) == null
                                       ? void 0 : _user$address.city;
                                     city = city != null ? city : 'n/a';
```

(Illustrative, not run — exact output varies by tool.)

How targets are chosen

Tools read a **browserslist** query (`package.json` or `.browserslistrc`), e.g. `"> 0.5%, last 2 versions, not dead"` or `"baseline widely available"` (working in all major browsers for at least ~2.5 years), and transform only what those browsers lack. Vite's default target is modern "baseline widely available" browsers, so little is transpiled; evergreen-only targets keep bundles small.

TypeScript note

TypeScript: Chapter 16. Most bundlers (esbuild, Vite, SWC) **strip types without checking them**; run `tsc --noEmit` separately (CI or editor) to type-check.

TL;DR

Transpile syntax, polyfill APIs; set targets with browserslist; bundlers strip TS types — type-check separately.

10.13 Linters and formatters

What it is

- A **linter** statically analyzes code to find likely **bugs**: unused variables, use-before-define, `==` for `===`, unreachable code, missing `await`, React hook rules. **ESLint** is the standard (Biome, Oxlint are faster Rust alternatives).
- A **formatter** rewrites **layout** (indentation, quotes, semicolons, wrapping) to one style so reviews focus on logic. **Prettier** is the standard.

Formatter owns style, linter owns correctness: ESLint deprecated its formatting core rules (v8.53, 2023), and `eslint-config-prettier` turns off remaining rules that would fight Prettier.

ESLint's flat config

Since ESLint 9 the config is **`eslint.config.js`** (a module exporting an array of config objects — "flat config"); ESLint 10 (February 2026) removed the old `.eslintrc.*` format.

```
// eslint.config.js  (not run)
import js from '@eslint/js';
import globals from 'globals';
import prettier from 'eslint-config-prettier';

export default [
  js.configs.recommended,                    // ESLint's recommended rule set
  {
    files: ['**/*.{js,mjs}'],
    languageOptions: {
      ecmaVersion: 'latest',
      sourceType: 'module',
      globals: { ...globals.browser, ...globals.node },
    },
    rules: {
      'no-unused-vars': ['error', { argsIgnorePattern: '^_' }],
      eqeqeq: ['error', 'always'],           // forbid == / !=
      'prefer-const': 'warn',
    },
  },
  { ignores: ['dist/**', 'coverage/**'] },
  prettier,                                  // last: disable style rules that conflict with Prettier
];
```

Objects target files via `files` and set language options, plugins, rules; severity `"off"` / `"warn"` / `"error"` (0/1/2); later objects override earlier ones for matching files.

Commands (not run):

```
npm init @eslint/config@latest     # interactive setup that writes eslint.config.js
npx eslint .                       # lint the project; exits non-zero on errors (fails CI)
npx eslint . --fix                 # auto-fix what can be fixed safely
npx prettier . --check             # report unformatted files (CI)
npx prettier . --write             # format files in place
```

Typical ESLint output (illustrative):

```
src/cart.js
   3:7   error    'total' is assigned a value but never used  no-unused-vars
  12:15  error    Expected '===' and instead saw '=='         eqeqeq

✖ 2 problems (2 errors, 0 warnings)
```

Minimal Prettier config `.prettierrc.json`: `{ "singleQuote": true, "semi": true, "printWidth": 100 }`. Prettier is deliberately "opinionated" (few options) to end style debates.

Wiring it into the workflow

1. **Editor** — extensions show problems as you type and format on save.
2. **Pre-commit hook** — husky + lint-staged run lint/format on **staged files** (changes marked with `git add`) before each commit.
3. **CI** — `npm run lint` and `prettier --check` fail the build on misses.

Common mistakes

- ESLint and Prettier fighting over formatting — use `eslint-config-prettier`.
- Disabling warnings to "get green" — fix the cause, or disable one line with a reason (`// eslint-disable-next-line no-console -- CLI output`).
- Old `.eslintrc.json` tutorials — that format no longer works in ESLint 10.

TL;DR

ESLint finds bugs (flat config `eslint.config.js`); Prettier formats; run both in editor, pre-commit and CI.

10.14 Putting it together — a modern project

```
notes-app/
├─ package.json          # "type": "module", scripts, deps
├─ package-lock.json     # committed
├─ eslint.config.js
├─ .prettierrc.json
├─ index.html            # <script type="module" src="/src/main.js">
├─ src/
│  ├─ main.js            # entry: imports app, starts it
│  ├─ app.js
│  ├─ features/notes/    # feature folders, named exports
│  └─ lib/               # shared helpers
├─ test/                 # node --test (Chapter 16)
└─ dist/                 # build output (git-ignored)
```

Daily loop: `npm run dev` (Vite + HMR) → edit → editor lints/formats on save → `npm test` → commit (pre-commit hook lints staged files) → CI runs `npm ci`, `npm run lint`, `npm test`, `npm run build` → deploy `dist/`.

Chapter 10 TL;DR

- **Modules** give files their own scope; **ES modules** are the standard in browsers and Node.

- **Named exports** (many, names match) vs **default** (one, importer names it); `import * as ns`; `export … from` for barrels.
- ESM loads **parse** → **link** → **evaluate**: imports **hoisted, live, read-only**; modules **strict singletons**; cycles work if top-level code avoids TDZ bindings.
- `import()` loads on demand; **top-level** `await` makes importers wait — fine in apps, risky in libraries.
- `import.meta.url` / `dirname` / `filename`; JSON via `with { type: 'json' }` (ES2025).
- Browsers: `<script type="module">` is deferred, strict, CORS-bound; **import maps** resolve bare names.
- Node: `.mjs` / `.cjs` / `"type"` choose the system; ESM imports CJS (default = `module.exports`); `require(esm)` unless top-level `await`; `"exports"` defines public entry points.
- **npm**: `dependencies` vs `devDependencies`, scripts, `npx`, `node --run`; **semver** `^` / `~`; **commit the lockfile**, `npm ci` in CI.
- **Bundlers** (Vite, esbuild, webpack, Rolldown): tree shaking, code splitting, minification, source maps, HMR. **Transpile syntax, polyfill APIs.**
- **ESLint** (flat config) finds bugs; **Prettier** formats; run both in editor, hooks and CI.

Next: when things go wrong — Chapter 11: Errors and Debugging.